(12) INTERNATIONAL APPLICATION PUBLISHED UNDER THE PATENT COOPERATION TREATY (PCT)

(19) World Intellectual Property Organization
International Bureau

(43) International Publication Date
27 September 2018 (27.09.2018)

WIPO | PCT

(10) International Publication Number
WO 2018/174935 A1

(51) International Patent Classification:
G06F 9/30 (2006.01)

(21) International Application Number:
PCT/US2017/040546

(22) International Filing Date:
01 July 2017 (01.07.2017)

(25) Filing Language: English

(26) Publication Language: English

(30) Priority Data:
62/473,732 20 March 2017 (20.03.2017) US

(71) Applicant: **INTEL CORPORATION** [US/US]; 2200 Mission College Boulevard, Santa Clara, California 95054 (US).

(72) Inventors: **VALENTINE, Robert**; Rechov Hadganiot 33-5, 36054 Kiryat Tivon (IL). **CHARNEY, Mark J.**; 610 Waltham St, Lexington, Massachusetts 02421 (US). **OULD-AHMED-VALL, Elmoustapha**; 5000 W Chandler Blvd, M/S: CH7-401, Chandler, Arizona 85226 (US). **BAUM, Dan**; Nizanim 28 A St, 34354 Haifa (IL). **SPERBER, Zeev**; 32nd Igal Alon st., 3092832 Zichron Yackov (IL). **CORBAL, Jesus**; Jordi Girona 1-3 Intel Labs, p3, Nexus II, 08034 Barcelona (ES). **TOLL, Bret L.**; 2868 NE Lorie Dr., Hillsboro, Oregon 97124 (US). **SADE, Raanan**; Zivony 12, 3658900 Kibutz Sarid (IL). **YANOVER, Igor**; Gilboa 10A, 20692 Yokneam Illit (IL). **GEBIL, Yuri**; Keren Hayesod 24/1, 2244771 Nahariya (IL). **RAPPOPORT, Rinat**; 14 Vardia st., 34657 Haifa (IL). **SHWARTSMAN, Stanislav**; st Ha-Hinnanit 6/29, 35849 Haifa (IL). **ADELMAN, Menachem**; Hatichon 31A Apt. 5, 3229624 Haifa (IL). **RUBANOVICH, Simon**; Egoz str. 10/4, 34792 Haifa (IL).

(74) Agent: **NICHOLSON, David F.**; Nicholson De Vos Webster & Elliott, LLP, 99 Almaden Boulevard, Suite 710, San Jose, California 95113 (US).

(81) Designated States *(unless otherwise indicated, for every kind of national protection available)*: AE, AG, AL, AM, AO, AT, AU, AZ, BA, BB, BG, BH, BN, BR, BW, BY, BZ, CA, CH, CL, CN, CO, CR, CU, CZ, DE, DJ, DK, DM, DO, DZ, EC, EE, EG, ES, FI, GB, GD, GE, GH, GM, GT, HN, HR, HU, ID, IL, IN, IR, IS, JO, JP, KE, KG, KH, KN, KP, KR, KW, KZ, LA, LC, LK, LR, LS, LU, LY, MA, MD, ME, MG, MK, MN, MW, MX, MY, MZ, NA, NG, NI, NO, NZ, OM, PA, PE, PG, PH, PL, PT, QA, RO, RS, RU, RW, SA, SC, SD, SE, SG, SK, SL, SM, ST, SV, SY, TH, TJ, TM, TN, TR, TT, TZ, UA, UG, US, UZ, VC, VN, ZA, ZM, ZW.

(84) Designated States *(unless otherwise indicated, for every kind of regional protection available)*: ARIPO (BW, GH, GM, KE, LR, LS, MW, MZ, NA, RW, SD, SL, ST, SZ, TZ, UG, ZM, ZW), Eurasian (AM, AZ, BY, KG, KZ, RU, TJ, TM), European (AL, AT, BE, BG, CH, CY, CZ, DE, DK, EE, ES, FI, FR, GB, GR, HR, HU, IE, IS, IT, LT, LU, LV, MC, MK, MT, NL, NO, PL, PT, RO, RS, SE, SI, SK, SM,

(54) Title: SYSTEMS, METHODS, AND APPARATUS FOR MATRIX OPERATIONS

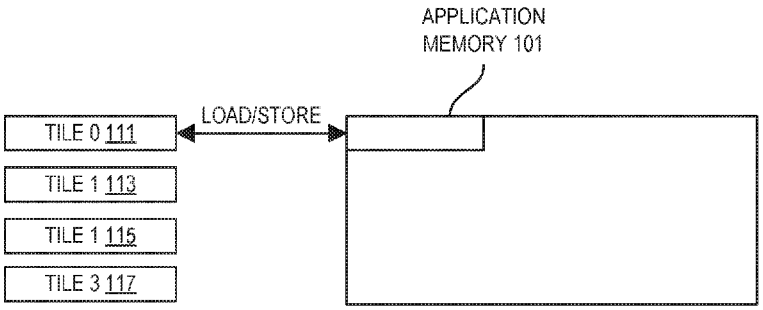


FIG. 1

(57) Abstract: Embodiments detailed herein relate to matrix (tile) operations. For example, decode circuitry to decode an instruction having fields for an opcode and a memory address; and execution circuitry to execute the decoded instruction to set a tile configuration for the processor to utilize tiles in matrix operations based on a description retrieved from the memory address, wherein a tile a set of 2-dimensional registers are discussed.

[Continued on next page]

WO 2018/174935 A1

TR), OAPI (BF, BJ, CF, CG, CI, CM, GA, GN, GQ, GW, KM, ML, MR, NE, SN, TD, TG).

Declarations under Rule 4.17:

— *of inventorship (Rule 4.17(iv))*

Published:

— *with international search report (Art. 21(3))*

SYSTEMS, METHODS, AND APPARATUS FOR MATRIX OPERATIONS

FIELD OF INVENTION

[0001] The field of invention relates generally to computer processor architecture, and, more specifically, to matrix manipulation.

BACKGROUND

[0002] Matrices are increasingly important in many computing tasks such as machine learning and other bulk data processing.

BRIEF DESCRIPTION OF THE DRAWINGS

[0003] The present invention is illustrated by way of example and not limitation in the figures of the accompanying drawings, in which like references indicate similar elements and in which:

[0004] FIG. 1 illustrates an embodiment of configured tiles;

[0005] FIG. 2 illustrates several examples of matrix storage;

[0006] FIG. 3 illustrates an embodiment of a system utilizing a matrix (tile) operations accelerator;

[0007] FIGS. 4 and 5 show different embodiments of how memory is shared using a matrix operations accelerator;

[0008] FIG. 6 illustrates an embodiment of matrix multiply accumulate operation using tiles ("TMMA");

[0009] FIG. 7 illustrates an embodiment of a subset of the execution of an iteration of a chained fused multiply accumulate instruction;

[0010] FIG. 8 illustrates an embodiment of a subset of the execution of an iteration of a chained fused multiply accumulate instruction;

[0011] FIG. 9 illustrates an embodiment of a subset of the execution of an iteration of a chained fused multiply accumulate instruction;

[0012] FIG. 10 illustrates an embodiment of a subset of the execution of an iteration of chained fused multiply accumulate instruction;

[0013] FIG. 11 illustrates power-of-two sized SIMD implementations wherein the accumulators use input sizes that are larger than the inputs to the multipliers according to an embodiment;

[0014] FIG. 12 illustrates an embodiment of a system utilizing matrix operations circuitry;

[0015] FIG. 13 illustrates an embodiment of a processor core pipeline supporting matrix operations using tiles;

[0016] FIG. 14 illustrates an embodiment of a processor core pipeline supporting matrix operations using tiles;

[0017] FIG. 15 illustrates an example of a matrix expressed in row major format and column major format;

[0018] FIG. 16 illustrates an example of usage of matrices (tiles);

[0019] FIG. 17 illustrates an embodiment a method of usage of matrices (tiles);

[0020] FIG. 18 illustrates an exemplary execution of a TILECONFIG instruction;

[0021] FIGS. 19(A)-(D) illustrate examples of register(s);

[0022] FIG. 20 illustrates an embodiment of a description of the matrices (tiles) to be supported;

[0023] FIG. 21 illustrates an embodiment of method performed by a processor to process a TILECONFIG instruction;

[0024] FIG. 22 illustrates a more detailed description of an execution of a TILECONFIG instruction using memory addressing;

[0025] FIG. 23 illustrates exemplary pseudocode for an execution of a TILECONFIG instruction;

[0026] FIG. 24 illustrates an exemplary execution of a TILELOAD instruction;

[0027] FIG. 25 illustrates an embodiment of method performed by a processor to process a TILELOAD instruction

[0028] FIG. 26 illustrates a more detailed description of an execution of a TILELOAD instruction;

[0029] FIGS. 27(A)-(C) illustrate examples of pseudocode representing a method of executing a TILELOAD instruction using words, doublewords, and quadwords;

[0030] Figure 28 illustrates an exemplary execution of a TILESTORE instruction;

[0031] Figure 29 illustrates an embodiment of method performed by a processor to process a TILESTORE instruction;

[0032] Figure 30 illustrates a more detailed description of an execution of a TILESTORE instruction;

[0033] FIGS. 31(A)-(C) illustrate examples of pseudocode representing methods of executing a TILESTORE instruction using words, doublewords, and quadwords;

[0034] FIG. 32 illustrates an exemplary execution of a TILEDIAGONAL instruction;

[0035] FIG. 33 illustrates an embodiment of method performed by a processor to process a TILEDIAGONAL instruction;

[0036] FIG. 34 illustrates a more detailed description of an execution of a TILEDIAGONAL instruction;

[0037] FIG. 35 is exemplary pseudocode describing an embodiment of a method performed by a processor to process a TILEDIAGONALD instruction;

[0038] FIG. 36 illustrates an exemplary execution of a TILETRANSPOSE instruction;

[0039] FIG. 37 illustrates an embodiment of method performed by a processor to process a TILETRANSPOSE instruction;

[0040] FIG. 38 illustrates a more detailed description of an execution of a TILETRANSPOSE instruction;

[0041] FIG. 39 is exemplary pseudocode describing an embodiment of a method performed by a processor to process a TILETRANSPOSED instruction;

[0042] FIG. 40 illustrates an exemplary execution of a TILEMOVE instruction;

[0043] FIG. 41 illustrates an embodiment of method performed by a processor to process a TILEMOVE instruction;

[0044] FIG. 42 illustrates a more detailed description of an execution of a TILEMOVE instruction;

[0045] FIG. 43 illustrates exemplary pseudocode for the execution of a TILEMOVE instruction;

[0046] FIG. 44 illustrates an exemplary execution of a TILEBROADCAST instruction;

[0047] FIG. 45 illustrates an embodiment of method performed by a processor to process a TILEBROADCAST instruction;

[0048] FIG. 46 illustrates a more detailed description of an execution of a TILEBROADCAST instruction;

[0049] FIG. 47 illustrates examples of pseudocode of methods for executing a TILEBROADCAST instruction;

[0050] FIG. 48 illustrates an exemplary execution of a TILEROWBROADCAST instruction;

[0051] FIG. 49 illustrates an embodiment of method performed by a processor to process a TILEROWBROADCAST instruction;

[0052] FIG. 50 illustrates a more detailed description of an execution of a TILEROWBROADCAST instruction;

[0053] FIG. 51 illustrates examples of pseudocode of methods for executing a TILECOLBROADCAST instruction;

[0054] FIG. 52 illustrates an exemplary execution of a TILECOLBROADCAST instruction;

[0055] FIG. 53 illustrates an embodiment of method performed by a processor to process a TILECOLBROADCAST instruction;

[0056] FIG. 54 illustrates a more detailed description of an execution of a TILECOLBROADCAST instruction;

[0057] FIG. 55 illustrates examples of pseudocode of methods for executing a TILECOLBROADCAST instruction;

[0058] FIG. 56 illustrates an exemplary execution of a TILEZERO instruction;

[0059] FIG. 57 illustrates an embodiment of method performed by a processor to process a TILEZERO instruction;

[0060] FIG. 58 depicts pseudocode of a method of execution of a TILEZERO instruction;

[0061] FIG. 59 illustrates an exemplary execution of a TILEADD instruction;

[0062] FIG. 60 illustrates an embodiment of method performed by a processor to process a TILEADD instruction;

[0063] FIG. 61 illustrates an example process describing a method performed by a processor to process a TILEADD instruction;

[0064] FIG. 62 illustrates an example method for performing a TILEADD operation when the source matrix (tile) operands contain single-precision elements;

[0065] FIG. 63 illustrates an exemplary execution of a TILESUB instruction;

[0066] FIG. 64 illustrates an embodiment of method performed by a processor to process a TILESUB instruction;

[0067] FIG. 65 illustrates an example process describing a method performed by a processor to process a TILESUB instruction;

[0068] FIG. 66 illustrates an example method for performing a TILESUB operation when the source matrix (tile) operands contain single-precision elements;

[0069] FIG. 67 illustrates an exemplary execution of a TILEMUL instruction;

[0070] FIG. 68 illustrates an embodiment of a method performed by a processor to process a TILEMUL instruction;

[0071] FIG. 69 illustrates an example process describing a method performed by a processor to process a TILEMUL instruction;

[0072] FIG. 70 illustrates an example method for performing a TILEMUL operation when the source matrix (tile) operands contain single-precision elements;

[0073] FIG. 71 illustrates an exemplary execution of a TMMA instruction using memory source operand;

[0074] FIG. 72 illustrates an embodiment of a method performed by a processor to process a TMMA instruction;

[0075] FIG. 73 illustrates a more detailed description of an execution of a TMMA instruction using register addressing;

[0076] FIG. 74 illustrates pseudocode for a method of implementing a TMMPS instruction;

[0077] FIG. 75 illustrates an exemplary execution of a TNMMA instruction using memory source operand;

[0078] FIG. 76 illustrates an embodiment of method performed by a processor to process a TNMMA instruction;

[0079] FIG. 77 illustrates a more detailed description of an execution of a TNMMA instruction using register addressing;

[0080] FIG. 78 illustrates an exemplary execution of a TILEDOTPRODUCT instruction;

[0081] FIG. 79 illustrates an embodiment of method performed by a processor to process a matrix (tile) dot product instruction;

[0082] FIG. 80 illustrates additional details related to an example method performed by a processor to execute a TILEDOTPRODUCT instruction;

[0083] FIGS. 81A-81G illustrate example methods for performing TILEDOTPRODUCT operations;

[0084] FIGS. 82(A)-(C) illustrate an exemplary instruction format;

[0085] FIG. 83 is a block diagram of a register architecture according to one embodiment of the invention;

[0086] FIGS. 84(A)-(B) illustrate the in-order pipeline and in-order core;

[0087] FIGS. 85(A)-(B) illustrate a block diagram of a more specific exemplary in-order core architecture, which core would be one of several logic blocks (including other cores of the same type and/or different types) in a chip;

[0088] FIG. 86 is a block diagram of a processor 8600 that may have more than one core, may have an integrated memory controller, and may have integrated graphics according to embodiments of the invention;

[0089] FIGS. 87-90 are block diagrams of exemplary computer architectures; and

[0090] FIG. 91 is a block diagram contrasting the use of a software instruction converter to convert binary instructions in a source instruction set to binary instructions in a target instruction set according to embodiments of the invention.

DETAILED DESCRIPTION

[0091] In the following description, numerous specific details are set forth. However, it is understood that embodiments of the invention may be practiced without these specific details. In other instances, well-known circuits, structures and techniques have not been shown in detail in order not to obscure the understanding of this description.

[0092] References in the specification to "one embodiment," "an embodiment," "an example embodiment," etc., indicate that the embodiment described may include a particular feature, structure, or characteristic, but every embodiment may not necessarily include the particular feature, structure, or characteristic. Moreover, such phrases are not necessarily referring to the same embodiment. Further, when a particular feature, structure, or characteristic is described in connection with an embodiment, it is submitted that it is within the knowledge of one skilled in the art to affect such feature, structure, or characteristic in connection with other embodiments whether or not explicitly described.

[0093] In many mainstream processors, handling matrices is a difficult and/or instruction intensive task. For example, rows of a matrix could be put into a plurality of packed data (e.g., SIMD or vector) registers and then operated on individually. For example, an add two 8x2 matrices may require a load or gather into four packed data registers depending upon data sizes. Then a first add of packed data registers corresponding to a first row from each matrix is performed and a second add of packed data registers corresponding to a second row from each matrix is performed. Then the resulting packed data registers are scattered back to memory. While for small matrices this scenario may be acceptable, it is often not acceptable with larger matrices.

[0094] I. High-Level Discussion

[0095] Described herein are mechanisms to support matrix operations in computer hardware such as central processing units (CPUs), graphic processing units (GPUs), and accelerators. The matrix operations utilize 2-dimensional (2-D) data structures representing one or more packed regions of memory such as registers. Throughout this description, these 2-D data structures are referred to as tiles. Note that a matrix may be smaller than a tile (use less than all of a tile), or utilize a plurality of tiles (the matrix is larger than the size of any one tile). Throughout the description, matrix (tile) language is used to indicate operations performed using tiles that impact a matrix; whether or not that matrix is larger than any one tile is not typically relevant.

[0096] Each tile may be acted upon by different operations such as those that are detailed herein and include, but are not limited to: matrix (tile) multiplication, tile add, tile subtract, tile diagonal, tile zero, tile transpose, tile dot product, tile broadcast, tile row broadcast, tile column broadcast, tile multiplication, tile multiplication and accumulation, tile move, etc. Additionally, support for operators such as the use of a scale and/or bias may be used with these operations or in support of non-numeric applications in the future, for instance, OpenCL "local memory," data compression/decompression, etc.

[0097] Portions of storage (such as memory (non-volatile and volatile), registers, cache, etc.) are arranged into tiles of different horizontal and vertical dimensions. For example, a tile may have horizontal dimension of 4 (e.g., four rows of a matrix) and a vertical dimension of 8 (e.g., 8 columns of the matrix). Typically, the horizontal dimension is related to element sizes (e.g., 2-, 4-, 8-, 16-, 32-, 64-, 128-bit, etc.). Multiple datatypes (single precision floating point, double precision floating point, integer, etc.) may be supported.

[0098] A. Exemplary usage of configured tiles

[0099] FIG. 1 illustrates an embodiment of configured tiles. As shown, there are four tiles 111, 113, 115, and 117 that are loaded from application memory 101. In this example, tiles T0 111 and T1 113 have M rows and N columns with 4 element bytes (e.g., single precision data). Tiles T2 115 and T3 117 have M rows and N/2 columns with 8 element bytes (e.g., double precision data). As the double precision operands are twice the width of single precision, this configuration is consistent with a palette, used to provide tile options, supplying at least 4 names with total storage of 16*N*M bytes. Depending upon the instruction encoding scheme used, the number of tiles available varies.

[0100] In some embodiments, tile parameters are definable. For example, a "palette" is used to provide tile options. Exemplary options include, but are not limited to: the number of tile names, the number of bytes in a row of storage, the number of rows and columns in a tile, etc. For example, a maximum "height" (number of rows) of a tile may be defined as:

[0101] Tile Max Rows = Architected Storage / (The Number of Palette Names * The Number of Bytes per row)

[0102] As such, an application can be written such that a fixed usage of names will be able to take advantage of different storage sizes across implementations.

[0103] Configuration of tiles is done using a tile configuration ("TILECONFIG") instruction, where a particular tile usage is defined in a selected palette. This declaration includes the number of tile names to be used, the requested number of rows and columns per name (tile), and, in some embodiments, the requested datatype of each tile. In some

embodiments, consistency checks are performed during the execution of a TILECONFIG instruction to determine that it matches the restrictions of the palette entry.

[0104] B. Exemplary tile storage types

[0105] FIG. 2 illustrates several examples of matrix storage. In (A), a tile is stored in memory. As shown, each "row" consists of four packed data elements. To get to the next "row," a stride value is used. Note that rows may be consecutively stored in memory. Strided memory accesses allows for access of one row to then next when the tile storage does not map the underlying memory array row width.

[0106] Tile loads from memory and stores to memory are typically strided accesses from the application memory to packed rows of data. Exemplary TILELOAD and TILESTORE instructions, or other instruction references to application memory as a TILE operand in load-op instructions, are, in some embodiments, restartable to handle (up to) 2*rows of page faults, unmasked floating point exceptions, and/or interrupts per instruction.

[0107] In (B), a matrix is stored in a tile comprised of a plurality of registers such as packed data registers (single instruction, multiple data (SIMD) or vector registers). In this example, the tile is overlaid on three physical registers. Typically, consecutive registers are used, however, this need not be the case.

[0108] In (C), a matrix is stored in a tile in non-register storage accessible to a fused multiple accumulate (FMA) circuit used in tile operations. This storage may be inside of a FMA, or adjacent to it. Additionally, in some embodiments, discussed below, the storage may be for a data element and not an entire row or tile.

[0109] The supported parameters for the TMMA architecture are reported via CPUID. In some embodiments, the list of information includes a maximum height and a maximum SIMD dimension. Configuring the TMMA architecture requires specifying the dimensions for each tile, the element size for each tile and the palette identifier. This configuration is done by executing the TILECONFIG instruction.

[0110] Successful execution of a TILECONFIG instruction enables subsequent TILE operators. A TILERELEASEALL instruction clears the tile configuration and disables the TILE operations (until the next TILECONFIG instructions executes). In some embodiments, XSAVE, XSTORE, etc. are used in context switching using tiles. In some embodiments, 2 XCR0 bits are used in XSAVE, one for TILECONFIF metadata and one bit corresponding to actual tile payload data.

[0111] TILECONFIG not only configures the tile usage, but also sets a state variable indicating that the program is in a region of code with tiles configured. An implementation

may enumerate restrictions on other instructions that can be used with a tile region such as no usage of an existing register set, etc.

[0112] Exiting a tile region is typically done with the TILERELEASEALL instruction. It takes no parameters and swiftly invalidates all tiles (indicating that the data no longer needs any saving or restoring) and clears the internal state corresponding to being in a tile region.

[0113] In some embodiments, tile operations will zero any rows and any columns beyond the dimensions specified by the tile configuration. For example, tile operations will zero the data beyond the configured number of columns (factoring in the size of the elements) as each row is written. For example, with 64 byte rows and a tile configured with 10 rows and 12 columns, an operation writing FP32 elements would write each of the first 10 rows with 12*4 bytes with output/result data and zero the remaining 4*4 bytes in each row. Tile operations also fully zero any rows after the first 10 configured rows. When using 1K tile with 64 byte rows, there would be 16 rows, so in this example, the last 6 rows would also be zeroed.

[0114] In some embodiments, a context restore (e.g., XRSTOR), when loading data, enforces that the data beyond the configured rows for a tile will be maintained as zero. If there is no valid configuration, all rows are zeroed. XRSTOR of tile data can load garbage in the columns beyond those configured. It should not be possible for XRSTOR to clear beyond the number of columns configured because there is not an element width associated with the tile configuration.

[0115] Context save (e.g., XSAVE) exposes the entire TILE storage area when writing it to memory. If XRSTOR loaded garbage data in to the rightmost part of a tile, that data will be saved by XSAVE. XSAVE will write zeros for rows beyond the number specified for each tile.

[0116] In some embodiments, tile instructions are restartable. The operations that access memory allow restart after page faults. The computational instructions that deal with floating point operations also allow for unmasked floating point exceptions, with the masking of the exceptions controlled by a control and/or status register.

[0117] To support restarting instructions after these events, the instructions store information in the start registers detailed below.

[0118] II. Matrix (Tile) Operation Systems

[0119] A. Exemplary Hardware Support

[0120] FIG. 3 illustrates an embodiment of a system utilizing a matrix (tile) operations accelerator. In this illustration, a host processor/processing system 301 communicates

commands 311 (e.g., matrix manipulation operations such as arithmetic or matrix manipulation operations, or load and store operations) to a matrix operations accelerator 307. However, this is shown this way for discussion purposes only. As detailed later, this accelerator 307 may be a part of a processing core. Typically, commands 311 that are tile manipulation operator instructions will refer to tiles as register-register ("reg-reg") or register-memory ("reg-mem") format. Other commands such as TILESTORE, TILELOAD, TILECONFIG, etc., do not perform data operations on a tile. Commands may be decoded instructions (e.g., micro-ops) or macro-instructions for the accelerator 307 to handle.

[0121] In this example, a coherent memory interface 303 is coupled to the host processor/processing system 301 and matrix operations accelerator 405 such that they can share memory. FIGS. 4 and 5 show different embodiments of how memory is shared using a matrix operations accelerator. As shown in FIG. 4, the host processor 401 and matrix operations accelerator circuitry 405 share the same memory 403. Figure 5 illustrates an embodiment where the host processor 501 and matrix operations accelerator 505 do not share memory, but can access each other's memory. For example, processor 501 can access tile memory 507 and utilize its host memory 503 as normal. Similarly, the matrix operations accelerator 505 can access host memory 503, but more typically uses its own memory 507. Note these memories may be of different types.

[0122] The matrix operations accelerator 307 includes a plurality of FMAs 309 coupled to data buffers 305 (in some implementations, one or more of these buffers 305 are stored in the FMAs of the grid as shown). The data buffers 305 buffer tiles loaded from memory and/or tiles to be stored to memory (e.g., using a tileload or tilestore instruction). Data buffers may be, for example, a plurality of registers. Typically, these FMAs are arranged as a grid of chained FMAs 309 which are able to read and write tiles. In this example, the matrix operations accelerator 307 is to perform a matrix multiply operation using tiles T0, T1, and T2. At least one of tiles is housed in the FMA grid 309. In some embodiments, all tiles in an operation are stored in the FMA grid 309. In other embodiments, only a subset are stored in the FMA grid 309. As shown, T1 is housed and T0 and T2 are not. Note that A, B, and C refer to the matrices of these tiles which may or may not take up the entire space of the tile.

[0123] FIG. 6 illustrates an embodiment of matrix multiply accumulate operation using tiles ("TMMA").

[0124] The number of rows in the matrix (TILE A 601) matches the number of serial (chained) FMAs comprising the computation's latency. An implementation is free to recirculate on a grid of smaller height, but the computation remains the same.

[0125] The source/destination vector comes from a tile of N rows (TILE C 605) and the grid of FMAs 611 performs N vector-matrix operations resulting in a complete instruction performing a matrix multiplication of tiles. Tile B 603 is the other vector source and supplies "broadcast" terms to the FMAs in each stage.

[0126] In operation, in some embodiments, the elements of matrix B (stored in a tile B 603) are spread across the rectangular grid of FMAs. Matrix B (stored in tile A 601) has its elements of a row transposed to match up with the columnar dimension of the rectangular grid of FMAs. At each FMA in the grid, an element of A and B are multiplied and added to the incoming summand (from above in the Figure) and the outgoing sum is passed to the next row of FMAs (or the final output).

[0127] The latency of a single step is proportional to K (row height of matrix B) and dependent TMMAs typically have enough source-destination rows (either in a single tile or across tile) to hide that latency. An implementation may also split the SIMD (packed data element) dimension M (row height of matrix A) across time steps, but this simply changes the constant that K is multiplied by. When a program specifies a smaller K than the maximum enumerated by the TMACC, an implementation is free to implement this with "masking" or "early outs."

[0128] The latency of an entire TMMA is proportional to N*K. The repeat rate is proportional to N. The number of MACs per TMMA instruction is N*K*M.

[0129] FIG. 7 illustrates an embodiment of a subset of the execution of an iteration of a chained fused multiply accumulate instruction. In particular, this illustrates execution circuitry of an iteration of one packed data element position of the destination. In this embodiment, the chained fused multiply accumulate is operating on signed sources wherein the accumulator is 2x the input data size.

[0130] A first signed source (source 1 701) and a second signed source (source 2 703) each have four packed data elements. Each of these packed data elements stores signed data such as floating point data. A third signed source (source 3 709) has two packed data elements, each of which stores signed data. The sizes of the first and second signed sources 701 and 703 are half that of the third signed source (initial value or previous result) 709. For example, the first and second signed sources 701 and 703 could have 32-bit packed data elements (e.g., single precision floating point) while the third signed source 709 could have 64-bit packed data elements (e.g., double precision floating point).

[0131] In this illustration, only the two most significant packed data element positions of the first and second signed sources 701 and 703 and the most significant packed data element

position of the third signed source 709 are shown. Of course, the other packed data element positions would also be processed.

[0132] As illustrated, packed data elements are processed in pairs. For example, the data of the most significant packed data element positions of the first and second signed sources 701 and 703 are multiplied using a multiplier circuit 705, and the data from second most significant packed data element positions of the first and second signed sources 701 and 703 are multiplied using a multiplier circuit 707. In some embodiments, these multiplier circuits 705 and 707 are reused for other packed data elements positions. In other embodiments, additional multiplier circuits are used so that the packed data elements are processed in parallel. In some contexts, parallel execution is done using lanes that are the size of the signed third source 709. The results of each of the multiplications are added using addition circuitry 711.

[0133] The result of the addition of the results of the multiplications is added to the data from most significant packed data element position of the signed source 3 709 (using a different adder 713 or the same adder 711).

[0134] Finally, the result of the second addition is either stored into the signed destination 715 in a packed data element position that corresponds to the packed data element position used from the signed third source 709, or passed on to the next iteration, if there is one. In some embodiments, a writemask is applied to this storage such that if a corresponding writemask (bit) is set, the storage happens, and, if not set, the storage does not happen.

[0135] FIG. 8 illustrates an embodiment of a subset of the execution of an iteration of a chained fused multiply accumulate instruction. In particular, this illustrates execution circuitry of an iteration of one packed data element position of the destination. In this embodiment, the chained fused multiply accumulate is operating on signed sources wherein the accumulator is 2x the input data size.

[0136] A first signed source (source 1 801) and a second signed source (source 2 803) each have four packed data elements. Each of these packed data elements stores signed data such as integer data. A third signed source (source 3 809) has two packed data elements, each of which stores signed data. The sizes of the first and second signed sources 801 and 803 are half that of the third signed source 809. For example, the first and second signed sources 801 and 803 could have 32-bit packed data elements (e.g., single precision floating point) the third signed source 809 could have 64-bit packed data elements (e.g., double precision floating point).

[0137] In this illustration, only the two most significant packed data element positions of the first and second signed sources 801 and 803 and the most significant packed data element position of the third signed source 809 are shown. Of course, the other packed data element positions would also be processed.

[0138] As illustrated, packed data elements are processed in pairs. For example, the data of the most significant packed data element positions of the first and second signed sources 801 and 803 are multiplied using a multiplier circuit 805, and the data from second most significant packed data element positions of the first and second signed sources 801 and 803 are multiplied using a multiplier circuit 807. In some embodiments, these multiplier circuits 805 and 807 are reused for other packed data elements positions. In other embodiments, additional multiplier circuits are used so that the packed data elements are processed in parallel. In some contexts, parallel execution is done using lanes that are the size of the signed third source (initial value or previous iteration result) 809. The results of each of the multiplications are added to the signed third source 809 using addition/saturation circuitry 811.

[0139] Addition/saturation (accumulator) circuitry 811 preserves a sign of an operand when the addition results in a value that is too big. In particular, saturation evaluation occurs on the infinite precision result between the multi-way-add and the write to the destination or next iteration. When the accumulator 811 is floating point and the input terms are integer, the sum of products and the floating point accumulator input value are turned into infinite precision values (fixed point numbers of hundreds of bits), the addition of the multiplication results and the third input is performed, and a single rounding to the actual accumulator type is performed.

[0140] Unsigned saturation means the output values are limited to a maximum unsigned number for that element width (all 1s). Signed saturation means a value is limited to the be in the range between a minimum negative number and a max positive number for that element width (for bytes for example, the range is from -128 (= - 2^7) to 127(=2^7-1)).

[0141] The result of the addition and saturation check is stored into the signed result 815 in a packed data element position that corresponds to the packed data element position used from the signed third source 809, or passed on to the next iteration if there is one. In some embodiments, a writemask is applied to this storage such that if a corresponding writemask (bit) is set, the storage happens, and, if not set, the storage does not happen.

[0142] FIG. 9 illustrates an embodiment of a subset of the execution of an iteration of a chained fused multiply accumulate instruction. In particular, this illustrates execution

circuitry of an iteration of one packed data element position of the destination. In this embodiment, the chained fused multiply accumulate is operating on a signed source and an unsigned source wherein the accumulator is 4x the input data size.

[0143] A first signed source (source 1 901) and a second unsigned source (source 2 903) each have four packed data elements. Each of these packed data elements has data such as floating point or integer data. A third signed source (initial value or result 915) has a packed data element of which stores signed data. The sizes of the first and second sources 901 and 903 are a quarter of the third signed source 915. For example, the first and second sources 901 and 903 could have 16-bit packed data elements (e.g., word) and the third signed source 915 could have 64-bit packed data elements (e.g., double precision floating point or 64-bit integer).

[0144] In this illustration, the four most significant packed data element positions of the first and second sources 901 and 903 and the most significant packed data element position of the third signed source 915 are shown. Of course, other packed data element positions would also be processed if there are any.

[0145] As illustrated, packed data elements are processed in quadruplets. For example, the data of the most significant packed data element positions of the first and second sources 901 and 903 are multiplied using a multiplier circuit 907, data from second most significant packed data element positions of the first and second sources 901 and 903 are multiplied using a multiplier circuit 907, data from third most significant packed data element positions of the first and second sources 901 and 903 are multiplied using a multiplier circuit 909, and data from the least significant packed data element positions of the first and second sources 901 and 903 are multiplied using a multiplier circuit 911. In some embodiments, the signed packed data elements of the first source 901 are sign extended and the unsigned packed data elements of the second source 903 are zero extended prior to the multiplications.

[0146] In some embodiments, these multiplier circuits 905-911 are reused for other packed data elements positions. In other embodiments, additional multiplier circuits are used so that the packed data elements are processed in parallel. In some contexts, parallel execution is done using lanes that are the size of the signed third source 915. The results of each of the multiplications are added using addition circuitry 911.

[0147] The result of the addition of the results of the multiplications is added to the data from most significant packed data element position of the signed source 3 915 (using a different adder 913 or the same adder 911).

[0148] Finally, the result 919 of the second addition is either stored into the signed destination in a packed data element position that corresponds to the packed data element position used from the signed third source 915, or passed to the next iteration. In some embodiments, a writemask is applied to this storage such that if a corresponding writemask (bit) is set, the storage happens, and, if not set, the storage does not happen.

[0149] FIG. 10 illustrates an embodiment of a subset of the execution of an iteration of chained fused multiply accumulate instruction. In particular, this illustrates execution circuitry of an iteration of one packed data element position of the destination. In this embodiment, the chained fused multiply accumulate is operating on a signed source and an unsigned source wherein the accumulator is 4x the input data size.

[0150] A first signed source (source 1 1001) and a second unsigned source (source 2 1003) each have four packed data elements. Each of these packed data elements stores data such as floating point or integer data. A third signed source (initial or previous result 1015) has a packed data element of which stores signed data. The sizes of the first and second sources 1001 and 1003 are a quarter of the third signed source 1015. For example, the first and second sources 1001 and 1003 could have 16-bit packed data elements (e.g., word) and the third signed source 1015 could have 64-bit packed data elements (e.g., double precision floating point or 64-bit integer).

[0151] In this illustration, the four most significant packed data element positions of the first and second sources 1001 and 1003 and the most significant packed data element position of the third signed source 1015 are shown. Of course, other packed data element positions would also be processed if there are any.

[0152] As illustrated, packed data elements are processed in quadruplets. For example, the data of the most significant packed data element positions of the first and second sources 1001 and 1003 are multiplied using a multiplier circuit 1007, data from second most significant packed data element positions of the first and second sources 1001 and 1003 are multiplied using a multiplier circuit 1007, data from third most significant packed data element positions of the first and second sources 1001 and 1003 are multiplied using a multiplier circuit 1009, and data from the least significant packed data element positions of the first and second sources 1001 and 1003 are multiplied using a multiplier circuit 1011. In some embodiments, the signed packed data elements of the first source 1001 are sign extended and the unsigned packed data elements of the second source 1003 are zero extended prior to the multiplications.

[0153] In some embodiments, these multiplier circuits 1005-1011 are reused for other packed data elements positions. In other embodiments, additional multiplier circuits are used so that the packed data elements are processed in parallel. In some contexts, parallel execution is done using lanes that are the size of the signed third source 1015. The result of the addition of the results of the multiplications is added to the data from most significant packed data element position of the signed source 3 1015 using addition/saturation circuitry 1013.

[0154] Addition/saturation (accumulator) circuitry 1013 preserves a sign of an operand when the addition results in a value that is too big or too small for signed saturation. In particular, saturation evaluation occurs on the infinite precision result between the multi-way-add and the write to the destination. When the accumulator 1013 is floating point and the input terms are integer, the sum of products and the floating point accumulator input value are turned into infinite precision values (fixed point numbers of hundreds of bits), the addition of the multiplication results and the third input is performed, and a single rounding to the actual accumulator type is performed.

[0155] The result 1019 of the addition and saturation check is stored into the signed destination in a packed data element position that corresponds to the packed data element position used from the signed third source 1015, or passed to the next iteration. In some embodiments, a writemask is applied to this storage such that if a corresponding writemask (bit) is set, the storage happens, and, if not set, the storage does not happen.

[0156] FIG. 11 illustrates power-of-two sized SIMD implementations wherein the accumulators use input sizes that are larger than the inputs to the multipliers according to an embodiment. Note the source (to the multipliers) and accumulator values may be signed or unsigned values. For an accumulator having 2X input sizes (in other words, the accumulator input value is twice the size of the packed data element sizes of the sources), table 1101 illustrates different configurations. For byte sized sources, the accumulator uses word or half-precision floating-point (HPFP) values that are 16-bit in size. For word sized sources, the accumulator uses 32-bit integer or single-precision floating-point (SPFP) values that are 32-bit in size. For SPFP or 32-bit integer sized sources, the accumulator uses 64-intenger or double-precision floating-point (DPFP) values that are 64-bit in size.

[0157] For an accumulator having 4X input sizes (in other words, the accumulator input value is four times the size of the packed data element sizes of the sources), table 1103 illustrates different configurations. For byte sized sources, the accumulator uses 32-bit integer or single-precision floating-point (SPFP) values that are 32-bit in size. For word

sized sources, the accumulator uses 64-bit integer or double-precision floating-point (DPFP) values that are 64-bit in size in some embodiments.

[0158] For an accumulator having 8X input sizes (in other words, the accumulator input value is eight times the size of the packed data element sizes of the sources), table 1105 illustrates a configuration. For byte sized sources, the accumulator uses 64-bit integer.

[0159] As hinted at earlier, matrix operations circuitry may be included in a core, or as an external accelerator. FIG. 12 illustrates an embodiment of a system utilizing matrix operations circuitry. In this illustration, a plurality of entities are coupled with a ring interconnect 1245.

[0160] A plurality of cores 1201, 1203, 1205, and 1207 provide non-tile based instruction support. In some embodiments, matrix operations circuitry is provided in a core 1203, and in other embodiments matrix operations circuitry 1211 and 1213 is accessible on the ring interconnect 1245.

[0161] Additionally, one or more memory controllers 1223-1225 are provided to communicate with memory 1233 and 1231 on behalf of the cores and/or matrix operations circuitry.

[0162] FIG. 13 illustrates an embodiment of a processor core pipeline supporting matrix operations using tiles. Branch prediction and decode circuitry 1303 performs branch predicting of instructions, decoding of instructions, and/or both from instructions stored in instruction storage 1301. For example, instructions detailed herein may be stored in instruction storage. In some implementations, separate circuitry is used for branch prediction and in some embodiments, at least some instructions are decoded into one or more micro-operations, micro-code entry points, microinstructions, other instructions, or other control signals using microcode 1305. The branch prediction and decode circuitry 1303 may be implemented using various different mechanisms. Examples of suitable mechanisms include, but are not limited to, look-up tables, hardware implementations, programmable logic arrays (PLAs), microcode read only memories (ROMs), etc.

[0163] The branch prediction and decode circuitry 1303 is coupled to a rename/allocator circuitry 1307 which is coupled, in some embodiments, to scheduler circuitry 1309. In some embodiments, these circuits provide register renaming, register allocation, and/or scheduling functionality by performing one or more of: 1) renaming logical operand values to physical operand values (e.g., a register alias table in some embodiments), 2) allocating status bits and flags to the decoded instruction, and 3) scheduling the decoded instruction for execution on

execution circuitry out of an instruction pool (e.g., using a reservation station in some embodiments).

[0164] The scheduler circuitry 1309 represents any number of different schedulers, including reservations stations, central instruction window, etc. The scheduler unit(s) scheduler circuitry 1309 is coupled to, or includes, physical register file(s) 1315. Each of the physical register file(s) 1315 represents one or more physical register files, different ones of which store one or more different data types, such as scalar integer, scalar floating point, packed integer, packed floating point, vector integer, vector floating point, status (e.g., an instruction pointer that is the address of the next instruction to be executed), tiles, etc. In one embodiment, the physical register file(s) 1315 comprises vector registers circuitry, write mask registers circuitry, and scalar registers circuitry. These register circuits may provide architectural vector registers, vector mask registers, and general purpose registers. The physical register file(s) 1315 is overlapped by a retirement circuit 1317 to illustrate various ways in which register renaming and out-of-order execution may be implemented (e.g., using a reorder buffer(s) and a retirement register file(s); using a future file(s), a history buffer(s), and a retirement register file(s); using a register maps and a pool of registers; etc.). The retirement circuit 1317 and the physical register file(s) 1315 are coupled to the execution circuit(s) 1311.

[0165] While register renaming is described in the context of out-of-order execution, it should be understood that register renaming may be used in an in-order architecture. While the illustrated embodiment of the processor may also include separate instruction and data cache units and a shared L2 cache unit, alternative embodiments may have a single internal cache for both instructions and data, such as, for example, a Level 1 (L1) internal cache, or multiple levels of internal cache. In some embodiments, the system may include a combination of an internal cache and an external cache that is external to the core and/or the processor. Alternatively, all of the cache may be external to the core and/or the processor.

[0166] The execution circuitry 1311 a set of one or more execution circuits 1321, 1323, and 1327 and a set of one or more memory access circuits 1325. The execution circuits 1321, 1323, and 1327 perform various operations (e.g., shifts, addition, subtraction, multiplication) and on various types of data (e.g., scalar floating point, packed integer, packed floating point, vector integer, vector floating point). While some embodiments may include a number of execution units dedicated to specific functions or sets of functions, other embodiments may include only one execution unit or multiple execution units that all perform all functions. The scalar circuitry 1321 performs scalar operations, the vector/SIMD circuitry 1323 performs

vector/SIMD operations, and matrix operations circuitry 1327 performs matrix (tile) operations detailed herein.

[0167] The set of memory access units 1364 is coupled to the memory unit 1370, which includes a data TLB unit 1372 coupled to a data cache unit 1374 coupled to a level 2 (L2) cache unit 1376. In one exemplary embodiment, the memory access units 1364 may include a load unit, a store address unit, and a store data unit, each of which is coupled to the data TLB unit 1372 in the memory unit 1370. The instruction cache unit 1334 is further coupled to a level 2 (L2) cache unit 1376 in the memory unit 1370. The L2 cache unit 1376 is coupled to one or more other levels of cache and eventually to a main memory.

[0168] By way of example, the exemplary register renaming, out-of-order issue/execution core architecture may implement a pipeline as follows: 1) an instruction fetch circuit performs fetch and length decoding stages; 2) the branch and decode circuitry 1303 performs a decode stage; 3) the rename/allocator circuitry 1307 performs an allocation stage and renaming stage; 4) the scheduler circuitry 1309 performs a schedule stage; 5) physical register file(s) (coupled to, or included in, the scheduler circuitry 1307 and rename/allocate circuitry 1307 and a memory unit perform a register read/memory read stage ; the execution circuitry 1311 performs an execute stage; 6) a memory unit and the physical register file(s) unit(s) perform a write back/memory write stage; 7) various units may be involved in the exception handling stage; and 8) a retirement unit and the physical register file(s) unit(s) perform a commit stage.

[0169] The core may support one or more instructions sets (e.g., the x86 instruction set (with some extensions that have been added with newer versions); the MIPS instruction set of MIPS Technologies of Sunnyvale, CA; the ARM instruction set (with optional additional extensions such as NEON) of ARM Holdings of Sunnyvale, CA), including the instruction(s) described herein. In one embodiment, the core 1390 includes logic to support a packed data instruction set extension (e.g., AVX1, AVX2), thereby allowing the operations used by many multimedia applications to be performed using packed data.

[0170] It should be understood that the core may support multithreading (executing two or more parallel sets of operations or threads), and may do so in a variety of ways including time sliced multithreading, simultaneous multithreading (where a single physical core provides a logical core for each of the threads that physical core is simultaneously multithreading), or a combination thereof (e.g., time sliced fetching and decoding and simultaneous multithreading thereafter such as in the Intel® Hyperthreading technology).

[0171] FIG. 14 illustrates an embodiment of a processor core pipeline supporting matrix operations using tiles. Branch prediction and decode circuitry 1403 performs branch predicting of instructions, decoding of instructions, and/or both from instructions stored in instruction storage 1401. For example, instructions detailed herein may be stored in instruction storage. In some implementations, separate circuitry is used for branch prediction and in some embodiments, at least some instructions are decoded into one or more micro-operations, micro-code entry points, microinstructions, other instructions, or other control signals using microcode 1405. The branch prediction and decode circuitry 1403 may be implemented using various different mechanisms. Examples of suitable mechanisms include, but are not limited to, look-up tables, hardware implementations, programmable logic arrays (PLAs), microcode read only memories (ROMs), etc.

[0172] The branch prediction and decode circuitry 1403 is coupled to a rename/allocator circuitry 1407 which is coupled, in some embodiments, to scheduler circuitry 1409. In some embodiments, these circuits provide register renaming, register allocation, and/or scheduling functionality by performing one or more of: 1) renaming logical operand values to physical operand values (e.g., a register alias table in some embodiments), 2) allocating status bits and flags to the decoded instruction, and 3) scheduling the decoded instruction for execution on execution circuitry out of an instruction pool (e.g., using a reservation station in some embodiments).

[0173] The scheduler circuitry 1409 represents any number of different schedulers, including reservations stations, central instruction window, etc. The scheduler unit(s) scheduler circuitry 1409 is coupled to, or includes, physical register file(s) 1415. Each of the physical register file(s) 1415 represents one or more physical register files, different ones of which store one or more different data types, such as scalar integer, scalar floating point, packed integer, packed floating point, vector integer, vector floating point, status (e.g., an instruction pointer that is the address of the next instruction to be executed), tiles, etc. In one embodiment, the physical register file(s) 1415 comprises vector registers circuitry, write mask registers circuitry, and scalar registers circuitry. These register circuits may provide architectural vector registers, vector mask registers, and general purpose registers. The physical register file(s) 1415 is overlapped by a retirement circuit 1417 to illustrate various ways in which register renaming and out-of-order execution may be implemented (e.g., using a reorder buffer(s) and a retirement register file(s); using a future file(s), a history buffer(s), and a retirement register file(s); using a register maps and a pool of registers; etc.). The

retirement circuit 1417 and the physical register file(s) 1415 are coupled to the execution circuit(s) 1411.

[0174] While register renaming is described in the context of out-of-order execution, it should be understood that register renaming may be used in an in-order architecture. While the illustrated embodiment of the processor may also include separate instruction and data cache units and a shared L2 cache unit, alternative embodiments may have a single internal cache for both instructions and data, such as, for example, a Level 1 (L1) internal cache, or multiple levels of internal cache. In some embodiments, the system may include a combination of an internal cache and an external cache that is external to the core and/or the processor. Alternatively, all of the cache may be external to the core and/or the processor.

[0175] The execution circuitry 1411 a set of one or more execution circuits 1427 and a set of one or more memory access circuits 1425. The execution circuits 1427 perform matrix (tile) operations detailed herein.

[0176] The set of memory access units 1464 is coupled to the memory unit 1470, which includes a data TLB unit 1472 coupled to a data cache unit 1474 coupled to a level 2 (L2) cache unit 1476. In one exemplary embodiment, the memory access units 1464 may include a load unit, a store address unit, and a store data unit, each of which is coupled to the data TLB unit 1472 in the memory unit 1470. The instruction cache unit 1434 is further coupled to a level 2 (L2) cache unit 1476 in the memory unit 1470. The L2 cache unit 1476 is coupled to one or more other levels of cache and eventually to a main memory.

[0177] By way of example, the exemplary register renaming, out-of-order issue/execution core architecture may implement a pipeline as follows: 1) an instruction fetch circuit performs fetch and length decoding stages; 2) the branch and decode circuitry 1403 performs a decode stage; 3) the rename/allocator circuitry 1407 performs an allocation stage and renaming stage; 4) the scheduler circuitry 1409 performs a schedule stage; 5) physical register file(s) (coupled to, or included in, the scheduler circuitry 1407 and rename/allocate circuitry 1407 and a memory unit perform a register read/memory read stage ; the execution circuitry 1411 performs an execute stage; 6) a memory unit and the physical register file(s) unit(s) perform a write back/memory write stage; 7) various units may be involved in the exception handling stage; and 8) a retirement unit and the physical register file(s) unit(s) perform a commit stage.

[0178] The core may support one or more instructions sets (e.g., the x86 instruction set (with some extensions that have been added with newer versions); the MIPS instruction set of MIPS Technologies of Sunnyvale, CA; the ARM instruction set (with optional additional

extensions such as NEON) of ARM Holdings of Sunnyvale, CA), including the instruction(s) described herein. In one embodiment, the core 1490 includes logic to support a packed data instruction set extension (e.g., AVX1, AVX2), thereby allowing the operations used by many multimedia applications to be performed using packed data.

[0179] It should be understood that the core may support multithreading (executing two or more parallel sets of operations or threads), and may do so in a variety of ways including time sliced multithreading, simultaneous multithreading (where a single physical core provides a logical core for each of the threads that physical core is simultaneously multithreading), or a combination thereof (e.g., time sliced fetching and decoding and simultaneous multithreading thereafter such as in the Intel® Hyperthreading technology).

[0180] B. Layout

[0181] Throughout this description, data is expressed using row major data layout. Column major users should translate the terms according to their orientation. FIG. 15 illustrates an example of a matrix expressed in row major format and column major format. As shown, matrix A is a 2x3 matrix. When this matrix is stored in row major format, the data elements of a row are consecutive. When this matrix is stored in column major format, the data elements of a column are consecutive. It is a well-known property of matrices that A^T * $B^T = (BA)^T$, where superscript T means transpose. Reading column major data as row major data results in the matrix looking like the transpose matrix.

[0182] In some embodiments, row-major semantics are utilized in hardware, and column major data is to swap the operand order with the result being transposes of matrix, but for subsequent column-major reads from memory it is the correct, non-transposed matrix.

[0183] For example, if there are two column-major matrices to multiply:

a b	g i k	ag+bh ai+bj ak+bl
c d *	h j l =	cg+dh ci+dj ck+dl
e f		eg+fh ei+fj ek+fl
(3x2)	(2x3)	(3x3)

[0184] The input matrices would be stored in linear memory (column-major) as:

a c e b d f

and

g h i j k l.

[0185] Reading those matrices as row-major with dimensions 2x3 and 3x2, they would appear as:

a c e and g h

b d f i j

k l

[0186] Swapping the order and matrix multiplying:

g h a c e ag+bh cg+dh eg+fh

i j * b d f = ai+bj ci+dj ei+fj

k l ak+bl ck+dl ek+fl

the transpose matrix is out and can then be stored in in row-major order:

ag+bh cg+dh eg+fh ai+bj ci+dj ei+fj ak+bl ck+dl ek+fl

and used in subsequent column major computations, it is the correct un-transposed matrix:

ag+bh ai+bj ak+bl

cg+dh ci+dj ck+dl

eg+fh ei+fj ek+fl

[0187] III. Exemplary Usage

[0188] FIG. 16 illustrates an example of usage of matrices (tiles). In this example, matrix C 1601 includes two tiles, matrix A 1603 includes one tile, and matrix B 1605 includes two tiles. This figure shows an example of the inner loop of an algorithm to compute a matrix multiplication. In this example, two result tiles, tmm0 and tmm1, from matrix C 1601 are used to accumulate the intermediate results. One tile from the A matrix 1603 (tmm2) is re-used twice as it multiplied by two tiles from the B matrix 1605. Pointers to load a new A tile and two new B tiles from the directions indicated by the arrows. An outer loop, not shown, adjusts the pointers for the C tiles.

[0189] The exemplary code as shown includes the usage of a tile configuration instruction and is executed to configure tile usage, load tiles, a loop to process the tiles, store tiles to memory, and release tile usage.

[0190] **Example 1** An apparatus comprising: matrix operations circuitry to execute one or more decoded matrix operation instructions on data stored in two-dimensional data structures; and storage to store the two-dimensional data structures.

[0191] **Example 2** The apparatus of example 1, wherein the storage is a plurality of packed data registers and the two-dimensional data structures are overlaid on these registers.

[0192] **Example 3** The apparatus of example 1, wherein the storage is a plurality of packed data registers and memory, and the two-dimensional data structures are overlaid on these registers and memory.

[0193] **Example 4** The apparatus of any of examples 1-3, wherein the matrix operations circuitry is a plurality of chained fused multiply accumulate circuits.

[0194] **Example 5** The apparatus of example 4, wherein each of the chained fused multiply accumulate circuits is to include storage for a portion of a two-dimensional data structure that the fused multiply accumulate circuit is to operate on.

[0195] **Example 6** The apparatus of any of examples 1-5, wherein matrix operations circuitry supports element matrix multiply, subtract, and add instructions.

[0196] **Example 7** The apparatus of any of examples 1-6, wherein matrix operations circuitry supports dot product and multiply accumulate operations.

[0197] **Example 8** The apparatus of any of examples 1-7, wherein matrix operations circuitry supports matrix transpose and diagonal operations.

[0198] **Example 9** A system comprising: a host processor; a matrix operations accelerator coupled to the host processor, wherein the matrix operations accelerator is to perform matrix operations on two-dimensional data structures using a computational grid based on commands received from the host processor.

[0199] **Example 10** The system of example 9, wherein the matrix operations accelerator further comprises a plurality of data buffers to buffer matrix data in two-dimensional data structures.

[0200] **Example 11** The system of any of examples 9-10, wherein the computational grid is to house at least one of the buffered matrix data from the plurality of data buffers during a matrix manipulation operation.

[0201] **Example 12** The system of any of examples 9-11, wherein the data buffers are a plurality of registers.

[0202] **Example 13** The system of example 12, wherein the registers are a plurality of packed data registers and the two-dimensional data structures are overlaid on these registers.

[0203] **Example 14** The system of example 12, wherein the storage is a plurality of packed data registers and memory, and the two-dimensional data structures are overlaid on these registers and the memory.

[0204] **Example 15** The system of any of examples 9-14, wherein the matrix operations circuitry is a plurality of chained fused multiply add circuits.

[0205] **Example 16** The system of example 15, wherein each of the chained fused multiply add circuits is to include storage for a portion of a two-dimensional data structure that the fused multiply add circuit is to operate on.

[0206] **Example 17** The system of any of examples 9-15, further comprising a coherent memory interface coupled to the matrix operations accelerator and host processor to provide access to shared memory between the host processor and matrix operations accelerator.

[0207] FIG. 17 illustrates an embodiment of usage of matrices (tiles). At 1701, tile usage is configured. For example, a TILECONFIG instruction is executed to configure tile usage including setting a numbers of rows and columns per tile. Typically, at least one matrix (tile) is loaded from memory at 1703.

[0208] IV. Exemplary Instructions

[0209] *A. Tile Configuration*

[0210] As discussed above, tile usage typically needs to be configured prior to use. For example, full usage of all rows and columns may not be needed. Not only does not configuring these rows and columns save power in some embodiments, but the configuration may be used to determine if an operation will generate an error. For example, a matrix multiplication of the form (NxM)*(L*N) will typically not work if M and L are not the same.

[0211] Detailed herein are embodiments of a matrix (tile) configuration ("TILECONFIG") instruction and its execution. Prior to using matrices using tiles, in some embodiments, tile support is to be configured. For example, how many rows and columns per tile, tiles that are to be used, etc. are configured. A TILECONFIG instruction is an improvement to a computer itself as it provides for support to configure the computer to use a matrix accelerator (either as a part of a processor core, or as an external device). In particular, an execution of the TILECONFIG instruction causes a configuration to be retrieved from memory and applied to matrix (tile) settings within a matrix accelerator.

[0212] *i. Exemplary Execution*

[0213] FIG. 18 illustrates an exemplary execution of a TILECONFIG instruction. The TILECONFIG instruction format includes fields for an opcode and a memory address.

[0214] As illustrated, the TILECONFIG instruction uses the address as a pointer to a memory 1801 location containing the description of the matrices (tiles) to be supported 1803.

[0215] Execution circuitry 1811 of a processor/core 1805 performs the TILECONFIG by retrieving the description 1803 from memory 1801 via a memory controller 1815, configuring tiles for a palette (setting the number of rows and columns) in a tile configuration 1817, and marking that matrix support is in use. In particular, instruction execution resources 1811 are configured to use tiles as specified by setting tile configurations 1817. The instruction execution resources may also include a machine specific register or configuration register to indicate tile usage.

[0216] Tile configurations 1817 are set to indicate parameters per tile as indicated by the tile description 1803 via the execution of the TILECONFIG instruction. The set parameters are the number of rows and columns per tile. Additional values such as in-use and start values are also set. The tile configurations 1817 utilize one or more registers 1819 to store tile usage and configuration information.

[0217] *ii. Exemplary tile storage*

[0218] FIGS. 19(A)-(D) illustrate examples of register(s) 1819. Fig. 19(A) illustrates a plurality of registers 1819. As shown each tile (TMM0 1901 ... TMMN 1903) has a separate register with each register storing a row and column size for that particular tile. StartK and StartM are stored in separate registers 1911 and 1913. One or more status registers 1915 are set (e.g., TILES_CONFIGURED = 1) to indicate tiles are configured for use.

[0219] Fig. 19(B) illustrates a plurality of registers 1819. As shown each tile has separate registers for its rows and columns. For example, TMM0 rows configuration 1921, TMM0 columns configuration 1923, StartK and StartM are stored in separate registers '1911 and 1913. One or more status registers 1915 are set (e.g., TILES_CONFIGURED = 1) to indicate tiles are configured for use.

[0220] Fig. 19(C) illustrates a single register 1819. As shown, this register stores tile configurations (rows and columns per tile) 1931, StartK 1933, and StartM 1933 are stored in single register as packed data registers. One or more status registers 1915 are set (e.g., TILES_CONFIGURED = 1) to indicate tiles are configured for use.

[0221] Fig. 19(D) illustrates a plurality of registers 1819. As shown, a single register stores tile configurations (rows and columns per tile) 1931. StartK and StartM are stored in separate registers 1911 and 1913. One or more status registers 1915 are set (e.g., TILES_CONFIGURED = 1) to indicate tiles are configured for use.

[0222] Other combinations are contemplated such as combining the start registers into a single register where they are shown separately, etc.

[0223] *iii. Exemplary Stored Matrix (Tile) Description*

[0224] FIG. 20 illustrates an embodiment of a description of the matrices (tiles) to be supported. In this example, each field is a byte. In byte[0], a palette ID 2001 is stored. The palette ID is used to index a palette table 1813 which stores, per palette ID, a number of bytes in a tile, and bytes per row of the tiles that are associated with this ID as defined by the configuration. Bytes 1-7 are reserved and are typically zero.

[0225] Bytes 8-9 store a value for a "startM" register 2003 and bytes 10-11 store a value for a "startK" register 2005. To support restarting instructions after these events, the

instructions store information these registers. The startM indicates a row that should be used for restart. The startK indicates a position in the inner-product for relevant operations. The position in the row (the column) is not needed. Two-dimensional operations like the element-wise addition/subtraction/multiplication only use startM. Three-dimensinal operations use values from both startM and startK. Typically, operations that only require startM will zero startK when writing startM.

[0226] Any time an interrupted tile instruction is not restarted, in some embodiments, it is the responsibility of software to zero the startM and startK values. For example, unmasked floating point exception handlers might decide to finish the operation in software and change the program counter value to another instruction, usually the next instruction. In this case the software exception handler must zero the startM and startK values in the exception frame presented to it by the operating system before resuming the program. The operating system will subsequently reload those values.

[0227] Bytes 16-17 store the number of rows 2013 and columns 2015 for tile 0, bytes 18-19 store the number of rows and columns for tile 1, etc. In other words, each 2 byte group specifies a number of rows and columns for a tile. If a group of 2 bytes is not used to specify tile parameters, they should have the value zero. Specifying tile parameters for more tiles than the implementation limit or the palette limit results in a fault. Unconfigured tiles are set to the INIT state with 0 rows, 0 columns.

[0228] Finally, the configuration in memory typically ends with an ending delineation such as all zeros for several consecutive bytes.

[0229] *iv. Exemplary Format(s)*

[0230] An embodiment of a format for a TILECONFIG instruction is TILECONFIG Address. In some embodiments, TILECONFIG is the opcode mnemonic of the instruction. Address is a pointer to a matrix (tile) description in memory. In some embodiments, the address field is a R/M value (such as 2446).

[0231] In embodiments, encodings of the instruction include a scale-index-base (SIB) type memory addressing operand that indirectly identifies multiple indexed destination locations in memory (e.g., field 2450). In one embodiment, an SIB type memory operand may include an encoding identifying a base address register. The contents of the base address register may represent a base address in memory from which the addresses of the particular destination locations in memory are calculated. For example, the base address may be the address of the first location in a block of potential destination locations for an extended vector instruction. In one embodiment, an SIB type memory operand may include an

encoding identifying an index register. Each element of the index register may specify an index or offset value usable to compute, from the base address, an address of a respective destination location within a block of potential destination locations. In one embodiment, an SIB type memory operand may include an encoding specifying a scaling factor to be applied to each index value when computing a respective destination address. For example, if a scaling factor value of four is encoded in the SIB type memory operand, each index value obtained from an element of the index register may be multiplied by four and then added to the base address to compute a destination address.

[0232] In one embodiment, an SIB type memory operand of the form vm32{x,y,z} may identify a vector array of memory operands specified using SIB type memory addressing. In this example, the array of memory addresses is specified using a common base register, a constant scaling factor, and a vector index register containing individual elements, each of which is a 32-bit index value. The vector index register may be a 128-bit register (e.g., XMM) register (vm32x), a 256-bit (e.g., YMM) register (vm32y), or a 512-bit (e.g., ZMM) register (vm32z). In another embodiment, an SIB type memory operand of the form vm64{x,y,z} may identify a vector array of memory operands specified using SIB type memory addressing. In this example, the array of memory addresses is specified using a common base register, a constant scaling factor, and a vector index register containing individual elements, each of which is a 64-bit index value. The vector index register may be a 128-bit register (e.g., XMM) register (vm64x), a 256-bit (e.g., YMM) register (vm64y) or a 512-bit (e.g., ZMM) register (vm64z).

[0233] *v. Exemplary Method(s) of Execution*

[0234] FIG. 21 illustrates an embodiment of method performed by a processor to process a TILECONFIG instruction.

[0235] At 2101, an instruction is fetched. For example, a TILECONFIG instruction is fetched. An embodiment of the TILECONFIG instruction includes fields for an opcode and a memory address operand.

[0236] The fetched instruction is decoded at 2103. For example, the fetched TILECONFIG instruction is decoded by decode circuitry such as that detailed herein.

[0237] A description found at the memory address of the memory address operand is are retrieved at 2105 and the decoded instruction is scheduled (as needed).

[0238] At 2107, the decoded instruction is executed by execution circuitry (hardware) such as that detailed herein. For the TILECONFIG instruction, the execution will cause execution circuitry to configure usage of tiles in a tile configuration (setting the number of

rows and columns) and marking that matrix (tile) support is in use (active). For example, configuration one or more registers 1819. Tile support usage (e.g., "TILES_CONFIGURED") is typically indicated by setting a bit in a status, control, or machine specific register. In particular, instruction execution resources 1811 are configured to use tiles as specified by the retrieved configuration.

[0239] In some embodiments, the instruction is committed or retired at 2109.

[0240] FIG. 22 illustrates a more detailed description of an execution of a TILECONFIG instruction using memory addressing. Typically, this is performed by execution circuitry such as that detailed above after the description has been retrieved from memory. While not illustrated, in some embodiments, a check is first performed to determine if tiles are supported. Support is usually found by a CPUID check.

[0241] At 2201, a determination of if the palette ID is supported is made. For example, does the CPUID state that this ID is supported? If not, then a general protection fault occurs at 2203.

[0242] At 2205, a first tile specific grouping is read. For example, the number of rows and columns for tile 0 (T0) is read.

[0243] A determination of if the read grouping is valid is made at 2207. For example, if one the number of rows or columns (not both) is set 0, then the grouping is not valid and the configuration halts and tiles are not considered to be in use at 2203. Invalid groups occur, for example, when one of rows or columns (not both) are zero. Additionally, when a value for the number of rows is greater than the maximum of rows supported (this is found by dividing the tile byte size of the palette ID with the number of bytes per row for the palette ID as found in the palette table) as fault occurs. Another potential fault is when there are more names than supported.

[0244] If the read grouping is valid, then the tile associated with the read grouping is configured to use the number of rows and columns specified by the grouping in a tile configuration at 2211. The size of the elements in the tile are set by the palette table entry for the palette ID.

[0245] A determination of if all tiles of the retrieved configuration have been configured is made at 2213. For example, have all of the possible tile names been processed? In some embodiments, when the rows and columns for a particular tile are both 0, then all tiles have been processed.

[0246] When all tiles have not been configured, at 2215, the tile number is incremented such that the next tile in the configuration will be evaluated.

[0247] At 2217, the incremented tile's grouping is read. For example, the number of rows and columns for tile 1 (T1) is read. A determination of if the read grouping is valid is made at 2207, etc.

[0248] When all tiles have been configured, then the instruction completes at 2209. The tiles will be marked as being in use for matrix operations, for example, by setting an in-use indicator in a register.

[0249] *vi. Exemplary Pseudocode*

[0250] FIG. 23 illustrates exemplary pseudocode for an execution of a TILECONFIG instruction.

[0251] *vii. Examples*

[0252] Example 1 A processor comprising decode circuitry to decode an instruction having fields for an opcode and a memory address; and execution circuitry to execute the decoded instruction to set a tile configuration for the processor to utilize tiles in matrix operations based on a description retrieved from the memory address, wherein a tile a set of 2-dimensional registers.

[0253] Example 2 The processor of example 1, wherein the configuration comprises a palette identifier and details regarding a number of rows and columns per tile of the palette.

[0254] Example 3 The processor of example 1, wherein the palette identifier is an index into a palette table defining a number of bytes per tile and bytes per row of the tile.

[0255] Example 4 The processor of any of examples 1-4, wherein the configuration is stored in a single packed data register.

[0256] Example 5 The processor of any of examples 1-4, wherein the configuration is stored in a plurality of packed data registers, one packed data register per configured tile.

[0257] Example 6 The processor of any of examples 1-5, wherein the execution circuitry to general a fault upon a value of a number of rows for a tile being zero and a number of columns for the row being non-zero.

[0258] Example 7 The processor of any of examples 1-6, wherein the description is 64 bytes in size.

[0259] Example 8 A method comprising: decoding an instruction having fields for an opcode and a memory address; and executing the decoded instruction to set a tile configuration for the processor to utilize tiles in matrix operations based on a description retrieved from the memory address, wherein a tile a set of 2-dimensional registers.

[0260] Example 9 The method of example 8, wherein the configuration comprises a palette identifier and details regarding a number of rows and columns per tile of the palette.

[0261] Example 10 The method of example 8, wherein the palette identifier is an index into a palette table defining a number of bytes per tile and bytes per row of the tile.

[0262] Example 11 The method of any of examples 8-10, wherein the configuration is stored in a single packed data register.

[0263] Example 12 The method of any of examples 8-10, wherein the configuration is stored in a plurality of packed data registers, one packed data register per configured tile.

[0264] Example 13 The method of any of examples 8-12, further comprising:

[0265] faulting upon a value of a number of rows for a tile being zero and a number of columns for the row being non-zero.

[0266] Example 14 The method of any of examples 8-13, wherein the description is 64 bytes in size.

[0267] Example 15 A non-transitory machine-readable medium storing an instruction which causes a processor to perform a method, the method comprising: decoding an instruction having fields for an opcode and a memory address; and executing the decoded instruction to set a tile configuration for the processor to utilize tiles in matrix operations based on a description retrieved from the memory address, wherein a tile a set of 2-dimensional registers.

[0268] Example 16 The non-transitory machine-readable medium of example 15, wherein the configuration comprises a palette identifier and details regarding a number of rows and columns per tile of the palette.

[0269] Example 17 The non-transitory machine-readable medium of example 15, wherein the palette identifier is an index into a palette table defining a number of bytes per tile and bytes per row of the tile.

[0270] Example 18 The non-transitory machine-readable medium of any of examples 15-17, wherein the configuration is stored in a single packed data register.

[0271] Example 19 The non-transitory machine-readable medium of any of examples 15-17, wherein the configuration is stored in a plurality of packed data registers, one packed data register per configured tile.

[0272] Example 20 The non-transitory machine-readable medium of any of examples 15-19, further comprising: faulting upon a value of a number of rows for a tile being zero and a number of columns for the row being non-zero.

[0273] Example 21 The non-transitory machine-readable medium of any of examples 15-20, wherein the description is 64 bytes in size.

[0274] *B. Tile Load*

[0275] As noted above, a common operation is to load a time from memory prior to performing an operation on it. Detailed herein are embodiments of a matrix (tile) load ("TILELOAD") instruction and its execution. A TILELOAD instruction is an improvement a computer itself as it provides for support to move data from one matrix (tile) to another matrix (tile) with a single instruction. In particular, the execution of the TILELOAD instruction causes data from memory to be loaded into a destination matrix (tile). The size of the data values to be loaded varies depending on the instruction and tile support. Exemplary sizes included, but are not limited to, 16-bit, 32-bit, 64-bit, 128-bit, 256-bit, etc. Typically, the data stored in memory is strided.

[0276] *i. Exemplary Execution*

[0277] Figure 24 illustrates an exemplary execution of a TILELOAD instruction. The TILELOAD instruction format includes fields for an opcode, a source memory address (shown as "SIBMEM" in the figure), and an identifier of a destination matrix (tile) operand (shown as "Destination Matrix (Tile)" in the figure).

[0278] As shown, the source is memory 2401. A plurality of data elements (shaded) is to be loaded from memory into a destination matrix (tile). The source address of the memory are provided by the initial memory address of the instruction (e.g., base plus displacement) and the stride value. (e.g., index << scale). In most embodiments, the execution circuitry is a memory (store) execution circuit. In some embodiments, the memory (store) execution circuit is also utilized for non-matrix (tile) memory operations.

[0279] The "stride" comes from an index register as indicated in the memory addressing scheme. The stride indicates an amount of memory to go from one group of data elements of memory to another group of data elements that are a "stride" away from the group. Depending upon the implementation, the stride is either from an address corresponding to an initial data element of a group to an initial data element of a subsequent group in memory, or from an address corresponding to a last data element of a group to an initial data element of a subsequent group in memory. Typically, strides are used to delineate rows, however, that in not necessarily true. The source memory address information includes a scale, index (stride value), and base (SIB). In a SIB addressing scheme, the stride is the index (I) shifted by the scale (S).

[0280] The destination matrix (tile) operand field represents a destination matrix (tile) to be stored in tile storage. A matrix (tile) may be stored in storage 2423 within execution circuitry in a collection of registers or other storage, or in storage external to execution resources 2421.

[0281] As shown, execution circuity 2407 of a processor, accelerator, or core 2405 executes a decoded TILELOAD instruction to store the source data of the source memory 2401 into matrix (tile) storage 2421 or 2423. The source addresses of the memory are provided by the initial memory address of the instruction (e.g., base plus displacement) and the stride value. (e.g., index << scale). In most embodiments, the execution circuitry is a memory (load) execution circuit. In some embodiments, the memory (load) execution circuit is also utilized for non-matrix (tile) memory operations.

[0282] *ii. Exemplary Format(s)*

[0283] An embodiment of a format for a TILELOAD instruction is TILELOAD{B/W/D/Q} TMM1, SIBMEM. In some embodiments, TILELOAD{B/W/D/Q} is the opcode mnemonic of the instruction where B/W/D/Q represent data element sizes (byte, word, double word, quadword) of the source and destination. TMM1 is a field for identifying a destination matrix (tile) operand. SIBMEM is a field storing address information for the initial memory address to be used for the destination. In some embodiments, the SIBMEM includes the use of a R/M value (such as 8246), SIB Byte 8250, and displacement 8262. In some embodiments, the destination matrix (tile) field is REG 8244, and the data element size is found in 8265.

[0284] In embodiments, encodings of the instruction include a scale-index-base (SIB) type memory addressing operand that indirectly identifies multiple indexed destination locations in memory (e.g., field 8250). In one embodiment, an SIB type memory operand may include an encoding identifying a base address register. The contents of the base address register may represent a base address in memory from which the addresses of the particular destination locations in memory are calculated. For example, the base address may be the address of the first location in a block of potential destination locations for an extended vector instruction. In one embodiment, an SIB type memory operand may include an encoding identifying an index register. Each element of the index register may specify an index or offset value usable to compute, from the base address, an address of a respective destination location within a block of potential destination locations. In one embodiment, an SIB type memory operand may include an encoding specifying a scaling factor to be applied to each index value when computing a respective destination address. For example, if a scaling factor value of four is encoded in the SIB type memory operand, each index value obtained from an element of the index register may be multiplied by four and then added to the base address to compute a destination address.

[0285] In one embodiment, an SIB type memory operand of the form vm32{x,y,z} may identify a vector array of memory operands specified using SIB type memory addressing. In this example, the array of memory addresses is specified using a common base register, a constant scaling factor, and a vector index register containing individual elements, each of which is a 32-bit index value. The vector index register may be a 128-bit register (e.g., XMM) register (vm32x), a 256-bit (e.g., YMM) register (vm32y), or a 512-bit (e.g., ZMM) register (vm32z). In another embodiment, an SIB type memory operand of the form vm64{x,y,z} may identify a vector array of memory operands specified using SIB type memory addressing. In this example, the array of memory addresses is specified using a common base register, a constant scaling factor, and a vector index register containing individual elements, each of which is a 64-bit index value. The vector index register may be a 128-bit register (e.g., XMM) register (vm64x), a 256-bit (e.g., YMM) register (vm64y) or a 512-bit (e.g., ZMM) register (vm64z).

[0286] *iii. Exemplary Method(s) of Execution*

[0287] Figure 25 illustrates an embodiment of method performed by a processor to process a TILELOAD instruction.

[0288] At 2501, an instruction is fetched. For example, a TILELOAD instruction is fetched. The TILELOAD instruction includes fields for an opcode, a source memory address, a stride value, and a destination matrix (tile) operand. In some embodiments, the instruction is fetched from an instruction cache. The opcode of the TILELOAD instruction indicates a load of rows from memory into a destination matrix (tile) operand is to occur.

[0289] The fetched instruction is decoded at 2503. For example, the fetched TILELOAD instruction is decoded by decode circuitry such as that detailed herein.

[0290] Data values associated with the strided source memory address of the decoded instruction are retrieved at 2505 and the decoded instruction is scheduled (as needed).

[0291] At 2507, the decoded instruction is executed by execution circuitry (hardware) such as that detailed herein. For the TILELOAD instruction, the execution will cause execution circuitry to a load groups of strided data elements from memory into configured rows and columns of the destination matrix (tile) operand.

[0292] In some embodiments, the instruction is committed or retired at 2509.

[0293] Figure 26 illustrates a more detailed description of an execution of a TILELOAD instruction. Typically, this is performed by execution circuitry such as that detailed above.

[0294] At 2601, an initial memory address is determined. For example, using SIB addressing, the base provided by the instruction is added to the displacement value.

[0295] At 2603, an initial stride is determined. For example, using SIB addressing, the index provided by the instruction shifted by the scale value of the instruction.

[0296] A counter used in the load operation is set to zero at 2605. The use of the counter allows for the load to be restarted at a particular row.

[0297] Data elements of the memory address are retrieved at 2607. For example, in an initial iteration, data elements at memory[initial memory address] are retrieved.

[0298] The retrieved data elements are loaded in a configured row of the destination matrix (tile) operand corresponding to the counter value at 2609. For example, for the initial iteration, row[counter=0, no stride] is filled with these data elements. In some embodiments, unconfigured columns are zeroed.

[0299] A determination of if the counter is at a maximum value is made at 2610. For example, is the counter less than the number of rows in the destination? Note that if the counter is initially set to 1, then this is a check if the counter is less than or equal to the number of rows in the destination.

[0300] If not, then the load is done and all unconfigured rows are zeroed. If the counter is not maxed out, then the counter is incremented at 2611. For example, the counter is increased by 1 which indicates the next row of the destination is to be loaded.

[0301] The memory address using the determined stride is updated at 2613. For example, the memory address used in the previous retrieval is updated to account for the row position (counter) and stride (e.g., counter * stride is added to the previously used address).

[0302] In some embodiments, if tiles are not configured for use, a fault is generated before any execution takes place.

[0303] To restart, the execution picks up from the last row to be loaded. No zeroing is done.

[0304] *iv. Exemplary Pseudocode*

[0305] FIGS. 27(A)-(C) illustrate examples of pseudocode representing a method of executing a TILELOAD instruction using words, doublewords, and quadwords.

[0306] *v. Examples*

[0307] **EXAMPLE 1** A processor comprising: decode circuitry to decode an instruction having fields for an opcode, a destination matrix operand identifier, and source memory information; and execution circuitry to execute the decoded instruction to load groups of strided data elements from memory into configured rows of the identified destination matrix operand to memory.

[0308] **EXAMPLE 2** The processor of example 1, wherein the opcode defines a size of each data element of the destination matrix operand.

[0309] **EXAMPLE 3** The processor of example 2, wherein the size of each data element of the destination matrix operand is a doubleword.

[0310] **EXAMPLE 4** The processor of example 2, wherein the size of each data element of the destination matrix operand is a word.

[0311] **EXAMPLE 5** The processor of any of examples 1-4, wherein the execution circuitry is to store each configured row into the identified destination matrix operand and update a counter value as each row is stored.

[0312] **EXAMPLE 6** The processor of any of examples 1-5, wherein the identified destination matrix operand is a plurality of registers configured to represent a matrix.

[0313] **EXAMPLE 7** The processor of any of examples 1-6, wherein the source memory information includes a scale, an index, a base, and a displacement.

[0314] **EXAMPLE 8** A method comprising: decoding an instruction having fields for an opcode, a destination matrix operand identifier, and source memory information; and executing the decoded instruction to load groups of strided data elements from memory into configured rows of the identified destination matrix operand to memory.

[0315] **EXAMPLE 9** The method of example 8, wherein the opcode defines a size of each data element of the destination matrix operand.

[0316] **EXAMPLE 10** The method of example 9, wherein the size of each data element of the destination matrix operand is a doubleword.

[0317] **EXAMPLE 11** The method of example 9, wherein the size of each data element of the destination matrix operand is a word.

[0318] **EXAMPLE 12** The method of any of examples 8-11, wherein the execution circuitry is to load each configured row of the identified destination matrix operand and update a counter value as each row is loaded.

[0319] **EXAMPLE 13** The method of any of examples 8-12, wherein the identified destination matrix operand is a plurality of registers configured to represent a matrix.

[0320] **EXAMPLE 14** The method of any of examples 8-13, wherein the source memory information includes a scale, an index, a base, and a displacement.

[0321] **EXAMPLE 15** A non-transitory machine-readable medium storing an instruction which causes a processor to perform a method, the method comprising: decoding an instruction having fields for an opcode, a destination matrix operand identifier, and source memory information; and executing the decoded instruction to load groups of strided data

elements from memory into configured rows of the identified destination matrix operand to memory.

[0322] **EXAMPLE 16** The non-transitory machine-readable medium of example 15, wherein the opcode defines a size of each data element of the destination matrix operand.

[0323] **EXAMPLE 17** The non-transitory machine-readable medium of example 16, wherein the size of each data element of the destination matrix operand is a doubleword.

[0324] **EXAMPLE 18** The non-transitory machine-readable medium of example 16, wherein the size of each data element of the destination matrix operand is a word.

[0325] **EXAMPLE 19** The non-transitory machine-readable medium of any of examples 15-18, wherein the execution circuitry is to load each configured row of the identified destination matrix operand and update a counter value as each row is loaded.

[0326] **EXAMPLE 20** The non-transitory machine-readable medium of any of examples 15-19, wherein the identified destination matrix operand is a plurality of registers configured to represent a matrix.

[0327] **EXAMPLE 21** The non-transitory machine-readable medium of any of examples 15-20, wherein the source memory information includes a scale, an index, a base, and a displacement.

[0328] **EXAMPLE 22** A system comprising: a processor; and an accelerator coupled to the processor, the accelerator including: decode circuitry to decode an instruction having fields for an opcode, a destination matrix operand identifier, and source memory information, and execution circuitry to execute the decoded instruction to load groups of strided data elements from memory into configured rows of the identified destination matrix operand to memory.

[0329] **EXAMPLE 23** The system of example 22, wherein the execution circuitry is to load each configured row of the identified destination matrix operand and update a counter value as each row is stored.

[0330] **EXAMPLE 24** The system of any of examples 22-23, wherein the identified source destination operand is a plurality of registers configured to represent a matrix.

[0331] **EXAMPLE 25** The system of any of examples 22-24, wherein the source memory information includes a scale, an index, a base, and a displacement.

[0332] *C. Tile Store*

[0333] As discussed above, matrices (tiles) may need to loaded from and stored to memory as there is typically not enough on die storage to indefinitely hold them. Detailed herein are embodiments of a matrix (tile) store ("TILESTORE") instruction and its execution.

A TILESTORE instruction is an improvement a computer itself as it provides for support to move data from one matrix (tile) to another matrix (tile) with a single instruction. In particular, the execution of the TILESTORE instruction causes data from source matrix (tile) to be stored in memory. The size of the data values to be stored varies depending on the instruction and tile support. Exemplary sizes included, but are not limited to, 16-bit, 32-bit, 64-bit, 128-bit, 256-bit, etc.

[0334] *i. Exemplary Execution*

[0335] Figure 28 illustrates an exemplary execution of a TILESTORE instruction. The TILESTORE instruction format includes fields for an opcode, a destination memory address information (shown as "SIBMEM" in the figure), and an identifier of source matrix (tile) operand (shown as "Source Matrix (Tile)" in the figure).

[0336] As shown, the destination is memory 2801. A plurality of data elements is to be stored from a source matrix (tile) in memory.

[0337] The source matrix (tile) operand field represents a source matrix (tile) to be stored in tile storage. The source matrix (tile) may be stored in storage 2823 within execution circuitry in a collection of registers or other storage, or in storage external to execution resources 2821.

[0338] As shown, execution circuity 2807 of a processor, accelerator, or core 2805 executes a decoded TILESTORE instruction to store the source data into the destination memory 2801 from configured rows of the source matrix (tile) storage 2821 or 2823. The destination addresses of the memory are provided by the initial memory address of the instruction (e.g., base plus displacement) and the stride value. (e.g., index << scale). In most embodiments, the execution circuitry is a memory (store) execution circuit. In some embodiments, the memory (store) execution circuit is also utilized for non-matrix (tile) memory operations.

[0339] The "stride" comes from an index register as indicated in the memory addressing scheme. The stride indicates an amount of memory to go from one group of data elements of memory to be stored to another group of data elements that are a "stride" away from the group. Depending upon the implementation, the stride is either from an address corresponding to an initial data element of a group to an initial data element of a subsequent group in memory, or from an address corresponding to a last data element of a group to an initial data element of a subsequent group in memory. Typically, strides are used to delineate rows, however, that is not necessarily true. The destination memory address information

includes a scale, index (stride value), and base (SIB). In a SIB addressing scheme, the stride is the index (I) shifted by the scale (S).

[0340] *ii. Exemplary Format(s)*

[0341] An embodiment of a format for a TILESTORE instruction is TILESTORE{B/W/D/Q} SIBMEM, TMM1. In some embodiments, TILESTORE{B/W/D/Q} is the opcode mnemonic of the instruction where B/W/D/Q represent data element sizes (byte, word, double word, quadword) of the source and destination. TMM1 is a field for identifying a source matrix (tile) operand. SIBMEM is a field storing address information for the initial memory address to be used for the destination. In some embodiments, the SIBMEM includes the use of a R/M value (such as 8246), SIB Byte 8250, and displacement 8262. In some embodiments, the source matrix (tile) field is REG 8244, and the data element size is found in 8265.

[0342] In embodiments, encodings of the instruction include a scale-index-base (SIB) type memory addressing operand that indirectly identifies multiple indexed destination locations in memory (e.g., field 8250). In one embodiment, an SIB type memory operand may include an encoding identifying a base address register. The contents of the base address register may represent a base address in memory from which the addresses of the particular destination locations in memory are calculated. For example, the base address may be the address of the first location in a block of potential destination locations for an extended vector instruction. In one embodiment, an SIB type memory operand may include an encoding identifying an index register. Each element of the index register may specify an index or offset value usable to compute, from the base address, an address of a respective destination location within a block of potential destination locations. In one embodiment, an SIB type memory operand may include an encoding specifying a scaling factor to be applied to each index value when computing a respective destination address. For example, if a scaling factor value of four is encoded in the SIB type memory operand, each index value obtained from an element of the index register may be multiplied by four and then added to the base address to compute a destination address.

[0343] In one embodiment, an SIB type memory operand of the form vm32{x,y,z} may identify a vector array of memory operands specified using SIB type memory addressing. In this example, the array of memory addresses is specified using a common base register, a constant scaling factor, and a vector index register containing individual elements, each of which is a 32-bit index value. The vector index register may be a 128-bit register (e.g., XMM) register (vm32x), a 256-bit (e.g., YMM) register (vm32y), or a 512-bit (e.g., ZMM)

register (vm32z). In another embodiment, an SIB type memory operand of the form vm64{x,y,z} may identify a vector array of memory operands specified using SIB type memory addressing. In this example, the array of memory addresses is specified using a common base register, a constant scaling factor, and a vector index register containing individual elements, each of which is a 64-bit index value. The vector index register may be a 128-bit register (e.g., XMM) register (vm64x), a 256-bit (e.g., YMM) register (vm64y) or a 512-bit (e.g., ZMM) register (vm64z).

[0344] *iii. Exemplary Method(s) of Execution*

[0345] Figure 29 illustrates an embodiment of method performed by a processor to process a TILESTORE instruction.

[0346] At 2901, an instruction is fetched. For example, a TILESTORE instruction is fetched. The TILESTORE instruction includes fields for an opcode, destination memory information, and an identifier of a source matrix (tile) operand. In some embodiments, the instruction is fetched from an instruction cache. The opcode of the TILESTORE instruction indicates a store of rows of data into memory from a source matrix (tile) operand is to occur.

[0347] The fetched instruction is decoded at 2903. For example, the fetched TILESTORE instruction is decoded by decode circuitry such as that detailed herein.

[0348] Data values associated with the source matrix (tile) of the decoded instruction are retrieved at 2905 and the decoded instruction is scheduled (as needed).

[0349] At 2907, the decoded instruction is executed by execution circuitry (hardware) such as that detailed herein. For the TILESTORE instruction, the execution will cause execution circuitry to a store groups of data elements into memory from configured rows and columns of the source matrix (tile) operand based on memory address information provided by the instruction.

[0350] In some embodiments, the instruction is committed or retired at 2909.

[0351] Figure 30 illustrates a more detailed description of an execution of a TILESTORE instruction. Typically, this is performed by execution circuitry such as that detailed above.

[0352] At 3001, an initial memory address is determined. For example, using SIB addressing, the base provided by the instruction is added to the displacement value.

[0353] At 3003, an initial stride is determined. For example, using SIB addressing, the index provided by the instruction is shifted by the scale value of the instruction. Additionally, a counter used in the store operation is set to zero. The use of the counter allows for the store to be restarted at a particular row.

[0354] Data elements of a first configured row of the source matrix (tile) are stored in memory at 3005. For example, in an initial iteration, data elements of a first configured row (not including columns that were not configured for use) are stored at memory[initial memory address].

[0355] A determination of if the last row of the source has been stored is made at 3007. For example, is the counter less than the number of rows in the destination? Note that if the counter is initially set to 1, then this is a check if the counter is less than or equal to the number of rows in the destination.

[0356] If yes, then the store is done. If the counter is not maxed out, then the counter is incremented at 3009. For example, the counter is increased by 1 which indicates the next row of the source is to be stored.

[0357] The memory address using the determined stride is updated at 3011. For example, the memory address used in the previous retrieval is updated to account for the row position (counter) and stride (e.g., counter * stride is added to the previously used address).

[0358] An immediately sequential row of the source matrix (tile) to what has last stored is stored at the updated memory address at 3013.

[0359] In some embodiments, if tiles are not configured for use, a fault is generated before any execution takes place. To restart, the execution picks up from the last row to be stored.

[0360] *iv. Exemplary Pseudocode*

[0361] FIGS. 31(A)-(C) illustrate examples of pseudocode representing methods of executing a TILESTORE instruction using words, doublewords, and quadwords.

[0362] *v. Examples*

[0363] **Example 1** A processor comprising: decode circuitry to decode an instruction having fields for an opcode, a source matrix operand identifier, and destination memory information; and execution circuitry to execute the decoded instruction to store each data element of configured rows of the identified source matrix operand to memory based on the destination memory information.

[0364] **Example 2** The processor of claim 1, wherein the opcode defines a size of each data element of the source matrix operand.

[0365] **Example 3** The processor of example 2, wherein the size of each data element of the source matrix operand is a doubleword.

[0366] **Example 4** The processor of example 2, wherein the size of each data element of the source matrix operand is a word.

[0367] **Example 5** The processor of any of examples 1-4, wherein the execution circuitry is to store each configured row of the identified source matrix operand and update a counter value as each row is stored.

[0368] **Example 6** The processor of any of examples 1-5, wherein the identified source matrix operand is a plurality of registers configured to represent a matrix.

[0369] **Example 7** The processor of any of examples 1-6, wherein the destination memory information includes a scale, an index, a base, and a displacement.

[0370] **Example 8** A method comprising: decoding an instruction having fields for an opcode, a source matrix operand identifier, and destination memory information; and executing the decoded instruction to store each data element of configured rows of the identified source matrix operand to memory based on the destination memory information.

[0371] **Example 9** The method of example 8, wherein the opcode defines a size of each data element of the source matrix operand.

[0372] **Example 10** The method of example 9, wherein the size of each data element of the source matrix operand is a doubleword.

[0373] **Example 11** The method of example 9, wherein the size of each data element of the source matrix operand is a word.

[0374] **Example 12** The method of any of examples 8-11, wherein the execution circuitry is to store each configured row of the identified source matrix operand and update a counter value as each row is stored.

[0375] **Example 13** The method of any of examples 8-12, wherein the identified source matrix operand is a plurality of registers configured to represent a matrix.

[0376] **Example 14** The method of any of examples 8-13, wherein the destination memory information includes a scale, an index, a base, and a displacement.

[0377] **Example 15** A non-transitory machine-readable medium storing an instruction which causes a processor to perform a method, the method comprising: decoding an instruction having fields for an opcode, a source matrix operand identifier, and destination memory information; and executing the decoded instruction to store each data element of configured rows of the identified source matrix operand to memory based on the destination memory information.

[0378] **Example 16** The non-transitory machine-readable medium of example 15, wherein the opcode defines a size of each data element of the source matrix operand.

[0379] **Example 17** The non-transitory machine-readable medium of example 16, wherein the size of each data element of the source matrix operand is a doubleword.

[0380] **Example 18** The non-transitory machine-readable medium of example 16, wherein the size of each data element of the source matrix operand is a word.

[0381] **Example 19** The non-transitory machine-readable medium of any of examples 15-18, wherein the execution circuitry is to store each configured row of the identified source matrix operand and update a counter value as each row is stored.

[0382] **Example 20** The non-transitory machine-readable medium of any of examples 15-19, wherein the identified source matrix operand is a plurality of registers configured to represent a matrix.

[0383] **Example 21** The non-transitory machine-readable medium of any of examples 15-20, wherein the destination memory information includes a scale, an index, a base, and a displacement.

[0384] **Example 22** A system comprising: a processor; and an accelerator coupled to the processor, the accelerator including: decode circuitry to decode an instruction having fields for an opcode, a source matrix operand identifier, and destination memory information, and execution circuitry to execute the decoded instruction to store each data element of configured rows of the identified source matrix operand to memory based on the destination memory information.

[0385] **Example 23** The system of example 22, wherein the execution circuitry is to store each configured row of the identified source matrix operand and update a counter value as each row is stored.

[0386] **Example 24** The system of any of examples 22-23, wherein the identified source matrix operand is a plurality of registers configured to represent a matrix.

[0387] **Example 25** The system of any of examples 22-24, wherein the destination memory information includes a scale, an index, a base, and a displacement.

[0388] *D. Tile Diagonal*

[0389] Detailed herein are embodiments of a matrix (tile) diagonal ("TILEDIAGONAL") instruction and its execution. A TILEDIAGONAL instruction is an improvement to a computer itself as it provides for support to populate the main diagonal of a matrix (tile) with a single instruction. In particular, the execution of the TILEDIAGONAL instruction causes execution circuitry to store the identified source operand to every element along the main diagonal of the destination matrix (tile) and zeros all other elements in configured rows. The size of the data values to be stored varies depending on the instruction and tile support. Exemplary sizes include, but are not limited to, 16-bit, 32-bit, 64-bit, 128-bit, 256-bit, etc. In some embodiments, elements of the destination matrix (tile) that are not on the diagonal are

zeroed. This instruction may be used, for example, to generate a diagonal matrix, scalar matrix, or identity matrix.

[0390] *i. Exemplary Execution*

[0391] Figure 32 illustrates an exemplary execution of a TILEDIAGONAL instruction. The TILEDIAGONAL instruction 3202 format includes fields for an opcode, a source operand identifier, and a destination matrix (tile) operand identifier (shown as "DESTINATION MATRIX (TILE)").

[0392] The source operand 3204, as shown, identifies a register such as, for example, a general purpose register of a processor's register file. The destination matrix (tile) operand fields represent a destination matrix (tile) 3210. As detailed earlier, a matrix (tile) may be stored in a collection of registers, locations in memory (e.g., as strided rows), or in other storage accessible to execution circuitry.

[0393] As shown, execution circuity 3206 executes a decoded TILEDIAGONAL instruction to store an identified source operand 3204 to every element along the main diagonal of destination matrix (tile) 3210.

[0394] Also shown are remaining (unconfigured) columns and rows being set to zero, which is done in some embodiments. In some embodiments, a matrix (tile) is configured to use only a subset of the rows and columns possible. For example, a matrix (tile) may have up to 16 rows and columns to use, but only uses 4 of each. The configuration of each matrix (tile) is typically done by the execution of a configuration instruction prior to matrix (tile) usage. In this example, there are N columns and M rows possible.

[0395] *ii. Exemplary Format(s)*

[0396] An embodiment of a format for a TILEDIAGONAL instruction is TILEDIAGONAL DESTINATION MATRIX (TILE) IDENTIFIER, SOURCE OPERAND IDENTIFIER. In some embodiments, TILEDIAGONAL [A]{B/W/D/Q} is the opcode mnemonic of the instruction where B/W/D/Q is an optional field to represent data element sizes (byte, word, double word, quadword) of the source scalar value and the destination matrix (tile) elements, and where A is an optional prefix that indicates that an antidiagonal is to be generated, rather than a main diagonal. DESTINATION MATRIX (TILE) IDENTIFIER is a field for the destination matrix (tile) operand. SOURCE OPERAND IDENTIFIER is a field for the source operand identifier. In some embodiments, the SOURCE OPERAND IDENTIFIER field is a R/M value (such as 82 46), the destination matrix (tile) field is REG 8244, and the data element size is found in 8265.

[0397] In some embodiments, encodings of the instruction include a scale-index-base (SIB) type memory addressing operand that indirectly identifies multiple indexed destination locations in memory. In one embodiment, an SIB type memory operand may include an encoding identifying a base address register. The contents of the base address register may represent a base address in memory from which the addresses of the particular destination locations in memory are calculated. For example, the base address may be the address of the first location in a block of potential destination locations for an extended vector instruction. In one embodiment, an SIB type memory operand may include an encoding identifying an index register. Each element of the index register may specify an index or offset value usable to compute, from the base address, an address of a respective destination location within a block of potential destination locations. In one embodiment, an SIB type memory operand may include an encoding specifying a scaling factor to be applied to each index value when computing a respective destination address. For example, if a scaling factor value of four is encoded in the SIB type memory operand, each index value obtained from an element of the index register may be multiplied by four and then added to the base address to compute a destination address.

[0398] In one embodiment, an SIB type memory operand of the form vm32{x, y, z} may identify a vector array of memory operands specified using SIB type memory addressing. In this example, the array of memory addresses is specified using a common base register, a constant scaling factor, and a vector index register containing individual elements, each of which is a 32-bit index value. The vector index register may be a 128-bit register (e.g., XMM) register (vm32x), a 256-bit (e.g., YMM) register (vm32y), or a 512-bit (e.g., ZMM) register (vm32z). In another embodiment, an SIB type memory operand of the form vm64{x, y, z} may identify a vector array of memory operands specified using SIB type memory addressing. In this example, the array of memory addresses is specified using a common base register, a constant scaling factor, and a vector index register containing individual elements, each of which is a 64-bit index value. The vector index register may be a 128-bit register (e.g., XMM) register (vm64x), a 256-bit (e.g., YMM) register (vm64y) or a 512-bit (e.g., ZMM) register (vm64z).

[0399] *iii. Exemplary Method(s) of Execution*

[0400] Figure 33 illustrates an embodiment of method performed by a processor to process a TILEDIAGONAL instruction.

[0401] At 3301, an instruction is fetched. For example, a TILEDIAGONAL instruction is fetched. The TILEDIAGONAL instruction includes fields for an opcode, a source operand

identifier, and a destination matrix (tile) operand identifier. In some embodiments, the instruction is fetched from an instruction cache. The opcode of the TILEDIAGONAL instruction indicates populating a main diagonal of an identified destination matrix (tile) operand is to occur, and a size of the data to be stored (written).

[0402] The fetched instruction is decoded at 3303. For example, the fetched TILEDIAGONAL instruction is decoded by decode circuitry such as that detailed herein.

[0403] Data values associated with the identified source operand of the decoded instruction are retrieved at 3305 and the decoded instruction is scheduled (as needed). For example, when the identified source operand is a memory location, the data from the indicated memory location is retrieved.

[0404] At 3307, the decoded instruction is executed by execution circuitry (hardware) such as that detailed herein. For the TILEDIAGONAL instruction, the execution will cause execution circuitry to store (write) the identified source operand to every element along the main diagonal of the destination matrix (tile). In some embodiments, unconfigured elements of rows of the destination matrix (tile) are zeroed as are elements not on the diagonal.

[0405] In some embodiments, the instruction is committed or retired at 3309.

[0406] Figure 34 illustrates a more detailed description of an execution of a TILEDIAGONAL instruction. Typically, this is performed by execution circuitry such as that detailed above.

[0407] At 3402, a determination is made as to whether the destination matrix (tile) elements have the same size as the identified source operand. If not, then a fault is raised at 3404. 3402 is optional, as signified by its dashed borders in Figure 34 (dashed borders are used herein to identify optional items.)

[0408] If a positive determination is made at 3402, the execution circuitry at 3406 loops with loop index x equal to zero (0) to the Minimum of (dest.rows and dest.columns). For example, if the destination matrix (tile) has four rows and five columns, x will go from zero to four. On each loop iteration, at 3408, the execution circuitry stores (writes) the identified source operand to the destination matrix (tile) at element [x][x]. At 3410 the execution circuit increments x, and determines whether at least one more row and at least one more column of the destination matrix (tile) remain. If so, the execution circuit returns to the start of the loop at 3406. If it is determined at 3410 that there is not at least one remaining row and at least one remaining column, the process ends. In this way, the execution circuit, over the course of the loop, stores (writes) the identified source operand to every element along the main diagonal of the destination matrix (tile).

[0409] *iv. Exemplary Pseudocode*

[0410] Figure 35 is exemplary pseudocode describing an embodiment of a method performed by a processor to process a TILEDIAGONALD instruction. As shown in pseudocode 3502, the TILEDIAGONALD instruction includes an opcode, a source operand identifier SRC, and a destination operand TDEST to identify a destination matrix (tile). As shown, the pseudocode 3502 first causes the execution circuitry to generate a fault if any of three error checks fails. Then the pseudocode causes the processor to loop for a number of LOOP_ITERATIONS equaling the MINIMUM of the number of rows and the number of columns of the destination matrix (tile). At each iteration, the processor sets the double word at destination element [x][x] to the value of the source operand. The TILEDIAGONALD opcode includes a "D" suffix, indicating that the elements of the destination matrix (tile) are each the size of a doubleword. Pseudocode 3504 operates similarly to pseudocode 3502, but has a "W" suffix, indicating that its destination matrix (tile) elements are each two bytes in size.

[0411] *v. Examples*

[0412] Example 1 A processor comprising: decode circuitry to decode an instruction having fields for an opcode, a source operand identifier, and a destination matrix operand identifier; and execution circuitry to execute the decoded instruction to write the identified source operand to each element along a main diagonal of the identified destination matrix operand.

[0413] Example 2 The processor of example 1, wherein the opcode defines a size of each data element of the destination matrix operand.

[0414] Example 3 The processor of example 2, wherein the size of each data element of the destination matrix operand is a doubleword.

[0415] Example 4 The processor of example 2, wherein the size of each data element of the destination matrix operand is a word.

[0416] Example 5 The processor of any of examples 1-4, wherein the execution circuitry is further to zero any remaining columns of the identified destination matrix operand and unconfigured rows of the identified destination matrix operand.

[0417] Example 6 The processor of any of examples 1-5, wherein the destination matrix operand is a plurality of registers to represent a matrix.

[0418] Example 7 The processor of any of examples 1-5, wherein the execution circuitry is to fault upon a determination of one of: the identified source operand having a different number of bytes than each element of the identified destination matrix operand, each element

of the destination matrix operand having a different size than a size identifier included in the opcode, and the identified destination matrix operand having zero configured elements.

[0419] Example 8 A method comprising: decoding an instruction having fields for an opcode, a source operand identifier, and a destination matrix operand identifier; and executing the decoded instruction to write the identified source operand to each element along a main diagonal of the identified destination matrix operand.

[0420] Example 9 The method of example 8, wherein the opcode defines a size of each data element of the destination matrix operand.

[0421] Example 10 The method of example 9, wherein the size of each data element of the destination matrix operands is a doubleword.

[0422] Example 11 The method of example 9, wherein the size of each data element of the destination matrix operands is a word.

[0423] Example 12 The method of any of examples 8-11, further comprising zeroing any remaining columns of the identified destination matrix operand and unconfigured rows of the identified destination matrix operand.

[0424] Example 13 The method of any of examples 8-12, wherein the identified destination matrix operand is a plurality of registers to represent a matrix.

[0425] Example 14 The method of any of examples 8-13, further comprising faulting upon a determination of one of: the identified source operand having a different number of bytes than each element of the identified destination matrix operand, each element of the destination matrix operand having a different size than a size identifier included in the opcode, and the identified destination matrix operand having zero configured elements.

[0426] Example 15 A non-transitory machine-readable medium storing an instruction which causes a processor to perform a method, the method comprising: decoding an instruction having fields for an opcode, a source operand identifier, and a destination matrix operand identifier; and executing the decoded instruction to write the identified source operand to each element along a main diagonal of the identified destination matrix operand.

[0427] Example 16 The non-transitory machine-readable medium of example 15, wherein the opcode defines a size of each data element of the destination matrix operand.

[0428] Example 17 The non-transitory machine-readable medium of example 16, wherein the size of each data element of the destination matrix operand is a doubleword.

[0429] Example 18 The non-transitory machine-readable medium of example 16, wherein the size of each data element of the destination matrix operand is a word.

[0430] Example 19 The non-transitory machine-readable medium of any of examples 15-18, wherein the method further comprises zeroing any remaining columns of the identified destination matrix operand and unconfigured rows of the identified destination matrix operand.

[0431] Example 20 The non-transitory machine-readable medium of any of examples 15-19, wherein the identified destination matrix operand is a plurality of registers to represent a matrix.

[0432] Example 21 The non-transitory machine-readable medium of any of examples 15-20, wherein the method further comprises faulting upon a determination of one of: the identified source operand having a different number of bytes than each element of the identified destination matrix operand, each element of the destination matrix operand having a different size than a size identifier included in the opcode, and the identified destination matrix operand having zero configured elements.

[0433] Example 22 A system comprising: a processor; and an accelerator coupled to the processor, the accelerator including: decode circuitry to decode an instruction having fields for an opcode, a source operand identifier, and a destination matrix operand identifier; and execution circuitry to execute the decoded instruction to write the identified source operand to each element along a main diagonal of the identified destination matrix operand.

[0434] Example 23 The system of example 22, wherein the opcode defines a size of each data element of the destination matrix operand.

[0435] Example 32 The system of any of examples 22-23, wherein the execution circuitry is further to zero any remaining columns of the identified destination matrix operand and unconfigured rows of the identified destination matrix operand.

[0436] Example 33 The system of any of examples 22-32, wherein the destination matrix operand is a plurality of registers to represent a matrix.

[0437] *E. Tile Transpose*

[0438] A common matrix operation is transpose. An example of a particular usage is to change format from row to column major and back. Detailed herein are embodiments of a TILETRANSPOSE instruction and its execution. A TILETRANSPOSE instruction is an improvement to a computer itself as it provides for support to transpose data within a matrix (tile) with a single instruction. In particular, the execution of the TILETRANSPOSE instruction causes the rows of a source matrix (tile) to be written as the columns of a destination matrix (tile). The size of the data values to be stored varies depending on the instruction and tile support. Exemplary sizes include, but are not limited to, 16-bit, 32-bit,

64-bit, 128-bit, 256-bit, etc. In some embodiments, elements of rows of the destination matrix (tile) that do not have corresponding columns in the source matrix (tile) are zeroed.

[0439] *i. Exemplary Execution*

[0440] **Figure 36** illustrates an exemplary execution of a TILETRANSPOSE instruction. The TILETRANSPOSE instruction format includes fields for an opcode, a source matrix (tile) operand (shown as "SOURCE MATRIX (TILE)"), and a destination matrix (tile) operand (shown as "DESTINATION MATRIX (TILE)").

[0441] The source matrix (tile) operand and destination matrix (tile) operand fields represent a source matrix (tile) 3604 and a destination matrix (tile) 3608. As detailed earlier, a matrix (tile) may be stored in a collection of registers, locations in memory, or in other storage accessible to execution circuitry.

[0442] As shown, execution circuity 3610 executes a decoded TILETRANSPOSE instruction to transpose the source data of the source matrix (tile) operand 3604 into configured rows of the destination matrix (tile) operand 3608.

[0443] Also shown are remaining (unconfigured) columns and rows being set to zero which is done in some embodiments. In some embodiments, a matrix (tile) is configured to use only a subset of the rows and columns possible. For example, a matrix (tile) may have up to 16 rows and columns to use, but only use 4 of each. The configuration of each matrix (tile) is typically done by the execution of a configuration instruction prior to matrix (tile) usage. In this example, there are N columns and M rows possible.

[0444] *ii. Exemplary Format(s)*

[0445] An embodiment of a format for a TILETRANSPOSE instruction is TILETRANSPOSE{B/W/D/Q} TMM1, TMM2. In some embodiments, TILETRANSPOSE{B/W/D/Q} is the opcode mnemonic of the instruction where B/W/D/Q represent data element sizes (byte, word, double word, quadword) of the source and destination. TMM1 is a field for the destination matrix (tile) operand identifier. TMM2 is a field for a source matrix (tile) operand identifier. In some embodiments, the TMM2 field is a R/M value (such as 8246), the TMM1 field is REG 8244, and the data element size is found in 8265.

[0446] In embodiments, encodings of the instruction include a scale-index-base (SIB) type memory addressing operand that indirectly identifies multiple indexed destination locations in memory (e.g., field 8250). In one embodiment, an SIB type memory operand may include an encoding identifying a base address register. The contents of the base address register may represent a base address in memory from which the addresses of the particular

destination locations in memory are calculated. For example, the base address may be the address of the first location in a block of potential destination locations for an extended vector instruction. In one embodiment, an SIB type memory operand may include an encoding identifying an index register. Each element of the index register may specify an index or offset value usable to compute, from the base address, an address of a respective destination location within a block of potential destination locations. In one embodiment, an SIB type memory operand may include an encoding specifying a scaling factor to be applied to each index value when computing a respective destination address. For example, if a scaling factor value of four is encoded in the SIB type memory operand, each index value obtained from an element of the index register may be multiplied by four and then added to the base address to compute a destination address.

[0447] In one embodiment, an SIB type memory operand of the form vm32{x,y,z} may identify a vector array of memory operands specified using SIB type memory addressing. In this example, the array of memory addresses is specified using a common base register, a constant scaling factor, and a vector index register containing individual elements, each of which is a 32-bit index value. The vector index register may be a 128-bit register (e.g., XMM) register (vm32x), a 256-bit (e.g., YMM) register (vm32y), or a 512-bit (e.g., ZMM) register (vm32z). In another embodiment, an SIB type memory operand of the form vm64{x,y,z} may identify a vector array of memory operands specified using SIB type memory addressing. In this example, the array of memory addresses is specified using a common base register, a constant scaling factor, and a vector index register containing individual elements, each of which is a 64-bit index value. The vector index register may be a 128-bit register (e.g., XMM) register (vm64x), a 256-bit (e.g., YMM) register (vm64y) or a 512-bit (e.g., ZMM) register (vm64z).

[0448] *iii. Exemplary Method(s) of Execution*

[0449] Figure 37 illustrates an embodiment of method performed by a processor to process a TILETRANSPOSE instruction.

[0450] At 3701, an instruction is fetched. For example, a TILETRANSPOSE instruction is fetched. The TILETRANSPOSE instruction includes fields for an opcode, a source matrix (tile) operand identifier, and a destination matrix (tile) operand identifier. In some embodiments, the instruction is fetched from an instruction cache. The opcode of the TILETRANSPOSE instruction indicates a transposition of the data from the identified source matrix (tile) operand to corresponding packed data element positions of the identified destination matrix (tile) operand is to occur, and a size of the data to be transposed.

[0451] The fetched instruction is decoded at 3703. For example, the fetched TILETRANSPOSE instruction is decoded by decode circuitry such as that detailed herein.

[0452] Data values associated with the source matrix (tile) operand of the decoded instruction are retrieved at 3705 and the decoded instruction is scheduled (as needed). For example, when the source matrix (tile) operand is a memory location, the data from the indicated memory location is retrieved.

[0453] At 3707, the decoded instruction is executed by execution circuitry (hardware) such as that detailed herein. For the TILETRANSPOSE instruction, the execution will cause execution circuitry to transpose the source data of the source matrix (tile) operand into the destination matrix (tile) operand. In some embodiments, unconfigured elements of rows of the destination matrix (tile) are zeroed. In some embodiments, instead of a write, the identified source matrix (tile) operand is renamed to be the identified destination matrix (tile) operand. This eliminates the need to do a transposition and instead uses a logical renaming.

[0454] In some embodiments, the instruction is committed or retired at 3709.

[0455] **Figure 38** illustrates a more detailed description of an execution of a TILETRANSPOSE instruction. Typically, this is performed by execution circuitry such as that detailed above.

[0456] At 3802, a determination of whether ALL of the following is true is made: 1) Does the number of columns in the source matrix (tile) equal the number of rows in the destination matrix (tile); 2) Does the number of rows in the source matrix (tile) equal the number of columns of the destination matrix (tile); and 3) Do the source and destination matrix (tile) operands have data elements of the same size? When any of these is not true, then a fault is generated at 3804.

[0457] When all of these conditions are true, then the execution circuitry at 3806 loops over each row M of the destination matrix (tile), starting with the first row. For each row, the execution circuitry executes an inner loop at 3808, looping over each column N of the destination tile, starting with the first column. For each of the elements of the inner loop, the execution circuitry determines at 3810 whether the destination tile element contains 2 bytes. If so, the execution circuitry at 3812 sets the word at destination[M][N] to the value of the word at source[N][M]. But, when the execution circuit at 3810 determines that the destination tile element does not contain 2 bytes, the execution circuit at 3814 determines whether the destination tile element contains 4 bytes. If so, the execution circuitry at 3816 sets the doubleword at destination[M][N] to the value of the doubleword at source[N][M].

As shown, when the execution circuit at 3814 determines that the destination tile element does not contain 4 bytes, a fault is generated at 3818.

[0458] After setting an element of the destination tile at either one of 3812 and 3816, the execution circuit at 3820 determines whether any columns remain in the loop, and, if so, processing of the inner loop returns to 3808. But when the determination at 3820 indicates that no rows remain, the execution circuitry at 3822 determines whether any rows remain in the loop, and, if so, processing returns to the outer loop at 3806. But, when the determination at 3822 indicates that no rows remain, the process ends.

[0459] *iv. Exemplary Pseudocode*

[0460] **Figure 39** is exemplary pseudocode describing an embodiment of a method performed by a processor to process a TILETRANSPOSED instruction. As shown in pseudocode 3902, the TILETRANSPOSED instruction includes an opcode, a source operand TSRC to identify a source matrix (tile), and a destination operand TDEST to identify a destination matrix (tile). As shown, the pseudocode 3902 first causes the execution circuitry to generate a fault if any of three error checks fails. Then the pseudocode causes the processor to loop over each row j and each column k of the destination tile. At each element, the processor sets the double word at destination tile[j][k] to the value of the double word at source tile[k][j]. The TILETRANSPOSED opcode includes a "D" suffix, indicating that the elements of the destination matrix (tile) are each the size of a doubleword. Pseudocode 3904 operates similarly to pseudocode 3902, but has a "W" suffix, indicating that its destination matrix (tile) elements are each two bytes in size.

[0461] *v. Examples*

[0462] **Example 1** A processor comprising: decode circuitry to decode an instruction having fields for an opcode, a source matrix operand identifier, and a destination matrix operand identifier; and execution circuitry to execute the decoded instruction to transpose each row of elements of the identified source matrix operand into a corresponding column of the identified destination matrix operand.

[0463] **Example 2** The processor of example 1, wherein the opcode defines a size of each data element of the source and destination matrix operands.

[0464] **Example 3** The processor of example 2, wherein the size of each data element of the source and destination matrix operands is a doubleword.

[0465] **Example 4** The processor of example 2, wherein the size of each data element of the source and destination matrix operands is a word.

[0466] **Example 5** The processor of any of examples 1-4, wherein the execution circuitry is to transpose each row of the identified source matrix operand to a corresponding column of the identified destination matrix operand and zero any remaining columns of the identified destination matrix operand and unconfigured rows of the identified destination matrix operand.

[0467] **Example 6** The processor of any of examples 1-5, wherein the source matrix operand is a plurality of registers to represent a matrix.

[0468] **Example 7** The processor of any of examples 1-5, wherein the execution circuitry is to fault upon a determination of one of: the identified source operand has a different number of rows than a number of columns of the identified destination operand, the identified source operand has a different number of columns than a number of rows in the identified destination operand, and the identified source and destination matrix operands have different sized data elements.

[0469] **Example 8** A method comprising: decoding an instruction having fields for an opcode, a source matrix operand identifier, and a destination matrix operand identifier; and executing the decoded instruction to transpose data elements of the identified source matrix operand into transposed data element positions of the identified destination matrix operand.

[0470] **Example 9** The method of example 8, wherein the opcode defines a size of each data element of the source and destination matrix operands.

[0471] **Example 10** The method of example 9, wherein the size of each data element of the source and destination matrix operands is a doubleword.

[0472] **Example 11** The method of example 9, wherein the size of each data element of the source and destination matrix operands is a word.

[0473] **Example 12** The method of any of examples 8-11, wherein each row of elements of the identified source matrix operand is transposed into a corresponding column of the identified destination matrix operand, the method further comprising zeroing any remaining columns of the identified destination matrix operand and unconfigured rows of the identified destination matrix operand.

[0474] **Example 13** The method of any of examples 8-12, wherein the source matrix operand is a plurality of registers to represent a matrix.

[0475] **Example 14** The method of any of examples 8-13, further comprising: faulting upon a determination of one of: the identified source operand has a different number of rows than a number of columns in the identified destination operand, the identified source operand has a different number of columns than a number of rows in the identified destination

operand, and the identified source and destination matrix operands have different sized data elements.

[0476] **Example 15** A non-transitory machine-readable medium storing an instruction which causes a processor to perform a method, the method comprising: decoding an instruction having fields for an opcode, a source matrix operand identifier, and a destination matrix operand identifier; and executing the decoded instruction to transpose data elements of the identified source matrix operand into transposed data element positions of the identified destination matrix operand.

[0477] **Example 16** The non-transitory machine-readable medium of example 15, wherein the opcode defines a size of each data element of the source and destination matrix operands.

[0478] **Example 17** The non-transitory machine-readable medium of example 16, wherein the size of each data element of the source and destination matrix operands is a doubleword.

[0479] **Example 18** The non-transitory machine-readable medium of example 16, wherein the size of each data element of the source and destination matrix operands is a word.

[0480] **Example 19** The non-transitory machine-readable medium of any of examples 15-18, wherein each row of elements of the identified source matrix operand is transposed into a corresponding column of the identified destination matrix operand, the method further comprising zeroing any remaining columns of the identified destination matrix operand and unconfigured rows of the identified destination matrix operand.

[0481] **Example 20** The non-transitory machine-readable medium of any of examples 15-19, wherein the source matrix operand is a plurality of registers to represent a matrix.

[0482] **Example 21** The non-transitory machine-readable medium of any of examples 15-20, further comprising: faulting upon a determination of one of: the identified source operand has a different number of rows than a number of columns of the identified destination operand, the identified source operand has a different number of columns than a number of rows in the identified destination operand, and the identified source and destination matrix operands have different sized data elements.

[0483] **Example 22** A system comprising: a processor; and an accelerator coupled to the processor, the accelerator including: decode circuitry to decode an instruction having fields for an opcode, a source matrix operand identifier, and a destination matrix operand identifier; and execution circuitry to execute the decoded instruction to transpose each row of elements of the identified source matrix operand into a corresponding column of the identified destination matrix operand.

[0484] **Example 23** The system of example 22, wherein the opcode defines a size of each data element of the source and destination matrix operands.

[0485] **Example 24** The system of any of examples 22-23, wherein the execution circuitry is to transpose each row of the identified source matrix operand to a corresponding column of the identified destination matrix operand and zero any remaining columns of the identified destination matrix operand and unconfigured rows of the identified destination matrix operand.

[0486] **Example 25** The system of any of examples 22-24, wherein the source matrix operand is a plurality of registers to represent a matrix.

[0487] *F. Tile Move*

[0488] *i. Exemplary Execution*

[0489] Detailed herein are embodiments of a matrix (tile) move ("TILEMOVE") instruction and its execution. A TILEMOVE instruction is an improvement a computer itself as it provides for support to move data from one matrix (tile) to another matrix (tile) with a single instruction. In particular, the execution of the TILEMOVE instruction causes all data from a source matrix (tile) to be stored into a corresponding data element position of a destination matrix (tile). The size of the data values to be stored varies depending on the instruction and tile support. Exemplary sizes included, but are not limited to, 16-bit, 32-bit, 64-bit, 128-bit, 256-bit, etc. In some embodiments, elements of rows of the destination matrix (tile) that do not have corresponding columns in the source matrix (tile) are zeroed.

[0490] Figure 40 illustrates an exemplary execution of a TILEMOVE instruction. The TILEMOVE instruction format includes fields for an opcode, a source matrix (tile) operand identifier (shown as "SOURCE MATRIX (TILE)"), and a destination matrix (tile) operand identifier (shown as "DESTINATION MATRIX (TILE)").

[0491] The source and destination matrix (tile) operand fields represent a source matrix (tile) 4001 and a destination matrix (tile) 4005. As detailed earlier, a matrix (tile) may be stored in a collection of registers, locations in memory (e.g., as strided rows), or in other storage accessible to execution circuitry.

[0492] As shown, execution circuity 4003 executes a decoded TILEMOVE instruction to move the source data of the source matrix (tile) operand 4001 into corresponding data element positions of the destination matrix (tile) operand 4005.

[0493] Also shown are remaining (unconfigured) columns and rows being set to zero which is done in some embodiments. In some embodiments, a matrix (tile) is configured to use only a subset of the rows and columns possible. For example, a matrix (tile) may have up

to 16 rows and columns to use, but only use 4 of each. The configuration of each matrix (tile) is typically done by the execution of a configuration instruction prior to matrix (tile) usage. In this example, there are N columns and M rows possible.

[0494] *ii. Exemplary Format(s)*

[0495] An embodiment of a format for a TILEMOVE instruction is TILEMOVE{B/W/D/Q} TMM1, TMM2. In some embodiments, TILEMOVE{B/W/D/Q} is the opcode mnemonic of the instruction where B/W/D/Q represent data element sizes (byte, word, double word, quadword) of the source and destination. TMM1 is a field for the destination matrix (tile) operand identifier. TMM2 is a field for a source matrix (tile) operand identifier. In some embodiments, the TMM2 field is a R/M value (such as 2846), the TMM1 field is REG 8244, and the data element size is found in 82865.

[0496] In embodiments, encodings of the instruction include a scale-index-base (SIB) type memory addressing operand that indirectly identifies multiple indexed destination locations in memory (e.g., field 8250). In one embodiment, an SIB type memory operand may include an encoding identifying a base address register. The contents of the base address register may represent a base address in memory from which the addresses of the particular destination locations in memory are calculated. For example, the base address may be the address of the first location in a block of potential destination locations for an extended vector instruction. In one embodiment, an SIB type memory operand may include an encoding identifying an index register. Each element of the index register may specify an index or offset value usable to compute, from the base address, an address of a respective destination location within a block of potential destination locations. In one embodiment, an SIB type memory operand may include an encoding specifying a scaling factor to be applied to each index value when computing a respective destination address. For example, if a scaling factor value of four is encoded in the SIB type memory operand, each index value obtained from an element of the index register may be multiplied by four and then added to the base address to compute a destination address.

[0497] In one embodiment, an SIB type memory operand of the form vm32{x,y,z} may identify a vector array of memory operands specified using SIB type memory addressing. In this example, the array of memory addresses is specified using a common base register, a constant scaling factor, and a vector index register containing individual elements, each of which is a 32-bit index value. The vector index register may be a 128-bit register (e.g., XMM) register (vm32x), a 416-bit (e.g., YMM) register (vm32y), or a 512-bit (e.g., ZMM) register (vm32z). In another embodiment, an SIB type memory operand of the form

vm64{x,y,z} may identify a vector array of memory operands specified using SIB type memory addressing. In this example, the array of memory addresses is specified using a common base register, a constant scaling factor, and a vector index register containing individual elements, each of which is a 64-bit index value. The vector index register may be a 128-bit register (e.g., XMM) register (vm64x), a 416-bit (e.g., YMM) register (vm64y) or a 512-bit (e.g., ZMM) register (vm64z).

[0498] *iii. Exemplary Method(s) of Execution*

[0499] Figure 41 illustrates an embodiment of method performed by a processor to process a TILEMOVE instruction.

[0500] At 4101, an instruction is fetched. For example, a TILEMOVE instruction is fetched. The TILEMOVE instruction includes fields for an opcode, a source matrix (tile) operand identifier, and a destination matrix (tile) operand identifier. In some embodiments, the instruction is fetched from an instruction cache. The opcode of the TILEMOVE instruction indicates a move of the data from the identified source matrix (tile) operand to corresponding packed data element positions of the identified destination matrix (tile) operand is to occur, and a size of the data to be moved.

[0501] The fetched instruction is decoded at 4103. For example, the fetched TILEMOVE instruction is decoded by decode circuitry such as that detailed herein.

[0502] Data values associated with the source matrix (tile) operand of the decoded instruction are retrieved at 4105 and the decoded instruction is scheduled (as needed). For example, when the source matrix (tile) operand is a memory location, the data from the indicated memory location is retrieved.

[0503] At 4107, the decoded instruction is executed by execution circuitry (hardware) such as that detailed herein. For the TILEMOVE instruction, the execution will cause execution circuitry to move (write) every configured element position of the identified destination matrix (tile) operand with data from corresponding element positions of the identified source matrix (tile) operand. In some embodiments, unconfigured elements of rows of the destination matrix (tile) are zeroed. In some embodiments, instead of a write, the identified source matrix (tile) operand is renamed to be the identified destination matrix (tile) operand. This eliminates the need to do a move and instead uses a logical renaming.

[0504] In some embodiments, the instruction is committed or retired at 4109.

[0505] Figure 42 illustrates a more detailed description of an execution of a TILEMOVE instruction. Typically, this is performed by execution circuitry such as that detailed above.

[0506] At 4201, a determination of if one of the following is true is made: 1) Do the identified source and destination matrix (tile) operands have the same number of rows?; 2) Do the identified source and destination matrix (tile) operands have the same number of columns?; and 3) Do the identified source and destination matrix (tile) operands have data elements of the same size? If any of these is not true, then a fault is raised at 4203.

[0507] If all of these conditions are true, then the execution circuitry moves (writes), for each configured row of the identified destination matrix (tile) operand, data values from a corresponding row of the identified source matrix (tile) operand in corresponding data element positions (columns) at 4205. In some embodiments, unconfigured elements of rows of the destination matrix (tile) that do not have corresponding columns in the source matrix (tile) are zeroed.

[0508] *iv. Exemplary Pseudocode*

[0509] FIG. 43 illustrates exemplary pseudocode for the execution of a TILEMOVE instruction.

[0510] v. Examples

[0511] **Example 1** A processor comprising: decode circuitry to decode an instruction having fields for an opcode, a source matrix operand identifier, and a destination matrix operand identifier; and execution circuitry to execute the decoded instruction to move each data element of the identified source matrix operand to corresponding data element position of the identified destination matrix operand.

[0512] **Example 2** The processor of example 1, wherein the opcode defines a size of each data element of the source and destination matrix operands.

[0513] **Example 3** The processor of example 2, wherein the size of each data element of the source and destination matrix operands is a doubleword.

[0514] **Example 4** The processor of example 2, wherein the size of each data element of the source and destination matrix operands is a word.

[0515] **Example 5** The processor of any of examples 1-4, wherein the execution circuitry is to move each row of the identified source matrix operand to a corresponding row of the destination matrix operand and zero any remaining columns of the identified destination matrix operand and unconfigured rows of the identified destination matrix operand.

[0516] **Example 6** The processor of any of examples 1-5, wherein the source matrix operand is a plurality of registers to represent a matrix.

[0517] **Example 7** The processor of any of examples 1-5, wherein the execution circuitry to fault upon a determination of one of: the identified source and destination matrix operands

have different number of rows, the identified source and destination matrix operands have different number of columns, and the identified source and destination matrix operands have different sized data elements.

[0518] **Example 8** A method comprising: decoding an instruction having fields for an opcode, a source matrix operand identifier, and a destination matrix operand identifier; and executing the decoded instruction to move each data element of the identified source matrix operand to corresponding data element position of the identified destination matrix operand.

[0519] **Example 9** The method of example 8, wherein the opcode defines a size of each data element of the source and destination matrix operands.

[0520] **Example 10** The method of example 9, wherein the size of each data element of the source and destination matrix operands is a doubleword.

[0521] **Example 11** The method of example 9, wherein the size of each data element of the source and destination matrix operands is a word.

[0522] **Example 12** The method of any of examples 8-11, wherein each row of the identified source matrix operand is moved to a corresponding row of the destination matrix operand and zero any remaining columns of the identified destination matrix operand and unconfigured rows of the identified destination matrix operand.

[0523] **Example 13**The method of any of examples 8-12, wherein the source matrix operand is a plurality of registers to represent a matrix.

[0524] **Example 14** The method of any of examples 8-13, further comprising: faulting upon a determination of one of: the identified source and destination matrix operands have different number of rows, the identified source and destination matrix operands have different number of columns, and the identified source and destination matrix operands have different sized data elements.

[0525] **Example 15** A non-transitory machine-readable medium storing an instruction which causes a processor to perform a method, the method comprising: decoding an instruction having fields for an opcode, a source matrix operand identifier, and a destination matrix operand identifier; and executing the decoded instruction to move each data element of the identified source matrix operand to corresponding data element position of the identified destination matrix operand.

[0526] **Example 16** The non-transitory machine-readable medium of example 15, wherein the opcode defines a size of each data element of the source and destination matrix operands.

[0527] **Example 17** The non-transitory machine-readable medium of examples 16, wherein the size of each data element of the source and destination matrix operands is a doubleword.

[0528] **Example 18** The non-transitory machine-readable medium of example 16, wherein the size of each data element of the source and destination matrix operands is a word.

[0529] **Example 19** The non-transitory machine-readable medium of any of examples 15-18, wherein each row of the identified source matrix operand is moved to a corresponding row of the destination matrix operand and zero any remaining columns of the identified destination matrix operand and unconfigured rows of the identified destination matrix operand.

[0530] **Example 20** The non-transitory machine-readable medium of any of examples 15-19, wherein the source matrix operand is a plurality of registers to represent a matrix.

[0531] **Example 21** The non-transitory machine-readable medium of any of examples 15-20, further comprising: faulting upon a determination of one of: the identified source and destination matrix operands have different number of rows, the identified source and destination matrix operands have different number of columns, and the identified source and destination matrix operands have different sized data elements.

[0532] **Example 22** A system comprising: a processor; and an accelerator coupled to the processor, the accelerator including: decode circuitry to decode an instruction having fields for an opcode, a source matrix operand identifier, and a destination matrix operand identifier; and execution circuitry to execute the decoded instruction to move each data element of the identified source matrix operand to corresponding data element position of the identified destination matrix operand.

[0533] **Example 23** The system of example 22, wherein the opcode defines a size of each data element of the source and destination matrix operands.

[0534] **Example 40** The system of any of examples 22-23, wherein the execution circuitry is to move each row of the identified source matrix operand to a corresponding row of the destination matrix operand and zero any remaining columns of the identified destination matrix operand and unconfigured rows of the identified destination matrix operand.

[0535] **Example 40** The system of any of examples 22-40, wherein the source matrix operand is a plurality of registers to represent a matrix.

[0536] *G. Tile Broadcast*

[0537] Detailed herein are embodiments of a matrix (tile) broadcast ("TILEBROADCAST") instruction and its execution. A TILEBROADCAST instruction is

an improvement a computer itself as it provides for support to broadcasting a data value into a matrix (tile) with a single instruction. In particular, an execution of the TILEBROADCAST instruction causes a data value from a source (e.g., a memory location or a register) to be stored into each configured data element position of a destination matrix (tile). This type of operation is referred to as a "broadcast." The size of the data value to be stored varies depending on the instruction and tile support. Exemplary sizes included, but are not limited to, 16-bit, 32-bit, 64-bit, 128-bit, 256-bit, etc.

[0538] *i. Exemplary Execution*

[0539] Figure 44 illustrates an exemplary execution of a TILEBROADCAST instruction. The TILEBROADCAST instruction format includes fields for an opcode, a source operand identifier (shown as "SOURCE"), and a destination matrix (tile) operand identifier (shown as "DESTINATION MATRIX (TILE)").

[0540] The source operand field represents a location of a source operand 4401 storing a source data value to be broadcast. This location may be a memory location (e.g., an address in memory such as a disk or RAM) or a register.

[0541] The destination matrix (tile) operand field represents a destination matrix (tile) 4407 to store the data from the data of the source operand. As detailed earlier, a matrix (tile) may be stored in a collection of registers, locations in memory (e.g., as strided rows), or within other storage accessible to execution circuitry.

[0542] As shown, execution circuity 4403 uses broadcast circuitry 4405 to execute a decoded TILEBROADCAST instruction to store the source data of the source operand 4401 into each data element position of the matrix (tile) destination operand 4407. In some embodiments, the broadcast circuitry 4405 is a crossbar switch. In this example, a value "A" is stored in each data element position of the matrix (tile) destination operand 4407.

[0543] Also shown are remaining (unconfigured) columns and rows being set to zero which is done in some embodiments. In some embodiments, a matrix (tile) is configured to use only a subset of the rows and columns possible. For example, a matrix (tile) may have up to 16 rows and columns to use, but only use 4 of each. The configuration of each matrix (tile) is typically done by the execution of a configuration instruction prior to matrix (tile) usage. In this example, there are N columns and M rows possible.

[0544] *ii. Exemplary Format(s)*

[0545] An embodiment of a format for a TILEBROADCAST instruction is TILEBROADCAST TMM1, m32. In some embodiments, TILEBROADCAST{B/W/D/Q} is the opcode mnemonic of the instruction where B/W/D/Q represent data element sizes

(byte, word, double word, quadword) of the destination. TMM1 is a field for a destination matrix (tile) operand identifier. m32 is a field for a source operand identifier such as a register and/or memory. In some embodiments, the m32 field is a R/M value (such as 8246), the destination matrix (tile) field is REG 8244, and the data element size is found in 8265.

[0546] In embodiments, encodings of the instruction include a scale-index-base (SIB) type memory addressing operand that indirectly identifies multiple indexed destination locations in memory (e.g., field 8250). In one embodiment, an SIB type memory operand may include an encoding identifying a base address register. The contents of the base address register may represent a base address in memory from which the addresses of the particular destination locations in memory are calculated. For example, the base address may be the address of the first location in a block of potential destination locations for an extended vector instruction. In one embodiment, an SIB type memory operand may include an encoding identifying an index register. Each element of the index register may specify an index or offset value usable to compute, from the base address, an address of a respective destination location within a block of potential destination locations. In one embodiment, an SIB type memory operand may include an encoding specifying a scaling factor to be applied to each index value when computing a respective destination address. For example, if a scaling factor value of four is encoded in the SIB type memory operand, each index value obtained from an element of the index register may be multiplied by four and then added to the base address to compute a destination address.

[0547] In one embodiment, an SIB type memory operand of the form vm32{x,y,z} may identify a vector array of memory operands specified using SIB type memory addressing. In this example, the array of memory addresses is specified using a common base register, a constant scaling factor, and a vector index register containing individual elements, each of which is a 32-bit index value. The vector index register may be a 128-bit register (e.g., XMM) register (vm32x), a 256-bit (e.g., YMM) register (vm32y), or a 512-bit (e.g., ZMM) register (vm32z). In another embodiment, an SIB type memory operand of the form vm64{x,y,z} may identify a vector array of memory operands specified using SIB type memory addressing. In this example, the array of memory addresses is specified using a common base register, a constant scaling factor, and a vector index register containing individual elements, each of which is a 64-bit index value. The vector index register may be a 128-bit register (e.g., XMM) register (vm64x), a 256-bit (e.g., YMM) register (vm64y) or a 512-bit (e.g., ZMM) register (vm64z).

[0548] *iii. Exemplary Method(s) of Execution*

[0549] Figure 45 illustrates an embodiment of method performed by a processor to process a TILEBROADCAST instruction.

[0550] At 4501, an instruction is fetched. For example, a TILEBROADCAST instruction is fetched. The TILEBROADCAST instruction includes fields for an opcode, a source operand, identifier, and a destination matrix (tile) operand identifier. In some embodiments, the instruction is fetched from an instruction cache. The identified destination operand consists of matrix of packed data. The opcode of the TILEBROADCAST instruction indicates a broadcast of the data from the identified source operand to the configured packed data element positions of the identified destination matrix (tile) operand is to occur. In some embodiments, unconfigured data element positions are set to zero.

[0551] The fetched instruction is decoded at 4503. For example, the fetched TILEBROADCAST instruction is decoded by decode circuitry such as that detailed herein.

[0552] Data values associated with the identified source operand of the decoded instruction are retrieved at 4505 and the decoded instruction is scheduled (as needed). For example, when the identified source operand is a memory operand, the data from the indicated memory location is retrieved.

[0553] At 4507, the decoded instruction is executed by execution circuitry (hardware) such as that detailed herein. For the TILEBROADCAST instruction, the execution will cause execution circuitry to write every configured element position of the identified destination matrix (tile) operand with data from the identified source operand. In some embodiments, unconfigured packed data element positions of the identified destination matrix (tile) operand are zeroed. For example, if the destination matrix (tile) is configured to only use 4 rows and 4 columns, but the identified destination matrix (tile) operand could store 16 rows and 16 columns, then data element positions from the fifth row and up are set to zero and all columns from the fifth and up are set to zero. A configuration for a matrix (tile) may be stored in one or more registers.

[0554] In some embodiments, the instruction is committed or retired at 4509.

[0555] Figure 46 illustrates a more detailed description of an execution of a TILEBROADCAST instruction. Typically, this is performed by execution circuitry such as that detailed above.

[0556] At 4601, for each configured row of the identified matrix (tile) destination, the same data value from the identified source operand is written into configured data element positions (columns) of the row and unconfigured columns of the row are zeroed.

[0557] At 4603, after the zeroing of these data elements, the execution circuitry (e.g., broadcast circuitry described above) writes, for each row of the destination matrix (tile) operand, the data value from the identified source operand into each data element position (column) of the row of the identified destination matrix (tile) operand.

[0558] *iv. Exemplary Pseudocode*

[0559] FIG. 47 illustrates examples of pseudocode of methods for executing a TILEBROADCAST instruction.

[0560] *v. Examples*

[0561] **Example 1** A processor comprising: decode circuitry to decode an instruction having fields for an opcode, an identifier for a source operand, and an identifier for a destination matrix operand; and execution circuitry to execute the decoded instruction to broadcast a single data element from the identified source operand into each configured data element of the identified destination matrix operand.

[0562] **Example 1** The processor of example 1, wherein the execution circuitry comprises a crossbar switch.

[0563] **Example 2** The processor of any of examples 1-2, wherein the source operand is stored in memory.

[0564] **Example 3** The processor of any of examples 1-2, wherein the source operand is stored in a packed data register.

[0565] **Example 4** The processor of any of examples 1-2, wherein the source operand is stored in a general-purpose data register.

[0566] **Example 5** The processor of any of examples 1-5, wherein the identified destination matrix operand is a plurality of registers to represent a two-dimensional matrix.

[0567] **Example 6** The processor of any of examples 1-6, wherein the data elements of the row of data are doubleword in size.

[0568] **Example 7** The processor of any of examples 1-6, wherein the data elements of the row of data are word in size.

[0569] **Example 8** The processor of any of examples 1-8, further comprising storage to store a configuration of the identified destination matrix operand.

[0570] **Example 9** The processor of any of examples 1-9, wherein the configuration is to indicate a number of rows and columns to be used by the identified destination matrix operand.

[0571] **Example 10** The processor of any of examples 1-10, wherein the execution circuitry is to further zero unconfigured data elements of the identified destination matrix operand.

[0572] **Example 11** A method comprising: decoding an instruction having fields for an opcode, an identifier for a source operand, and an identifier for a destination matrix operand; and executing the decoded instruction to broadcast a single data element from the identified source operand into each configured data element of the identified destination matrix operand.

[0573] **Example 12** The method of example 12, wherein the source operand is stored in memory.

[0574] **Example 13** The method of example 12, wherein the source operand is stored in a register.

[0575] **Example 14** The method of any of examples 12-14, wherein the identified destination matrix operand is a plurality of registers to represent a two-dimensional matrix.

[0576] **Example 15** The method of any of examples 12-15, wherein the data elements of the row of data are doubleword in size.

[0577] **Example 16** The method of any of examples 12-15, wherein the data elements of the row of data are word in size.

[0578] **Example 17** The method of any of examples 12-17, further comprising: zeroing unconfigured data elements of the identified destination matrix operand.

[0579] **Example 18** The method of any of examples 12-18, further comprising: translating the instruction from a first instruction set to a second instruction set prior to decoding.

[0580] **Example 19** A non-transitory machine-readable medium storing an instruction which causes a processor to perform a method, the method comprising: decoding the instruction having fields for an opcode, an identifier for a source operand, and an identifier for a destination matrix operand; and executing the decoded instruction to broadcast a single data element from the identified source operand into each configured data element of the identified destination matrix operand.

[0581] **Example 20** The non-transitory machine-readable medium of example 19, wherein the source operand is stored in memory.

[0582] **Example 21** The non-transitory machine-readable medium of example 19, wherein the source operand is stored in a register.

[0583] **Example 22** The non-transitory machine-readable medium of any of examples 19-21, wherein the identified destination matrix operand is a plurality of registers to represent a two-dimensional matrix.

[0584] **Example 23** The non-transitory machine-readable medium of any of examples 19-22, further comprising: zeroing unconfigured data elements of the identified destination matrix operand.

[0585] **Example 24** The non-transitory machine-readable medium of any of examples 19-23, further comprising: translating the instruction from a first instruction set to a second instruction set prior to decoding.

[0586] **Example 25** A system comprising: a processor; an accelerator coupled to the processor, the accelerator including: decode circuitry to decode an instruction having fields for an opcode, an identifier for a source operand, and an identifier for a destination matrix operand; and execution circuitry to execute the decoded instruction to broadcast a single data element from the identified source operand into each configured data element of the identified destination matrix operand.

[0587] *H. Tile Row Broadcast*

[0588] Detailed herein are embodiments of a tile row broadcast ("TILEROWBROADCAST") instruction and its execution. A TILEROWBROADCAST instruction is an improvement a computer itself as it provides for support to broadcasting a data value into a matrix (tile) with a single instruction. In particular, an execution of the TILEROWBROADCAST instruction causes a read of a row of data from memory and a write of the read row to every row of the destination matrix (tile) identified by the instruction. This type of operation is referred to as a "broadcast." The size of the data value to be stored varies depending on the instruction and tile support. Exemplary sizes included, but are not limited to, 16-bit, 32-bit, 64-bit, 128-bit, 256-bit, etc.

[0589] *i. Exemplary Execution*

[0590] Figure 48 illustrates an exemplary execution of a TILEROWBROADCAST instruction. The TILEROWBROADCAST instruction format includes fields for an opcode, a memory source operand identifier (shown as "MEMSOURCE"), and a destination matrix (tile) operand identifier (shown as "DESTINATION MATRIX (TILE)").

[0591] The source operand field represents a location of a source operand 4801 storing a source data value to be broadcast. This location is a memory location (e.g., an address in memory such as a disk or RAM).

[0592] The destination matrix (tile) operand field represents a destination matrix (tile) 4807 to store the data from the data of the source operand. As detailed earlier, a matrix (tile) may be stored in a collection of registers, locations in memory (e.g., as strided rows), or within other storage accessible to execution circuitry.

[0593] As shown, execution circuity 4803 uses broadcast circuitry 4805 to execute a decoded TILEROWBROADCAST instruction to store a row source data of memory (source operand 4801) into each row of the matrix (tile) destination operand 4807. In some embodiments, the broadcast circuitry 4805 is a crossbar switch. In this example, the values "A," "B," "C," and "D" of a "row" in memory are stored as a row in corresponding data element positions of each row of the matrix (tile) destination operand 4807. Also shown is remaining columns being set to zero which is done in some embodiments.

[0594] Also shown is remaining (unconfigured) columns and rows being set to zero which is done in some embodiments. In some embodiments, a matrix (tile) is configured to use only a subset of the rows and columns possible. For example, a matrix (tile) may have up to 16 rows and columns to use, but only use 4 of each. The configuration of each matrix (tile) is typically done by the execution of a configuration instruction prior to matrix (tile) usage. In this example, there are N columns and M rows possible.

[0595] *ii. Exemplary Format(s)*

[0596] An embodiment of a format for a TILEROWBROADCAST instruction is TILEROWBROADCAST DST, SRC. In some embodiments, TILEROWBROADCAST{B/W/D/Q} is the opcode mnemonic of the instruction where B/W/D/Q represent data element sizes (byte, word, double word, quadword) of the destination. DST is a field for a destination matrix (tile) operand identifier. SRC is a field for a source operand identifier such as a memory location. In some embodiments, the SRC field is a R/M value (such as 8246), the destination matrix (tile) field is REG 8244, and the data element size is found in 8265.

[0597] In embodiments, encodings of the instruction include a scale-index-base (SIB) type memory addressing operand that indirectly identifies multiple indexed destination locations in memory (e.g., field 8250). In one embodiment, an SIB type memory operand may include an encoding identifying a base address register. The contents of the base address register may represent a base address in memory from which the addresses of the particular destination locations in memory are calculated. For example, the base address may be the address of the first location in a block of potential destination locations for an extended vector instruction. In one embodiment, an SIB type memory operand may include an

encoding identifying an index register. Each element of the index register may specify an index or offset value usable to compute, from the base address, an address of a respective destination location within a block of potential destination locations. In one embodiment, an SIB type memory operand may include an encoding specifying a scaling factor to be applied to each index value when computing a respective destination address. For example, if a scaling factor value of four is encoded in the SIB type memory operand, each index value obtained from an element of the index register may be multiplied by four and then added to the base address to compute a destination address.

[0598] In one embodiment, an SIB type memory operand of the form vm32{x,y,z} may identify a vector array of memory operands specified using SIB type memory addressing. In this example, the array of memory addresses is specified using a common base register, a constant scaling factor, and a vector index register containing individual elements, each of which is a 32-bit index value. The vector index register may be a 128-bit register (e.g., XMM) register (vm32x), a 256-bit (e.g., YMM) register (vm32y), or a 512-bit (e.g., ZMM) register (vm32z). In another embodiment, an SIB type memory operand of the form vm64{x,y,z} may identify a vector array of memory operands specified using SIB type memory addressing. In this example, the array of memory addresses is specified using a common base register, a constant scaling factor, and a vector index register containing individual elements, each of which is a 64-bit index value. The vector index register may be a 128-bit register (e.g., XMM) register (vm64x), a 256-bit (e.g., YMM) register (vm64y) or a 512-bit (e.g., ZMM) register (vm64z).

[0599] *iii. Exemplary Method(s) of Execution*

[0600] Figure 49 illustrates an embodiment of method performed by a processor to process a TILEROWBROADCAST instruction.

[0601] At 4901, an instruction is fetched. For example, a TILEROWBROADCAST instruction is fetched. The TILEROWBROADCAST instruction includes fields for an opcode, a source operand, identifier, and a destination matrix (tile) operand identifier. In some embodiments, the instruction is fetched from an instruction cache. The identified destination operand consists of matrix of packed data. The opcode of the TILEROWBROADCAST instruction indicates a broadcast of the row data from the identified source operand to each configured row (in packed data element positions) of the identified destination matrix (tile) operand is to occur. In some embodiments, unconfigured data element positions are set to zero.

[0602] The fetched instruction is decoded at 4903. For example, the fetched TILEROWBROADCAST instruction is decoded by decode circuitry such as that detailed herein.

[0603] Data values associated with the identified source operand of the decoded instruction are retrieved at 4905 and the decoded instruction is scheduled (as needed). For example, when the identified source operand is a memory operand, the data from the indicated memory location is retrieved.

[0604] At 4907, the decoded instruction is executed by execution circuitry (hardware) such as that detailed herein. For the TILEROWBROADCAST instruction, the execution will cause execution circuitry to write each configured row of the identified destination matrix (tile) operand with the same row of data from the identified source operand. In some embodiments, unconfigured packed data element positions of the identified destination matrix (tile) operand are zeroed. For example, if the destination matrix (tile) is configured to only use 4 rows and 4 columns, but the identified destination matrix (tile) operand could store 16 rows and 16 columns, then data element positions from the fifth row and up are set to zero and all columns from the fifth and up are set to zero. A configuration for a matrix (tile) may be stored in one or more registers.

[0605] In some embodiments, the instruction is committed or retired at 4909.

[0606] Figure 50 illustrates a more detailed description of an execution of a TILEROWBROADCAST instruction. Typically, this is performed by execution circuitry such as that detailed above.

[0607] At 5001, each read element from the identified source is copied as its own whole temporary row.

[0608] At 5003, each temporary row is written into consecutive configured rows of the identified destination matrix (tile) operand and unconfigured columns of these rows are zeroed.

[0609] At 5005, unconfigured rows of the identified destination matrix (tile) operand are zeroed.

[0610] *iv. Exemplary Pseudocode*

[0611] FIG. 51 illustrates examples of pseudocode of methods for executing a TILECOLBROADCAST instruction.

[0612] *v. Examples*

[0613] **Example 1** A processor comprising: decode circuitry to decode an instruction having fields for an opcode, an identifier for a source operand, and an identifier for a

destination matrix operand; and execution circuitry to execute the decoded instruction to broadcast a row of data from the identified source operand into each configured row of the identified destination matrix operand.

[0614] **Example 2** The processor of example 1, wherein the execution circuitry comprises a crossbar switch.

[0615] **Example 3** The processor of any of examples 1-2, wherein the source operand is stored in memory.

[0616] **Example 4** The processor of any of examples 1-2, wherein the source operand is stored in a packed data register.

[0617] **Example 5** The processor of any of examples 1-4, wherein the execution circuitry is further to zero unconfigured columns of the identified destination matrix operand that were written to by the row of data.

[0618] **Example 6** The processor of any of examples 1-5, wherein the identified destination matrix operand is a plurality of registers to represent a two-dimensional matrix.

[0619] **Example 7** The processor of any of examples 1-6, wherein the data elements of the row of data are doubleword in size.

[0620] **Example 8** The processor of any of examples 1-7, wherein the data elements of the row of data are word in size.

[0621] **Example 9** A method comprising: decoding an instruction having fields for an opcode, an identifier for a source operand, and an identifier for a destination matrix operand; and executing the decoded instruction to broadcast a row of data from the identified source operand into each configured row of the identified destination matrix operand.

[0622] **Example 10** The method of example 9, wherein the source operand is stored in memory.

[0623] **Example 11** The method of example 9, wherein the source operand is stored in a packed data register.

[0624] **Example 12** The method of any of examples 9-11, further comprising: zeroing unconfigured columns of the identified destination matrix operand that were written to by the row of data.

[0625] **Example 13** The method of any of examples 9-12, wherein the identified destination matrix operand is a plurality of registers to represent a two-dimensional matrix.

[0626] **Example 14** The method of any of examples 9-13, wherein the data elements of the row of data are doubleword in size.

[0627] **Example 15** The method of any of examples 9-13, wherein the data elements of the row of data are word in size.

[0628] **Example 16** A non-transitory machine-readable medium storing an instruction which causes a processor to perform a method, the method comprising: decoding an instruction having fields for an opcode, an identifier for a source operand, and an identifier for a destination matrix operand; and executing the decoded instruction to broadcast a row of data from the identified source operand into each configured row of the identified destination matrix operand.

[0629] **Example 17** The non-transitory machine-readable medium of example 16, wherein the source operand is stored in memory.

[0630] **Example 18** The non-transitory machine-readable medium of example 16, wherein the source operand is stored in a packed data register.

[0631] **Example 19** The non-transitory machine-readable medium of any of examples 16-18, further comprising: zeroing unconfigured columns of the identified destination matrix operand that were written to by the row of data.

[0632] **Example 20** The non-transitory machine-readable medium of any of examples 16-19, wherein the identified destination matrix operand is a plurality of registers to represent a two-dimensional matrix.

[0633] **Example 21** The non-transitory machine-readable medium of any of examples 16-20, wherein the data elements of the row of data are doubleword in size.

[0634] **Example 22** The non-transitory machine-readable medium of any of examples 16-20, wherein the data elements of the row of data are word in size.

[0635] **Example 23** A system comprising: a processor; an accelerator coupled to the processor, the accelerator including: decode circuitry to decode an instruction having fields for an opcode, an identifier for a source operand, and an identifier for a destination matrix operand; and execution circuitry to execute the decoded instruction to broadcast a row of data from the identified source operand into each configured row of the identified destination matrix operand.

[0636] **Example 24** The system of example 23, wherein the execution circuitry is further to zero columns of the identified destination matrix operand that were written to by the row of data.

[0637] **Example 25** The system of any of example 23-24, wherein the identified destination matrix operand is a plurality of registers to represent a two-dimensional matrix.

[0638] *I. Tile Column Broadcast*

[0639] Detailed herein are embodiments of a tile column broadcast ("TILECOLBROADCAST") instruction and its execution. A TILECOLBROADCAST instruction is an improvement a computer itself as it provides for support to broadcasting a data value into a matrix (tile) with a single instruction. In particular, an execution of the TILECOLBROADCAST instruction causes a read of a column of data from memory and a write to every configured column of the destination matrix (tile) identified by the instruction. This type of operation is referred to as a "broadcast." The size of the data value to be stored varies depending on the instruction and tile support. Exemplary sizes included, but are not limited to, 16-bit, 32-bit, 64-bit, 128-bit, 256-bit, etc.

[0640] *i. Exemplary Execution*

[0641] Figure 52 illustrates an exemplary execution of a TILECOLBROADCAST instruction. The TILECOLBROADCAST instruction format includes fields for an opcode, a memory source operand identifier (shown as "MEMSOURCE"), and a destination matrix (tile) operand identifier (shown as "DESTINATION MATRIX (TILE)").

[0642] The source operand field represents a location of a source operand 5201 storing a source data value to be broadcast. This location is a memory location (e.g., an address in memory such as a disk or RAM).

[0643] The destination matrix (tile) operand field represents a destination matrix (tile) 5207 to store the data from the data of the source operand. As detailed earlier, a matrix (tile) may be stored in a collection of registers, locations in memory (e.g., as strided rows), or within other storage accessible to execution circuitry.

[0644] As shown, execution circuity 5203 uses broadcast circuitry 5205 to execute a decoded TILECOLBROADCAST instruction to store a column source data of memory (source operand 5201) into each column of the matrix (tile) destination operand 5207. In some embodiments, the broadcast circuitry 5205 is a crossbar switch. In this example, the values "A," "B," "C," and "D" of a "column" in memory are stored as a column in corresponding data element positions of each column of the matrix (tile) destination operand 5207.

[0645] Also shown are remaining (unconfigured) columns and rows being set to zero which is done in some embodiments. In some embodiments, a matrix (tile) is configured to use only a subset of the rows and columns possible. For example, a matrix (tile) may have up to 16 rows and columns to use, but only use 4 of each. The configuration of each matrix (tile) is typically done by the execution of a configuration instruction prior to matrix (tile) usage. In this example, there are N columns and M rows possible.

[0646] *ii. Exemplary Format(s)*

[0647] An embodiment of a format for a TILECOLBROADCAST instruction is TILECOLBROADCAST DST, SRC. In some embodiments, TILECOLBROADCAST{B/W/D/Q} is the opcode mnemonic of the instruction where B/W/D/Q represent data element sizes (byte, word, double word, quadword) of the destination. DST is a field for a destination matrix (tile) operand identifier. SRC is a field for a source operand identifier such as a memory location. In some embodiments, the SRC field is a R/M value (such as 8246), the destination matrix (tile) field is REG 8244, and the data element size is found in 8265.

[0648] In embodiments, encodings of the instruction include a scale-index-base (SIB) type memory addressing operand that indirectly identifies multiple indexed destination locations in memory (e.g., field 8250). In one embodiment, an SIB type memory operand may include an encoding identifying a base address register. The contents of the base address register may represent a base address in memory from which the addresses of the particular destination locations in memory are calculated. For example, the base address may be the address of the first location in a block of potential destination locations for an extended vector instruction. In one embodiment, an SIB type memory operand may include an encoding identifying an index register. Each element of the index register may specify an index or offset value usable to compute, from the base address, an address of a respective destination location within a block of potential destination locations. In one embodiment, an SIB type memory operand may include an encoding specifying a scaling factor to be applied to each index value when computing a respective destination address. For example, if a scaling factor value of four is encoded in the SIB type memory operand, each index value obtained from an element of the index register may be multiplied by four and then added to the base address to compute a destination address.

[0649] In one embodiment, an SIB type memory operand of the form vm32{x,y,z} may identify a vector array of memory operands specified using SIB type memory addressing. In this example, the array of memory addresses is specified using a common base register, a constant scaling factor, and a vector index register containing individual elements, each of which is a 32-bit index value. The vector index register may be a 128-bit register (e.g., XMM) register (vm32x), a 256-bit (e.g., YMM) register (vm32y), or a 512-bit (e.g., ZMM) register (vm32z). In another embodiment, an SIB type memory operand of the form vm64{x,y,z} may identify a vector array of memory operands specified using SIB type memory addressing. In this example, the array of memory addresses is specified using a

common base register, a constant scaling factor, and a vector index register containing individual elements, each of which is a 64-bit index value. The vector index register may be a 128-bit register (e.g., XMM) register (vm64x), a 256-bit (e.g., YMM) register (vm64y) or a 512-bit (e.g., ZMM) register (vm64z).

[0650] *iii. Exemplary Method(s) of Execution*

[0651] Figure 53 illustrates an embodiment of method performed by a processor to process a TILECOLBROADCAST instruction.

[0652] At 5301, an instruction is fetched. For example, a TILECOLBROADCAST instruction is fetched. The TILECOLBROADCAST instruction includes fields for an opcode, a source operand, identifier, and a destination matrix (tile) operand identifier. In some embodiments, the instruction is fetched from an instruction cache. The identified destination operand consists of matrix of packed data. The opcode of the TILECOLBROADCAST instruction indicates a broadcast of the column data from the identified source operand to each configured column (in packed data element positions) of the identified destination matrix (tile) operand is to occur. In some embodiments, unconfigured data element positions are set to zero.

[0653] The fetched instruction is decoded at 5303. For example, the fetched TILECOLBROADCAST instruction is decoded by decode circuitry such as that detailed herein.

[0654] Data values associated with the identified source operand of the decoded instruction are retrieved at 5305 and the decoded instruction is scheduled (as needed). For example, when the identified source operand is a memory operand, the data from the indicated memory location is retrieved.

[0655] At 5307, the decoded instruction is executed by execution circuitry (hardware) such as that detailed herein. For the TILECOLBROADCAST instruction, the execution will cause execution circuitry to write each configured column of the identified destination matrix (tile) operand with the same column of data from the identified source operand. In some embodiments, columns and rows of the destination matrix (tile) that were not configured are zeroed.

[0656] In some embodiments, the instruction is committed or retired at 5309.

[0657] Figure 54 illustrates a more detailed description of an execution of a TILECOLBROADCAST instruction. Typically, this is performed by execution circuitry such as that detailed above.

[0658] At 5401, each read data element from the identified source operand is copied into a temporary row. For example, rows of "A," "B," "C," and "D" are created.

[0659] At 5403, the execution circuitry (e.g., broadcast circuitry described above) writes each temporary row to the configured rows of the identified destination matrix (tile) operand and zeros unconfigured columns of the configured rows.

[0660] At 5405, the execution circuitry zeros unconfigured rows of the identified destination matrix (tile) operand.

[0661] *iv. Exemplary Pseudocode*

[0662] FIG. 55 illustrates examples of pseudocode of methods for executing a TILECOLBROADCAST instruction.

[0663] *v. Examples*

[0664] **Example 1** A processor comprising: decode circuitry to decode an instruction having fields for an opcode, an identifier for a source operand, and an identifier for a destination matrix operand; and execution circuitry to execute the decoded instruction to broadcast a column of data from the identified source operand into each configured column of the identified destination matrix operand.

[0665] **Example 2** The processor of example 1, wherein the execution circuitry comprises a crossbar switch.

[0666] **Example 3** The processor of any of examples 1-2, wherein the source operand is stored in memory.

[0667] **Example 4** The processor of examples 1-2, wherein the source operand is stored in a packed data register.

[0668] **Example 5** The processor of any of examples 1-2, wherein the execution circuitry is further to zero unconfigured columns of the identified destination matrix operand that were written to by the row of data.

[0669] **Example 6** The processor of any of examples 1-5, wherein the identified destination matrix operand is a plurality of registers to represent a two-dimensional matrix.

[0670] **Example 7** The processor of any of examples 1-6, wherein the execution circuitry is to create a temporary row of each data element of the identified source operand and store each temporary row as a column of the identified destination matrix operand.

[0671] **Example 8** The processor of any of examples 1-7, wherein the data elements of the row of data are doubleword in size.

[0672] **Example 9** The processor of any of examples 1-7, wherein the data elements of the row of data are word in size.

[0673] **Example 10** A method comprising: decoding an instruction having fields for an opcode, an identifier for a source operand, and an identifier for a destination matrix operand; and executing the decoded instruction to broadcast a column of data from the identified source operand into each configured column of the identified destination matrix operand.

[0674] **Example 11** The method of example 10, wherein the source operand is stored in memory.

[0675] **Example 12** The method of example 10, wherein the source operand is stored in a packed data register.

[0676] **Example 13** The method of any of examples 10-12, further comprising zeroing unconfigured columns of the identified destination matrix operand that were written to by the row of data.

[0677] **Example 14** The method of any of examples 10-13, wherein the identified destination matrix operand is a plurality of registers to represent a two-dimensional matrix..

[0678] **Example 15** The method of any of examples 10-14, further comprising creating a temporary row of each data element of the identified source operand and store each temporary row as a column of the identified destination matrix operand.

[0679] **Example 16** The method of any of examples 10-15, wherein the data elements of the row of data are doubleword in size.

[0680] **Example 17** The method of any of examples 10-15, wherein the data elements of the row of data are word in size.

[0681] **Example 18** A non-transitory machine-readable medium storing an instruction which causes a processor to perform a method, the method comprising: decoding an instruction having fields for an opcode, an identifier for a source operand, and an identifier for a destination matrix operand; and executing the decoded instruction to broadcast a column of data from the identified source operand into each configured column of the identified destination matrix operand.

[0682] **Example 19** The non-transitory machine-readable medium of example 18, wherein the source operand is stored in memory.

[0683] **Example 20** The non-transitory machine-readable medium of example 18, wherein the source operand is stored in a packed data register.

[0684] **Example 21** The non-transitory machine-readable medium of any of examples 18-20, further comprising zeroing unconfigured columns of the identified destination matrix operand that were written to by the row of data.

[0685] **Example 22** The non-transitory machine-readable medium of any of examples 18-21, wherein the identified destination matrix operand is a plurality of registers to represent a two-dimensional matrix..

[0686] **Example 23** The non-transitory machine-readable medium of any of examples 18-22, further comprising creating a temporary row of each data element of the identified source operand and store each temporary row as a column of the identified destination matrix operand.

[0687] **Example 24** The non-transitory machine-readable medium of any of examples 18-23, wherein the data elements of the row of data are doubleword in size.

[0688] **Example 25** A system comprising: a processor; an accelerator coupled to the processor, the accelerator including: decode circuitry to decode an instruction having fields for an opcode, an identifier for a source operand, and an identifier for a destination matrix operand; and execution circuitry to execute the decoded instruction to broadcast a column of data from the identified source operand into each configured column of the identified destination matrix operand.

[0689] *J. Tile Zero*

[0690] There are times when a matrix of all zeros is desirable for a given operation. Detailed herein are embodiments of a matrix (tile) zeroing ("TILEZERO") instruction and its execution to generate that matrix. A TILEZERO instruction is an improvement a computer itself as it provides for support to zero data of a matrix (tile). In particular, the execution of the TILEZERO instruction causes all data from a source matrix (tile) to be set to zero.

[0691] *i. Exemplary Execution*

[0692] FIG. 56 illustrates an exemplary execution of a TILEZERO instruction. The TILEZERO instruction format includes fields for an opcode and a source/destination matrix (tile) operand (shown as "SOURCE/DESTINATION MATRIX (TILE)").

[0693] The source/destination matrix (tile) operand field represents a source matrix (tile) 5601 and an updated version of the source matrix (tile) as the destination 5607. As detailed earlier, a matrix may be stored in a collection of registers (tile), locations in memory, or in other storage accessible to execution circuitry.

[0694] As shown, execution circuity 5603 executes a decoded TILEZERO instruction to zero the source data of the source matrix (tile) operand 5601 and store that data in the source matrix (tile) operand as a destination of the instruction 5607.

[0695] *ii. Exemplary Format(s)*

[0696] An embodiment of a format for a TILEZERO instruction is TILEZERO TMM1. In some embodiments, TILEZERO is the opcode mnemonic of the instruction. SRC is a field for a source matrix (tile) operand. In some embodiments, the SRC/DST field is a R/M value (such as 2746) and in others the field is REG 2744.

[0697] In embodiments, encodings of the instruction include a scale-index-base (SIB) type memory addressing operand that indirectly identifies multiple indexed destination locations in memory (e.g., field 2750). In one embodiment, an SIB type memory operand may include an encoding identifying a base address register. The contents of the base address register may represent a base address in memory from which the addresses of the particular destination locations in memory are calculated. For example, the base address may be the address of the first location in a block of potential destination locations for an extended vector instruction. In one embodiment, an SIB type memory operand may include an encoding identifying an index register. Each element of the index register may specify an index or offset value usable to compute, from the base address, an address of a respective destination location within a block of potential destination locations. In one embodiment, an SIB type memory operand may include an encoding specifying a scaling factor to be applied to each index value when computing a respective destination address. For example, if a scaling factor value of four is encoded in the SIB type memory operand, each index value obtained from an element of the index register may be multiplied by four and then added to the base address to compute a destination address.

[0698] In one embodiment, an SIB type memory operand of the form vm32{x,y,z} may identify a vector array of memory operands specified using SIB type memory addressing. In this example, the array of memory addresses is specified using a common base register, a constant scaling factor, and a vector index register containing individual elements, each of which is a 32-bit index value. The vector index register may be a 128-bit register (e.g., XMM) register (vm32x), a 576-bit (e.g., YMM) register (vm32y), or a 512-bit (e.g., ZMM) register (vm32z). In another embodiment, an SIB type memory operand of the form vm64{x,y,z} may identify a vector array of memory operands specified using SIB type memory addressing. In this example, the array of memory addresses is specified using a common base register, a constant scaling factor, and a vector index register containing individual elements, each of which is a 64-bit index value. The vector index register may be a 128-bit register (e.g., XMM) register (vm64x), a 576-bit (e.g., YMM) register (vm64y) or a 512-bit (e.g., ZMM) register (vm64z).

[0699] *iii. Exemplary Method(s) of Execution*

[0700] FIG. 57 illustrates an embodiment of method performed by a processor to process a TILEZERO instruction.

[0701] At 5701, an instruction is fetched. For example, a TILEZERO instruction is fetched. The TILEZERO instruction includes fields for an opcode and a source/destination matrix (tile) operand. In some embodiments, the instruction is fetched from an instruction cache. The opcode of the TILEZERO instruction indicates a zeroing of the data from the source/destination matrix (tile) operand is to occur.

[0702] The fetched instruction is decoded at 5703. For example, the fetched TILEZERO instruction is decoded by decode circuitry such as that detailed herein.

[0703] Data values associated with the source/destination matrix (tile) operand of the decoded instruction are retrieved at 5705 and the decoded instruction is scheduled (as needed).

[0704] At 5707, the decoded instruction is executed by execution circuitry (hardware) such as that detailed herein. For the TILEZERO instruction, the execution will cause execution circuitry to write every element position of the source/destination matrix (tile) operand with a zero. In some embodiments, each row of the source/destination matrix (tile) operand is set to zero at a time. In some embodiments, an actual writing of zeroes does not occur, rather, a status bit for the matrix (tile) is set to indicate that all of the data is zero.

[0705] In some embodiments, the instruction is committed or retired at 5709.

[0706] *iv. Exemplary Pseudocode*

[0707] FIG. 58 depicts pseudocode of a method of execution of a TILEZERO instruction.

[0708] *v. Examples*

[0709] **EXAMPLE 1** A processor comprising: decode circuitry to decode an instruction having fields for an opcode and a source/destination matrix operand identifier; and execution circuitry to execute the decoded instruction to zero each data element of the identified source/destination matrix.

[0710] **EXAMPLE 2** The processor of example 1, wherein the execution circuitry is to zero each row of the source/destination matrix operand.

[0711] **EXAMPLE 3** The processor of any of examples 1-2, wherein the source/destination matrix operand is a plurality of registers configured to represent a matrix.

[0712] **EXAMPLE 4** The processor of any of examples 1-2, wherein the execution circuitry to fault upon a determination the source/destination matrix operand identifier is not configured as a matrix.

[0713] **EXAMPLE 5** The processor of any of examples 1 and 3-5, wherein the execution circuitry is to set a status bit regarding the identified source/destination matrix as being zero.

[0714] **EXAMPLE 6** A method comprising: decoding an instruction having fields for an opcode and a source/destination matrix operand identifier; and executing the decoded instruction to zero each data element of the identified source/destination matrix.

[0715] **EXAMPLE 7** The method of example 6, wherein the zeroing is done a row of the source/destination matrix operand at a time.

[0716] **EXAMPLE 8** The method of any of examples 6-7, wherein the source/destination matrix operand is a plurality of registers configured to represent a matrix.

[0717] **EXAMPLE 9** The method of any of examples 6-8, further comprising faulting upon a determination the source/destination matrix operand identifier is not configured as a matrix.

[0718] **EXAMPLE 10** The method of any of examples 6 and 8-9, further comprising setting a status bit regarding the identified source/destination matrix as being zero.

[0719] **EXAMPLE 11** A non-transitory machine-readable medium storing an instruction which causes a processor to perform a method, the method comprising: decoding an instruction having fields for an opcode and a source/destination matrix operand identifier; and executing the decoded instruction to zero each data element of the identified source/destination matrix.

[0720] **EXAMPLE 12** The non-transitory machine-readable medium of example 11, wherein the zeroing is done a row of the source/destination matrix operand at a time.

[0721] **EXAMPLE 13** The non-transitory machine-readable medium of any of examples 11-12, wherein the source/destination matrix operand is a plurality of registers configured to represent a matrix.

[0722] **EXAMPLE 14** The non-transitory machine-readable medium of any of examples 11-13, further comprising faulting upon a determination the source/destination matrix operand identifier is not configured as a matrix.

[0723] **EXAMPLE 15** The non-transitory machine-readable medium of any of examples 11 and 13-14, further comprising setting a status bit regarding the identified source/destination matrix as being zero.

[0724] **EXAMPLE 16** A system comprising: a processor; and an accelerator coupled to the processor, the accelerator including: decode circuitry to decode an instruction having fields for an opcode and a source/destination matrix operand identifier; and execution

circuitry to execute the decoded instruction to zero each data element of the identified source/destination matrix.

[0725] **EXAMPLE 17** The system of example 16, wherein the execution circuitry is to zero each row of the source/destination matrix operand.

[0726] **EXAMPLE 18** The system of any of examples 16-17, wherein the source/destination matrix operand is a plurality of registers configured to represent a matrix.

[0727] **EXAMPLE 19** The system of any of examples 16-18, wherein the execution circuitry to fault upon a determination the source/destination matrix operand identifier is not configured as a matrix.

[0728] *K. Tile Add*

[0729] Detailed herein are embodiments of a matrix (tile) addition ("TILEADD") instruction and its execution. A TILEADD instruction is an improvement to a computer itself as it provides for support to add matrices (tiles) of data values with a single instruction. In particular, the execution of the TILEADD instruction causes elementwise addition of elements of a first source matrix (tile) with corresponding elements of a second source matrix (tile) and storage of the result in corresponding data element positions of a destination matrix (tile). The size of the data values in the source and destination matrices varies depending on the instruction and tile support. Exemplary sizes include, but are not limited to, 16-bit, 32-bit, 64-bit, 128-bit, and 256-bit. In some embodiments, elements of rows of the destination matrix (tile) that do not have corresponding elements in the source matrices (tiles) are zeroed.

[0730] *i. Exemplary Execution*

[0731] Figure 59 illustrates an exemplary execution of a TILEADD instruction. The TILEADD instruction format includes fields for an opcode (e.g., shown as "TADDP{H,S}" in the figure), a destination operand (e.g., shown as "DESTINATION MATRIX (TILE)" in the figure) and two source operands (e.g., shown as "FIRST SOURCE MATRIX (TILE)" and "SECOND SOURCE MATRIX (TILE)" in the figure).

[0732] The source and destination matrix (tiles) operand fields represent a first source matrix (tile) 5901, a second source matrix (tile) 5903, and a destination matrix (tile) 5907. As detailed earlier, a matrix (tile) may be stored in a collection of registers, locations in memory (for example, as strided rows), or in other storage accessible to execution circuitry.

[0733] As shown, execution circuitry 5905 executes a decoded TADD instruction to perform elementwise addition of the elements of the first source matrix (tile) 5901 and the second source matrix (tile) 5903 and stores the result in corresponding data element positions of the destination matrix (tile) 5907. In some embodiments, a matrix (tile) is configured to

use only a subset of the rows and columns possible. For example, a matrix (tile) may have up to 16 rows and columns to use, but only use 4 of each. The configuration of each matrix (tile) is typically done by the execution of a configuration instruction prior to matrix (tile) usage. In this example, there are N columns and M rows possible. Each of the first source matrix (tile) 5901 and the second source matrix (tile) 5903 uses 3 rows and 3 columns. Although the example of Figure 59 illustrates an example of adding two 3 x 3 matrices, in general, a TILEADD instruction can operate on any two matrices (tiles) having the same dimensions (that is, source matrices (tiles) having a same number of columns N and a same number of rows M).

[0734] In some embodiments, execution circuitry 5905 uses a grid of fused multiply adders (FMAs) to execute a decoded TILEADD instruction by storing the result of adding the two source matrix (tile) operands into corresponding data element positions of the destination matrix (tile) 5907. In particular, the grid of FMAs generates, for each data element position[row, column] of a first source matrix (tile) 5901, a sum of the value at that data element position and the value at a corresponding data element position[row, column] of the second source matrix (tile) 5903. In reference to the example source matrix (tile) operands 5901, 5903, the execution circuitry 5905 generates a sum of the values first source matrix (tile) 5901 position[0,0] and second source matrix (tile) 5903 position[0,0] (A+J) and stores the result in the position[0,0] of the destination matrix (tile) 5907, generates a sum of the values first source matrix (tile) 5901 position[0,1] and second source matrix (tile) 5903 position[0,1] (B+K) and stores the result in the position[0,1] of the destination matrix (tile) 5907, and so forth.

[0735] *ii. Exemplary Format(s)*

[0736] An embodiment of a format for a TILEADD instruction is TADDP{H/S} TMM1, TMM2, TMM3. In some embodiments, TADDP{H/S} is the opcode mnemonic of the instruction, where the S or H identifier indicates whether the source matrices (tile) comprise single-precision (PS) or half-precision (PH) floating-point data values. TMM1 is a field for the destination matrix (tile) operand. TMM2 and TMM3 are fields for the matrix (tile) source operands. In some embodiments, the TMM3 field is a R/M value (such as 8246), the TMM1 field is REG 8244, and the data element size is found in 8265.

[0737] In embodiments, encodings of the instruction include a scale-index-base (SIB) type memory addressing operand that indirectly identifies multiple indexed destination locations in memory. In one embodiment, an SIB type memory operand may include an encoding identifying a base address register. The contents of the base address register may represent a

base address in memory from which the addresses of the particular destination locations in memory are calculated. For example, the base address may be the address of the first location in a block of potential destination locations for an extended vector instruction. In one embodiment, an SIB type memory operand may include an encoding identifying an index register. Each element of the index register may specify an index or offset value usable to compute, from the base address, an address of a respective destination location within a block of potential destination locations. In one embodiment, an SIB type memory operand may include an encoding specifying a scaling factor to be applied to each index value when computing a respective destination address. For example, if a scaling factor value of four is encoded in the SIB type memory operand, each index value obtained from an element of the index register may be multiplied by four and then added to the base address to compute a destination address.

[0738] In one embodiment, an SIB type memory operand of the form vm32{x,y,z} may identify a vector array of memory operands specified using SIB type memory addressing. In this example, the array of memory addresses is specified using a common base register, a constant scaling factor, and a vector index register containing individual elements, each of which is a 32-bit index value. The vector index register may be a 128-bit register (e.g., XMM) register (vm32x), a 256-bit (e.g., YMM) register (vm32y), or a 512-bit (e.g., ZMM) register (vm32z). In another embodiment, an SIB type memory operand of the form vm64{x,y,z} may identify a vector array of memory operands specified using SIB type memory addressing. In this example, the array of memory addresses is specified using a common base register, a constant scaling factor, and a vector index register containing individual elements, each of which is a 64-bit index value. The vector index register may be a 128-bit register (e.g., XMM) register (vm64x), a 256-bit (e.g., YMM) register (vm64y) or a 512-bit (e.g., ZMM) register (vm64z).

[0739] *iii. Exemplary Method(s) of Execution*

[0740] Figure 60 illustrates an embodiment of method performed by a processor to process a TILEADD instruction.

[0741] At 6001, an instruction is fetched. For example, a TILEADD instruction is fetched. The TILEADD instruction includes fields for an opcode, a first and a second source matrix (tile) operand identifier, and a destination matrix (tile) operand identifier. In some embodiments, the instruction is fetched from an instruction cache. The source operands and destination operand consist of packed data. The opcode of the TILEADD instruction indicates that a sum of the source operands is to be generated. In an embodiment, the opcode

further indicates whether the source operands consist of half-precision floating-point values or single-precision floating-point values.

[0742] The fetched instruction is decoded at 6003. For example, the fetched TILEADD instruction is decoded by decode circuitry such as that detailed herein.

[0743] Data values associated with the source matrix (tile) operands of the decoded instruction are retrieved at 6005 and the decoded instruction is schedule (as needed). For example, when one or more of the source matrix (tile) operands are memory operands, the data from the indicated memory location is retrieved.

[0744] At 6007, the decoded instruction is executed by execution circuitry (hardware) such as that detailed herein. For the TILEADD instruction, the execution will cause execution circuitry to perform an elementwise matrix addition operation on the source data. In some embodiments, the execution of a decoded matrix addition operation instruction causes an execution circuit to, for each data element position of the first source matrix operand: add a first data value at that data element position to a second data value at a corresponding data element position of the second source matrix operand, and store a result of the addition into a corresponding data element position of the destination matrix operand.

[0745] In some embodiments, a fault is generated when a number of data element rows associated with the first source matrix (tile) operand is different than a number of data element rows associated with the second source matrix (tile) operand or is different than a number of data element rows associated with the destination matrix (tile) operand. Similarly, a fault is generated when a number of data element columns associated with the first source matrix (tile) operand is different than a number of data element columns associated with the second source matrix (tile) operand or is different than a number of data element columns associated with the destination matrix (tile) operand. As described elsewhere herein, the dimensions for each matrix, element size for the data elements each matrix, and other configuration can be set by executing a TILECONFIG instruction.

[0746] As described elsewhere herein, successful execution of a TILECONFIG instruction enables subsequent TILE operators and sets a state variable indicating that the corresponding code is in a region with TILES configured. In some embodiments, a fault is generated as part of executing a TILE ADD instruction if the TILES mode is determined to be inactive. For example, the execution circuitry can check whether the state variable set as part of the successful execution of a TILECONFIG instruction indicates that a TILES mode is active.

[0747] In some embodiments, the instruction is committed or retired at 6009.

[0748] Figure 61 illustrates an example process describing a method performed by a processor to process a TILEADD instruction. For example, process 6101 illustrates an example method for performing a TILEADD operation when the source matrix (tile) operands contain half-precision elements. Process 6201 illustrates an example method for performing a TILEADD operation when the source matrix (tile) operands contain single-precision elements.

[0749] As shown, the process of 6101 determines whether any of the following is true: 1) is a TILES mode not active?; 2) do the destination matrix (tile) operand and the first source matrix (tile) operand have a different number of columns?; 3) do the destination matrix (tile) operand and the second source matrix (tile) operand have a different number of columns?; 4) do the destination matrix (tile) operand and the first source matrix (tile) operand have a different number of row?; 5) do the destination matrix (tile) operand and the second source matrix (tile) operand have a different number of rows?; 6) does the destination matrix (tile) operand have more than a specified maximum number of columns?; 7) does the first source matrix (tile) operand have more than a specified maximum number of columns?; 8) does the second source matrix (tile) operand have more than a specified maximum number of columns? If any of these is true, then a fault is raised.

[0750] If none of the conditions above is true, then the execution circuitry writes, for each configured row and column of the identified destination matrix (tile) operand, the sum of corresponding element values from the first source matrix (tile) operand and the second source matrix (tile) operand into a corresponding data element position of the destination matrix (tile) operand. In some embodiments, unconfigured elements of rows of the destination matrix (tile) that do not have corresponding columns in the source matrix (tile) are zeroed.

[0751] *v. Examples*

[0752] **Example 1** A processor comprising: decode circuitry to decode an instruction having fields for an opcode, a first source matrix operand identifier, a second source matrix operand identifier, and a destination matrix operand identifier; and execution circuitry to execute the decoded instruction to, for each data element position of the identified first source matrix operand: add a first data value at that data element position to a second data value at a corresponding data element position of the identified second source matrix operand, and store a result of the addition into a corresponding data element position of the identified destination matrix operand.

[0753] **Example 2** The processor of Example 1, wherein the first source matrix operand is a packed data register and the second source matrix operand is a memory location.

[0754] **Example 3** The processor of Example 1, wherein the first source matrix operand is a packed data register and the second source matrix operand is a packed data register.

[0755] **Example 4** The processor of any of Examples 1-3, wherein the execution circuitry comprises a plurality of fused-multiply adders.

[0756] **Example 5** The processor of Example 1, wherein the opcode indicates that each of the first source matrix operand, the second source matrix operand, and the destination matrix operand comprises half-precision floating-point values.

[0757] **Example 6** The processor of Example 1, wherein the opcode indicates that each of the first source matrix operand, the second source matrix operand, and the destination matrix operand comprises single-precision floating-point values.

[0758] **Example 7** The processor of any of Examples 1-6, wherein a fault is generated when the first source matrix operand has a different number of data elements than the second source matrix operand.

[0759] **Example 8** The processor of any of Examples 1-7, wherein a fault is generated when a number of rows associated with the first source matrix operand is different than a number of rows associated with the second source matrix operand.

[0760] **Example 9** The processor of any of Examples 1-8, wherein a fault is generated when a number of columns associated with the first source matrix operand is different than a number of columns associated with the second source matrix operand.

[0761] **Example 10** The processor of any of Examples 1-9, wherein the execution circuitry further checks a state variable indicating whether a matrix operations mode is active, and wherein a fault is generated when it is determined that the matrix operations mode is not active.

[0762] **Example 11** provides a method comprising: decoding an instruction having fields for an opcode, a first source matrix operand identifier, a second source matrix operand identifier, and a destination matrix operand identifier; and executing the decoded instruction to, for each data element position of the identified first source matrix operand: add a first data value at that data element position to a second data value at a corresponding data element position of the identified second source matrix operand, and store a result of the addition into a corresponding data element position of the identified destination matrix operand.

[0763] **Example 12** The method of Example 11, wherein the first source matrix operand is a packed data register and the second source matrix operand is a memory location.

[0764] **Example 13** The method of Example 11, wherein the first source matrix operand is a packed data register and the second source matrix operand is a packed data register.

[0765] **Example 14** The method of any of Examples 11-13, wherein the execution circuitry comprises a plurality of fused-multiply adders.

[0766] **Example 15** The method of Example 11, wherein the opcode indicates that each of the first source matrix operand, the second source matrix operand, and the destination matrix operand comprises half-precision floating-point values.

[0767] **Example 16** The method of Example 11, wherein the opcode indicates that each of the first source matrix operand, the second source matrix operand, and the destination matrix operand comprises single-precision floating-point values.

[0768] **Example 17** The method of any of Examples 11-16, wherein a fault is generated when the first source matrix operand has a different number of data elements than the second source matrix operand.

[0769] **Example 18** The method of any of Examples 11-17, wherein a fault is generated when a number of rows associated with the first source matrix operand is different than a number of rows associated with the second source matrix operand.

[0770] **Example 19** The method of Example 11-18, wherein a fault is generated when a number of columns associated with the first source matrix operand is different than a number of columns associated with the second source matrix operand.

[0771] **Example 20** The method of any of Examples 11-19, wherein executing the decoded instruction includes checking a state variable indicating whether a matrix operations mode is active, and wherein a fault is generated when it is determined that the matrix operations mode is not active.

[0772] **Example 21** provides a non-transitory machine-readable medium storing an instruction which when executed by a processor causes the processor to perform a method, the method comprising: decoding an instruction having fields for an opcode, a first source matrix operand identifier, a second source matrix operand identifier, and a destination matrix operand identifier, and executing the decoded instruction to, for each data element position of the identified first source matrix operand: add a first data value at that data element position to a second data value at a corresponding data element position of the identified second source matrix operand, and store a result of the addition into a corresponding data element position of the identified destination matrix operand.

[0773] **Example 22** The non-transitory machine-readable medium of Example 21, wherein the first source matrix operand is a packed data register and the second source matrix operand is a memory location.

[0774] **Example 23** The non-transitory machine-readable medium of Example 21, wherein the first source matrix operand is a packed data register and the second source matrix operand is a packed data register.

[0775] **Example 24** The non-transitory machine-readable medium of Example 21, wherein the execution circuitry comprises a plurality of fused-multiply adders.

[0776] **Example 25** The non-transitory machine-readable medium of Example 21, wherein the opcode indicates that each of the first source matrix operand, the second source matrix operand, and the destination matrix operand comprises half-precision floating-point values.

[0777] **Example 26** The non-transitory machine-readable medium of Example 21, wherein the opcode indicates that each of the first source matrix operand, the second source matrix operand, and the destination matrix operand contains single-precision floating-point values.

[0778] **Example 27** The non-transitory machine-readable medium of any of Examples 21-26, wherein a fault is generated when the first source matrix operand has a different number of data elements than the second source matrix operand.

[0779] **Example 28** The non-transitory machine-readable medium of any of Examples 21-27, wherein a fault is generated when a number of rows associated with the first source matrix operand is different than a number of rows associated with the second source matrix operand.

[0780] **Example 29** The non-transitory machine-readable medium of any of Examples 21-28, wherein a fault is generated when a number of columns associated with the first source matrix operand is different than a number of columns associated with the second source matrix operand.

[0781] **Example 30** The non-transitory machine-readable medium of any of Examples 21-29, wherein executing the decoded instruction includes checking a state variable indicating whether a matrix operations mode is active, and wherein a fault is generated when it determines that the matrix operations mode is not active.

[0782] **Example 31** provides a system comprising: a processor; and an accelerator coupled to the processor, the accelerator including: decode circuitry to decode an instruction having fields for an opcode, a first source matrix operand identifier, a second source matrix

operand identifier, and a destination matrix operand identifier; and execution circuitry to execute the decoded instruction to, for each data element position of the identified first source matrix operand: add a first data value at that data element position to a second data value at a corresponding data element position of the identified second source matrix operand, and store a result of the addition into a corresponding data element position of the identified destination matrix operand.

[0783] **Example 32** The system of Example 31, wherein the first source matrix operand is a packed data register and the second source matrix operand is a memory location.

[0784] **Example 33** The system of Example 31, wherein the first source matrix operand is a packed data register and the second source matrix operand is a packed data register.

[0785] **Example 34** The system of any of Examples 31-33, wherein the execution circuitry comprises a plurality of fused-multiply adders.

[0786] **Example 35** The system of Example 31, wherein the opcode indicates that each of the first source matrix operand, the second source matrix operand, and the destination matrix operand comprises half-precision floating-point values.

[0787] **Example 36** The system of Example 31, wherein the opcode indicates that each of the first source matrix operand, the second source matrix operand, and the destination matrix operand comprises single-precision floating-point values.

[0788] **Example 37** The system of any of Examples 31-36, wherein a fault is generated when the first source matrix operand has a different number of data elements than the second source matrix operand.

[0789] **Example 38** The system of any of Examples 31-37, wherein a fault is generated when a number of rows associated with the first source matrix operand is different than a number of rows associated with the second source matrix operand.

[0790] **Example 39** The system of any of Examples 31-38, wherein a fault is generated when a number of columns associated with the first source matrix operand is different than a number of columns associated with the second source matrix operand.

[0791] **Example 40** The system of any of Examples 31-39, wherein the execution circuitry further checks a state variable indicating whether a matrix operations mode is active, and wherein a fault is generated when it is determined that the matrix operations mode is not active.

[0792] *L. Tile Subtract*

[0793] Detailed herein are embodiments of a matrix (tile) subtraction ("TILESUB") instruction and its execution. A TILESUB instruction is an improvement to a computer itself

as it provides for support to subtract matrices (tiles) of data values with a single instruction. In particular, the execution of the TILESUB instruction causes elementwise subtraction of elements of a second source matrix (tile) from corresponding elements of a first source matrix (tile) and storage of the result in corresponding data element positions of a destination matrix (tile). The size of the data values in the source and destination matrices varies depending on the instruction and tile support. Exemplary sizes include, but are not limited to, 16-bit, 32-bit, 64-bit, 128-bit, and 256-bit. In some embodiments, elements of rows of the destination matrix (tile) that do not have corresponding elements in the source matrices (tiles) are zeroed.

[0794] *i. Exemplary Execution*

[0795] Figure 63 illustrates an exemplary execution of a TILESUB instruction. The TILESUB instruction format includes fields for an opcode (e.g., shown as "TSUBP{H/S}" in the figure), a destination operand (e.g., shown as "DESTINATION MATRIX (TILE)" in the figure) and two source operands (e.g., shown as "FIRST SOURCE MATRIX (TILE)" and "SECOND SOURCE MATRIX (TILE)" in the figure).

[0796] The source and destination matrix (tiles) operand fields represent a first source matrix (tile) 6301, a second source matrix (tile) 6303, and a destination matrix (tile) 6307. As detailed earlier, a matrix (tile) may be stored in a collection of registers, locations in memory (for example, as strided rows), or in other storage accessible to execution circuitry.

[0797] As shown, execution circuitry 6305 executes a decoded TSUB instruction to perform elementwise subtraction of the elements of the first source matrix (tile) 6301 and the second source matrix (tile) 6303 and stores the result in corresponding data element positions of the destination matrix (tile) 6307. In some embodiments, a matrix (tile) is configured to use only a subset of the rows and columns possible. For example, a matrix (tile) may have up to 16 rows and columns to use, but only use 4 of each. The configuration of each matrix (tile) is typically done by the execution of a configuration instruction prior to matrix (tile) usage. In this example, there are N columns and M rows possible. Each of the first source matrix (tile) 6301 and the second source matrix (tile) 6303 uses 3 rows and 3 columns. Although the example of Figure 63 illustrates an example of subtracting two 3 x 3 matrices, in general, a TILESUB instruction can operate on any two matrices (tiles) having the same dimensions (that is, source matrices (tiles) having a same number of columns N and a same number of rows M).

[0798] In some embodiments, execution circuitry 6305 uses a grid of fused multiply adders (FMAs) to execute a decoded TILESUB instruction by storing the result of subtracting the two source matrix (tile) operands into corresponding data element positions of the

destination matrix (tile) 6307. In particular, the grid of FMAs generates, for each data element position[row, column] of a first source matrix (tile) 6301, a result value indicating the difference between the value at that data element position and the value at a corresponding data element position[row, column] of the second source matrix (tile) 6303. In reference to the example source matrix (tile) operands 6301 and 6303, the execution circuitry 6305 subtracts the value of the second source matrix (tile) 6303 at position[0,0] from the value of the first source matrix (tile) 6301 at position[0,0] (A-J) and stores the result in the position[0,0] of the destination matrix (tile) 6307, subtracts the value of the second source matrix (tile) 6303 position[0,1] from the value of the first source matrix (tile) 6303 at position[0,1] (B-K) and stores the result in the position[0,1] of the destination matrix (tile) 6307, and so forth.

[0799] *ii. Exemplary Format(s)*

[0800] An embodiment of a format for a TILESUB instruction is TSUBP{H/S} TMM1, TMM2, TMM3. In some embodiments, TSUBP{H/S} is the opcode mnemonic of the instruction, where the S or H identifier indicates whether the source matrices (tile) comprise single-precision (PS) or half-precision (PH) floating-point data values. TMM1 is a field for the destination matrix (tile) operand. TMM2 and TMM3 are fields for the matrix (tile) source operands. In some embodiments, the TMM3 field is a R/M value (such as 8246), the TMM1 field is REG 8244, and the data element size is found in 8265.

[0801] In embodiments, encodings of the instruction include a scale-index-base (SIB) type memory addressing operand that indirectly identifies multiple indexed destination locations in memory. In one embodiment, an SIB type memory operand may include an encoding identifying a base address register. The contents of the base address register may represent a base address in memory from which the addresses of the particular destination locations in memory are calculated. For example, the base address may be the address of the first location in a block of potential destination locations for an extended vector instruction. In one embodiment, an SIB type memory operand may include an encoding identifying an index register. Each element of the index register may specify an index or offset value usable to compute, from the base address, an address of a respective destination location within a block of potential destination locations. In one embodiment, an SIB type memory operand may include an encoding specifying a scaling factor to be applied to each index value when computing a respective destination address. For example, if a scaling factor value of four is encoded in the SIB type memory operand, each index value obtained from an element of the

index register may be multiplied by four and then added to the base address to compute a destination address.

[0802] In one embodiment, an SIB type memory operand of the form vm32{x,y,z} may identify a vector array of memory operands specified using SIB type memory addressing. In this example, the array of memory addresses is specified using a common base register, a constant scaling factor, and a vector index register containing individual elements, each of which is a 32-bit index value. The vector index register may be a 128-bit register (e.g., XMM) register (vm32x), a 256-bit (e.g., YMM) register (vm32y), or a 512-bit (e.g., ZMM) register (vm32z). In another embodiment, an SIB type memory operand of the form vm64{x,y,z} may identify a vector array of memory operands specified using SIB type memory addressing. In this example, the array of memory addresses is specified using a common base register, a constant scaling factor, and a vector index register containing individual elements, each of which is a 64-bit index value. The vector index register may be a 128-bit register (e.g., XMM) register (vm64x), a 256-bit (e.g., YMM) register (vm64y) or a 512-bit (e.g., ZMM) register (vm64z).

[0803] *iii. Exemplary Method(s) of Execution*

[0804] Figure 64 illustrates an embodiment of method performed by a processor to process a TILESUB instruction.

[0805] At 6401, an instruction is fetched. For example, a TILESUB instruction is fetched. The TILESUB instruction includes fields for an opcode, a first and a second source matrix (tile) operand identifier, and a destination matrix (tile) operand identifier. In some embodiments, the instruction is fetched from an instruction cache. The source operands and destination operand consist of packed data. The opcode of the TILESUB instruction indicates that elementwise subtraction of the source operands is to be generated. In an embodiment, the opcode further indicates whether the source operands consist of half-precision floating-point values or single-precision floating-point values.

[0806] The fetched instruction is decoded at 6403. For example, the fetched TILESUB instruction is decoded by decode circuitry such as that detailed herein.

[0807] Data values associated with the source matrix (tile) operands of the decoded instruction are retrieved at 6405 and the decoded instruction is schedule (as needed). For example, when one or more of the source matrix (tile) operands are memory operands, the data from the indicated memory location is retrieved.

[0808] At 6407, the decoded instruction is executed by execution circuitry (hardware) such as that detailed herein. For the TILESUB instruction, the execution will cause execution

circuitry to perform an elementwise matrix subtraction operation on the source data. In some embodiments, the execution of a decoded matrix subtraction operation instruction causes an execution circuit to, for each data element position of the first source matrix operand: subtract from a first data value at that data element position a second data value at a corresponding data element position of the identified second source matrix operand, and store a result of the subtraction into a corresponding data element position of the destination matrix operand.

[0809] In some embodiments, a fault is generated when a number of data element rows associated with the first source matrix (tile) operand is different than a number of data element rows associated with the second source matrix (tile) operand or is different than a number of data element rows associated with the destination matrix (tile) operand. Similarly, a fault is generated when a number of data element columns associated with the first source matrix (tile) operand is different than a number of data element columns associated with the second source matrix (tile) operand or is different than a number of data element columns associated with the destination matrix (tile) operand. As described elsewhere herein, the dimensions for each matrix, element size for the data elements each matrix, and other configuration can be set by executing a TILECONFIG instruction.

[0810] As described elsewhere herein, successful execution of a TILECONFIG instruction enables subsequent TILE operators and sets a state variable indicating that the corresponding code is in a region with TILES configured. In some embodiments, a fault is generated as part of executing a TILE SUBTRACT instruction if the TILES mode is determined to be inactive. For example, the execution circuitry can check whether the state variable set as part of the successful execution of a TILECONFIG instruction indicates that a TILES mode is active.

[0811] In some embodiments, the instruction is committed or retired at 6409.

[0812] Figure 65 illustrates an example process describing a method performed by a processor to process a TILESUB instruction. For example, process 6501 illustrates an example method for performing a TILESUB operation when the source matrix (tile) operands contain half-precision elements. Process 6601 illustrates an example method for performing a TILESUB operation when the source matrix (tile) operands contain single-precision elements.

[0813] As shown, the process of 6501 determines whether any of the following is true: 1) is a TILES mode not active?; 2) do the destination matrix (tile) operand and the first source matrix (tile) operand have a different number of columns?; 3) do the destination matrix (tile) operand and the second source matrix (tile) operand have a different number of columns?; 4) do the destination matrix (tile) operand and the first source matrix (tile) operand have a

different number of row?; 5) do the destination matrix (tile) operand and the second source matrix (tile) operand have a different number of rows?; 6) does the destination matrix (tile) operand have more than a specified maximum number of columns?; 7) does the first source matrix (tile) operand have more than a specified maximum number of columns?; 8) does the second source matrix (tile) operand have more than a specified maximum number of columns? If any of these is true, then a fault is raised.

[0814] If none of the conditions above is true, then the execution circuitry writes, for each configured row and column of the identified destination matrix (tile) operand, the sum of corresponding element values from the first source matrix (tile) operand and the second source matrix (tile) operand into a corresponding data element position of the destination matrix (tile) operand. In some embodiments, unconfigured elements of rows of the destination matrix (tile) that do not have corresponding columns in the source matrix (tile) are zeroed.

[0100] **Figure 66** illustrates an example process describing a method performed by a processor to process a TILESUB instruction for single precision data elements.

[0815] *iv. Examples*

[0816] **Example 1** A processor comprising: decode circuitry to decode an instruction having fields for an opcode, a first source matrix operand identifier, a second source matrix operand identifier, and a destination matrix operand identifier; and execution circuitry to execute the decoded instruction to, for each data element position of the identified first source matrix operand: subtract from a first data value at that data element position a second data value at a corresponding data element position of the identified second source matrix operand, and store a result of the subtraction into a corresponding data element position of the identified destination matrix operand.

[0817] **Example 2** The processor of Example 1, wherein the first source matrix operand is a packed data register and the second source matrix operand is a memory location.

[0818] **Example 3** The processor of Example 1, wherein the first source matrix operand is a packed data register and the second source matrix operand is a packed data register.

[0819] **Example 4** The processor of any of Examples 1-3, wherein the execution circuitry comprises a plurality of fused-multiply adders.

[0820] **Example 5** The processor of Example 1, wherein the opcode indicates that each of the first source matrix operand, the second source matrix operand, and the destination matrix operand comprises half-precision floating-point values.

[0821] **Example 6** The processor of Example 1, wherein the opcode indicates that each of the first source matrix operand, the second source matrix operand, and the destination matrix operand comprises single-precision floating-point values.

[0822] **Example 7** The processor of any of Examples 1-6, wherein a fault is generated when the first source matrix operand has a different number of data elements than the second source matrix operand.

[0823] **Example 8** The processor of any of Examples 1-7, wherein a fault is generated when a number of rows associated with the first source matrix operand is different than a number of rows associated with the second source matrix operand.

[0824] **Example 9** The processor of any of Examples 1-8, wherein a fault is generated when a number of columns associated with the first source matrix operand is different than a number of columns associated with the second source matrix operand.

[0825] **Example 10** The processor of any of Examples 1-9, wherein the execution circuitry further checks a state variable indicating whether a matrix operations mode is active, and wherein a fault is generated when it is determined that the matrix operations mode is not active.

[0826] **Example 11** provides a method comprising: decoding an instruction having fields for an opcode, a first source matrix operand identifier, a second source matrix operand identifier, and a destination matrix operand identifier; and executing the decoded instruction to, for each data element position of the identified first source matrix operand: subtract from a first data value at that data element position a second data value at a corresponding data element position of the identified second source matrix operand, and store a result of the subtraction into a corresponding data element position of the identified destination matrix operand.

[0827] **Example 12** The method of Example 11, wherein the first source matrix operand is a packed data register and the second source matrix operand is a memory location.

[0828] **Example 13** The method of Example 11, wherein the first source matrix operand is a packed data register and the second source matrix operand is a packed data register.

[0829] **Example 14** The method of any of Examples 11-13, wherein the execution circuitry comprises a plurality of fused-multiply adders.

[0830] **Example 15** The method of Example 11, wherein the opcode indicates that each of the first source matrix operand, the second source matrix operand, and the destination matrix operand comprises half-precision floating-point values.

[0831] **Example 16** The method of Example 11, wherein the opcode indicates that each of the first source matrix operand, the second source matrix operand, and the destination matrix operand comprises single-precision floating-point values.

[0832] **Example 17** The method of any of Examples 11-16, wherein a fault is generated when the first source matrix operand has a different number of data elements than the second source matrix operand.

[0833] **Example 18** The method of any of Examples 11-17, wherein a fault is generated when a number of rows associated with the first source matrix operand is different than a number of rows associated with the second source matrix operand.

[0834] **Example 19** The method of Example 11-18, wherein a fault is generated when a number of columns associated with the first source matrix operand is different than a number of columns associated with the second source matrix operand.

[0835] **Example 20** The method of any of Examples 11-19, wherein executing the decoded instruction includes checking a state variable indicating whether a matrix operations mode is active, and wherein a fault is generated when it is determined that the matrix operations mode is not active.

[0836] **Example 21** provides a non-transitory machine-readable medium storing an instruction which when executed by a processor causes the processor to perform a method, the method comprising: decoding an instruction having fields for an opcode, a first source matrix operand identifier, a second source matrix operand identifier, and a destination matrix operand identifier, and executing the decoded instruction to, for each data element position of the identified first source matrix operand: subtract from a first data value at that data element position a second data value at a corresponding data element position of the identified second source matrix operand, and store a result of the subtraction into a corresponding data element position of the identified destination matrix operand.

[0837] **Example 22** The non-transitory machine-readable medium of Example 21, wherein the first source matrix operand is a packed data register and the second source matrix operand is a memory location.

[0838] **Example 23** The non-transitory machine-readable medium of Example 21, wherein the first source matrix operand is a packed data register and the second source matrix operand is a packed data register.

[0839] **Example 24** The non-transitory machine-readable medium of Example 21, wherein the execution circuitry comprises a plurality of fused-multiply adders.

[0840] **Example 25** The non-transitory machine-readable medium of Example 21, wherein the opcode indicates that each of the first source matrix operand, the second source matrix operand, and the destination matrix operand comprises half-precision floating-point values.

[0841] **Example 26** The non-transitory machine-readable medium of Example 21, wherein the opcode indicates that each of the first source matrix operand, the second source matrix operand, and the destination matrix operand contains single-precision floating-point values.

[0842] **Example 27** The non-transitory machine-readable medium of any of Examples 21-26, wherein a fault is generated when the first source matrix operand has a different number of data elements than the second source matrix operand.

[0843] **Example 28** The non-transitory machine-readable medium of any of Examples 21-27, wherein a fault is generated when a number of rows associated with the first source matrix operand is different than a number of rows associated with the second source matrix operand.

[0844] **Example 29** The non-transitory machine-readable medium of any of Examples 21-28, wherein a fault is generated when a number of columns associated with the first source matrix operand is different than a number of columns associated with the second source matrix operand.

[0845] **Example 30** The non-transitory machine-readable medium of any of Examples 21-29, wherein executing the decoded instruction includes checking a state variable indicating whether a matrix operations mode is active, and wherein a fault is generated when it determines that the matrix operations mode is not active.

[0846] **Example 31** provides a system comprising: a processor; and an accelerator coupled to the processor, the accelerator including: decode circuitry to decode an instruction having fields for an opcode, a first source matrix operand identifier, a second source matrix operand identifier, and a destination matrix operand identifier; and execution circuitry to execute the decoded instruction to, for each data element position of the identified first source matrix operand: subtract from a first data value at that data element position a second data value at a corresponding data element position of the identified second source matrix operand, and store a result of the subtraction into a corresponding data element position of the identified destination matrix operand.

[0847] **Example 32** The system of Example 31, wherein the first source matrix operand is a packed data register and the second source matrix operand is a memory location.

[0848] **Example 33** The system of Example 31, wherein the first source matrix operand is a packed data register and the second source matrix operand is a packed data register.

[0849] **Example 34** The system of any of Examples 31-33, wherein the execution circuitry comprises a plurality of fused-multiply adders.

[0850] **Example 35** The system of Example 31, wherein the opcode indicates that each of the first source matrix operand, the second source matrix operand, and the destination matrix operand comprises half-precision floating-point values.

[0851] **Example 36** The system of Example 31, wherein the opcode indicates that each of the first source matrix operand, the second source matrix operand, and the destination matrix operand comprises single-precision floating-point values.

[0852] **Example 37** The system of any of Examples 31-36, wherein a fault is generated when the first source matrix operand has a different number of data elements than the second source matrix operand.

[0853] **Example 38** The system of any of Examples 31-37, wherein a fault is generated when a number of rows associated with the first source matrix operand is different than a number of rows associated with the second source matrix operand.

[0854] **Example 39** The system of any of Examples 31-38, wherein a fault is generated when a number of columns associated with the first source matrix operand is different than a number of columns associated with the second source matrix operand.

[0855] **Example 40** The system of any of Examples 31-39, wherein the execution circuitry further checks a state variable indicating whether a matrix operations mode is active, and wherein a fault is generated when it is determined that the matrix operations mode is not active.

[0856] *M. Tile Multiply*

[0857] Detailed herein are embodiments of a matrix (tile) multiplication ("TILEMUL") instruction and its execution. A TILEMUL instruction is an improvement to a computer itself as it provides for support to perform elementwise multiplication of matrices (tiles) of data values with a single instruction. In particular, the execution of the TILEMUL instruction causes elementwise multiplication of elements of a first source matrix (tile) with corresponding elements of a second source matrix (tile) and storage of the result in corresponding data element positions of a destination matrix (tile). The size of the data values in the source and destination matrices varies depending on the instruction and tile support. Exemplary sizes include, but are not limited to, 16-bit, 32-bit, 64-bit, 128-bit, and 256-bit. In

some embodiments, elements of rows of the destination matrix (tile) that do not have corresponding elements in the source matrices (tiles) are zeroed.

[0858] *i. Exemplary Execution*

[0859] Figure 67 illustrates an exemplary execution of a TILEMUL instruction. The TILEMUL instruction format includes fields for an opcode (e.g., shown as "TMULP{H,S}" in the figure), a destination operand (e.g., shown as "DESTINATION MATRIX (TILE)" in the figure) and two source operands (e.g., shown as "FIRST SOURCE MATRIX (TILE)" and "SECOND SOURCE MATRIX (TILE)" in the figure).

[0860] The source and destination matrix (tiles) operand fields represent a first source matrix (tile) 6701, a second source matrix (tile) 6703, and a destination matrix (tile) 6707. As detailed earlier, a matrix (tile) may be stored in a collection of registers, locations in memory (for example, as strided rows), or in other storage accessible to execution circuitry.

[0861] As shown, execution circuitry 6705 executes a decoded TMUL instruction to perform elementwise multiplication of the elements of the first source matrix (tile) 6701 and the second source matrix (tile) 6703 and stores the result in corresponding data element positions of the destination matrix (tile) 6707. In some embodiments, a matrix (tile) is configured to use only a subset of the rows and columns possible. For example, a matrix (tile) may have up to 16 rows and columns to use, but only use 4 of each. The configuration of each matrix (tile) is typically done by the execution of a configuration instruction prior to matrix (tile) usage. In this example, there are N columns and M rows possible. Each of the first source matrix (tile) 6701 and the second source matrix (tile) 6703 uses 3 rows and 3 columns. Although the example of Figure 67illustrates an example of performing elementwise multiplication of two 3 x 3 matrices, in general, a TILEMUL instruction can operate on any two matrices (tiles) having the same dimensions (that is, source matrices (tiles) having a same number of columns N and a same number of rows M).

[0862] In some embodiments, execution circuitry 6705 uses a grid of fused multiply adders (FMAs) to execute a decoded TILEMUL instruction by storing the result of performing elementwise multiplication of the two source matrix (tile) operands into corresponding data element positions of the destination matrix (tile) 6707. In particular, the grid of FMAs generates, for each data element position[row, column] of a first source matrix (tile) 6701, a multiplication of the value at that data element position by the value at a corresponding data element position[row, column] of the second source matrix (tile) 6703. In reference to the example source matrix (tile) operands 6701, 6703, the execution circuitry 6705 multiplies the value at first source matrix (tile) 6701 position[0,0] by the value at

second source matrix (tile) 6703 position[0,0] (A*J) and stores the result in the position[0,0] of the destination matrix (tile) 6707, multiplies the value at first source matrix (tile) 6701 position[0,1] by the value at second source matrix (tile) 6703 position[0,1] (B*K) and stores the result in the position[0,1] of the destination matrix (tile) 6707, and so forth.

[0863] *ii. Exemplary Format(s)*

[0864] An embodiment of a format for a TILEMUL instruction is TMULP{H/S} TMM1, TMM2, TMM3. In some embodiments, TMULP{H/S} is the opcode mnemonic of the instruction, where the S or H identifier indicates whether the source matrices (tile) comprise single-precision (PS) or half-precision (PH) floating-point data values. TMM1 is a field for the destination matrix (tile) operand. TMM2 and TMM3 are fields for the matrix (tile) source operands. In some embodiments, the TMM3 field is a R/M value (such as 8246), the TMM1 field is REG 8244, and the data element size is found in 8265.

[0865] In embodiments, encodings of the instruction include a scale-index-base (SIB) type memory addressing operand that indirectly identifies multiple indexed destination locations in memory. In one embodiment, an SIB type memory operand may include an encoding identifying a base address register. The contents of the base address register may represent a base address in memory from which the addresses of the particular destination locations in memory are calculated. For example, the base address may be the address of the first location in a block of potential destination locations for an extended vector instruction. In one embodiment, an SIB type memory operand may include an encoding identifying an index register. Each element of the index register may specify an index or offset value usable to compute, from the base address, an address of a respective destination location within a block of potential destination locations. In one embodiment, an SIB type memory operand may include an encoding specifying a scaling factor to be applied to each index value when computing a respective destination address. For example, if a scaling factor value of four is encoded in the SIB type memory operand, each index value obtained from an element of the index register may be multiplied by four and then added to the base address to compute a destination address.

[0866] In one embodiment, an SIB type memory operand of the form vm32{x,y,z} may identify a vector array of memory operands specified using SIB type memory addressing. In this example, the array of memory addresses is specified using a common base register, a constant scaling factor, and a vector index register containing individual elements, each of which is a 32-bit index value. The vector index register may be a 128-bit register (e.g., XMM) register (vm32x), a 256-bit (e.g., YMM) register (vm32y), or a 512-bit (e.g., ZMM)

register (vm32z). In another embodiment, an SIB type memory operand of the form vm64{x,y,z} may identify a vector array of memory operands specified using SIB type memory addressing. In this example, the array of memory addresses is specified using a common base register, a constant scaling factor, and a vector index register containing individual elements, each of which is a 64-bit index value. The vector index register may be a 128-bit register (e.g., XMM) register (vm64x), a 256-bit (e.g., YMM) register (vm64y) or a 512-bit (e.g., ZMM) register (vm64z).

[0867] *iii. Exemplary Method(s) of Execution*

[0868] Figure 68 illustrates an embodiment of method performed by a processor to process a TILEMUL instruction.

[0869] At 6801, an instruction is fetched. For example, a TILEMUL instruction is fetched. The TILEMUL instruction includes fields for an opcode, a first and a second source matrix (tile) operand identifier, and a destination matrix (tile) operand identifier. In some embodiments, the instruction is fetched from an instruction cache. The source operands and destination operand consist of packed data. The opcode of the TILEMUL instruction indicates that a sum of the source operands is to be generated. In an embodiment, the opcode further indicates whether the source operands consist of half-precision floating-point values or single-precision floating-point values.

[0870] The fetched instruction is decoded at 6803. For example, the fetched TILEMUL instruction is decoded by decode circuitry such as that detailed herein.

[0871] Data values associated with the source matrix (tile) operands of the decoded instruction are retrieved at 6805 and the decoded instruction is schedule (as needed). For example, when one or more of the source matrix (tile) operands are memory operands, the data from the indicated memory location is retrieved.

[0872] At 6807, the decoded instruction is executed by execution circuitry (hardware) such as that detailed herein. For the TILEMUL instruction, the execution will cause execution circuitry to perform an elementwise matrix multiplication operation on the source data. In some embodiments, the execution of a decoded matrix multiplication operation instruction causes an execution circuit to, for each data element position of the first source matrix operand: multiply a first data value at that data element position by a second data value at a corresponding data element position of the second source matrix operand, and store a result of the multiplication into a corresponding data element position of the destination matrix operand.

[0873] In some embodiments, a fault is generated when a number of data element rows associated with the first source matrix (tile) operand is different than a number of data element rows associated with the second source matrix (tile) operand or is different than a number of data element rows associated with the destination matrix (tile) operand. Similarly, a fault is generated when a number of data element columns associated with the first source matrix (tile) operand is different than a number of data element columns associated with the second source matrix (tile) operand or is different than a number of data element columns associated with the destination matrix (tile) operand. As described elsewhere herein, the dimensions for each matrix, element size for the data elements each matrix, and other configuration can be set by executing a TILECONFIG instruction.

[0874] As described elsewhere herein, successful execution of a TILECONFIG instruction enables subsequent TILE operators and sets a state variable indicating that the corresponding code is in a region with TILES configured. In some embodiments, a fault is generated as part of executing a TILEMUL instruction if the TILES mode is determined to be inactive. For example, the execution circuitry can check whether the state variable set as part of the successful execution of a TILECONFIG instruction indicates that a TILES mode is active.

[0875] In some embodiments, the instruction is committed or retired at 6809.

[0876] Figure 69 illustrates an example process describing a method performed by a processor to process a TILEMUL instruction. For example, process 6901 illustrates an example method for performing a TILEMUL operation when the source matrix (tile) operands contain half-precision elements. Process 7001 as shown in Figure 70 illustrates an example method for performing a TILEMUL operation when the source matrix (tile) operands contain single-precision elements.

[0877] As shown, the process of 6901 determines whether any of the following is true: 1) is a TILES mode not active?; 2) do the destination matrix (tile) operand and the first source matrix (tile) operand have a different number of columns?; 3) do the destination matrix (tile) operand and the second source matrix (tile) operand have a different number of columns?; 4) do the destination matrix (tile) operand and the first source matrix (tile) operand have a different number of row?; 5) do the destination matrix (tile) operand and the second source matrix (tile) operand have a different number of rows?; 6) does the destination matrix (tile) operand have more than a specified maximum number of columns?; 7) does the first source matrix (tile) operand have more than a specified maximum number of columns?; 8) does the second source matrix (tile) operand have more than a specified maximum number of columns? If any of these is true, then a fault is raised.

[0878] If none of the conditions above is true, then the execution circuitry writes, for each configured row and column of the identified destination matrix (tile) operand, the sum of corresponding element values from the first source matrix (tile) operand and the second source matrix (tile) operand into a corresponding data element position of the destination matrix (tile) operand. In some embodiments, unconfigured elements of rows of the destination matrix (tile) that do not have corresponding columns in the source matrix (tile) are zeroed.

[0879] *iv. Examples*

[0880] **Example 1** A processor comprising: decode circuitry to decode an instruction having fields for an opcode, a first source matrix operand identifier, a second source matrix operand identifier, and a destination matrix operand identifier; and execution circuitry to execute the decoded instruction to, for each data element position of the identified first source matrix operand: multiply a first data value at that data element position by a second data value at a corresponding data element position of the identified second source matrix operand, and store a result of the multiplication into a corresponding data element position of the identified destination matrix operand.

[0881] **Example 2** The processor of Example 1, wherein the first source matrix operand is a packed data register and the second source matrix operand is a memory location.

[0882] **Example 3** The processor of Example 1, wherein the first source matrix operand is a packed data register and the second source matrix operand is a packed data register.

[0883] **Example 4** The processor of any of Examples 1-3, wherein the execution circuitry comprises a plurality of fused-multiply adders.

[0884] **Example 5** The processor of Example 1, wherein the opcode indicates that each of the first source matrix operand, the second source matrix operand, and the destination matrix operand comprises half-precision floating-point values.

[0885] **Example 6** The processor of Example 1, wherein the opcode indicates that each of the first source matrix operand, the second source matrix operand, and the destination matrix operand comprises single-precision floating-point values.

[0886] **Example 7** The processor of any of Examples 1-6, wherein a fault is generated when the first source matrix operand has a different number of data elements than the second source matrix operand.

[0887] **Example 8** The processor of any of Examples 1-7, wherein a fault is generated when a number of rows associated with the first source matrix operand is different than a number of rows associated with the second source matrix operand.

[0888] **Example 9** The processor of any of Examples 1-8, wherein a fault is generated when a number of columns associated with the first source matrix operand is different than a number of columns associated with the second source matrix operand.

[0889] **Example 10** The processor of any of Examples 1-9, wherein the execution circuitry further checks a state variable indicating whether a matrix operations mode is active, and wherein a fault is generated when it is determined that the matrix operations mode is not active.

[0890] **Example 11** provides a method comprising: decoding an instruction having fields for an opcode, a first source matrix operand identifier, a second source matrix operand identifier, and a destination matrix operand identifier; and executing the decoded instruction to, for each data element position of the identified first source matrix operand: multiply a first data value at that data element position by a second data value at a corresponding data element position of the identified second source matrix operand, and store a result of the multiplication into a corresponding data element position of the identified destination matrix operand.

[0891] **Example 12** The method of Example 11, wherein the first source matrix operand is a packed data register and the second source matrix operand is a memory location.

[0892] **Example 13** The method of Example 11, wherein the first source matrix operand is a packed data register and the second source matrix operand is a packed data register.

[0893] **Example 14** The method of any of Examples 11-13, wherein the execution circuitry comprises a plurality of fused-multiply adders.

[0894] **Example 15** The method of Example 11, wherein the opcode indicates that each of the first source matrix operand, the second source matrix operand, and the destination matrix operand comprises half-precision floating-point values.

[0895] **Example 16** The method of Example 11, wherein the opcode indicates that each of the first source matrix operand, the second source matrix operand, and the destination matrix operand comprises single-precision floating-point values.

[0896] **Example 17** The method of any of Examples 11-16, wherein a fault is generated when the first source matrix operand has a different number of data elements than the second source matrix operand.

[0897] **Example 18** The method of any of Examples 11-17, wherein a fault is generated when a number of rows associated with the first source matrix operand is different than a number of rows associated with the second source matrix operand.

[0898] **Example 19** The method of Example 11-18, wherein a fault is generated when a number of columns associated with the first source matrix operand is different than a number of columns associated with the second source matrix operand.

[0899] **Example 20** The method of any of Examples 11-19, wherein executing the decoded instruction includes checking a state variable indicating whether a matrix operations mode is active, and wherein a fault is generated when it is determined that the matrix operations mode is not active.

[0900] **Example 21** provides a non-transitory machine-readable medium storing an instruction which when executed by a processor causes the processor to perform a method, the method comprising: decoding an instruction having fields for an opcode, a first source matrix operand identifier, a second source matrix operand identifier, and a destination matrix operand identifier, and executing the decoded instruction to, for each data element position of the identified first source matrix operand: multiply a first data value at that data element position to a second data value by a corresponding data element position of the identified second source matrix operand, and store a result of the multiplication into a corresponding data element position of the identified destination matrix operand.

[0901] **Example 22** The non-transitory machine-readable medium of Example 21, wherein the first source matrix operand is a packed data register and the second source matrix operand is a memory location.

[0902] **Example 23** The non-transitory machine-readable medium of Example 21, wherein the first source matrix operand is a packed data register and the second source matrix operand is a packed data register.

[0903] **Example 24** The non-transitory machine-readable medium of Example 21, wherein the execution circuitry comprises a plurality of fused-multiply adders.

[0904] **Example 25** The non-transitory machine-readable medium of Example 21, wherein the opcode indicates that each of the first source matrix operand, the second source matrix operand, and the destination matrix operand comprises half-precision floating-point values.

[0905] **Example 26** The non-transitory machine-readable medium of Example 21, wherein the opcode indicates that each of the first source matrix operand, the second source matrix operand, and the destination matrix operand contains single-precision floating-point values.

[0906] **Example 27** The non-transitory machine-readable medium of any of Examples 21-26, wherein a fault is generated when the first source matrix operand has a different number of data elements than the second source matrix operand.

[0907] **Example 28** The non-transitory machine-readable medium of any of Examples 21-27, wherein a fault is generated when a number of rows associated with the first source matrix operand is different than a number of rows associated with the second source matrix operand.

[0908] **Example 29** The non-transitory machine-readable medium of any of Examples 21-28, wherein a fault is generated when a number of columns associated with the first source matrix operand is different than a number of columns associated with the second source matrix operand.

[0909] **Example 30** The non-transitory machine-readable medium of any of Examples 21-29, wherein executing the decoded instruction includes checking a state variable indicating whether a matrix operations mode is active, and wherein a fault is generated when it determines that the matrix operations mode is not active.

[0910] **Example 31** provides a system comprising: a processor; and an accelerator coupled to the processor, the accelerator including: decode circuitry to decode an instruction having fields for an opcode, a first source matrix operand identifier, a second source matrix operand identifier, and a destination matrix operand identifier; and execution circuitry to execute the decoded instruction to, for each data element position of the identified first source matrix operand: multiply a first data value at that data element position by a second data value at a corresponding data element position of the identified second source matrix operand, and store a result of the multiplication into a corresponding data element position of the identified destination matrix operand.

[0911] **Example 32** The system of Example 31, wherein the first source matrix operand is a packed data register and the second source matrix operand is a memory location.

[0912] **Example 33** The system of Example 31, wherein the first source matrix operand is a packed data register and the second source matrix operand is a packed data register.

[0913] **Example 34** The system of any of Examples 31-33, wherein the execution circuitry comprises a plurality of fused-multiply adders.

[0914] **Example 35** The system of Example 31, wherein the opcode indicates that each of the first source matrix operand, the second source matrix operand, and the destination matrix operand comprises half-precision floating-point values.

[0915] **Example 36** The system of Example 31, wherein the opcode indicates that each of the first source matrix operand, the second source matrix operand, and the destination matrix operand comprises single-precision floating-point values.

[0916] **Example 37** The system of any of Examples 31-36, wherein a fault is generated when the first source matrix operand has a different number of data elements than the second source matrix operand.

[0917] **Example 38** The system of any of Examples 31-37, wherein a fault is generated when a number of rows associated with the first source matrix operand is different than a number of rows associated with the second source matrix operand.

[0918] **Example 39** The system of any of Examples 31-38, wherein a fault is generated when a number of columns associated with the first source matrix operand is different than a number of columns associated with the second source matrix operand.

[0919] **Example 40** The system of any of Examples 31-39, wherein the execution circuitry further checks a state variable indicating whether a matrix operations mode is active, and wherein a fault is generated when it is determined that the matrix operations mode is not active.

[0920] *M. Tile Multiply Accumulate*

[0921] Detailed herein are embodiments of a matix (tile) multiply accumulate ("TMMA") instruction and its execution. A TMMA instruction is an improvement to a computer itself as it provides for support to perform matrix-matrix multiplication and accumulation (addition) using a single instruction. In particular, an execution of the TMMA instruction causes data from a first source matrix (tile) to be multiplied by data from a second source matrix (tile) and added to data from a destination matrix (tile), and the result of the multiply-add is stored in the destination matrix (tile). The size of the data values to be stored varies depending on the instruction and tile support. Exemplary sizes included, but are not limited to, 16-bit, 32-bit, 64-bit, 128-bit, 256-bit, etc.

[0922] *i. Exemplary Execution*

[0923] Figure 71 illustrates an exemplary execution of a TMMA instruction using memory source operand. The TMMA instruction format includes fields for an opcode, a destination matrix (tile) operand (shown as "Tile Destination"), an identifier of a first source matrix (tile) operand (shown as "FIRST TILE SOURCE"), an identifier of a second source matrix (tile) operand (shown as "SECOND TILE SOURCE"), and, in some embodiments, an identifier of a counter register. In some implementations, when the second source matrix

(tile) operand is in memory, a field for a register to be used in progress tracking is also included.

[0924] When one of the sources for the instruction is memory, the memory is accessed according to a scale-index-base (SIB) type memory addressing operand that indirectly identifies multiple indexed destination locations in memory, however, other memory addressing schemes may be utilized. As detailed, each "row" of a matrix (tile) source or destination is a group of elements. In memory, these groups are separated by a "stride" value. As will be detailed, the "index" of the SIB may be utilized to dictate this stride. Depending upon the implementation, the stride is either from an address corresponding to an initial data element of a group to an initial data element of a subsequent group in memory, or from an address corresponding to a last data element of a group to an initial data element of a subsequent group in memory. Typically, strides are used to delineate rows, however, that in not necessarily true.

[0925] One or both of the sources for the instruction is a matrix (tile) stored in a plurality register or in matrix (tile) data structure. This source is encoded in the instruction as if it was a single register. When there are two such matrices, both are encoded as if they were single registers.

[0926] The final source is the destination matrix (tile) 7109.

[0927] While the illustrated matrices (tiles) are shown as 2x2 matrices, this is an arbitrary illustration to help with understanding and different matrix (tile) sizes may be used. The TMMA operation is Source 1 * Source 2 + Destination. As such, (NxM)*(MxK)+(NxK) matrices are supported.

[0928] The matrix (tile) sources 7101 and 7103, and the destination matrix (tile) 7109 are provided to execution circuitry 7105 for the TMMA operation. In some embodiments, a grid of FMAs 7107 is utilized to execute this operation on a per data element position of the matrices (tiles) basis. A grid of FMAs 7107 has previously been described. In some implementations, one or more of the matrix (tile) source 7101 and the destination matrix (tile) 7109 are stored in the grid of FMAs 7107.

[0929] The execution circuitry 7105 performs the TMMA by performing a multiply on the sources on a per data element basis (using matrix multiplication of row X column) and adds data from a corresponding data element position of the destination matrix. The result of TMMA is stored into the corresponding data element position of the destination matrix as shown in 7111.

[0930] As FIG. 71 is simplified, it does not illustrate the use of a counter register which functions as a progress tracker. The counter is updated as each "row" of the destination is written. This allows for the TMMA operation to be restarted if needed by using the counter to determine where the operation left off. Note also, that in some embodiments, a counter function is not utilized.

[0931] *ii. Exemplary Format(s)*

[0932] An embodiment of a format for a TMMA instruction is TMMAP{S,H} TMM1, TMM2, TMM3. In some embodiments, TMMAP{S,H} is the opcode mnemonic of the instruction where S,H represent single precision (S) floating point data elements and half precision floating point data elements. TMM1 is a field for an identifier of a source/destination matrix (tile), and TMM3 is a field for an identifier of a second source matrix (tile), and TMM2 is a field for an identifier of a first source matrix (tile). In some embodiments, the TMM2 identifier is field R/M value (such as 8246), TMM3 is field VVVV 8220, the source/destination matrix (tile) identifier is field 8244. Note, if a counter is not used, SRC3 is not included in the instruction format.

[0933] *iii. Exemplary Method(s) of Execution*

[0934] Figure 72 illustrates an embodiment of method performed by a processor to process a TMMA instruction.

[0935] At 7201, an instruction is fetched. For example, a TMMA instruction is fetched. An embodiment of the TMMA instruction includes fields for an opcode, a destination matrix (tile) operand identifier, a first source matrix (tile) operand identifier, and a second source matrix (tile) operand identifier (e.g., stored in memory, or accessed as a register). In some embodiments, a register to store a counter value identifier is also included.

[0936] The fetched instruction is decoded at 7203. For example, the fetched TMMA instruction is decoded by decode circuitry such as that detailed herein.

[0937] Data values associated with the sources are retrieved at 7205 and the decoded instruction is scheduled (as needed).

[0938] At 7207, the decoded instruction is executed by execution circuitry (hardware) such as that detailed herein. For the TMMA instruction, the execution will cause execution circuitry to perform: 1) a matrix multiplication of the identified first source matrix (tile) operand with the identified second source matrix (tile) (from memory, or register accessed); and 2) add a result of the matrix multiplication to corresponding data element positions of the identified destination matrix (tile) operand. In some embodiments, data element positions of

the identified destination matrix (tile) operand that were not subject to an addition are zeroed (unconfigured columns).

[0939] In some embodiments, the instruction is committed or retired at 7209.

[0940] Figure 73 illustrates a more detailed description of an execution of a TMMA instruction using register addressing. Typically, this is performed by execution circuitry such as that detailed above.

[0941] At 7301, a value in a first, second, and third counter are set. For example, startK and startM are set. Typically, this was done during configuration, but upon a first instance of this instruction these are usually set to 0.

[0942] A determination of if the first counter value (e.g., startM) is less than a number of configured rows of the destination is made at 7303. If not, then the instruction has completed and all unconfigured rows are zeroed at 7305.

[0943] If yes, then a row from the destination is written into a temporary location at 7307. This row is at position[first counter] (or the value of startM).

[0944] A determination of if the second counter value (e.g., startK) is less than a number of configured columns of the first source is made at 7309.

[0945] If no, then the row of the temporary location is written to the destination in a corresponding row position at 7311. Typically, unconfigured columns of that row are also zeroed. The second and third counters are reset, and the first counter is incremented at 7317. Essentially, the next row is set to be processed beginning at 7303 again.

[0946] If yes, then there is potentially more work to do for that row. A determination of if the third counter value (e.g., n) is less than a number of configured columns of the destination is made at 7312. If yes, then a multiplication of a data element of the first source at position row[first counter, second counter] by a data element of the second source at position row[second counter, third counter] is made, and a result of that multiplication is added to the temporary value at position temporary[third counter] at 7313. The third counter is incremented at 7315 to move the loop along and the determination of 7312 is made again.

[0947] If not, then the second counter is incremented at 7316 and the determination of 7309 is made again.

[0948] *iv. Exemplary Pseudocode*

[0949] FIG. 74 illustrates pseudocode for a method of implementing a TMMPS instruction. The FMAOP may be a negated version when the opcode calls for it (TNMMPS).

[0950] *v. Examples*

[0951] **Example 1** A processor comprising: decode circuitry to decode an instruction having fields for an opcode, an identifier for a first source matrix operand, an identifier of a second source matrix operand, and an identifier for a source/destination matrix operand; and execution circuitry to execute the decoded instruction to multiply the identified first source matrix operand by the identified second source matrix operand, add a result of the multiplication to the identified source/destination matrix operand, and store a result of the addition in the identified source/destination matrix operand.

[0952] **Example 2** The processor of example 1, wherein the execution circuitry comprises a grid of mused multiply accumulators.

[0953] **Example 3** The processor of any of examples 1-2, wherein identified second source matrix operand is stored in memory.

[0954] **Example 4** The processor of any of examples 1-3, wherein the multiplication is per row of the identified first source matrix operand and per column of the identified second source matrix operand.

[0955] **Example 5** The processor of any of examples 1-4, wherein at least one of the operands is a plurality of registers configured to represent a matrix.

[0956] **Example 6** The processor of any of examples 1-5, wherein the data elements are single precision floating point data elements.

[0957] **Example 7** The processor of any of examples 1-5, wherein the data elements are half precision floating point data elements.

[0958] **Example 8** A method comprising: decoding an instruction having fields for an opcode, an identifier for a first source matrix operand, an identifier of a second source matrix operand, and an identifier for a source/destination matrix operand; and executing the decoded instruction to multiply the identified first source matrix operand by the identified second source matrix operand, add a result of the multiplication to the identified source/destination matrix operand, and store a result of the addition in the identified source/destination matrix operand.

[0959] **Example 9** The method of example 8, wherein the executing uses a grid of mused multiply accumulators.

[0960] **Example 10** The method of any of examples 8-9, wherein identified second source matrix operand is stored in memory.

[0961] **Example 11** The method of any of examples 8-10, wherein the multiplication is per row of the identified first source matrix operand and per column of the identified second source matrix operand.

[0962] **Example 12** The method of any of examples 8-11, wherein at least one of the operands is a plurality of registers configured to represent a matrix.

[0963] **Example 13** The method of any of examples 8-12, wherein the data elements are single precision floating point data elements.

[0964] **Example 14** The method of any of examples 8-12, wherein the data elements are half precision floating point data elements.

[0965] **Example 15** A non-transitory machine-readable medium storing an instruction which causes a processor to perform a method, the method comprising: decoding an instruction having fields for an opcode, an identifier for a first source matrix operand, an identifier of a second source matrix operand, and an identifier for a source/destination matrix operand; and executing the decoded instruction to multiply the identified first source matrix operand by the identified second source matrix operand, add a result of the multiplication to the identified source/destination matrix operand, and store a result of the addition in the identified source/destination matrix operand.

[0966] **Example 16** The non-transitory machine-readable medium of example 15, wherein the executing uses comprises a grid of mused multiply accumulators.

[0967] **Example 17** The non-transitory machine-readable medium of any of examples 15-16, wherein identified second source matrix operand is stored in memory.

[0968] **Example 18** The non-transitory machine-readable medium of any of examples 15-17, wherein the multiplication is per row of the identified first source matrix operand and per column of the identified second source matrix operand.

[0969] **Example 19** The non-transitory machine-readable medium of any of examples 15-18, wherein at least one of the operands is a plurality of registers configured to represent a matrix.

[0970] **Example 20** The non-transitory machine-readable medium of any of examples 15-19, wherein the data elements are single precision floating point data elements.

[0971] **Example 21** The non-transitory machine-readable medium of any of examples 15-19, wherein the data elements are half precision floating point data elements.

[0972] **Example 22** A system comprising: a processor; and an accelerator coupled to the processor, the accelerator including: decode circuitry to decode an instruction having fields for an opcode, an identifier for a first source matrix operand, an identifier of a second source matrix operand, and an identifier for a source/destination matrix operand; and execution circuitry to execute the decoded instruction to multiply the identified first source matrix operand by the identified second source matrix operand, add a result of the multiplication to

the identified source/destination matrix operand, and store a result of the addition in the identified source/destination matrix operand and zero unconfigured columns of identified source/destination matrix operand.

[0973] **Example 23** The system of example 22, wherein the execution circuitry comprises a grid of mused multiply accumulators.

[0974] **Example 24** The system of any of examples 22-23, wherein at least one of the operands is a plurality of registers configured to represent a matrix.

[0975] *N. Tile Negated Multiply Accumulate*

[0976] Detailed herein are embodiments of a matrix (tile) negated multiply accumulate ("TNMMA") instruction and its execution. A TNMMA instruction is an improvement to a computer itself as it provides for support to perform matrix-matrix multiplication and negated accumulation (subtraction) using a single instruction. In particular, an execution of the TNMMA instruction causes data from a first source matrix (tile) to be multiplied by data from a second source matrix (tile) and subtracted from data from a destination matrix (tile), and the result of the multiply-subtract is stored in the destination matrix (tile). The size of the data values to be stored varies depending on the instruction and tile support. Exemplary sizes included, but are not limited to, 16-bit, 32-bit, 64-bit, 128-bit, 256-bit, etc.

[0977] *i. Exemplary Execution*

[0978] Figure 75 illustrates an exemplary execution of a TNMMA instruction using memory source operand. The TNMMA instruction format includes fields for an opcode, a source/destination matrix (tile) operand (shown as "Tile Destination"), an identifier of a first source matrix (tile) operand (shown as "FIRST TILE SOURCE"), an identifier of a second source matrix (tile) operand (shown as "SECOND TILE SOURCE"), and, in some embodiments, an identifier of a counter register. In some implementations, when the second source matrix (tile) operand is in memory, a field for a register to be used in progress tracking is also included.

[0979] When one of the sources for the instruction is memory, the memory is accessed according to a scale-index-base (SIB) type memory addressing operand that indirectly identifies multiple indexed destination locations in memory, however, other memory addressing schemes may be utilized. As detailed, each "row" of a matrix (tile) source or destination is a group of elements. In memory, these groups are separated by a "stride" value. As will be detailed, the "index" of the SIB may be utilized to dictate this stride. Depending upon the implementation, the stride is either from an address corresponding to an initial data element of a group to an initial data element of a subsequent group in memory, or

from an address corresponding to a last data element of a group to an initial data element of a subsequent group in memory. Typically, strides are used to delineate rows, however, that in not necessarily true.

[0980] One or both of the sources for the instruction is a matrix (tile) stored in a plurality register or in matrix (tile) data structure. This source is encoded in the instruction as if it was a single register. When there are two such matrices, both are encoded as if they were single registers.

[0981] The final source is the destination matrix (tile) 7509.

[0982] While this illustrated matrices (tiles) are shown as 2x2 matrices, this is an arbitrary illustration to help with understanding and different matrix (tile) sizes may be used. The TNMMA operation is Source 1 * Source 2 + Destination As such, (NxK)-(NxM)*(MxK) matrices are supported.

[0983] The matrix (tile) sources 7501 and 7503, and the destination matrix (tile) 7509 are provided to execution circuitry 7505 for the TNMMA operation. In some embodiments, a grid of FMAs 7507 is utilized to execute this operation on a per data element position of the matrices (tiles) basis. A grid of FMAs 7507 has previously been described. In some implementations, one or more of the matrix (tile) source 7501 and the destination matrix (tile) 7509 are stored in the grid of FMAs 7507.

[0984] The execution circuitry 7505 performs the TNMMA by performing a multiply on the sources on a per data element basis (using matrix multiplication of row X column) and a subtract from a corresponding data element position of the destination matrix. The result of TNMMA is stored into the corresponding data element position of the destination matrix as shown in 7511.

[0985] As FIG. 75 is simplified, it does not illustrate the use of a counter register which functions as a progress tracker. The counter is updated as each "row" of the destination is written. This allows for the TNMMA operation to be restarted if needed by using the counter to determine where the operation left off. Note also, that in some embodiments, a counter function is not utilized.

[0986] *ii. Exemplary Format(s)*

[0987] An embodiment of a format for a TNMMA instruction is TNMMA{S,H} TMM1, TMM2, TMM3. In some embodiments, TNMMAP{S,H} is the opcode mnemonic of the instruction where S,H represent single precision (S) floating point data elements and half precision floating point data elements. TMM1 is a field for an identifier of a source/destination matrix (tile), and TMM3 is a field for an identifier of a second source

matrix (tile), and TMM2 is a field for an identifier of a first source matrix (tile). In some embodiments, the TMM2 identifier is field R/M value (such as 8246), TMM3 is field VVVV 8220, the source/destination matrix (tile) identifier is field 8244. Note, if a counter is not used, SRC3 is not included in the instruction format.

[0988] *iii. Exemplary Method(s) of Execution*

[0989] Figure 76 illustrates an embodiment of method performed by a processor to process a TNMMA instruction.

[0990] At 7601, an instruction is fetched. For example, a TNMMA instruction is fetched. An embodiment of the TNMMA instruction includes fields for an opcode, a source/destination matrix (tile) operand identifier, a first source matrix (tile) operand identifier, and a second source matrix (tile) operand identifier (e.g., stored in memory, or accessed as a register). In some embodiments, a register to store a counter value identifier is also included.

[0991] The fetched instruction is decoded at 7603. For example, the fetched TNMMA instruction is decoded by decode circuitry such as that detailed herein.

[0992] Data values associated with the sources are retrieved at 7605 and the decoded instruction is scheduled (as needed).

[0993] At 7607, the decoded instruction is executed by execution circuitry (hardware) such as that detailed herein. For the TNMMA instruction, the execution will cause execution circuitry to perform: 1) a matrix multiplication of the identified first source matrix (tile) operand with the identified second source matrix (tile) (from memory, or register accessed); and 2) subtract a result of the matrix multiplication from corresponding data element positions of the identified destination matrix (tile) operand. In some embodiments, data element positions of the identified source/destination matrix (tile) operand that were not subject to a subtraction are zeroed (unconfigured columns).

[0994] In some embodiments, the instruction is committed or retired at 7609.

[0995] Figure 77 illustrates a more detailed description of an execution of a TNMMA instruction using register addressing. Typically, this is performed by execution circuitry such as that detailed above

[0996] At 7701, a value in a first, second, and third counter are set. For example, startK and startM are set. Typically, this was done during configuration, but upon a first instance of this instruction these are usually set to 0.

[0997] A determination of if the first counter value (e.g., startM) is less than a number of configured rows of the destination is made at 7703. If not, then the instruction has completed and all unconfigured rows are zeroed at 7705.

[0998] If yes, then a row from the destination is written into a temporary location at 7707. This row is at position[first counter] (or the value of startM).

[0999] A determination of if the second counter value (e.g., startK) is less than a number of configured columns of the first source is made at 7709.

[1000] If no, then the row of the temporary location is written to the destination in a corresponding row position at 7711. Typically, unconfigured columns of that row are also zeroed. The second and third counters are reset, and the first counter is incremented at 7717. Essentially, the next row is set to be processed beginning at 7703 again.

[1001] If yes, then there is potentially more work to do for that row. A determination of if the third counter value (e.g., n) is less than a number of configured columns of the destination is made at 7712. If yes, then a multiplication of a data element of the first source at position row[first counter, second counter] by a data element of the second source at position row[second counter, third counter] is made, and a result of that multiplication is subtracted from the temporary value at position temporary[third counter] at 7713. The third counter is incremented at 7715 to move the loop along and the determination of 7712 is made again.

[1002] If not, then the second counter is incremented at 7716 and the determination of 7709 is made again.

[1003] *iv. Examples*

[1004] **Example 1** A processor comprising: decode circuitry to decode an instruction having fields for an opcode, an identifier for a first source matrix operand, an identifier of a second source matrix operand, and an identifier for a source/destination matrix operand; and execution circuitry to execute the decoded instruction to multiply the identified first source matrix operand by the identified second source matrix operand, subtract a result of the multiplication to the identified source/destination matrix operand, and store a result of the subtraction in the identified source/destination matrix operand.

[1005] **Example 2** The processor of example 1, wherein the execution circuitry comprises a grid of mused multiply accumulators.

[1006] **Example 3** The processor of any of examples 1-2, wherein identified second source matrix operand is stored in memory.

[1007] **Example 4** The processor of any of examples 1-3, wherein the multiplication is per row of the identified first source matrix operand and per column of the identified second source matrix operand.

[1008] **Example 5** The processor of any of examples 1-4, wherein at least one of the operands is a plurality of registers configured to represent a matrix.

[1009] **Example 6** The processor of any of examples 1-5, wherein the data elements are single precision floating point data elements.

[1010] **Example 7** The processor of any of examples 1-5, wherein the data elements are half precision floating point data elements.

[1011] **Example 8** A method comprising: decoding an instruction having fields for an opcode, an identifier for a first source matrix operand, an identifier of a second source matrix operand, and an identifier for a source/destination matrix operand; and executing the decoded instruction to multiply the identified first source matrix operand by the identified second source matrix operand, subtract a result of the multiplication to the identified source/destination matrix operand, and store a result of the subtraction in the identified source/destination matrix operand.

[1012] **Example 9** The method of example 8, wherein the executing uses a grid of mused multiply accumulators.

[1013] **Example 10** The method of any of examples 8-9, wherein identified second source matrix operand is stored in memory.

[1014] **Example 11** The method of any of examples 8-10, wherein the multiplication is per row of the identified first source matrix operand and per column of the identified second source matrix operand.

[1015] **Example 12** The method of any of examples 8-11, wherein at least one of the operands is a plurality of registers configured to represent a matrix.

[1016] **Example 13** The method of any of examples 8-12, wherein the data elements are single precision floating point data elements.

[1017] **Example 14** The method of any of examples 8-12, wherein the data elements are half precision floating point data elements.

[1018] **Example 15** A non-transitory machine-readable medium storing an instruction which causes a processor to perform a method, the method comprising: decoding an instruction having fields for an opcode, an identifier for a first source matrix operand, an identifier of a second source matrix operand, and an identifier for a source/destination matrix operand; and executing the decoded instruction to multiply the identified first source matrix

operand by the identified second source matrix operand, subtract a result of the multiplication to the identified source/destination matrix operand, and store a result of the subtraction in the identified source/destination matrix operand.

[1019] **Example 16** The non-transitory machine-readable medium of example 15, wherein the executing uses comprises a grid of mused multiply accumulators.

[1020] **Example 17** The non-transitory machine-readable medium of any of examples 15-16, wherein identified second source matrix operand is stored in memory.

[1021] **Example 18** The non-transitory machine-readable medium of any of examples 15-17, wherein the multiplication is per row of the identified first source matrix operand and per column of the identified second source matrix operand.

[1022] **Example 19** The non-transitory machine-readable medium of any of examples 15-18, wherein at least one of the operands is a plurality of registers configured to represent a matrix.

[1023] **Example 20** The non-transitory machine-readable medium of any of examples 15-19, wherein the data elements are single precision floating point data elements.

[1024] **Example 21** The non-transitory machine-readable medium of any of examples 15-19, wherein the data elements are half precision floating point data elements.

[1025] **Example 22** A system comprising: a processor; and an accelerator coupled to the processor, the accelerator including: decode circuitry to decode an instruction having fields for an opcode, an identifier for a first source matrix operand, an identifier of a second source matrix operand, and an identifier for a source/destination matrix operand; and execution circuitry to execute the decoded instruction to multiply the identified first source matrix operand by the identified second source matrix operand, subtract a result of the multiplication to the identified source/destination matrix operand, and store a result of the subtraction in the identified source/destination matrix operand and zero unconfigured columns of identified source/destination matrix operand.

[1026] **Example 23** The system of example 22, wherein the execution circuitry comprises a grid of mused multiply accumulators.

[1027] **Example 24** The system of any of examples 22-23, wherein at least one of the operands is a plurality of registers configured to represent a matrix.

[1028] *O. Tile Dot Product*

[1029] Detailed herein are embodiments of matrix (tile) dot product ("TILEDOTPRODUCT") instructions and their execution. A TILEDOTPRODUCT instruction is an improvement to a computer itself as it provides for support to perform dot

product operations involving two matrices (tiles) of data values with a single instruction. In particular, the execution of a TILEDOTPRODUCT instruction causes performance of dot product operations on elements from two source matrices (tiles) of data values and accumulation of the result into corresponding data element positions of a destination matrix (tile). The size of the data elements in the source matrices (tiles) varies depending on the instruction and tile support. Exemplary sizes of the data elements contained in the source matrices (tiles) include, but are not limited to, 4-bit, 8-bit, 16-bit, 32-bit, 64-bit, 128-bit, 256-bit, and so forth. In some embodiments, elements of rows and columns of the destination matrix (tile) that do not have corresponding elements in the source matrices (tiles) are zeroed.

[1030] *i. Exemplary Execution*

[1031] Figure 78 illustrates an exemplary execution of a TILEDOTPRODUCT instruction. The TILEDOTPRODUCT instruction format includes fields for an opcode (e.g., shown as "TDP" in the figure), a destination accumulator operand (e.g., shown as "DESTINATION MATRIX (TILE)" in the figure), and two source operands (e.g., shown as "FIRST SOURCE MATRIX (TILE)" and "SECOND SOURCE MATRIX (TILE)" in the figure). In an embodiment, the destination accumulator operand is used to accumulate the data resulting from performing dot product operations on elements of the first and second source matrix (tile) operands. An example destination matrix (tile) operand 7801 is shown in Figure 78, initially storing a matrix of doubleword-sized data elements.

[1032] The two source matrix (tile) operand fields represent a first source matrix (tile) operand 7803 and a second source matrix (tile) operand 7805, respectively. As detailed earlier, a matrix (tile) may be stored in a collection of registers, locations in memory (for example, as strided rows), or in storage accessible to execution circuitry.

[1033] In Figure 78, each of the destination matrix (tile) accumulator operand 7801, the first source matrix (tile) operand 7803, and the second source matrix (tile) operand 7805 comprises a 2 x 2 matrix of data elements. The dimensions of the matrices in Figure 78 are used for illustrative purposes only; in general, a TILEDOTPRODUCT instruction can operate on any two source matrix (tile) operands where the number of columns associated with a first matrix (tile) operand is the same as the number of rows of a second matrix (tile) operand (that is, where the dimensions of a first matrix (tile) operand is *M* rows x *K* columns and the dimensions of a second matrix (tile) operand is *K* rows x *N* columns, as shown in Figure 78). The destination matrix (tile) accumulator operand in this example has *M* rows x *N* columns such that the number of rows in the destination matrix (tile) is the same as the number of

rows in the first matrix (tile) operand and the number of columns in the destination matrix (tile) is the same as the number of columns in the second matrix (tile) operand.

[1034] As shown, execution circuitry 7807 uses a grid of fused multiply adders (FMAs) 7809 to execute a decoded TILEDOTPRODUCT instruction by performing dot product operations on elements of the two source matrix (tile) operands 7803 and 7805 and accumulating the result into corresponding data element positions of the destination matrix (tile) accumulator operand 7801.

[1035] Referring to the example destination matrix (tile) accumulator operand 7801 and source matrix (tile) operands 7803 and 7805, the execution circuitry 7807 generates dot product values using the first row of the first source matrix (tile) operand 'TDP103 and the first column of the second source matrix (tile) operand 7805 and accumulates the result in the [0,0] data element position of the destination matrix (tile) operand 7801. In Figure 78, for example, the [0,0] data element position of the destination matrix (tile) operand 'TDP101 accumulates the initially stored value W with dot product values computed using the first row of the first source matrix (tile) operand 7803 (the elements [A,B] and [C,D]) and the first column of the second source matrix (tile) operand 7805 (the elements [I,J] and [M,N]), that is, W+DP([A,B], [I,J]) + DP ([C,D], [M,N]).

[1036] The execution circuitry 7807 further computes dot product values using the first row of the first source matrix (tile) operand 7803 and the second column of the second source matrix (tile) operand 7805 and accumulates the result in the [0,1] data element position of the destination matrix (tile) operand 7801. The execution circuitry 7807 further generates dot product values using the second row of the first source matrix (tile) operand 7803 and the first column of the second matrix (tile) operand 7805 and accumulates the result in the [1,0] data element position of the destination matrix (tile) operand 7801. The execution circuitry 7807 further generates dot product values using the second row of the first source matrix (tile) operand 7803 and the second column of the second source matrix (tile) operand 7805 and accumulates the result in the [1,1] data element position of the destination matrix (tile) operand 7801.

[1037] *ii. Exemplary Format(s)*

[1038] One embodiment of a format for a TILEDOTPRODUCT instruction is TDPWSSDS TMM1, TMM2, TMM3. In some embodiments, TDPWSSDS is the opcode mnemonic of the instruction, where the "TDP" part of the mnemonic indicates a TILEDOTPRODUCT operation and the "WSSDS" part of the mnemonic indicates that the instruction computes the dot product of source matrix (tile) operands comprising signed

word-sized elements and accumulates the result into a destination matrix (tile) comprising doubleword-sized elements with saturation. In this instruction format and the instruction formats below, TMM1 is a field for the destination matrix (tile) operand. TMM2 and TMM3 are fields for the matrix (tile) source operands. In some embodiments, the TMM3 field is a R/M value (such as 8246), the TMM1 field is REG 8244, and the data element size is found in 8265.

[1039] Another embodiment of a format for a TILEDOTPRODUCT instruction is TDPWSSQS TMM1, TMM2, TMM3. In some embodiments, TDPWSSQS is the opcode mnemonic of the instruction, where the "WSSQS" part of the mnemonic indicates that the instruction computes the dot product of source matrix (tile) operands comprising signed word-sized elements and accumulates the result into a destination matrix (tile) comprising quadword-sized elements with saturation.

[1040] Another embodiment of a format for a TILEDOTPRODUCT instruction is TDPB[SS/UU/US/SU]DS TMM1, TMM2, TMM3. In some embodiments, TDPB[SS/UU/US/SU]DS is the opcode mnemonic of the instruction, where the "B" and "D" parts of the mnemonic indicate that the instruction computes the dot product of source matrix (tile) operands comprising byte-sized elements and accumulates the result into a destination matrix (tile) accumulator operand comprising doubleword-sized elements with saturation.

[1041] In an embodiment, the [SS/UU/US/SU] part of the mnemonic in the instruction above, and similarly in the instructions below, indicates whether each of the two source matrix (tile) operands comprises signed or unsigned data values. The first letter of the [SS/UU/US/SU] mnemonic part corresponds to the first source matrix (tile) operand and the second letter corresponds to the second source matrix (tile) operand. For example, "SS" indicates that both source matrix (tile) operands are signed, "UU" indicates that both source matrix (tile) operands are unsigned, "US" indicates that the first source matrix (tile) operand is unsigned and the second source matrix (tile) operand is signed, and "SU" indicates that the first source matrix (tile) operand is signed and the second source matrix (tile) operand is unsigned. The destination matrix (tile) operand is signed if either of the first source matrix (tile) operand or second source matrix (tile) is signed; otherwise, the destination matrix (tile) is unsigned when both source matrix (tile) operands are unsigned. If either of the source matrix (tile) operands is signed, the result output saturation is signed saturation; otherwise, the result output saturation is unsigned saturation.

[1042] Another embodiment of a format for a TILEDOTPRODUCT instruction is TDP8B[SS/UU/US/SU]4BITDS TMM1, TMM2, TMM3. In some embodiments,

TDP8B[SS/UU/US/SU]4BITDS is the opcode mnemonic of the instruction, where the "8B" and "4BITD" identifiers indicate that the instruction computes the dot product of doubleword source matrix (tile) operands, one operand containing byte-sized elements and the other operand containing 4-bit (nibble) sized elements and accumulates the result into a destination matrix (tile) comprising doubleword-sized elements.

[1043] Another embodiment of a format for a TILEDOTPRODUCT instruction is TDP4BIT[S,U][S,U]DS TMM1, TMM2, TMM3. In some embodiments, TDP4BIT[S,U][S,U]DS is the opcode mnemonic of the instruction, where the "4BIT" and "D" parts of the mnemonic indicate that the instruction computes the dot product of doubleword source matrix (tile) operands, each source matrix (tile) operand comprising 4-bit (nibble) sized elements and accumulates the result into a destination matrix (tile) comprising doubleword-sized elements.

[1044] In embodiments, encodings of the instruction include a scale-index-base (SIB) type memory addressing operand that indirectly identifies multiple indexed destination locations in memory. In one embodiment, an SIB type memory operand may include an encoding identifying a base address register. The contents of the base address register may represent a base address in memory from which the addresses of the particular destination locations in memory are calculated. For example, the base address may be the address of the first location in a block of potential destination locations for an extended vector instruction. In one embodiment, an SIB type memory operand may include an encoding identifying an index register. Each element of the index register may specify an index or offset value usable to compute, from the base address, an address of a respective destination location within a block of potential destination locations. In one embodiment, an SIB type memory operand may include an encoding specifying a scaling factor to be applied to each index value when computing a respective destination address. For example, if a scaling factor value of four is encoded in the SIB type memory operand, each index value obtained from an element of the index register may be multiplied by four and then added to the base address to compute a destination address.

[1045] In one embodiment, an SIB type memory operand of the form vm32{x,y,z} may identify a vector array of memory operands specified using SIB type memory addressing. In this example, the array of memory addresses is specified using a common base register, a constant scaling factor, and a vector index register containing individual elements, each of which is a 32-bit index value. The vector index register may be a 128-bit register (e.g., XMM) register (vm32x), a 256-bit (e.g., YMM) register (vm32y), or a 512-bit (e.g., ZMM)

register (vm32z). In another embodiment, an SIB type memory operand of the form vm64{x,y,z} may identify a vector array of memory operands specified using SIB type memory addressing. In this example, the array of memory addresses is specified using a common base register, a constant scaling factor, and a vector index register containing individual elements, each of which is a 64-bit index value. The vector index register may be a 128-bit register (e.g., XMM) register (vm64x), a 256-bit (e.g., YMM) register (vm64y) or a 512-bit (e.g., ZMM) register (vm64z).

[1046] *iii. Exemplary Method(s) of Execution*

[1047] Figure 79 illustrates an embodiment of method performed by a processor to process a matrix (tile) dot product instruction.

[1048] At 7901, an instruction is fetched. For example, a TILEDOTPRODUCT instruction is fetched. The TILEDOTPRODUCT instruction includes fields for an opcode, a first and a second source matrix (tile) operand, and a destination matrix (tile) operand. In some embodiments, the instruction further includes a field for a writemask. In some embodiments, the instruction is fetched from an instruction cache. The source operands and destination operand consist of packed data. The opcode of the TILEDOTPRODUCT instruction indicates that a dot product operation is to be performed on the source matrix (tile) operands. In some embodiments, the opcode further indicates whether each of the first source matrix (tile) operand and second source matrix (tile) operand consist signed or unsigned values. In some embodiments, the opcode further indicates a size (for example, a specified number of bits, bytes, quadwords, doublewords, and so forth) of the matrix (tile) data values comprising each of the first source matrix (tile) operand, the second source matrix (tile) operand, and the destination matrix (tile) operand.

[1049] The fetched instruction is decoded at 7903. For example, the fetched TILEDOTPRODUCT instruction is decoded by decode circuitry such as that detailed herein.

[1050] Data values associated with the source matrix (tile) operands of the decoded instruction are retrieved at 7905 and the decoded instruction is scheduled (as needed). For example, when one or more of the source matrix (tile) operands are memory operands, the data from the indicated memory location is retrieved.

[1051] At 7907, the decoded instruction is executed by execution circuitry (hardware) such as that detailed herein. For the TILEDOTPRODUCT instruction, the execution causes execution circuitry to perform a dot product operation on source data. In some embodiments, the execution of a decoded matrix dot product instruction causes an execution circuit to: compute a result by performing dot product operations on elements from the first source

matrix and the second source matrix operand; and accumulate the result into elements of the destination matrix operand.

[1052] In some embodiments, a fault is generated when one or more of the following is true: a number of columns associated with the first source matrix operand is different than a number of rows associated with the second source matrix operand; a number of rows associated with the destination matrix (tile) operand is different than a number of rows associated with the first source matrix (tile) operand; and a number of columns associated with the destination matrix (tile) operand is different than a number of columns associated with the second source matrix (tile) operand.

[1053] In some embodiments, the instruction is committed or retired at 7909.

[1054] Figure 80 illustrates additional detailed related to an example method performed by a processor to execute a TILEDOTPRODUCT instruction, where the instruction has fields for a first source matrix (tile) operand, a second source matrix (tile) operand, and a destination matrix (tile) accumulator operand.

[1055] At 8001, execution circuitry sets a first counter with the value zero. At 8002, it is determined whether the first counter is less than a number of configured rows of the destination matrix (tile) operand. If the first counter is not less than the number of configured rows of the destination matrix (tile) operand, the process ends.

[1056] At 8003, if the first counter is less than the number of configured rows of the destination matrix (tile) operand, a second counter is set with the value zero. At 8004, it is determined whether the second counter is less than a number of configured columns of a first source matrix (tile) operand. If not, the first counter is incremented at 8012 and the process returns to 8002.

[1057] At 8005, if the second counter is less than a number of configured columns of the first source matrix (tile) operand, a row from the destination matrix (tile) operand identified by the first counter is written to a temporary location. For example, if the first counter is currently set to zero, then row[first counter] identifies the first row of the destination matrix (tile) operand. Similarly, if the first counter is currently set to one, then row[first counter] identifies the second row of the destination matrix (tile) accumulator operand, and so forth. At 8006, a third counter is set with the value zero.

[1058] At 8007, it is determined whether the third counter is less than a number of configured columns of the destination matrix operand. If the third counter is not less than the number of configured columns of the destination matrix (tile) accumulator operand, at 8010, the data values stored at the temporary location are written to a row of the destination matrix

(tile) operand identified by the first counter (that is, the row[first counter] of the destination matrix(tile) operand). At 8011, the second counter is incremented and the process returns to 8004.

[1059] If the third counter is less than the number of configured columns of the destination matrix (tile) accumulator operand, at 8008, the execution circuitry performs a dot product operation involving data elements of the first source matrix (tile) operand at position row[first counter, second counter] and data elements of the second source matrix (tile) operand at position row[second counter, third counter] and accumulates the result at the element position[third counter] of the temporary location. In reference to Figure 78, and assuming each of the first counter, the second counter, and the third counter is currently set to zero, a dot product operation is performed at 8008 involving the data elements of the first source matrix (tile) operand 7802 at position row[0,0] (the element values [A,B]) and data elements of the second source matrix (tile) operand 7803 at position row[0,0] (the element values [I,J]) and accumulates the result at the element position[0] of the temporary location (currently storing the value W from the row[0,0] of the destination matrix (tile) accumulator operand 7801).

[1060] At 8009, the third counter is incremented and the process returns to 8007. The result of the process described in Figure 80 is the performance of dot product operations on elements from the first source matrix (tile) operand and the second source matrix (tile) operand and the accumulation of the results into elements of the destination matrix (tile) accumulator operand, as illustrated in the destination matrix (tile) operand 7801 of Figure 78.

[1061] Figures 81A-81G illustrate example methods for performing TILEDOTPRODUCT operations, as described above. For example, the steps shown in 8101 and 8103 illustrate an example process for performing a TILEDOTPRODUCT operation involving source matrices of signed word-sized elements accumulated into doubleword-sized elements with saturation (for example, based on an instruction of the example format TDPWSSDS TMM1, TMM2, TMM3). In particular, 8101 illustrates an example helper process DPWSS(c, x, y) used to perform a multiply and add operation on doubleword arguments. As shown in the accompanying process 8103, the multiply and add operation illustrated in 8101 is used as part of the dot product calculations performed on the rows and columns of the source matrix (tile) operands.

[1062] The steps of 8103 indicate that a fault is generated if any of the following is true: the matrix (tile) architecture is not currently configured; the number of columns in the first source matrix (tile) is different than the number of rows in the second source matrix (tile); the

number of rows in the destination matrix (tile) is different than the number of rows in the first source matrix (tile); the number of columns in the destination matrix (tile) is different than the number of columns in the second source matrix (tile); or the number of rows or the number of columns in the second source matrix (tile) exceeds a configured limit.

[1063] The process of 8103 further proceeds to iterate through the rows and columns of the destination matrix (tile) and source matrices (tiles). In particular, the example process computes a result by performing dot product operations on elements from a first source matrix (tile) operand ("tsrc1") and a second source matrix (tile) operand ("tsrc2") and accumulates the result into elements of the destination matrix (tile) accumulator operand ("tsrcdest").

[1064] The example processes shown in Figures 81B-81G illustrate processes performed to implement other example TILEDOTPRODUCT instruction formats. For example, the processes shown in 8105 and 8107 in Figures 81B-81C illustrate an example a helper function DPWSSQ(c, x, y) performing a multiply and add operation on quadword arguments. The example shown in 8107 illustrates a process for performing a TILEDOTPRODUCT operation involving matrices of signed word-sized elements accumulated into doubleword-sized elements (for example, based on an instruction in the format TDPWSSQS TMM1, TMM2, TMM3, as described above).

[1065] The processes shown in 8109 in Figure 81D illustrates an example a helper function DPBD(c, x, y) used to perform a multiply and add operation on doubleword arguments. The example shown in 8111 illustrates a process for performing a TILEDOTPRODUCT operation involving matrices of byte-sized elements accumulated into doubleword-sized elements (for example, based on an instruction in the format TDPB[SS,UU,US,SU]DS TMM1, TMM2, TMM3, as described above).

[1066] The processes shown in 8113 in Figures 81E-81F illustrates a process for performing a TILEDOTPRODUCT operation involving doubleword source matrix (tile) operands, where one operand contains byte-sized elements and the other operand contains 4-bit (nibble) sized elements and the result is accumulated into a destination matrix (tile) comprising doubleword-sized elements (for example, based on an instruction in the format TDP8B[SS,UU,US,SU]4BITDS TMM1, TMM2, TMM3, as described above).

[1067] The processes shown in 8115 in Figure 81G illustrates a process for performing a TILEDOTPRODUCT operation involving doubleword source matrix (tile) operands, where each operand contains 4-bit (nibble) sized elements and the result is accumulated into a destination matrix (tile) comprising doubleword-sized elements (for example, based on an

instruction in the format TDP4BIT[SS,UU,US,SU]DS TMM1, TMM2, TMM3, as described above).

[1068] *iv. Examples*

[1069] **Example 1** A processor comprising: decode circuitry to decode an instruction having fields for a first source matrix operand, a second source matrix operand, and a destination matrix operand; and execution circuitry to execute the decoded instruction to: compute a result by performing dot product operations on elements from the first source matrix operand and the second source matrix operand, and accumulate the result into data element positions of the destination matrix operand.

[1070] **Example 2** The processor of Example 1, wherein the elements of the first source matrix operand and the second source matrix operand are signed word elements, and wherein the elements of the destination matrix operand are signed doublewords.

[1071] **Example 3** The processor of Example 1, wherein the elements of the first source matrix operand and the second source matrix operand are signed word-sized elements, and wherein the elements of the destination matrix operand are signed quadword-sized elements.

[1072] **Example 4** The processor of Example 1, wherein the elements of the first source matrix operand and the second source matrix operand are byte-sized elements, and wherein the elements of the destination matrix operand are doubleword-sized elements.

[1073] **Example 5** The processor of Example 1, wherein the elements of the first source matrix operand are byte-sized elements and the elements of the second source matrix operand are 4-bit-sized elements, and wherein the elements of the destination matrix are doubleword-sized elements.

[1074] **Example 6** The processor of Example 1, wherein the elements of the first source matrix operand and the elements of the second source matrix operand are 4-bit-sized elements, and wherein the elements of the destination matrix are doubleword-sized elements.

[1075] **Example 7** The processor of any of Examples 1-6, wherein the result is computed with saturation.

[1076] **Example 8** The processor of any of Examples 1-7, wherein the execution circuitry comprises a plurality of fused-multiply adders.

[1077] **Example 9** The processor of any of Examples 1-8, wherein the instruction indicates that least one of the first source matrix operand the second source matrix operand contains unsigned data values.

[1078] **Example 10** The processor of any of Examples 1-9, wherein a fault is generated when the first source matrix operand has a number of columns that is different number than a number of rows the second source matrix operand.

[1079] **Example 11** The processor of any of Examples 1-10, wherein a fault is generated when a number of rows of the destination matrix operand is different than a number of rows of the first source matrix operand.

[1080] **Example 12** The processor of any of Examples 1-11, wherein a fault is generated when a number of columns of the destination matrix operand is different than a number of columns of the second source matrix operand.

[1081] **Example 13** A method comprising: decoding an instruction having fields for a first source matrix operand, a second source matrix operand, and a destination matrix operand; and executing the decoded instruction to: compute a result by performing dot product operations on elements from the first source matrix operand and the second source matrix operand, and accumulate the result into data element positions of the destination matrix operand.

[1082] **Example 14** The method of Example 13, wherein the elements of the first source matrix operand and the second source matrix operand are signed word elements, and wherein the elements of the destination matrix operand are signed doublewords.

[1083] **Example 15** The method of Example 13, wherein the elements of the first source matrix operand and the second source matrix operand are signed word-sized elements, and wherein the elements of the destination matrix operand are signed quadword-sized elements.

[1084] **Example 16** The method of Example 13, wherein the elements of the first source matrix operand and the second source matrix operand are byte-sized elements, and wherein the elements of the destination matrix operand are doubleword-sized elements.

[1085] **Example 17** The method of Example 13, wherein the elements of the first source matrix operand are byte-sized elements and the elements of the second source matrix operand are 4-bit-sized elements, and wherein the elements of the destination matrix are doubleword-sized elements.

[1086] **Example 18** The method of Example 13, wherein the elements of the first source matrix operand and the elements of the second source matrix operand are 4-bit-sized elements, and wherein the elements of the destination matrix are doubleword-sized elements.

[1087] **Example 19** The method of any of Examples 13-18, wherein the result is computed with saturation.

[1088] **Example 20** The method of any of Examples 13-19, wherein the execution circuitry comprises a plurality of fused-multiply adders.

[1089] **Example 21** The method of any of Examples 13-20, wherein the instruction indicates that least one of the first source matrix operand the second source matrix operand contains unsigned data values.

[1090] **Example 22** The method of any of Examples 13-21, wherein a fault is generated when the first source matrix operand has a number of columns that is different number than a number of rows the second source matrix operand.

[1091] **Example 23** The method of any of Examples 13-22, wherein a fault is generated when a number of rows of the destination matrix operand is different than a number of rows of the first source matrix operand.

[1092] **Example 24** The method of any of Examples 13-23, wherein a fault is generated when a number of columns of the destination matrix operand is different than a number of columns of the second source matrix operand.

[1093] **Example 25** provides a non-transitory machine-readable medium storing an instruction which when executed by a processor causes the processor to perform a method, the method comprising: decoding an instruction having fields for a first and a second packed data source operand, and a packed data destination operand; and executing the decoded instruction to: compute a result by performing dot product operations on elements from the first source matrix operand and the second source matrix operand, and accumulate the result into data element positions of the destination matrix operand.

[1094] **Example 26** The non-transitory machine-readable medium of Example 25, wherein the elements of the first source matrix operand and the second source matrix operand are signed word elements, and wherein the elements of the destination matrix operand are signed doublewords.

[1095] **Example 27** The non-transitory machine-readable medium of Example 25, wherein the elements of the first source matrix operand and the second source matrix operand are signed word-sized elements, and wherein the elements of the destination matrix operand are signed quadword-sized elements.

[1096] **Example 28** The non-transitory machine-readable medium of Example 25, wherein the elements of the first source matrix operand and the second source matrix operand are byte-sized elements, and wherein the elements of the destination matrix operand are doubleword-sized elements.

[1097] **Example 29** The non-transitory machine-readable medium of Example 25, wherein the elements of the first source matrix operand are byte-sized elements and the elements of the second source matrix operand are 4-bit-sized elements, and wherein the elements of the destination matrix are doubleword-sized elements.

[1098] **Example 30** The non-transitory machine-readable medium of Example 25, wherein the elements of the first source matrix operand and the elements of the second source matrix operand are 4-bit-sized elements, and wherein the elements of the destination matrix are doubleword-sized elements.

[1099] **Example 31** The non-transitory machine-readable medium of any of Examples 25-30, wherein the result is computed with saturation.

[1100] **Example 32** The non-transitory machine-readable medium of any of Examples 25-31, wherein the execution circuitry comprises a plurality of fused-multiply adders.

[1101] **Example 33** The non-transitory machine-readable medium of any of Examples 25-32, wherein the instruction indicates that least one of the first source matrix operand the second source matrix operand contains unsigned data values.

[1102] **Example 34** The non-transitory machine-readable medium of any of Examples 25-33, wherein a fault is generated when the first source matrix operand has a number of columns that is different number than a number of rows the second source matrix operand.

[1103] **Example 35** The non-transitory machine-readable medium of any of Examples 25-34, wherein a fault is generated when a number of rows of the destination matrix operand is different than a number of rows of the first source matrix operand.

[1104] **Example 36** The non-transitory machine-readable medium of any of Examples 25-35, wherein a fault is generated when a number of columns of the destination matrix operand is different than a number of columns of the second source matrix operand.

[1105] **Example 37** provides a system comprising: a processor; and an accelerator coupled to the processor, the accelerator including: decode circuitry to decode an instruction having fields for a first source matrix operand, a second source matrix operand, and a destination matrix operand; and execution circuitry to execute the decoded instruction to: compute a result by performing dot product operations on elements from the first source matrix operand and the second source matrix operand, and accumulate the result into data element positions of the destination matrix operand.

[1106] **Example 38** The system of Example 37, wherein the elements of the first source matrix operand and the second source matrix operand are signed word elements, and wherein the elements of the destination matrix operand are signed doublewords.

[1107] **Example 39** The system of Example 37, wherein the elements of the first source matrix operand and the second source matrix operand are signed word-sized elements, and wherein the elements of the destination matrix operand are signed quadword-sized elements.

[1108] **Example 40** The system of Example 37, wherein the elements of the first source matrix operand and the second source matrix operand are byte-sized elements, and wherein the elements of the destination matrix operand are doubleword-sized elements.

[1109] **Example 41** The system of Example 37, wherein the elements of the first source matrix operand are byte-sized elements and the elements of the second source matrix operand are 4-bit-sized elements, and wherein the elements of the destination matrix are doubleword-sized elements.

[1110] **Example 42** The system of Example 37, wherein the elements of the first source matrix operand and the elements of the second source matrix operand are 4-bit-sized elements, and wherein the elements of the destination matrix are doubleword-sized elements.

[1111] **Example 43** The system of any of Examples 37-42, wherein the result is computed with saturation.

[1112] **Example 44** The system of any of Examples 37-43, wherein the execution circuitry comprises a plurality of fused-multiply adders.

[1113] **Example 45** The system of any of Examples 37-44, wherein the instruction indicates that least one of the first source matrix operand the second source matrix operand contains unsigned data values.

[1114] **Example 46** The system of any of Examples 37-45, wherein a fault is generated when the first source matrix operand has a number of columns that is different number than a number of rows the second source matrix operand.

[1115] **Example 47** The system of any of Examples 37-46, wherein a fault is generated when a number of rows of the destination matrix operand is different than a number of rows of the first source matrix operand.

[1116] **Example 48** The system of any of Examples 37-47, wherein a fault is generated when a number of columns of the destination matrix operand is different than a number of columns of the second source matrix operand.

[1117] V. Detailed Exemplary Systems, Processors, and Emulation

[1118] Detailed herein are examples of hardware, software, etc. to execute the above described instructions. For example, what is described below details aspects of instruction execution including various pipeline stages such as fetch, decode, schedule, execute, retire, etc.

[1119] An instruction set includes one or more instruction formats. A given instruction format defines various fields (number of bits, location of bits) to specify, among other things, the operation to be performed (opcode) and the operand(s) on which that operation is to be performed. Some instruction formats are further broken down though the definition of instruction templates (or subformats). For example, the instruction templates of a given instruction format may be defined to have different subsets of the instruction format's fields (the included fields are typically in the same order, but at least some have different bit positions because there are less fields included) and/or defined to have a given field interpreted differently. Thus, each instruction of an ISA is expressed using a given instruction format (and, if defined, in a given one of the instruction templates of that instruction format) and includes fields for specifying the operation and the operands. For example, an exemplary ADD instruction has a specific opcode and an instruction format that includes an opcode field to specify that opcode and operand fields to select operands (source1/destination and source2); and an occurrence of this ADD instruction in an instruction stream will have specific contents in the operand fields that select specific operands.

[1120] A. Exemplary Instruction Formats

[1121] Embodiments of the instruction(s) described herein may be embodied in different formats. Additionally, exemplary systems, architectures, and pipelines are detailed below. Embodiments of the instruction(s) may be executed on such systems, architectures, and pipelines, but are not limited to those detailed.

[1122] VEX Instruction Format

[1123] VEX encoding allows instructions to have more than two operands, and allows SIMD vector registers to be longer than 128 bits. The use of a VEX prefix provides for three-operand (or more) syntax. For example, previous two-operand instructions performed operations such as A = A + B, which overwrites a source operand. The use of a VEX prefix enables operands to perform nondestructive operations such as A = B + C.

[1124] Figure 82A illustrates an exemplary instruction format including a VEX prefix 8202, real opcode field 8230, Mod R/M byte 8240, SIB byte 8250, displacement field 8262, and IMM8 8272. Figure 82B illustrates which fields from Figure 82A make up a full opcode field 8274 and a base operation field 8241. Figure 82C illustrates which fields from Figure 82A make up a register index field 8244.

[1125] VEX Prefix (Bytes 0-2) 8202 is encoded in a three-byte form. The first byte is the Format Field 8290 (VEX Byte 0, bits [7:0]), which contains an explicit C4 byte value (the

unique value used for distinguishing the C4 instruction format). The second-third bytes (VEX Bytes 1-2) include a number of bit fields providing specific capability. Specifically, REX field 8205 (VEX Byte 1, bits [7-5]) consists of a VEX.R bit field (VEX Byte 1, bit [7] – R), VEX.X bit field (VEX byte 1, bit [6] – X), and VEX.B bit field (VEX byte 1, bit[5] – B). Other fields of the instructions encode the lower three bits of the register indexes as is known in the art (rrr, xxx, and bbb), so that Rrrr, Xxxx, and Bbbb may be formed by adding VEX.R, VEX.X, and VEX.B. Opcode map field 8215 (VEX byte 1, bits [4:0] – mmmmm) includes content to encode an implied leading opcode byte. W Field 8264 (VEX byte 2, bit [7] – W) - is represented by the notation VEX.W, and provides different functions depending on the instruction. The role of VEX.vvvv 8220 (VEX Byte 2, bits [6:3]-vvvv) may include the following: 1) VEX.vvvv encodes the first source register operand, specified in inverted (1s complement) form and is valid for instructions with 2 or more source operands; 2) VEX.vvvv encodes the destination register operand, specified in 1s complement form for certain vector shifts; or 3) VEX.vvvv does not encode any operand, the field is reserved and should contain 1111b. If VEX.L 8268 Size field (VEX byte 2, bit [2]-L) = 0, it indicates 128 bit vector; if VEX.L = 1, it indicates 256 bit vector. Prefix encoding field 8225 (VEX byte 2, bits [1:0]-pp) provides additional bits for the base operation field 8241.

[1126] Real Opcode Field 8230 (Byte 3) is also known as the opcode byte. Part of the opcode is specified in this field.

[1127] MOD R/M Field 8240 (Byte 4) includes MOD field 8242 (bits [7-6]), Reg field 8244 (bits [5-3]), and R/M field 8246 (bits [2-0]). The role of Reg field 8244 may include the following: encoding either the destination register operand or a source register operand (the rrr of Rrrr), or be treated as an opcode extension and not used to encode any instruction operand. The role of R/M field 8246 may include the following: encoding the instruction operand that references a memory address, or encoding either the destination register operand or a source register operand.

[1128] Scale, Index, Base (SIB) - The content of Scale field 8250 (Byte 5) includes SS8252 (bits [7-6]), which is used for memory address generation. The contents of SIB.xxx 8254 (bits [5-3]) and SIB.bbb 8256 (bits [2-0]) have been previously referred to with regard to the register indexes Xxxx and Bbbb.

[1129] The Displacement Field 8262 and the immediate field (IMM8) 8272 contain data.

[1130] B. Exemplary Register Architecture

[1131] Figure 83 is a block diagram of a register architecture 8300 according to one embodiment of the invention. In the embodiment illustrated, there are 32 vector registers

8310 that are 512 bits wide; these registers are referenced as zmm0 through zmm31. The lower order 256 bits of the lower 86 zmm registers are overlaid on registers ymm0-15. The lower order 128 bits of the lower 86 zmm registers (the lower order 128 bits of the ymm registers) are overlaid on registers xmm0-15.

[1132] General-purpose registers 8325 - in the embodiment illustrated, there are sixteen 64-bit general-purpose registers that are used along with the existing x86 addressing modes to address memory operands. These registers are referenced by the names RAX, RBX, RCX, RDX, RBP, RSI, RDI, RSP, and R8 through R15.

[1133] Scalar floating point stack register file (x87 stack) 8345, on which is aliased the MMX packed integer flat register file 8350 - in the embodiment illustrated, the x87 stack is an eight-element stack used to perform scalar floating-point operations on 32/64/80-bit floating point data using the x87 instruction set extension; while the MMX registers are used to perform operations on 64-bit packed integer data, as well as to hold operands for some operations performed between the MMX and XMM registers.

[1134] In some embodiments, tiles 8320 are supported using an overlay over physical registers. For example, a tile may utilize 16 1,024-bit registers, 32 512-bit registers, etc. depending on the implementation.

[1135] Alternative embodiments of the invention may use wider or narrower registers. Additionally, alternative embodiments of the invention may use more, less, or different register files and registers.

[1136] Exemplary Core Architectures, Processors, and Computer Architectures

[1137] Processor cores may be implemented in different ways, for different purposes, and in different processors. For instance, implementations of such cores may include: 1) a general purpose in-order core intended for general-purpose computing; 2) a high performance general purpose out-of-order core intended for general-purpose computing; 3) a special purpose core intended primarily for graphics and/or scientific (throughput) computing. Implementations of different processors may include: 1) a CPU including one or more general purpose in-order cores intended for general-purpose computing and/or one or more general purpose out-of-order cores intended for general-purpose computing; and 2) a coprocessor including one or more special purpose cores intended primarily for graphics and/or scientific (throughput). Such different processors lead to different computer system architectures, which may include: 1) the coprocessor on a separate chip from the CPU; 2) the coprocessor on a separate die in the same package as a CPU; 3) the coprocessor on the same die as a CPU (in which case, such a coprocessor is sometimes referred to as special purpose

logic, such as integrated graphics and/or scientific (throughput) logic, or as special purpose cores); and 4) a system on a chip that may include on the same die the described CPU (sometimes referred to as the application core(s) or application processor(s)), the above described coprocessor, and additional functionality. Exemplary core architectures are described next, followed by descriptions of exemplary processors and computer architectures. Detailed herein are circuits (units) that comprise exemplary cores, processors, etc.

[1138] C. Exemplary Core Architectures

[1139] In-order and out-of-order core block diagram

[1140] Figure 84A is a block diagram illustrating both an exemplary in-order pipeline and an exemplary register renaming, out-of-order issue/execution pipeline according to embodiments of the invention. Figure 84B is a block diagram illustrating both an exemplary embodiment of an in-order architecture core and an exemplary register renaming, out-of-order issue/execution architecture core to be included in a processor according to embodiments of the invention. The solid lined boxes in Figures 84A-B illustrate the in-order pipeline and in-order core, while the optional addition of the dashed lined boxes illustrates the register renaming, out-of-order issue/execution pipeline and core. Given that the in-order aspect is a subset of the out-of-order aspect, the out-of-order aspect will be described.

[1141] In Figure 84A, a processor pipeline 8400 includes a fetch stage 8402, a length decode stage 8404, a decode stage 8406, an allocation stage 8408, a renaming stage 8410, a scheduling (also known as a dispatch or issue) stage 8412, a register read/memory read stage 8414, an execute stage 8416, a write back/memory write stage 8418, an exception handling stage 8422, and a commit stage 8424.

[1142] Figure 84B shows processor core 8490 including a front end unit 8430 coupled to an execution engine unit 8450, and both are coupled to a memory unit 8470. The core 8490 may be a reduced instruction set computing (RISC) core, a complex instruction set computing (CISC) core, a very long instruction word (VLIW) core, or a hybrid or alternative core type. As yet another option, the core 8490 may be a special-purpose core, such as, for example, a network or communication core, compression engine, coprocessor core, general purpose computing graphics processing unit (GPGPU) core, graphics core, or the like.

[1143] The front end unit 8430 includes a branch prediction unit 8432 coupled to an instruction cache unit 8434, which is coupled to an instruction translation lookaside buffer (TLB) 8436, which is coupled to an instruction fetch unit 8438, which is coupled to a decode unit 8440. The decode unit 8440 (or decoder) may decode instructions, and generate as an output one or more micro-operations, micro-code entry points, microinstructions, other

instructions, or other control signals, which are decoded from, or which otherwise reflect, or are derived from, the original instructions. The decode unit 8440 may be implemented using various different mechanisms. Examples of suitable mechanisms include, but are not limited to, look-up tables, hardware implementations, programmable logic arrays (PLAs), microcode read only memories (ROMs), etc. In one embodiment, the core 8490 includes a microcode ROM or other medium that stores microcode for certain macroinstructions (e.g., in decode unit 8440 or otherwise within the front end unit 8430). The decode unit 8440 is coupled to a rename/allocator unit 8452 in the execution engine unit 8450.

[1144] The execution engine unit 8450 includes the rename/allocator unit 8452 coupled to a retirement unit 8454 and a set of one or more scheduler unit(s) 8456. The scheduler unit(s) 8456 represents any number of different schedulers, including reservations stations, central instruction window, etc. The scheduler unit(s) 8456 is coupled to the physical register file(s) unit(s) 8458. Each of the physical register file(s) units 8458 represents one or more physical register files, different ones of which store one or more different data types, such as scalar integer, scalar floating point, packed integer, packed floating point, vector integer, vector floating point, status (e.g., an instruction pointer that is the address of the next instruction to be executed), etc. In one embodiment, the physical register file(s) unit 8458 comprises a vector registers unit and a scalar registers unit. These register units may provide architectural vector registers, vector mask registers, and general purpose registers. The physical register file(s) unit(s) 8458 is overlapped by the retirement unit 8454 to illustrate various ways in which register renaming and out-of-order execution may be implemented (e.g., using a reorder buffer(s) and a retirement register file(s); using a future file(s), a history buffer(s), and a retirement register file(s); using a register maps and a pool of registers; etc.). The retirement unit 8454 and the physical register file(s) unit(s) 8458 are coupled to the execution cluster(s) 8460. The execution cluster(s) 8460 includes a set of one or more execution units 8462 and a set of one or more memory access units 8464. The execution units 8462 may perform various operations (e.g., shifts, addition, subtraction, multiplication) and on various types of data (e.g., scalar floating point, packed integer, packed floating point, vector integer, vector floating point). While some embodiments may include a number of execution units dedicated to specific functions or sets of functions, other embodiments may include only one execution unit or multiple execution units that all perform all functions. The scheduler unit(s) 8456, physical register file(s) unit(s) 8458, and execution cluster(s) 8460 are shown as being possibly plural because certain embodiments create separate pipelines for certain types of data/operations (e.g., a scalar integer pipeline, a scalar floating point/packed

integer/packed floating point/vector integer/vector floating point pipeline, and/or a memory access pipeline that each have their own scheduler unit, physical register file(s) unit, and/or execution cluster – and in the case of a separate memory access pipeline, certain embodiments are implemented in which only the execution cluster of this pipeline has the memory access unit(s) 8464). It should also be understood that where separate pipelines are used, one or more of these pipelines may be out-of-order issue/execution and the rest in-order.

[1145] The set of memory access units 8464 is coupled to the memory unit 8470, which includes a data TLB unit 8472 coupled to a data cache unit 8474 coupled to a level 2 (L2) cache unit 8476. In one exemplary embodiment, the memory access units 8464 may include a load unit, a store address unit, and a store data unit, each of which is coupled to the data TLB unit 8472 in the memory unit 8470. The instruction cache unit 8434 is further coupled to a level 2 (L2) cache unit 8476 in the memory unit 8470. The L2 cache unit 8476 is coupled to one or more other levels of cache and eventually to a main memory.

[1146] By way of example, the exemplary register renaming, out-of-order issue/execution core architecture may implement the pipeline 8400 as follows: 1) the instruction fetch 8438 performs the fetch and length decoding stages 8402 and 8404; 2) the decode unit 8440 performs the decode stage 8406; 3) the rename/allocator unit 8452 performs the allocation stage 8408 and renaming stage 8410; 4) the scheduler unit(s) 8456 performs the schedule stage 8412; 5) the physical register file(s) unit(s) 8458 and the memory unit 8470 perform the register read/memory read stage 8414; the execution cluster 8460 perform the execute stage 8416; 6) the memory unit 8470 and the physical register file(s) unit(s) 8458 perform the write back/memory write stage 8418; 7) various units may be involved in the exception handling stage 8422; and 8) the retirement unit 8454 and the physical register file(s) unit(s) 8458 perform the commit stage 8424.

[1147] The core 8490 may support one or more instructions sets (e.g., the x86 instruction set (with some extensions that have been added with newer versions); the MIPS instruction set of MIPS Technologies of Sunnyvale, CA; the ARM instruction set (with optional additional extensions such as NEON) of ARM Holdings of Sunnyvale, CA), including the instruction(s) described herein. In one embodiment, the core 8490 includes logic to support a packed data instruction set extension (e.g., AVX1, AVX2), thereby allowing the operations used by many multimedia applications to be performed using packed data.

[1148] It should be understood that the core may support multithreading (executing two or more parallel sets of operations or threads), and may do so in a variety of ways including time

sliced multithreading, simultaneous multithreading (where a single physical core provides a logical core for each of the threads that physical core is simultaneously multithreading), or a combination thereof (e.g., time sliced fetching and decoding and simultaneous multithreading thereafter such as in the Intel® Hyperthreading technology).

[1149] While register renaming is described in the context of out-of-order execution, it should be understood that register renaming may be used in an in-order architecture. While the illustrated embodiment of the processor also includes separate instruction and data cache units 8434/8474 and a shared L2 cache unit 8476, alternative embodiments may have a single internal cache for both instructions and data, such as, for example, a Level 1 (L1) internal cache, or multiple levels of internal cache. In some embodiments, the system may include a combination of an internal cache and an external cache that is external to the core and/or the processor. Alternatively, all of the cache may be external to the core and/or the processor.

[1150] Specific Exemplary In-Order Core Architecture

[1151] Figures 85A-B illustrate a block diagram of a more specific exemplary in-order core architecture, which core would be one of several logic blocks (including other cores of the same type and/or different types) in a chip. The logic blocks communicate through a high-bandwidth interconnect network (e.g., a ring network) with some fixed function logic, memory I/O interfaces, and other necessary I/O logic, depending on the application.

[1152] Figure 85A is a block diagram of a single processor core, along with its connection to the on-die interconnect network 8502 and with its local subset of the Level 2 (L2) cache 8504, according to embodiments of the invention. In one embodiment, an instruction decoder 8500 supports the x86 instruction set with a packed data instruction set extension. An L1 cache 8506 allows low-latency accesses to cache memory into the scalar and vector units. While in one embodiment (to simplify the design), a scalar unit 8508 and a vector unit 8510 use separate register sets (respectively, scalar registers 8512 and vector registers 8514) and data transferred between them is written to memory and then read back in from a level 1 (L1) cache 8506, alternative embodiments of the invention may use a different approach (e.g., use a single register set or include a communication path that allow data to be transferred between the two register files without being written and read back).

[1153] The local subset of the L2 cache 8504 is part of a global L2 cache that is divided into separate local subsets, one per processor core. Each processor core has a direct access path to its own local subset of the L2 cache 8504. Data read by a processor core is stored in its L2 cache subset 8504 and can be accessed quickly, in parallel with other processor cores accessing their own local L2 cache subsets. Data written by a processor core is stored in its

own L2 cache subset 8504 and is flushed from other subsets, if necessary. The ring network ensures coherency for shared data. The ring network is bi-directional to allow agents such as processor cores, L2 caches and other logic blocks to communicate with each other within the chip. Each ring data-path is 1024-bits wide per direction in some embodiments.

[1154] Figure 85B is an expanded view of part of the processor core in Figure 85A according to embodiments of the invention. Figure 85B includes an L1 data cache 8506A part of the L1 cache 8504, as well as more detail regarding the vector unit 8510 and the vector registers 8514. Specifically, the vector unit 8510 is a 86-wide vector processing unit (VPU) (see the 16-wide ALU 8528), which executes one or more of integer, single-precision float, and double-precision float instructions. The VPU supports swizzling the register inputs with swizzle unit 8520, numeric conversion with numeric convert units 8522A-B, and replication with replication unit 8524 on the memory input.

[1155] Processor with integrated memory controller and graphics

[1156] Figure 86 is a block diagram of a processor 8600 that may have more than one core, may have an integrated memory controller, and may have integrated graphics according to embodiments of the invention. The solid lined boxes in Figure 86 illustrate a processor 8600 with a single core 8602A, a system agent 8610, a set of one or more bus controller units 8616, while the optional addition of the dashed lined boxes illustrates an alternative processor 8600 with multiple cores 8602A-N, a set of one or more integrated memory controller unit(s) 8614 in the system agent unit 8610, and special purpose logic 8608.

[1157] Thus, different implementations of the processor 8600 may include: 1) a CPU with the special purpose logic 8608 being integrated graphics and/or scientific (throughput) logic (which may include one or more cores), and the cores 8602A-N being one or more general purpose cores (e.g., general purpose in-order cores, general purpose out-of-order cores, a combination of the two); 2) a coprocessor with the cores 8602A-N being a large number of special purpose cores intended primarily for graphics and/or scientific (throughput); and 3) a coprocessor with the cores 8602A-N being a large number of general purpose in-order cores. Thus, the processor 8600 may be a general-purpose processor, coprocessor or special-purpose processor, such as, for example, a network or communication processor, compression engine, graphics processor, GPGPU (general purpose graphics processing unit), a high-throughput many integrated core (MIC) coprocessor (including 30 or more cores), embedded processor, or the like. The processor may be implemented on one or more chips. The processor 8600 may be a part of and/or may be implemented on one or more substrates using any of a number of process technologies, such as, for example, BiCMOS, CMOS, or NMOS.

[1158] The memory hierarchy includes one or more levels of cache within the cores 8604A-N, a set or one or more shared cache units 8606, and external memory (not shown) coupled to the set of integrated memory controller units 8614. The set of shared cache units 8606 may include one or more mid-level caches, such as level 2 (L2), level 3 (L3), level 4 (L4), or other levels of cache, a last level cache (LLC), and/or combinations thereof. While in one embodiment a ring based interconnect unit 8612 interconnects the integrated graphics logic 8608, the set of shared cache units 8606, and the system agent unit 8610/integrated memory controller unit(s) 8614, alternative embodiments may use any number of well-known techniques for interconnecting such units. In one embodiment, coherency is maintained between one or more cache units 8606 and cores 8602-A-N.

[1159] In some embodiments, one or more of the cores 8602A-N are capable of multi-threading. The system agent 8610 includes those components coordinating and operating cores 8602A-N. The system agent unit 8610 may include for example a power control unit (PCU) and a display unit. The PCU may be or include logic and components needed for regulating the power state of the cores 8602A-N and the integrated graphics logic 8608. The display unit is for driving one or more externally connected displays.

[1160] The cores 8602A-N may be homogenous or heterogeneous in terms of architecture instruction set; that is, two or more of the cores 8602A-N may be capable of execution the same instruction set, while others may be capable of executing only a subset of that instruction set or a different instruction set.

[1161] D. Exemplary Computer Architectures

[1162] Figures 87-90 are block diagrams of exemplary computer architectures. Other system designs and configurations known in the arts for laptops, desktops, handheld PCs, personal digital assistants, engineering workstations, servers, network devices, network hubs, switches, embedded processors, digital signal processors (DSPs), graphics devices, video game devices, set-top boxes, micro controllers, cell phones, portable media players, hand held devices, and various other electronic devices, are also suitable. In general, a huge variety of systems or electronic devices capable of incorporating a processor and/or other execution logic as disclosed herein are generally suitable.

[1163] Referring now to Figure 87, shown is a block diagram of a system 8700 in accordance with one embodiment of the present invention. The system 8700 may include one or more processors 8710, 8715, which are coupled to a controller hub 8720. In one embodiment, the controller hub 8720 includes a graphics memory controller hub (GMCH) 8790 and an Input/Output Hub (IOH) 8750 (which may be on separate chips); the GMCH

8790 includes memory and graphics controllers to which are coupled memory 8740 and a coprocessor 8745; the IOH 8750 is couples input/output (I/O) devices 8760 to the GMCH 8790. Alternatively, one or both of the memory and graphics controllers are integrated within the processor (as described herein), the memory 8740 and the coprocessor 8745 are coupled directly to the processor 8710, and the controller hub 8720 in a single chip with the IOH 8750.

[1164] The optional nature of additional processors 8715 is denoted in Figure 87 with broken lines. Each processor 8710, 8715 may include one or more of the processing cores described herein and may be some version of the processor 8600.

[1165] The memory 8740 may be, for example, dynamic random access memory (DRAM), phase change memory (PCM), or a combination of the two. For at least one embodiment, the controller hub 8720 communicates with the processor(s) 8710, 8715 via a multi-drop bus, such as a frontside bus (FSB), point-to-point interface, or similar connection 8795.

[1166] In one embodiment, the coprocessor 8745 is a special-purpose processor, such as, for example, a high-throughput MIC processor, a network or communication processor, compression engine, graphics processor, GPGPU, embedded processor, or the like. In one embodiment, controller hub 8720 may include an integrated graphics accelerator.

[1167] There can be a variety of differences between the physical resources 8710, 87155 in terms of a spectrum of metrics of merit including architectural, microarchitectural, thermal, power consumption characteristics, and the like.

[1168] In one embodiment, the processor 8710 executes instructions that control data processing operations of a general type. Embedded within the instructions may be coprocessor instructions. The processor 8710 recognizes these coprocessor instructions as being of a type that should be executed by the attached coprocessor 8745. Accordingly, the processor 8710 issues these coprocessor instructions (or control signals representing coprocessor instructions) on a coprocessor bus or other interconnect, to coprocessor 8745. Coprocessor(s) 8745 accept and execute the received coprocessor instructions.

[1169] Referring now to Figure 88, shown is a block diagram of a first more specific exemplary system 8800 in accordance with an embodiment of the present invention. As shown in Figure 88, multiprocessor system 8800 is a point-to-point interconnect system, and includes a first processor 8870 and a second processor 8880 coupled via a point-to-point interconnect 8850. Each of processors 8870 and 8880 may be some version of the processor 8600. In one embodiment of the invention, processors 8870 and 8880 are respectively

processors 8710 and 8715, while coprocessor 8838 is coprocessor 8745. In another embodiment, processors 8870 and 8880 are respectively processor 8710 coprocessor 8745.

[1170] Processors 8870 and 8880 are shown including integrated memory controller (IMC) units 8872 and 8882, respectively. Processor 8870 also includes as part of its bus controller units point-to-point (P-P) interfaces 8876 and 8878; similarly, second processor 8880 includes P-P interfaces 8886 and 8888. Processors 8870, 8880 may exchange information via a point-to-point (P-P) interface 8850 using P-P interface circuits 8878, 8888. As shown in Figure 88, IMCs 8872 and 8882 couple the processors to respective memories, namely a memory 8832 and a memory 8834, which may be portions of main memory locally attached to the respective processors.

[1171] Processors 8870, 8880 may each exchange information with a chipset 8890 via individual P-P interfaces 8852, 8854 using point to point interface circuits 8876, 8894, 8886, 8898. Chipset 8890 may optionally exchange information with the coprocessor 8838 via a high-performance interface 8892. In one embodiment, the coprocessor 8838 is a special-purpose processor, such as, for example, a high-throughput MIC processor, a network or communication processor, compression engine, graphics processor, GPGPU, embedded processor, or the like.

[1172] A shared cache (not shown) may be included in either processor or outside of both processors, yet connected with the processors via P-P interconnect, such that either or both processors' local cache information may be stored in the shared cache if a processor is placed into a low power mode.

[1173] Chipset 8890 may be coupled to a first bus 8816 via an interface 8896. In one embodiment, first bus 8816 may be a Peripheral Component Interconnect (PCI) bus, or a bus such as a PCI Express bus or another I/O interconnect bus, although the scope of the present invention is not so limited.

[1174] As shown in Figure 88, various I/O devices 8814 may be coupled to first bus 8816, along with a bus bridge 8818 which couples first bus 8816 to a second bus 8820. In one embodiment, one or more additional processor(s) 8815, such as coprocessors, high-throughput MIC processors, GPGPU's, accelerators (such as, e.g., graphics accelerators or digital signal processing (DSP) units), field programmable gate arrays, or any other processor, are coupled to first bus 8816. In one embodiment, second bus 8820 may be a low pin count (LPC) bus. Various devices may be coupled to a second bus 8820 including, for example, a keyboard and/or mouse 8822, communication devices 8827 and a storage unit 8828 such as a disk drive or other mass storage device which may include instructions/code

and data 8830, in one embodiment. Further, an audio I/O 8824 may be coupled to the second bus 8816. Note that other architectures are possible. For example, instead of the point-to-point architecture of Figure 88, a system may implement a multi-drop bus or other such architecture.

[1175] Referring now to Figure 89, shown is a block diagram of a second more specific exemplary system 8900 in accordance with an embodiment of the present invention. Like elements in Figures 88 and 89 bear like reference numerals, and certain aspects of Figure 88 have been omitted from Figure 89 in order to avoid obscuring other aspects of Figure 89.

[1176] Figure 89 illustrates that the processors 8870, 8880 may include integrated memory and I/O control logic ("CL") 8972 and 8982, respectively. Thus, the CL 8972, 8982 include integrated memory controller units and include I/O control logic. Figure 89 illustrates that not only are the memories 8832, 8834 coupled to the CL 8972, 8982, but also that I/O devices 8914 are also coupled to the control logic 8872, 8882. Legacy I/O devices 8915 are coupled to the chipset 8890.

[1177] Referring now to Figure 90, shown is a block diagram of a SoC 9000 in accordance with an embodiment of the present invention. Similar elements in Figure 86 bear like reference numerals. Also, dashed lined boxes are optional features on more advanced SoCs. In Figure 90, an interconnect unit(s) 9002 is coupled to: an application processor 9010 which includes a set of one or more cores 902A-N, cache units 8604A-N, and shared cache unit(s) 8606; a system agent unit 8610; a bus controller unit(s) 8616; an integrated memory controller unit(s) 8614; a set or one or more coprocessors 9020 which may include integrated graphics logic, an image processor, an audio processor, and a video processor; an static random access memory (SRAM) unit 9030; a direct memory access (DMA) unit 9032; and a display unit 9040 for coupling to one or more external displays. In one embodiment, the coprocessor(s) 9020 include a special-purpose processor, such as, for example, a network or communication processor, compression engine, GPGPU, a high-throughput MIC processor, embedded processor, or the like.

[1178] Embodiments of the mechanisms disclosed herein may be implemented in hardware, software, firmware, or a combination of such implementation approaches. Embodiments of the invention may be implemented as computer programs or program code executing on programmable systems comprising at least one processor, a storage system (including volatile and non-volatile memory and/or storage elements), at least one input device, and at least one output device.

[1179] Program code, such as code 8830 illustrated in Figure 88, may be applied to input instructions to perform the functions described herein and generate output information. The output information may be applied to one or more output devices, in known fashion. For purposes of this application, a processing system includes any system that has a processor, such as, for example; a digital signal processor (DSP), a microcontroller, an application specific integrated circuit (ASIC), or a microprocessor.

[1180] The program code may be implemented in a high level procedural or object oriented programming language to communicate with a processing system. The program code may also be implemented in assembly or machine language, if desired. In fact, the mechanisms described herein are not limited in scope to any particular programming language. In any case, the language may be a compiled or interpreted language.

[1181] One or more aspects of at least one embodiment may be implemented by representative instructions stored on a machine-readable medium which represents various logic within the processor, which when read by a machine causes the machine to fabricate logic to perform the techniques described herein. Such representations, known as "IP cores" may be stored on a tangible, machine readable medium and supplied to various customers or manufacturing facilities to load into the fabrication machines that actually make the logic or processor.

[1182] Such machine-readable storage media may include, without limitation, non-transitory, tangible arrangements of articles manufactured or formed by a machine or device, including storage media such as hard disks, any other type of disk including floppy disks, optical disks, compact disk read-only memories (CD-ROMs), compact disk rewritables (CD-RWs), and magneto-optical disks, semiconductor devices such as read-only memories (ROMs), random access memories (RAMs) such as dynamic random access memories (DRAMs), static random access memories (SRAMs), erasable programmable read-only memories (EPROMs), flash memories, electrically erasable programmable read-only memories (EEPROMs), phase change memory (PCM), magnetic or optical cards, or any other type of media suitable for storing electronic instructions.

[1183] Accordingly, embodiments of the invention also include non-transitory, tangible machine-readable media containing instructions or containing design data, such as Hardware Description Language (HDL), which defines structures, circuits, apparatuses, processors and/or system features described herein. Such embodiments may also be referred to as program products.

[1184] E. Emulation (including binary translation, code morphing, etc.)

[1185] In some cases, an instruction converter may be used to convert an instruction from a source instruction set to a target instruction set. For example, the instruction converter may translate (e.g., using static binary translation, dynamic binary translation including dynamic compilation), morph, emulate, or otherwise convert an instruction to one or more other instructions to be processed by the core. The instruction converter may be implemented in software, hardware, firmware, or a combination thereof. The instruction converter may be on processor, off processor, or part on and part off processor.

[1186] Figure 91 is a block diagram contrasting the use of a software instruction converter to convert binary instructions in a source instruction set to binary instructions in a target instruction set according to embodiments of the invention. In the illustrated embodiment, the instruction converter is a software instruction converter, although alternatively the instruction converter may be implemented in software, firmware, hardware, or various combinations thereof. Figure 91 shows a program in a high level language 9102 may be compiled using a first compiler 9104 to generate a first binary code (e.g., x86) 9106 that may be natively executed by a processor with at least one first instruction set core 9116. In some embodiments, the processor with at least one first instruction set core 9116 represents any processor that can perform substantially the same functions as an Intel processor with at least one x86 instruction set core by compatibly executing or otherwise processing (1) a substantial portion of the instruction set of the Intel x86 instruction set core or (2) object code versions of applications or other software targeted to run on an Intel processor with at least one x86 instruction set core, in order to achieve substantially the same result as an Intel processor with at least one x86 instruction set core. The first compiler 9104 represents a compiler that is operable to generate binary code of the first instruction set 9106 (e.g., object code) that can, with or without additional linkage processing, be executed on the processor with at least one first instruction set core 9116. Similarly, Figure 91 shows the program in the high level language 9102 may be compiled using an alternative instruction set compiler 9108 to generate alternative instruction set binary code 9110 that may be natively executed by a processor without at least one first instruction set core 9114 (e.g., a processor with cores that execute the MIPS instruction set of MIPS Technologies of Sunnyvale, CA and/or that execute the ARM instruction set of ARM Holdings of Sunnyvale, CA). The instruction converter 9112 is used to convert the first binary code 9106 into code that may be natively executed by the processor without a first instruction set core 9114. This converted code is not likely to be the same as the alternative instruction set binary code 9110 because an instruction converter capable of this is difficult to make; however, the converted code will accomplish the general

operation and be made up of instructions from the alternative instruction set. Thus, the instruction converter 9112 represents software, firmware, hardware, or a combination thereof that, through emulation, simulation or any other process, allows a processor or other electronic device that does not have a first instruction set processor or core to execute the first binary code 9106.

CLAIMS:

We claim:

1. An apparatus comprising:

matrix operations circuitry to execute one or more decoded matrix operation instructions on data stored in two-dimensional data structures; and

storage to store the two-dimensional data structures.

2. The apparatus of claim 1, wherein the storage is a plurality of packed data registers and the two-dimensional data structures are overlaid on these registers.

3. The apparatus of claim 1, wherein the storage is a plurality of packed data registers and memory, and the two-dimensional data structures are overlaid on these registers and memory.

4. The apparatus of any of claims 1-3, wherein the matrix operations circuitry is a plurality of chained fused multiply accumulate circuits.

5. The apparatus of claim 4, wherein each of the chained fused multiply accumulate circuits is to include storage for a portion of a two-dimensional data structure that the fused multiply accumulate circuit is to operate on.

6. The apparatus of any of claims 1-5, wherein matrix operations circuitry supports element matrix multiply, subtract, and add instructions.

7. The apparatus of any of claims 1-6, wherein matrix operations circuitry supports dot product and multiply accumulate operations.

8. The apparatus of any of claims 1-7, wherein matrix operations circuitry supports matrix transpose and diagonal operations.

9. A system comprising:

a host processor;

a matrix operations accelerator coupled to the host processor, wherein the matrix operations accelerator is to perform matrix operations on two-dimensional data structures using a computational grid based on commands received from the host processor.

10. The system of claim 9, wherein the matrix operations accelerator further comprises a plurality of data buffers to buffer matrix data in two-dimensional data structures.

11. The system of any of claims 9-10, wherein the computational grid is to house at least one of the buffered matrix data from the plurality of data buffers during a matrix manipulation operation.

12. The system of any of claims 9-11, wherein the data buffers are a plurality of registers.

13. The system of claim 12, wherein the registers are a plurality of packed data registers and the two-dimensional data structures are overlaid on these registers.

14. The system of claim 12, wherein the storage is a plurality of packed data registers and memory, and the two-dimensional data structures are overlaid on these registers and the memory.

15. The system of any of claims 9-14, wherein the matrix operations circuitry is a plurality of chained fused multiply add circuits.

16. The system of claim 15, wherein each of the chained fused multiply add circuits is to include storage for a portion of a two-dimensional data structure that the fused multiply add circuit is to operate on.

17. The system of any of claims 9-15, further comprising a coherent memory interface coupled to the matrix operations accelerator and host processor to provide access to shared memory between the host processor and matrix operations accelerator.

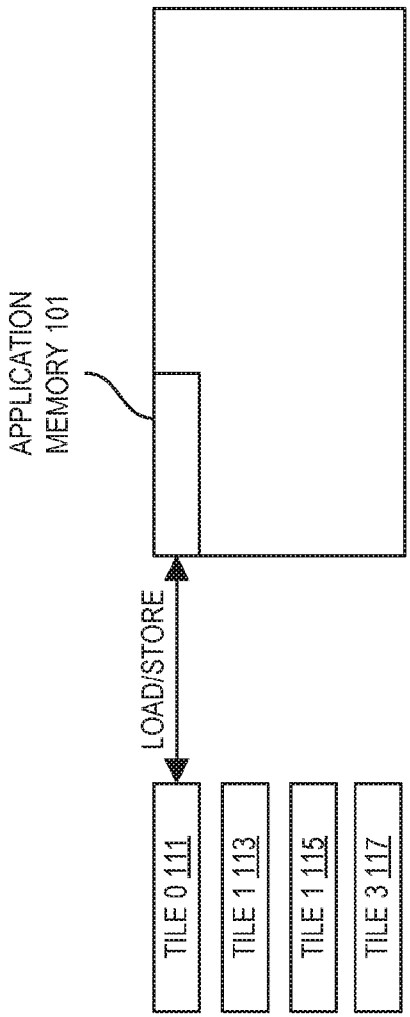

APPLICATION MEMORY 101
LOAD/STORE
TILE 0 111
TILE 1 113
TILE 1 115
TILE 3 117

FIG. 1

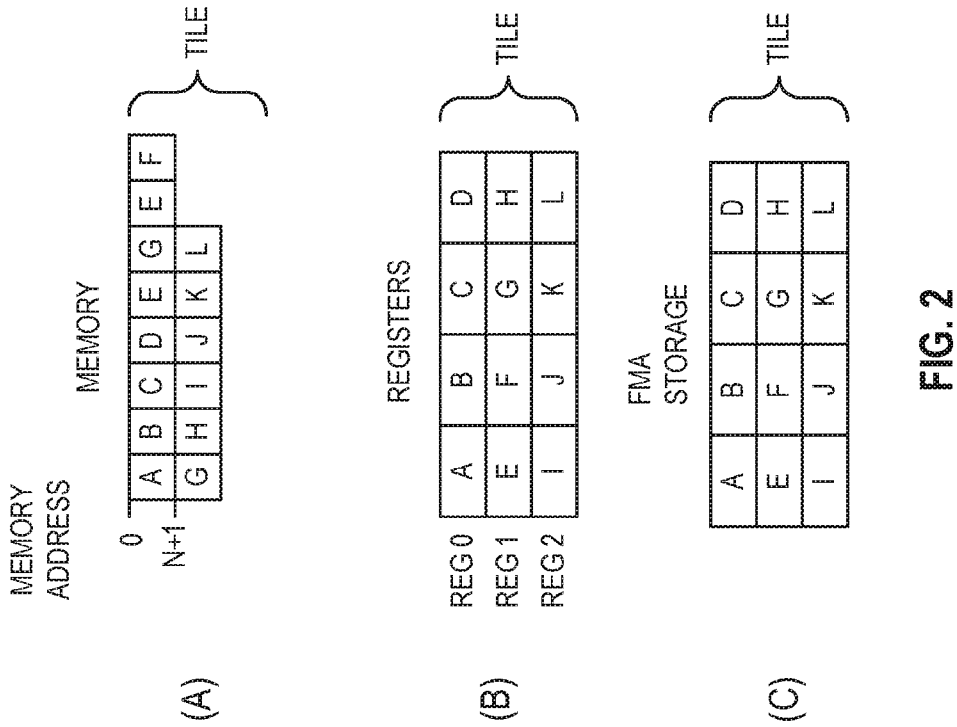

MEMORY ADDRESS
MEMORY
0
N+1
A B C D E F
G H I J K L
TILE
(A)
REGISTERS
REG 0
REG 1
REG 2
A B C D
E F G H
I J K L
TILE
(B)
FMA STORAGE
A B C D
E F G H
I J K L
TILE
(C)

FIG. 2

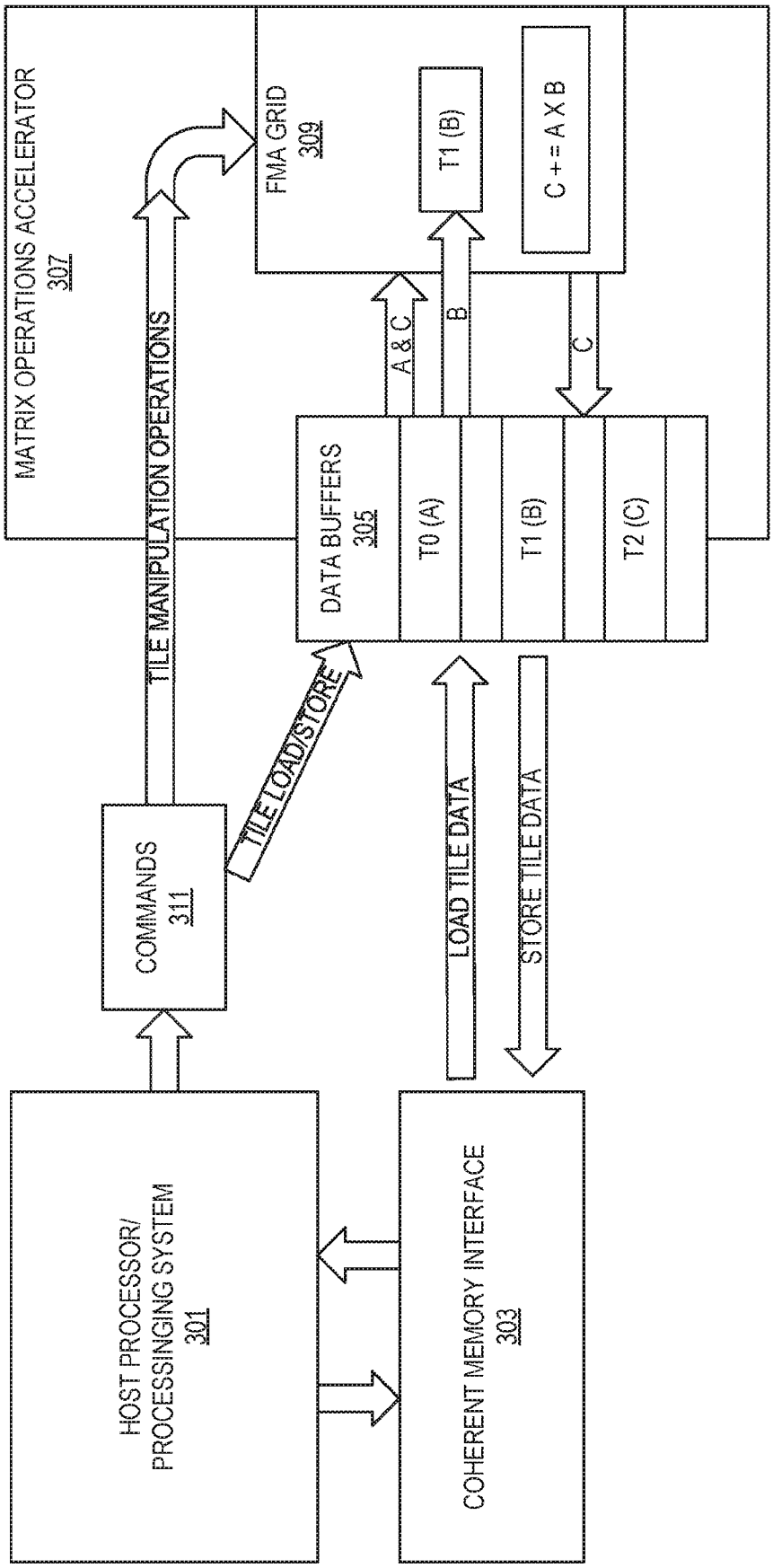

MATRIX OPERATIONS ACCELERATOR
307
FMA GRID
309
T1 (B)
C + = A X B
A & C
B
C
TILE MANIPULATION OPERATIONS
DATA BUFFERS
305
T0 (A)
T1 (B)
T2 (C)
TILE LOAD/STORE
COMMANDS
311
LOAD TILE DATA
STORE TILE DATA
HOST PROCESSOR/
PROCESSINGING SYSTEM
301
COHERENT MEMORY INTERFACE
303

FIG. 3

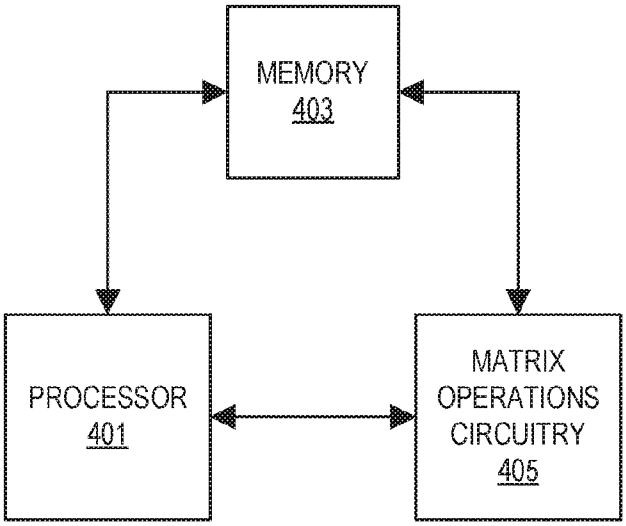

MEMORY
403
PROCESSOR
401
MATRIX
OPERATIONS
CIRCUITRY
405

FIG. 4

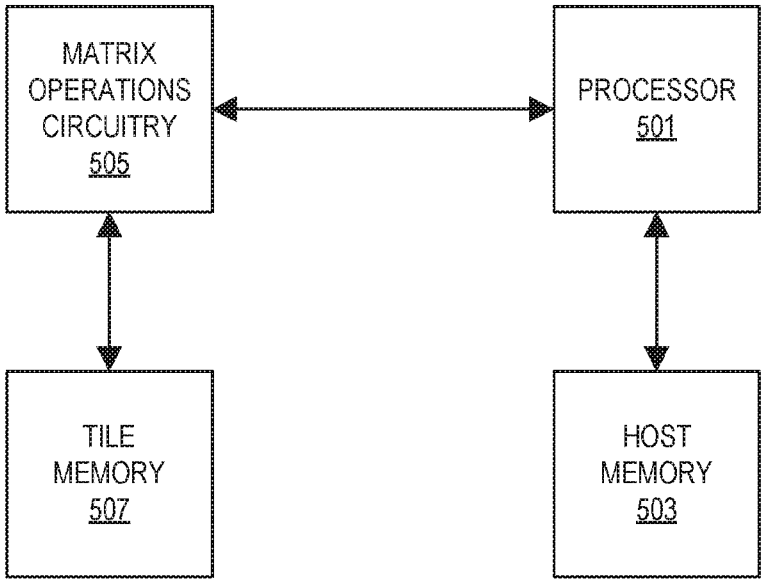

MATRIX
OPERATIONS
CIRCUITRY
505
PROCESSOR
501
TILE
MEMORY
507
HOST
MEMORY
503

FIG. 5

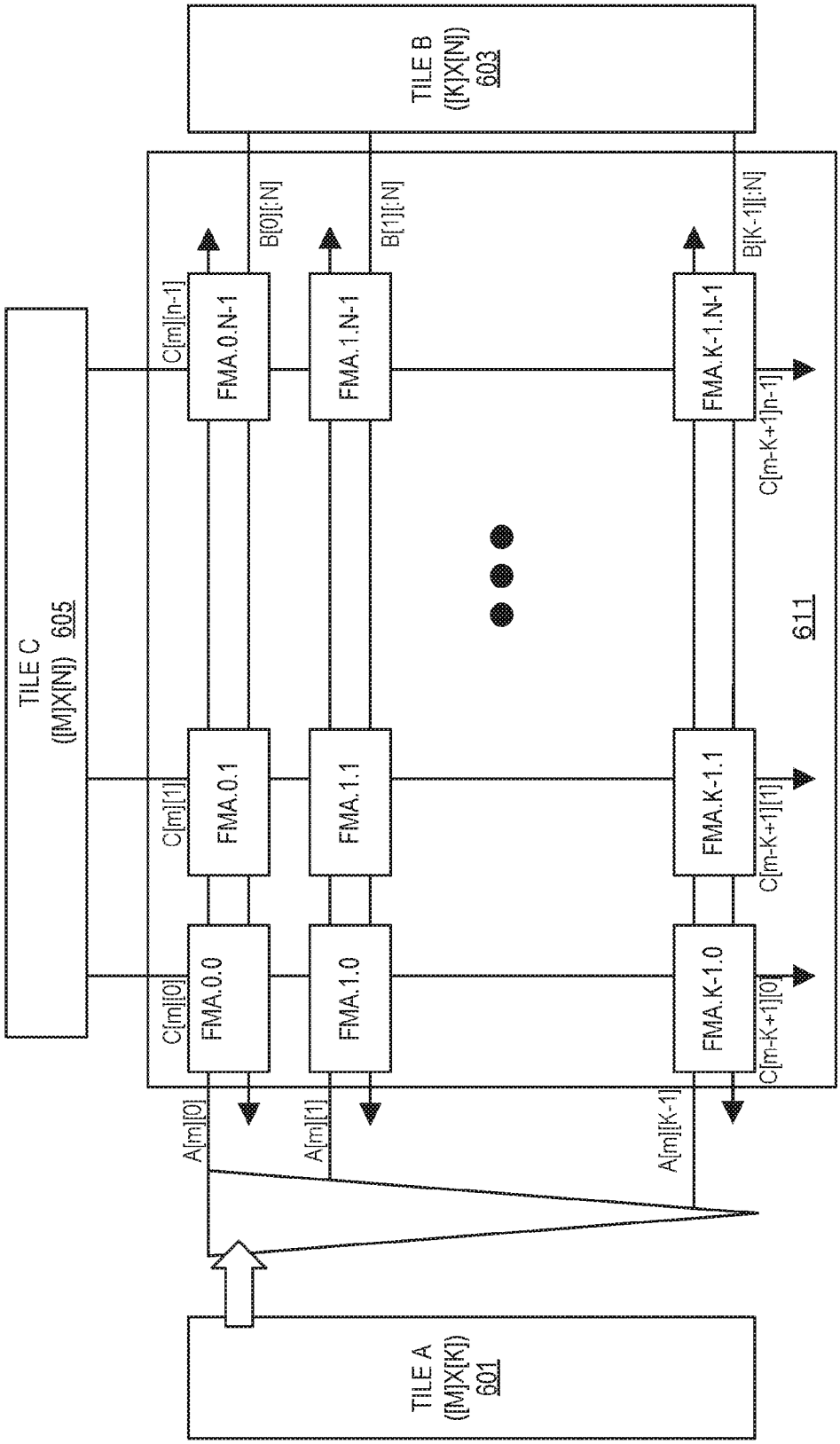

TILE B
([K]X[N])
603
TILE C
([M]X[N]) 605
TILE A
([M]X[K])
601
611
B[0][:N]
B[1][:N]
B[K-1][:N]
C[m][n-1]
C[m][1]
C[m][0]
FMA.0.N-1
FMA.1.N-1
FMA.K-1.N-1
FMA.0.1
FMA.1.1
FMA.K-1.1
FMA.0.0
FMA.1.0
FMA.K-1.0
C[m-K+1][n-1]
C[m-K+1][1]
C[m-K+1][0]
A[m][0]
A[m][1]
A[m][K-1]

FIG. 6

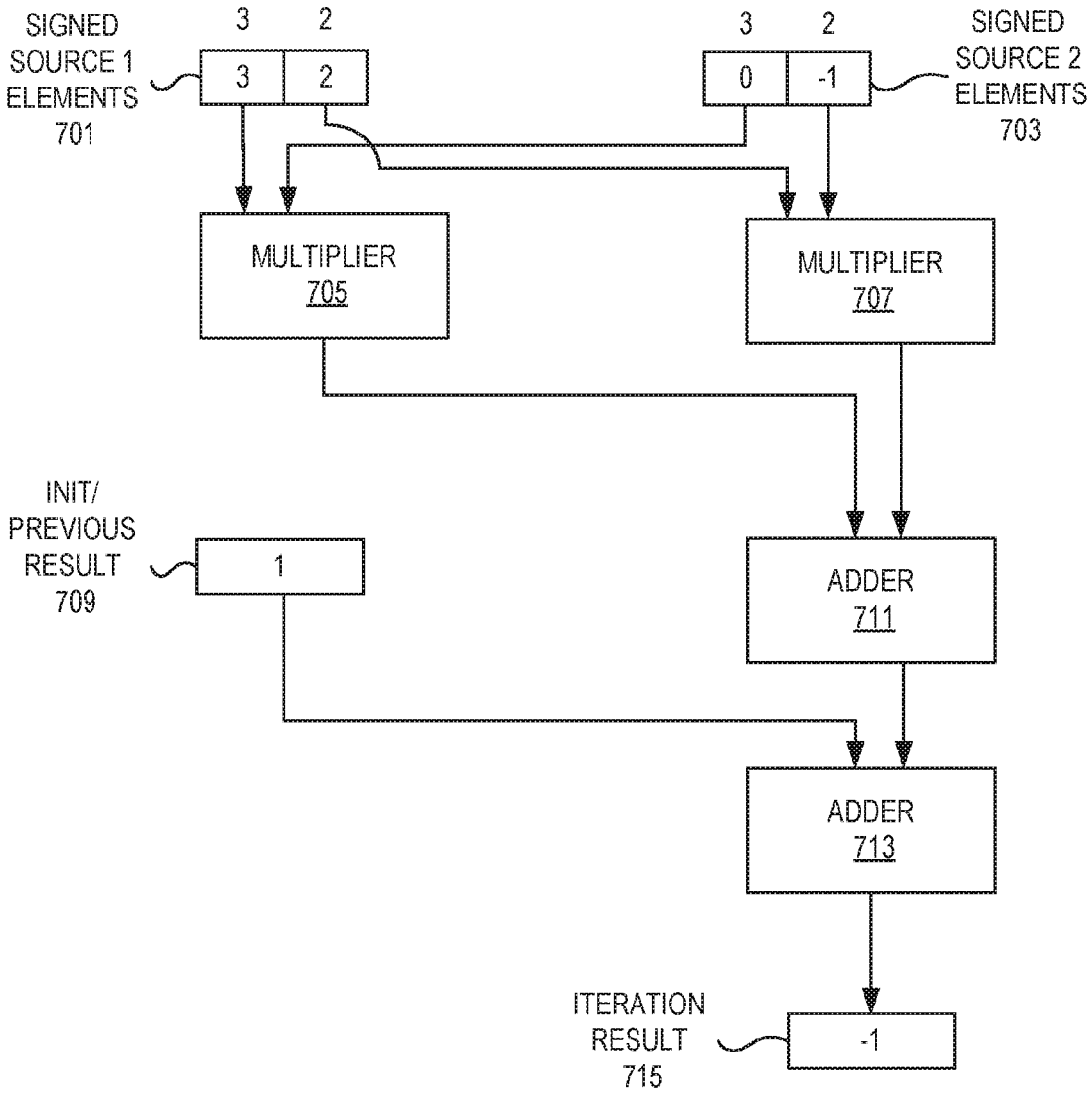

SIGNED SOURCE 1 ELEMENTS 701
3
2
3
2
3
2
0
-1
SIGNED SOURCE 2 ELEMENTS 703
MULTIPLIER 705
MULTIPLIER 707
INIT/ PREVIOUS RESULT 709
1
ADDER 711
ADDER 713
ITERATION RESULT 715
-1

FIG. 7

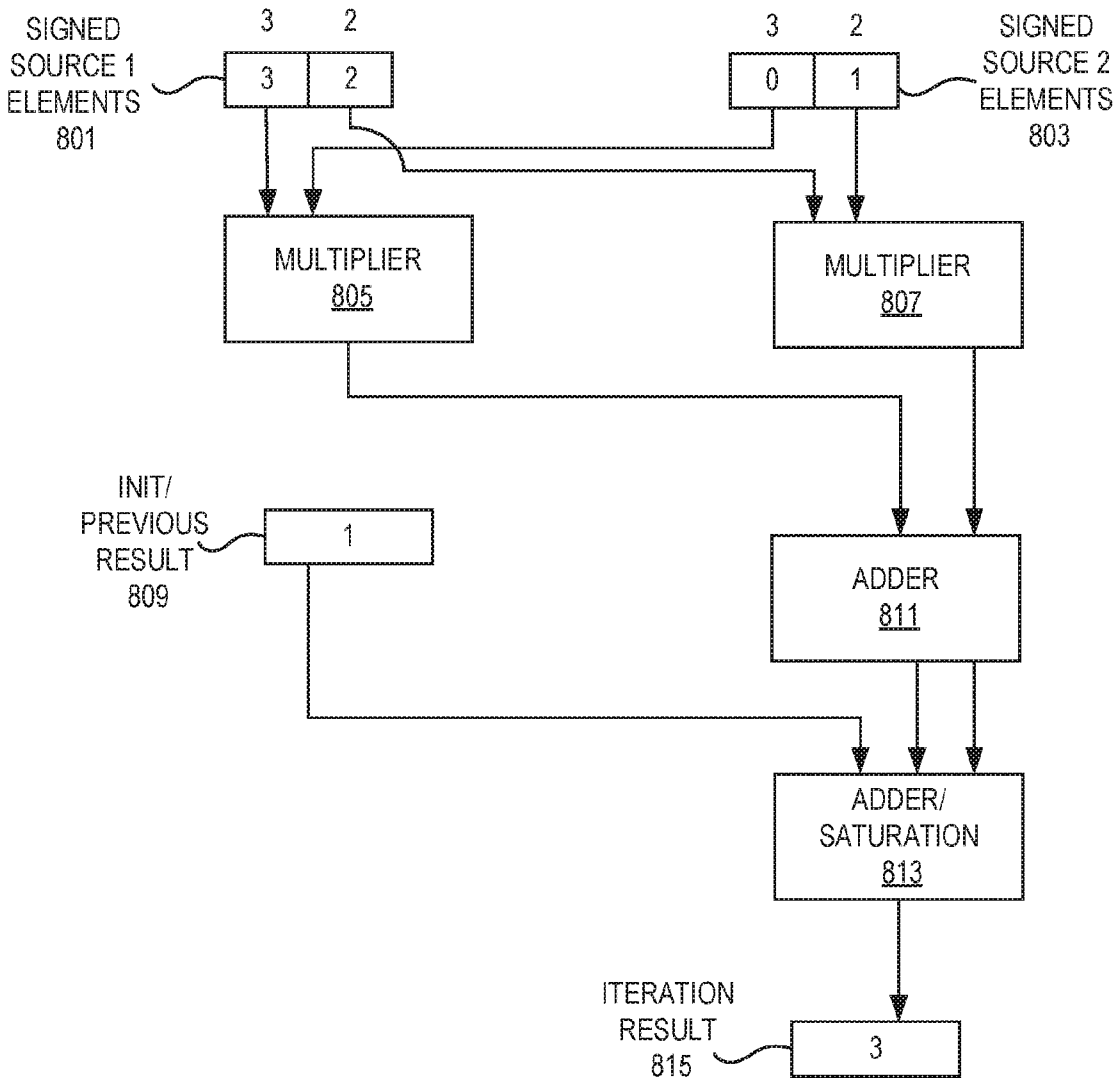

SIGNED SOURCE 1 ELEMENTS 801
3
2
3
2
SIGNED SOURCE 2 ELEMENTS 803
3
2
0
1
MULTIPLIER 805
MULTIPLIER 807
INIT/ PREVIOUS RESULT 809
1
ADDER 811
ADDER/ SATURATION 813
ITERATION RESULT 815
3

FIG. 8

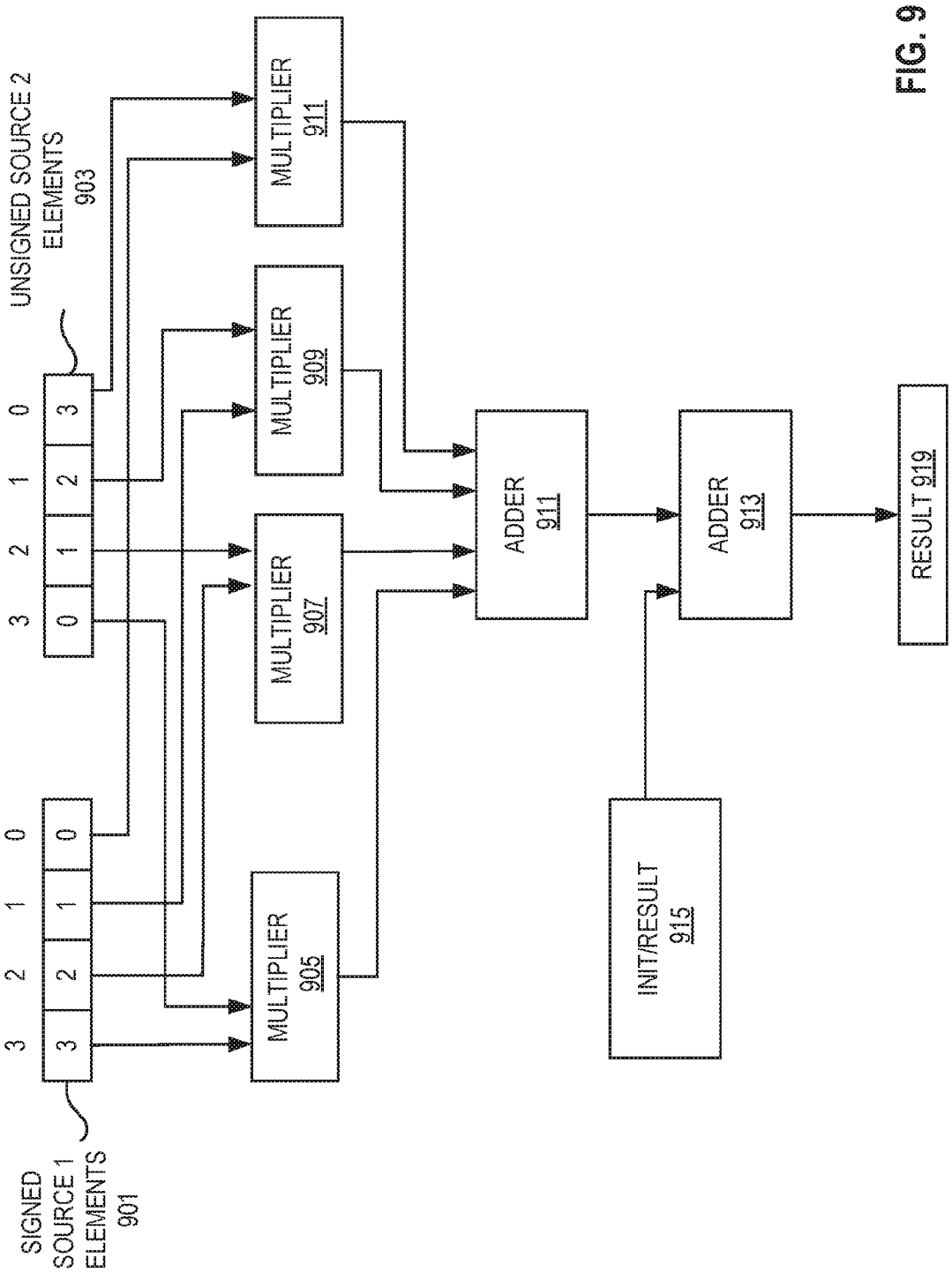

SIGNED SOURCE 1 ELEMENTS 901
UNSIGNED SOURCE 2 ELEMENTS 903
3 2 1 0
MULTIPLIER 905
MULTIPLIER 907
MULTIPLIER 909
MULTIPLIER 911
ADDER 911
ADDER 913
INIT/RESULT 915
RESULT 919

FIG. 9

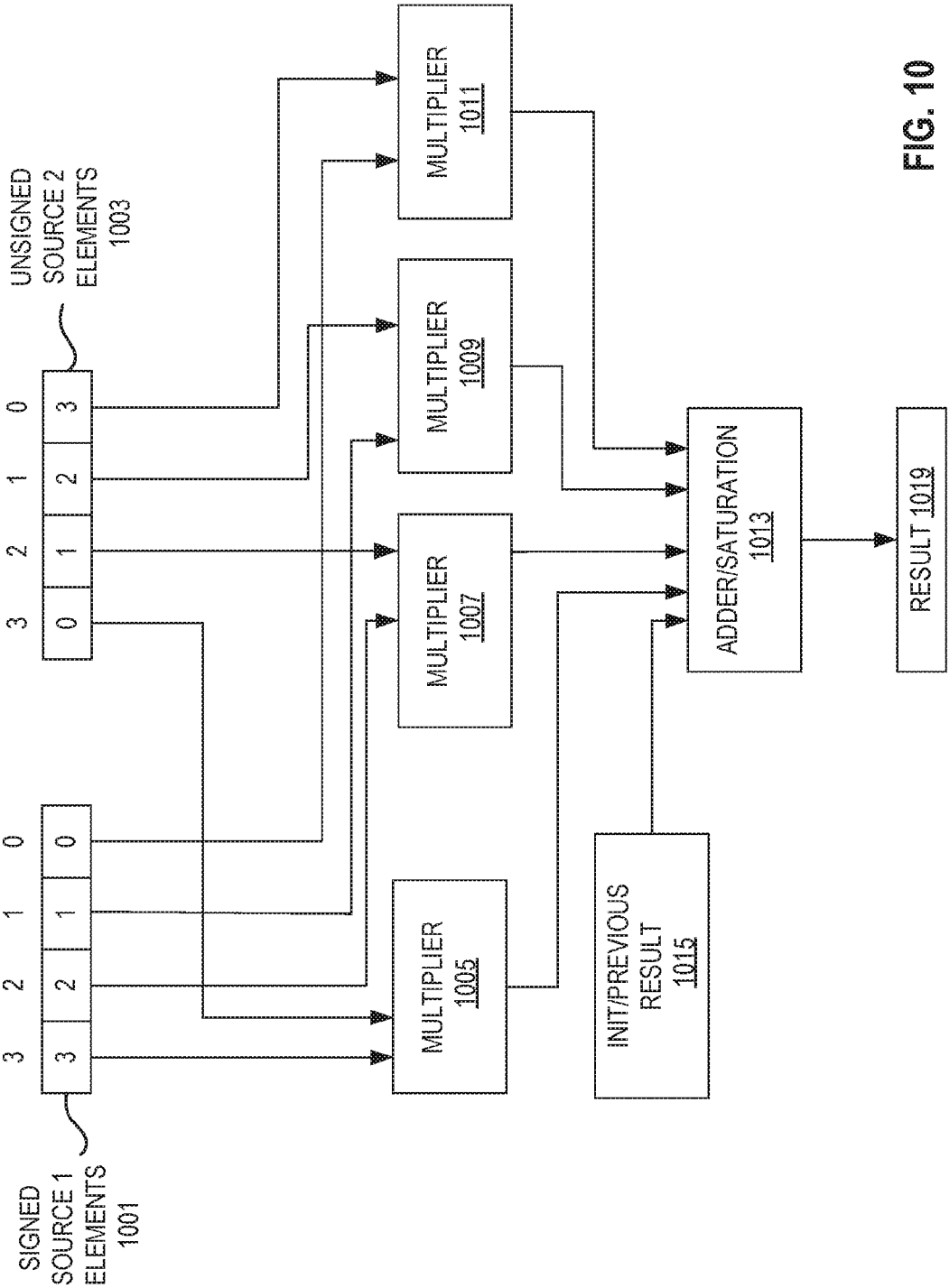

SIGNED SOURCE 1 ELEMENTS 1001
UNSIGNED SOURCE 2 ELEMENTS 1003
3 2 1 0
MULTIPLIER 1005
MULTIPLIER 1007
MULTIPLIER 1009
MULTIPLIER 1011
INIT/PREVIOUS RESULT 1015
ADDER/SATURATION 1013
RESULT 1019

FIG. 10

ACCUMULATOR 2X INPUT SIZES 1101

SOURCES	BITS	ACCUMULATOR	BITS
BYTE	8	WORD/HPFP	16
WORD	16	INT32/SPFP	32
SPFP/INT32	32	INT64/DPFP	64

ACCUMULATOR 4X INPUT SIZES 1103

SOURCES	BITS	ACCUMULATOR	BITS
BYTE	8	INT32/SPFP	32
WORD	16	INT64/DPFP	64

ACCUMULATOR 8X INPUT SIZES 1105

SOURCES	BITS	ACCUMULATOR	BITS
BYTE	8	INT64/DPFP	64

FIG. 11

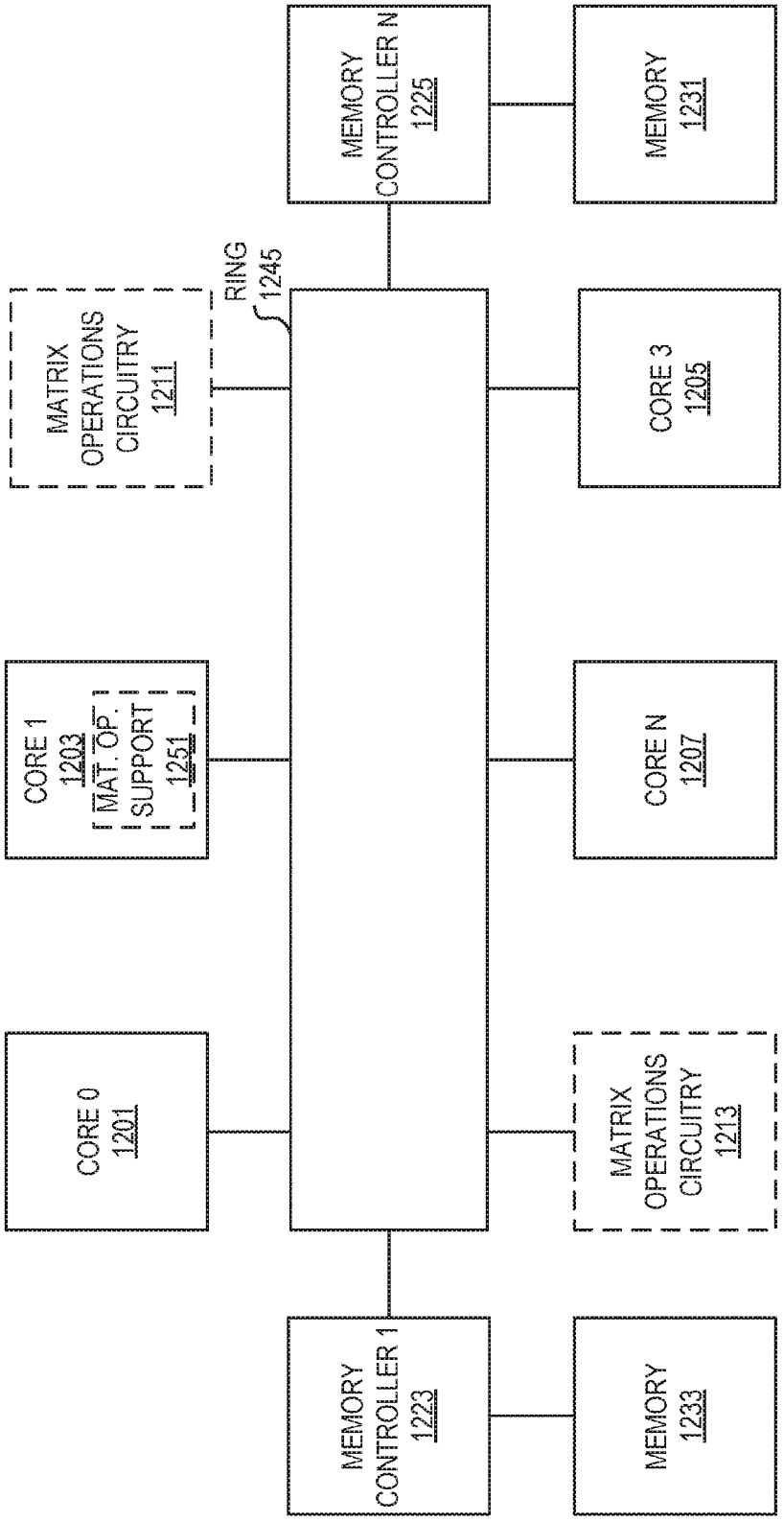

MEMORY CONTROLLER N 1225
MEMORY 1231
RING 1245
MATRIX OPERATIONS CIRCUITRY 1211
CORE 3 1205
CORE 1 1203
MAT. OP. SUPPORT 1251
CORE N 1207
CORE 0 1201
MATRIX OPERATIONS CIRCUITRY 1213
MEMORY CONTROLLER 1 1223
MEMORY 1233

FIG. 12

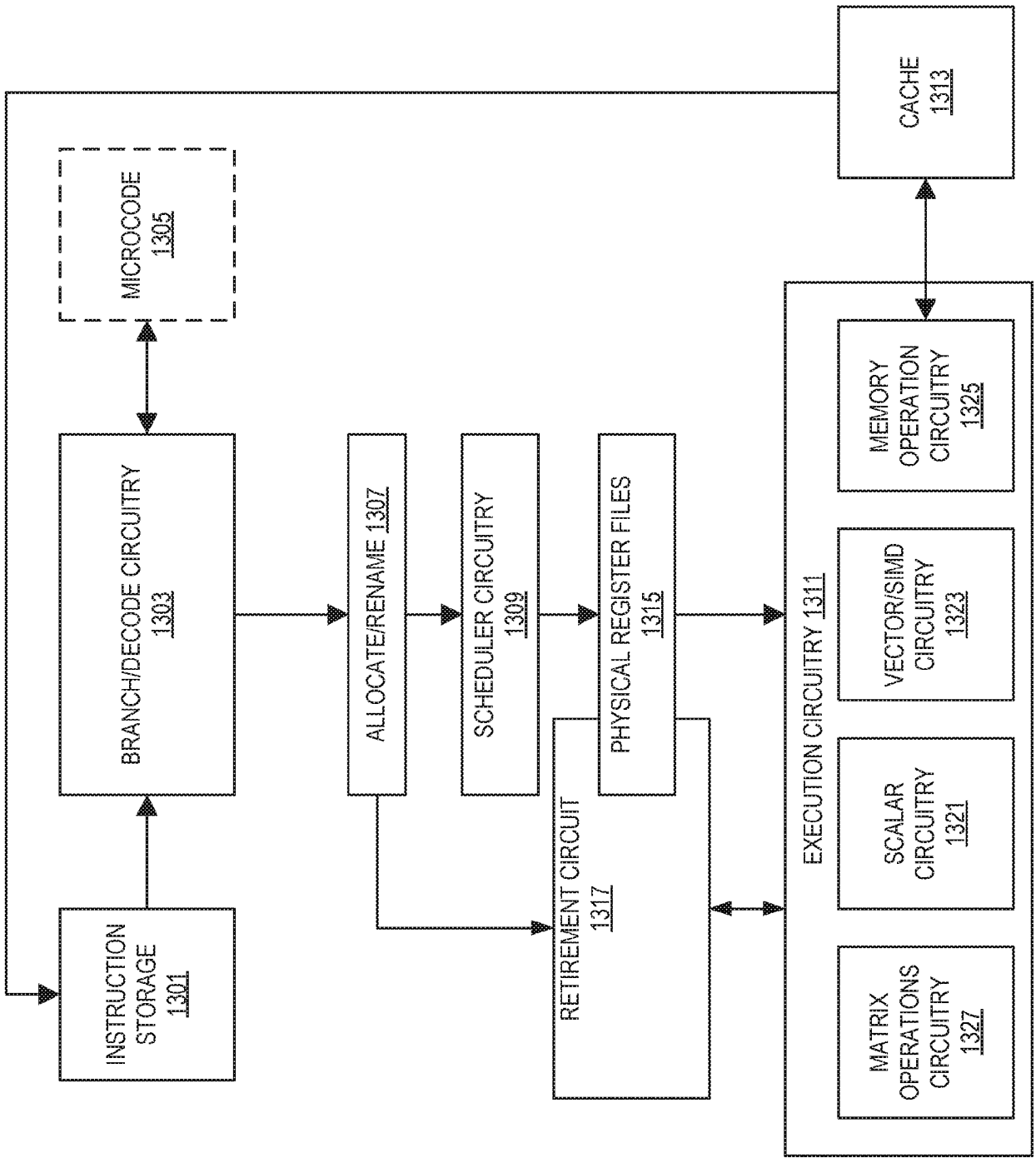

INSTRUCTION STORAGE 1301
BRANCH/DECODE CIRCUITRY 1303
MICROCODE 1305
ALLOCATE/RENAME 1307
SCHEDULER CIRCUITRY 1309
PHYSICAL REGISTER FILES 1315
RETIREMENT CIRCUIT 1317
CACHE 1313
EXECUTION CIRCUITRY 1311
MEMORY OPERATION CIRCUITRY 1325
VECTOR/SIMD CIRCUITRY 1323
SCALAR CIRCUITRY 1321
MATRIX OPERATIONS CIRCUITRY 1327

FIG. 13

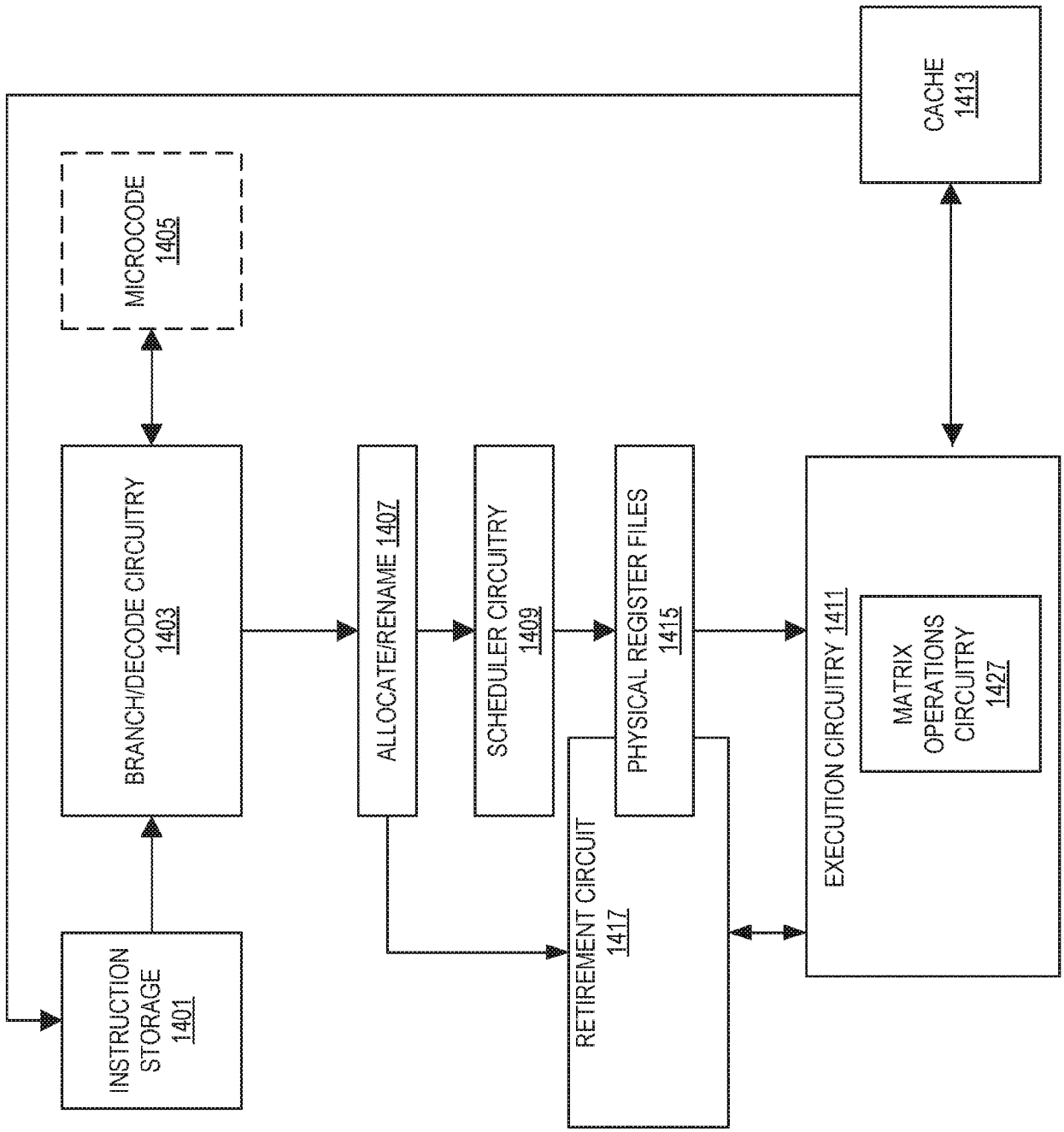

CACHE 1413
MICROCODE 1405
BRANCH/DECODE CIRCUITRY 1403
ALLOCATE/RENAME 1407
SCHEDULER CIRCUITRY 1409
PHYSICAL REGISTER FILES 1415
EXECUTION CIRCUITRY 1411
MATRIX OPERATIONS CIRCUITRY 1427
INSTRUCTION STORAGE 1401
RETIREMENT CIRCUIT 1417

FIG. 14

$$A = \begin{bmatrix} A_{11} & A_{12} & A_{13} \\ A_{21} & A_{22} & A_{23} \end{bmatrix}$$

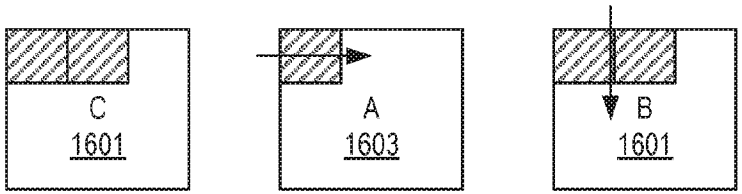
ADDR	VALUE
0	A_{11}
1	A_{12}
2	A_{13}
3	A_{21}
4	A_{22}
5	A_{23}

ROW MAJOR

ADDR	VALUE
0	A_{11}
1	A_{21}
2	A_{12}
3	A_{22}
4	A_{13}
5	A_{23}

COLUMN MAJOR

FIG. 15




```
TILECONFIG [RAX]
// ASSUME SOME OUTER LOOPS DRIVING THE CACHE TILING (NOT SHOWN)
{
TILELOAD TMM0, RSI+RDI // SRCDST, RSI POINTS TO C, RDI HAS
TILELOAD TMM1, RSI+RDI+N // SECOND TILE OF C, UNROLLING IN SIMD DIMENSION N
MOV KK, 0
LOOP:
TILELOAD TMM2, R8+R9 // SRC2 IS STRIDED LOAD OF A, REUSED FOR 2 TMMA INSTR.
TILELOAD TMM3, R10+R11 // SRC1 IS STRIDED LOAD OF B
TMMAPS TMM0, TMM2, TMM3 // UPDATE LEFT TILE OF C
TILELOAD TMM3, R10+R11+N // SRC1 LOADED WITH B FROM NEXT RIGHTMOST TILE
TMMAPS TMM1, TMM2, TMM3 // UPDATE RIGHT TILE OF C
ADD R8, K // UPDATE POINTERS BY CONSTANTS KNOWN OUTSIDE OF LOOP
ADD R10, K*R11
ADD KK, K
CMP KK, LIMIT
JNE LOOP
TILESTORE RSI+RDI, TMM0 // UPDATE THE C MATRIX IN MEMORY
TILESTORE RSI+RDI+M, TMM1
} // END OF OUTER LOOP
TILERELEASE // RETURN TILES TO INIT STATE
```

FIG. 16

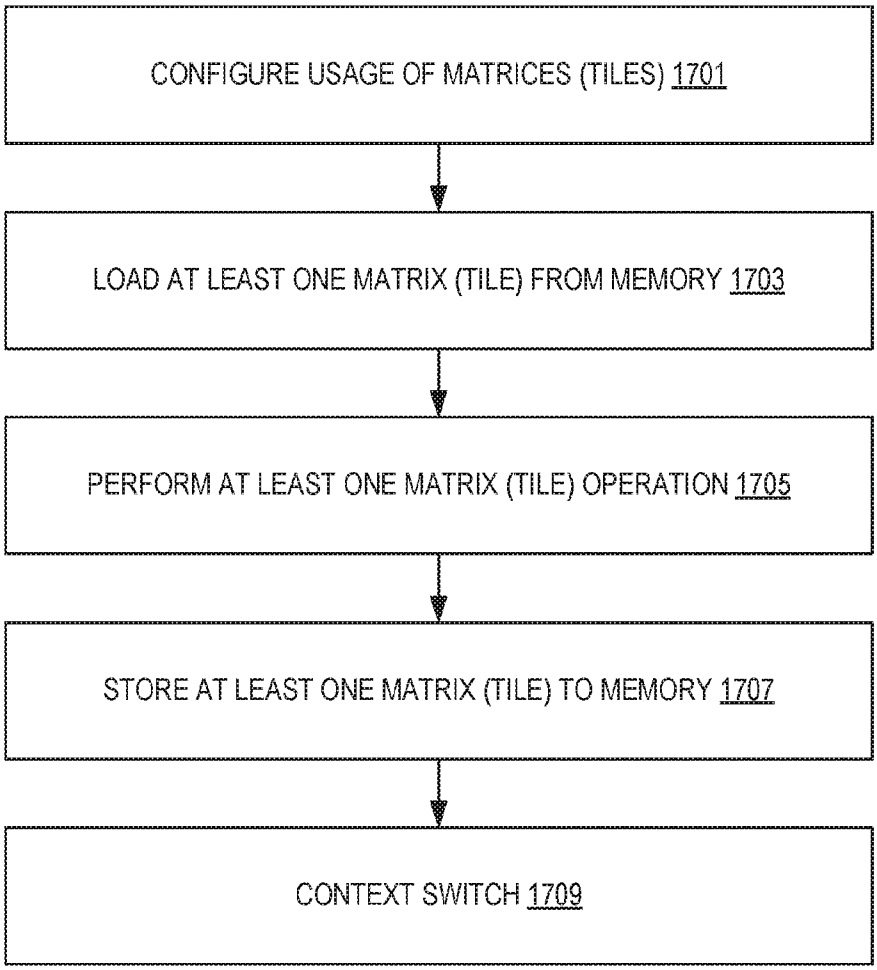

CONFIGURE USAGE OF MATRICES (TILES) 1701
LOAD AT LEAST ONE MATRIX (TILE) FROM MEMORY 1703
PERFORM AT LEAST ONE MATRIX (TILE) OPERATION 1705
STORE AT LEAST ONE MATRIX (TILE) TO MEMORY 1707
CONTEXT SWITCH 1709

FIG. 17

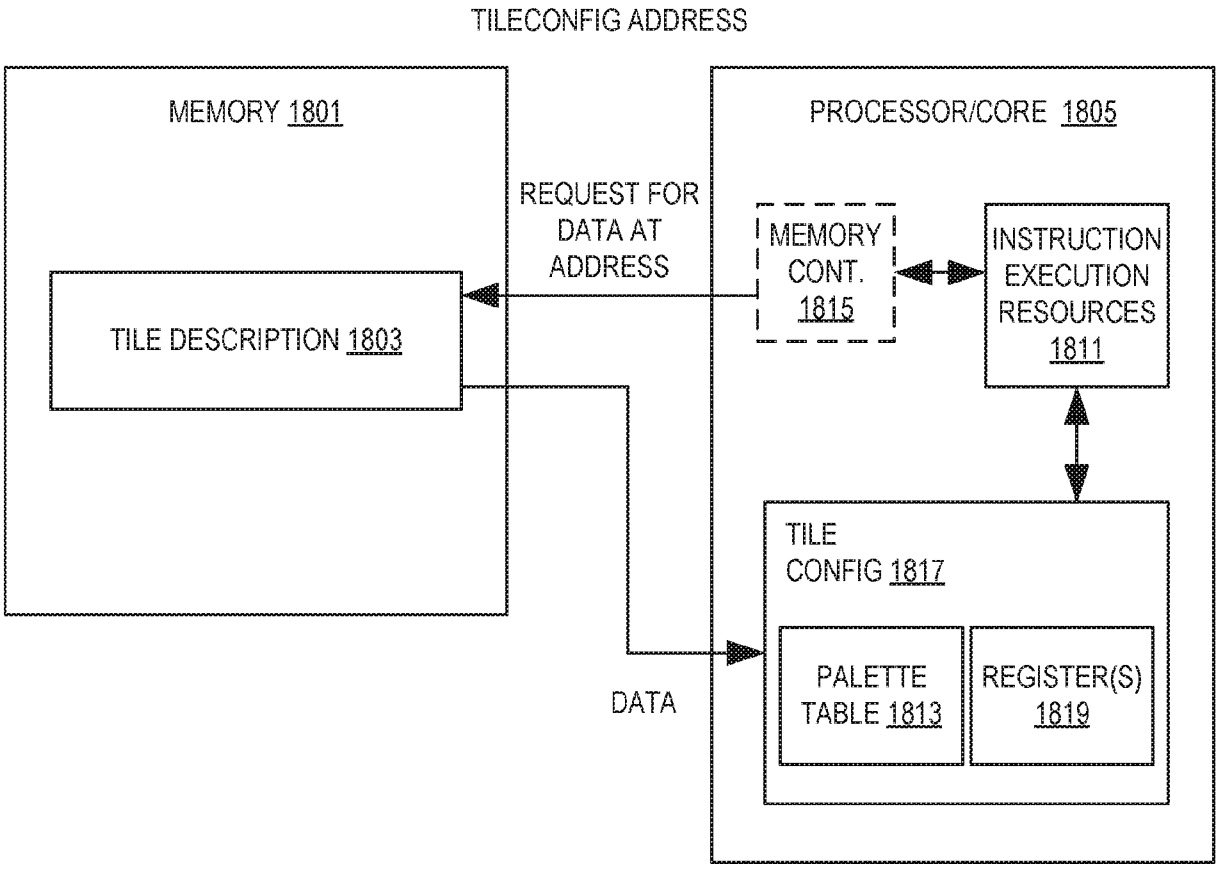

TILECONFIG ADDRESS
MEMORY 1801
TILE DESCRIPTION 1803
REQUEST FOR DATA AT ADDRESS
DATA
PROCESSOR/CORE 1805
MEMORY CONT. 1815
INSTRUCTION EXECUTION RESOURCES 1811
TILE CONFIG 1817
PALETTE TABLE 1813
REGISTER(S) 1819

FIG. 18

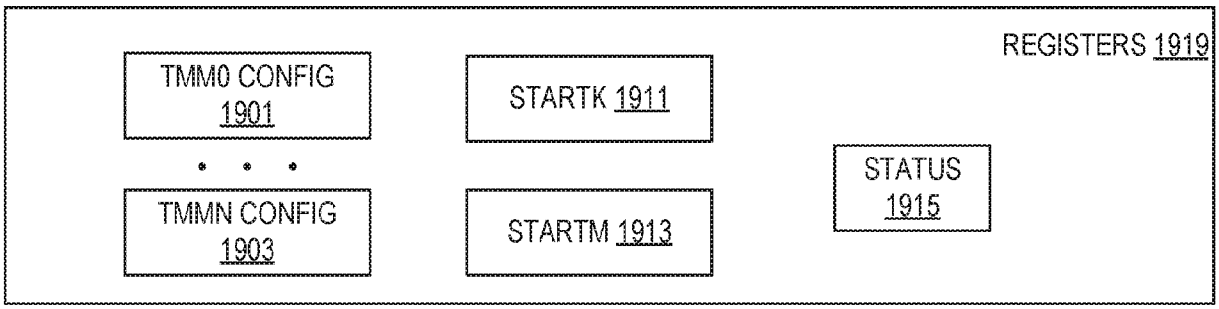


FIG. 19(A)

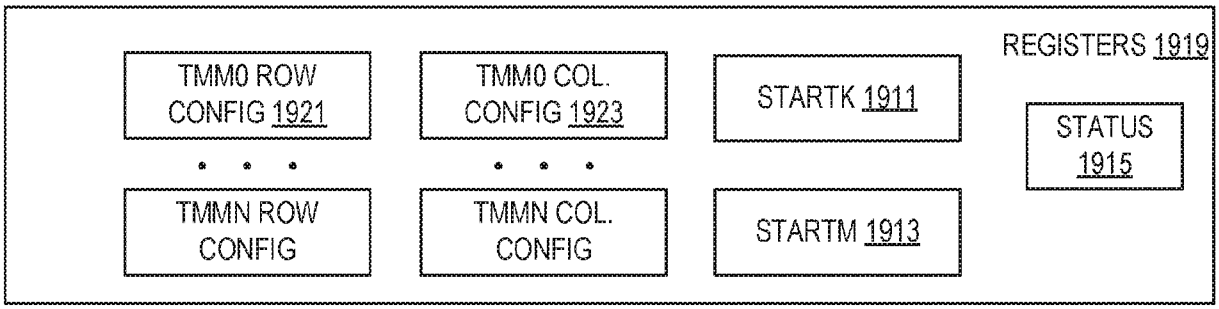


FIG. 19(B)

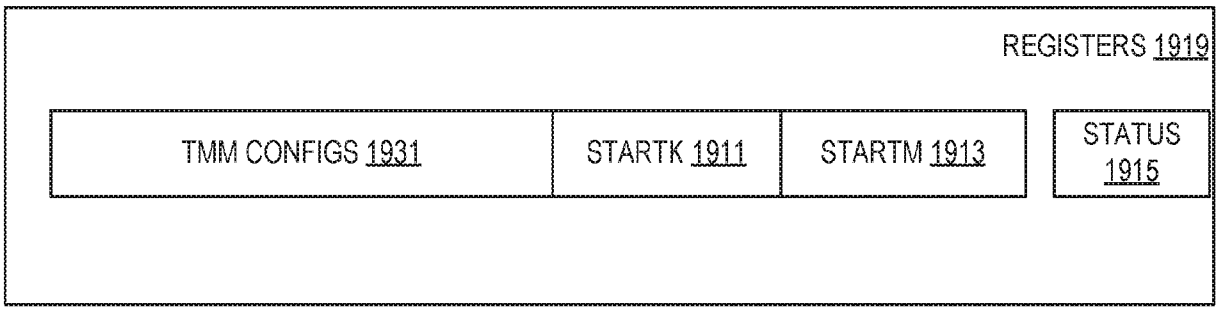


FIG. 19(C)

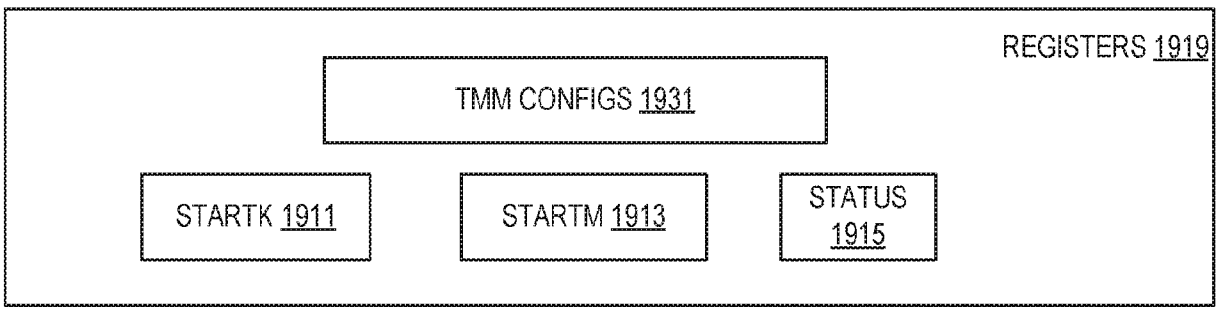


FIG. 19(D)

PALETTE ID 2001	0
0	0
0	0
0	0
STARTM 2003	
STARTK 2005	
0	0
TMM0 ROWS 2013	TMM0 COLUMNS 2015
TMM1 ROWS	TMM1 COLUMNS
• • •	
TMM15 ROWS	TMM15 COLUMNS
0	

FIG. 20

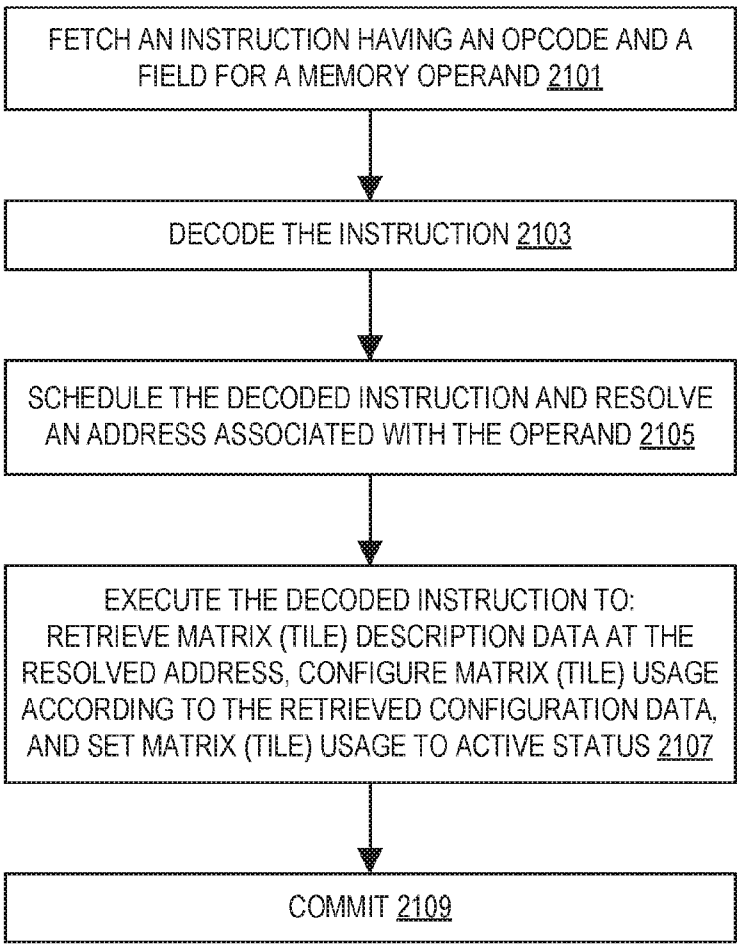

FETCH AN INSTRUCTION HAVING AN OPCODE AND A FIELD FOR A MEMORY OPERAND 2101
DECODE THE INSTRUCTION 2103
SCHEDULE THE DECODED INSTRUCTION AND RESOLVE AN ADDRESS ASSOCIATED WITH THE OPERAND 2105
EXECUTE THE DECODED INSTRUCTION TO: RETRIEVE MATRIX (TILE) DESCRIPTION DATA AT THE RESOLVED ADDRESS, CONFIGURE MATRIX (TILE) USAGE ACCORDING TO THE RETRIEVED CONFIGURATION DATA, AND SET MATRIX (TILE) USAGE TO ACTIVE STATUS 2107
COMMIT 2109

FIG. 21

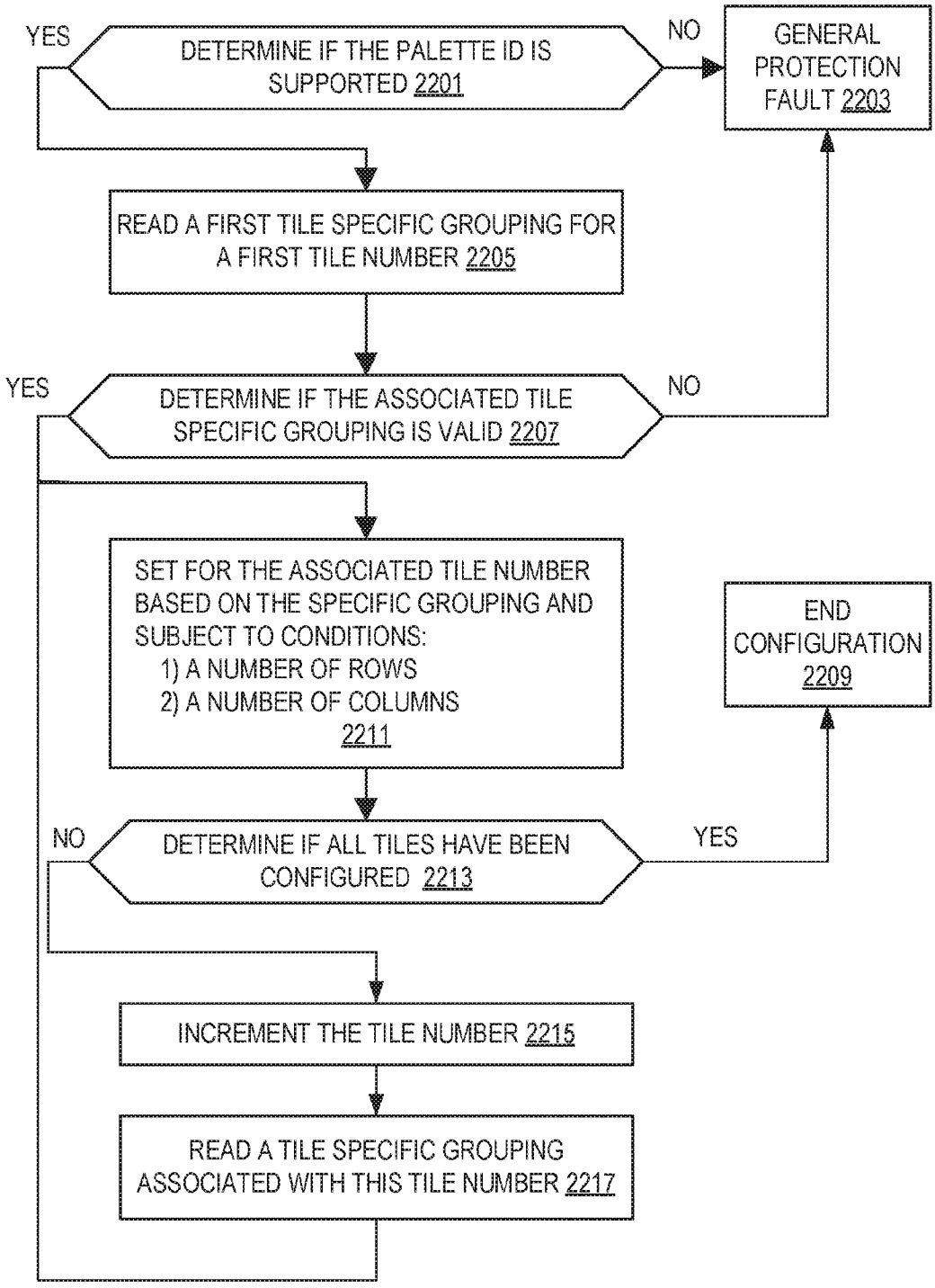

YES
DETERMINE IF THE PALETTE ID IS SUPPORTED 2201
NO
GENERAL PROTECTION FAULT 2203
READ A FIRST TILE SPECIFIC GROUPING FOR A FIRST TILE NUMBER 2205
YES
DETERMINE IF THE ASSOCIATED TILE SPECIFIC GROUPING IS VALID 2207
NO
SET FOR THE ASSOCIATED TILE NUMBER BASED ON THE SPECIFIC GROUPING AND SUBJECT TO CONDITIONS:
1) A NUMBER OF ROWS
2) A NUMBER OF COLUMNS
2211
END CONFIGURATION 2209
NO
DETERMINE IF ALL TILES HAVE BEEN CONFIGURED 2213
YES
INCREMENT THE TILE NUMBER 2215
READ A TILE SPECIFIC GROUPING ASSOCIATED WITH THIS TILE NUMBER 2217

FIG. 22

```
// FORMAT OF MEMORY PAYLOAD. EACH FIELD IS ABYTE
// 0: PALETTE_ID
// 1-7: RESERVED (MUST BE ZERO)
// 8-9: STARTM(16B)
// 10-11: STARTK(16B)
// 12-15: RESERVED, (MUST BE ZERO)
// 16-17: TILE0ROWSTILE0COLS
// 18-19: TILE1ROWSTILE1COLS
// 20-21: TILE2ROWSTILE2COLS
// ...
// 46-47: TILE15ROWS TILE15COLS
// 48-63: 16B RESERVED, (MUST BE ZERO)
TILECONFIG MEM
PALETTE_ID := MEM.BYTE[0]
#GP IF PALETTE_ID IS AN UNSUPPORTED PALETTE // FROM CPUID
#GP IF MEM.BYTE[1..7 ISNONZERO
STARTM := MEM.WORD[4] // BYTES 8..9
STARTK := MEM.WORD[5] // BYTES 10..11
#GP IF MEM.BYTE[12..15] ISNONZERO

MAX_NAMES := IMPL.MAX_TILE_BYTES /PALETTE_TABLE[I].TILE_BYTES
MAX_NUM_ROWS := PALETTE_TABLE[PALETTE_ID].TILE_BYTES/
PALETTE_TABLE[PALETTE_ID].BYTES_PER_ROW
P := 16
FOR N IN 0 ...MAX_NAMES-1:
T[N].ROWS :=MEM.BYTE[P++]
T[N].COLS :=MEM.BYTE[P++]
#GP IF T[N].ROWS ==0
#GP IF T[N].COLS ==0
#GP IF T[N].ROWS >MAX_NUM_ROWS
// ALL INSTRUCTIONS CHECK COLUMNS LIMITS BASED ON THEIR ELEMENT
// WIDTHS SO WE DO NOT ENFORCE COLUMN WITH RESTRICTION HERE.

WHILE P < 64: // CONFIRM REST OF BYTES IN 64B CHUNK ARE ZERO
#GP IF MEM.BYTE[P] !=0
P := P +1
FOR N IN 0 ...MAX_NAMES-1:
TILEZERO TILE[N]
TILES_CONFIGURED := 1
```

FIG. 23

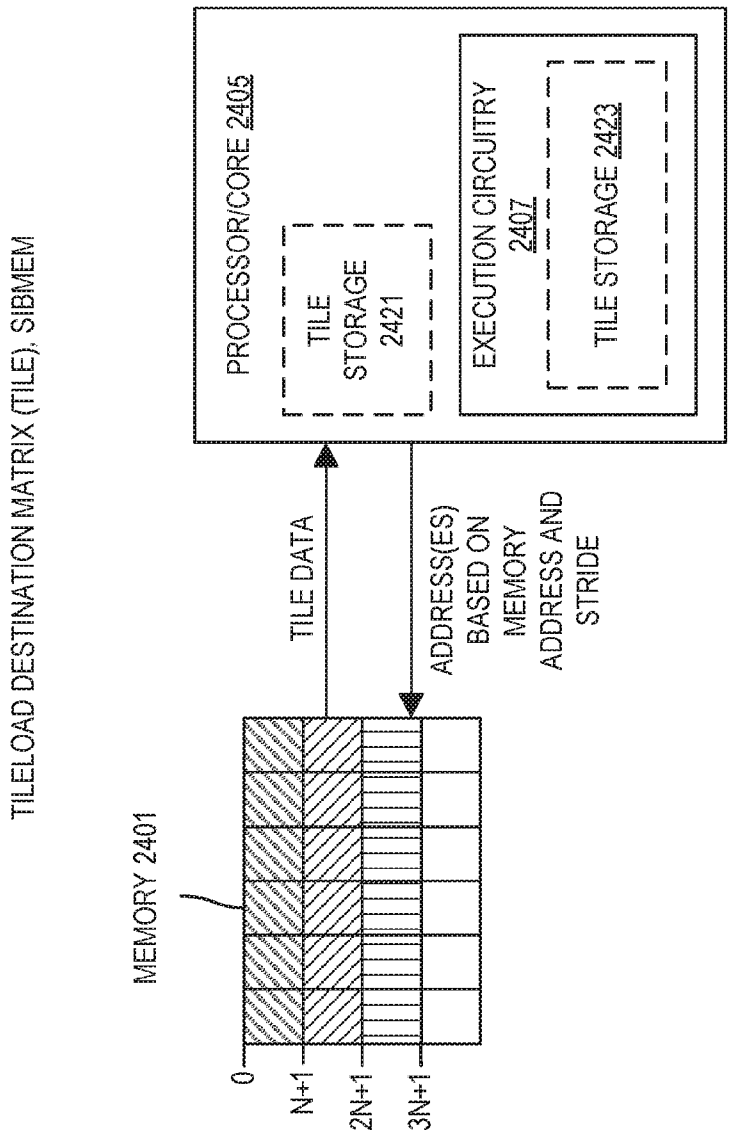

TILELOAD DESTINATION MATRIX (TILE), SIBMEM
MEMORY 2401
0
N+1
2N+1
3N+1
TILE DATA
ADDRESS(ES) BASED ON MEMORY ADDRESS AND STRIDE
PROCESSOR/CORE 2405
TILE STORAGE 2421
EXECUTION CIRCUITRY 2407
TILE STORAGE 2423

FIG. 24

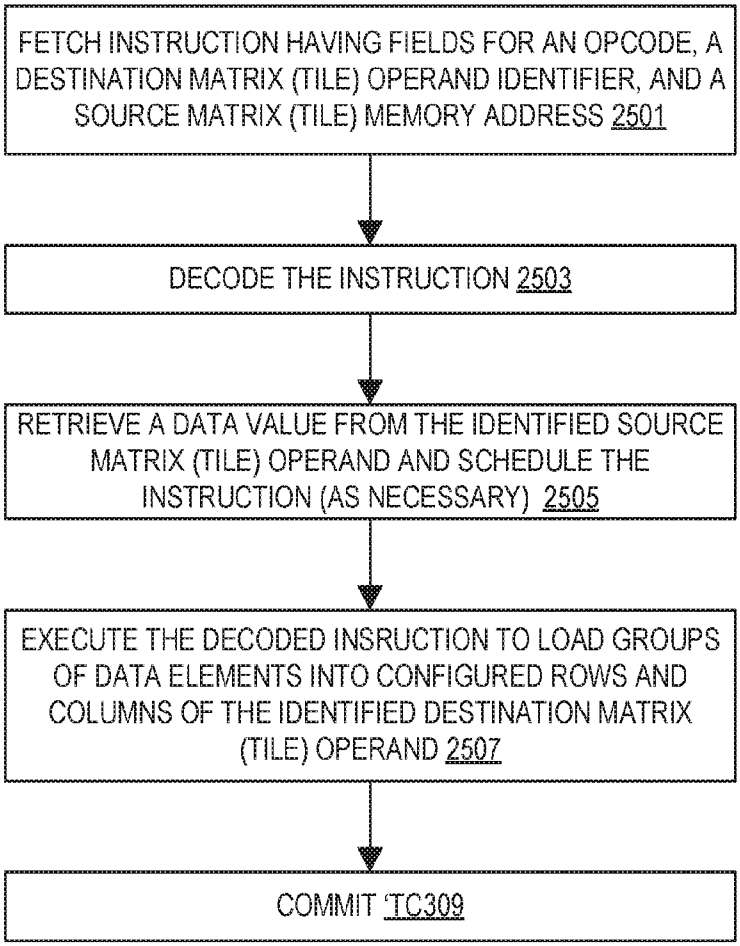

FETCH INSTRUCTION HAVING FIELDS FOR AN OPCODE, A DESTINATION MATRIX (TILE) OPERAND IDENTIFIER, AND A SOURCE MATRIX (TILE) MEMORY ADDRESS 2501
DECODE THE INSTRUCTION 2503
RETRIEVE A DATA VALUE FROM THE IDENTIFIED SOURCE MATRIX (TILE) OPERAND AND SCHEDULE THE INSTRUCTION (AS NECESSARY) 2505
EXECUTE THE DECODED INSRUCTION TO LOAD GROUPS OF DATA ELEMENTS INTO CONFIGURED ROWS AND COLUMNS OF THE IDENTIFIED DESTINATION MATRIX (TILE) OPERAND 2507
COMMIT 'TC309

FIG. 25

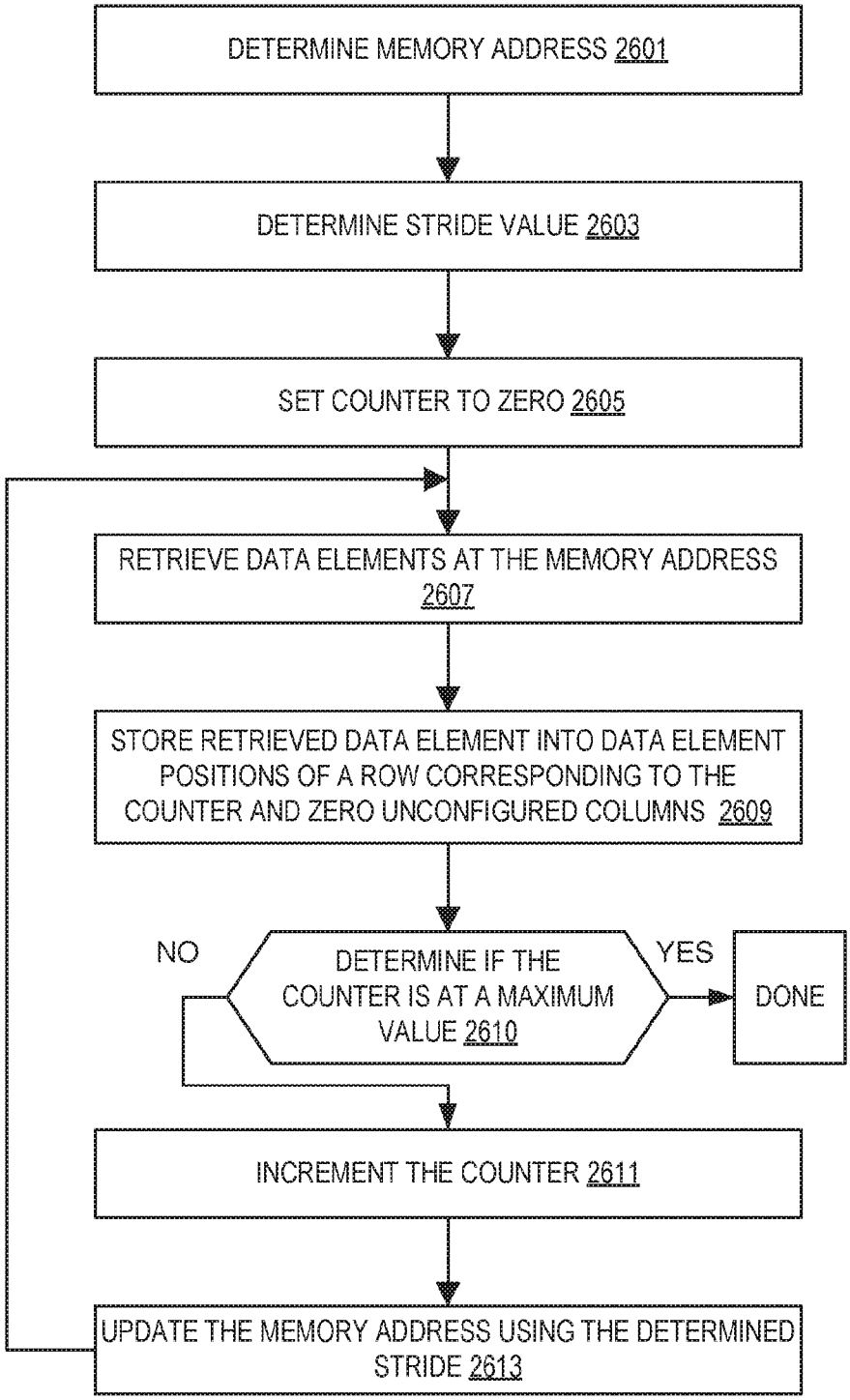

DETERMINE MEMORY ADDRESS 2601
DETERMINE STRIDE VALUE 2603
SET COUNTER TO ZERO 2605
RETRIEVE DATA ELEMENTS AT THE MEMORY ADDRESS 2607
STORE RETRIEVED DATA ELEMENT INTO DATA ELEMENT POSITIONS OF A ROW CORRESPONDING TO THE COUNTER AND ZERO UNCONFIGURED COLUMNS 2609
NO
DETERMINE IF THE COUNTER IS AT A MAXIMUM VALUE 2610
YES
DONE
INCREMENT THE COUNTER 2611
UPDATE THE MEMORY ADDRESS USING THE DETERMINED STRIDE 2613

FIG. 26

```
TILELOADQ TDEST, TSIB

 #GP IF TILES_CONFIGURED ==0
 #GP IF TDEST.COLS * 8 > PALETTE_TABLE[PALETTE_ID].BYTES_PER_ROW
 STARTM := TILECONFIG.STARTM
 #GP IF START >=TDEST.ROWS
 IF START == 0: // NOT RESTARTING, ZERO INCOMINGSTATE
 TILEZERO TDEST
 MEMBEGIN := TSIB.BASE +DISPLACEMENT
 STRIDE := TSIB.INDEX <<TSIB.SCALE
 NBYTES := TDEST.COLS *8
 WHILE START < TDEST.ROWS:
 MEMPTR := MEMBEGIN + START *STRIDE
 WRITE_ROW_AND_ZERO(TDEST,
 START,
 READ_MEMORY(MEMPTR, NBYTES),
 NBYTES)
 START := START +1
 ZERO_UPPER_ROWS(TDEST, TDEST.ROWS)
 ZERO_TILECONFIG_START()
// IN THE CASE OF A MEMORY FAULT IN THE MIDDLE OF AN INSTRUCTION
//TILECONFIG.STARTM := START
```

FIG. 27(A)

```
TILELOADW TDEST, TSIB

 #GP IF TILES_CONFIGURED ==0
 #GP IF TDEST.COLS * 2 > PALETTE_TABLE[PALETTE_ID].BYTES_PER_ROW
 STARTM := TILECONFIG.STARTM
 #GP IF START >=TDEST.ROWS
 IF START == 0: // NOT RESTARTING, ZERO INCOMINGSTATE
 TILEZERO TDEST
 MEMBEGIN := TSIB.BASE +DISPLACEMENT
 STRIDE := TSIB.INDEX <<TSIB.SCALE
 NBYTES := TDEST.COLS *2
 WHILE START < TDEST.ROWS:
 MEMPTR := MEMBEGIN + START *STRIDE
 WRITE_ROW_AND_ZERO(TDEST,
 START,
 READ_MEMORY(MEMPTR, NBYTES),
 NBYTES)
 START := START +1
 ZERO_UPPER_ROWS(TDEST, TDEST.ROWS)
 ZERO_TILECONFIG_START()
// IN THE CASE OF A MEMORY FAULT IN THE MIDDLE OF AN INSTRUCTION
//TILECONFIG.STARTM := START
```

FIG. 27(B)

```
TILELOADD TDEST, TSIB

 #GP IF TILES_CONFIGURED ==0
 #GP IF TDEST.COLS * 4 > PALETTE_TABLE[PALETTE_ID].BYTES_PER_ROW
 STARTM := TILECONFIG.STARTM
 #GP IF START >=TDEST.ROWS
 IF START == 0: // NOT RESTARTING, ZERO INCOMINGSTATE
 TILEZERO TDEST
 MEMBEGIN := TSIB.BASE +DISPLACEMENT
 STRIDE := TSIB.INDEX <<TSIB.SCALE
 NBYTES := TDEST.COLS *4
 WHILE START < TDEST.ROWS:
 MEMPTR := MEMBEGIN + START *STRIDE
 WRITE_ROW_AND_ZERO(TDEST,
 START,
 READ_MEMORY(MEMPTR, NBYTES),
 NBYTES)
 START := START +1
 ZERO_UPPER_ROWS(TDEST, TDEST.ROWS)
 ZERO_TILECONFIG_START()
// IN THE CASE OF A MEMORY FAULT IN THE MIDDLE OF AN INSTRUCTION
//TILECONFIG.STARTM := START
```

FIG. 27(C)

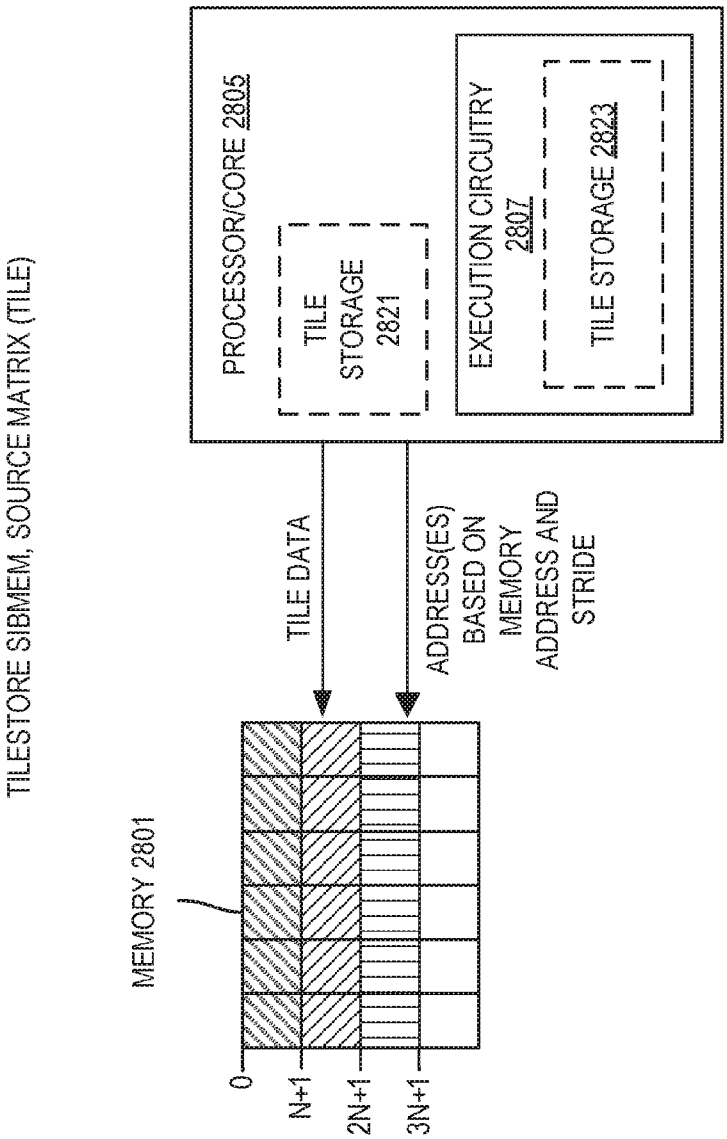

TILESTORE SIBMEM, SOURCE MATRIX (TILE)
PROCESSOR/CORE 2805
TILE STORAGE 2821
EXECUTION CIRCUITRY 2807
TILE STORAGE 2823
TILE DATA
ADDRESS(ES) BASED ON MEMORY ADDRESS AND STRIDE
MEMORY 2801
0
N+1
2N+1
3N+1

FIG. 28

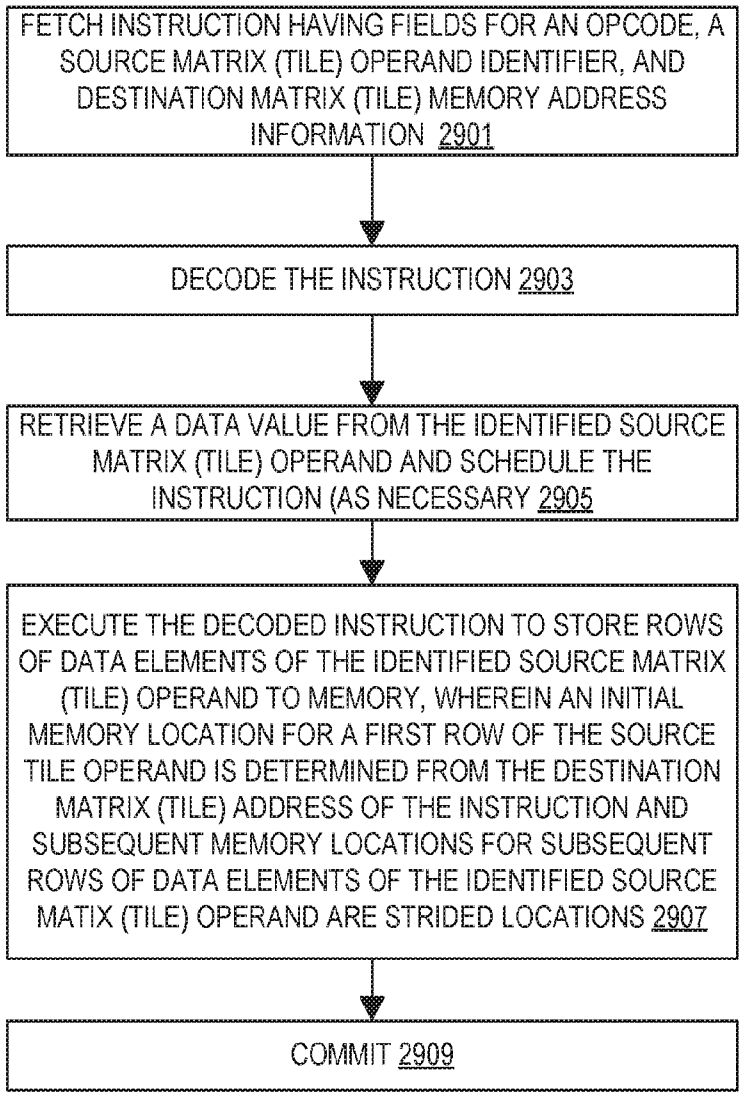

FETCH INSTRUCTION HAVING FIELDS FOR AN OPCODE, A SOURCE MATRIX (TILE) OPERAND IDENTIFIER, AND DESTINATION MATRIX (TILE) MEMORY ADDRESS INFORMATION 2901
DECODE THE INSTRUCTION 2903
RETRIEVE A DATA VALUE FROM THE IDENTIFIED SOURCE MATRIX (TILE) OPERAND AND SCHEDULE THE INSTRUCTION (AS NECESSARY 2905
EXECUTE THE DECODED INSTRUCTION TO STORE ROWS OF DATA ELEMENTS OF THE IDENTIFIED SOURCE MATRIX (TILE) OPERAND TO MEMORY, WHEREIN AN INITIAL MEMORY LOCATION FOR A FIRST ROW OF THE SOURCE TILE OPERAND IS DETERMINED FROM THE DESTINATION MATRIX (TILE) ADDRESS OF THE INSTRUCTION AND SUBSEQUENT MEMORY LOCATIONS FOR SUBSEQUENT ROWS OF DATA ELEMENTS OF THE IDENTIFIED SOURCE MATIX (TILE) OPERAND ARE STRIDED LOCATIONS 2907
COMMIT 2909

FIG. 29

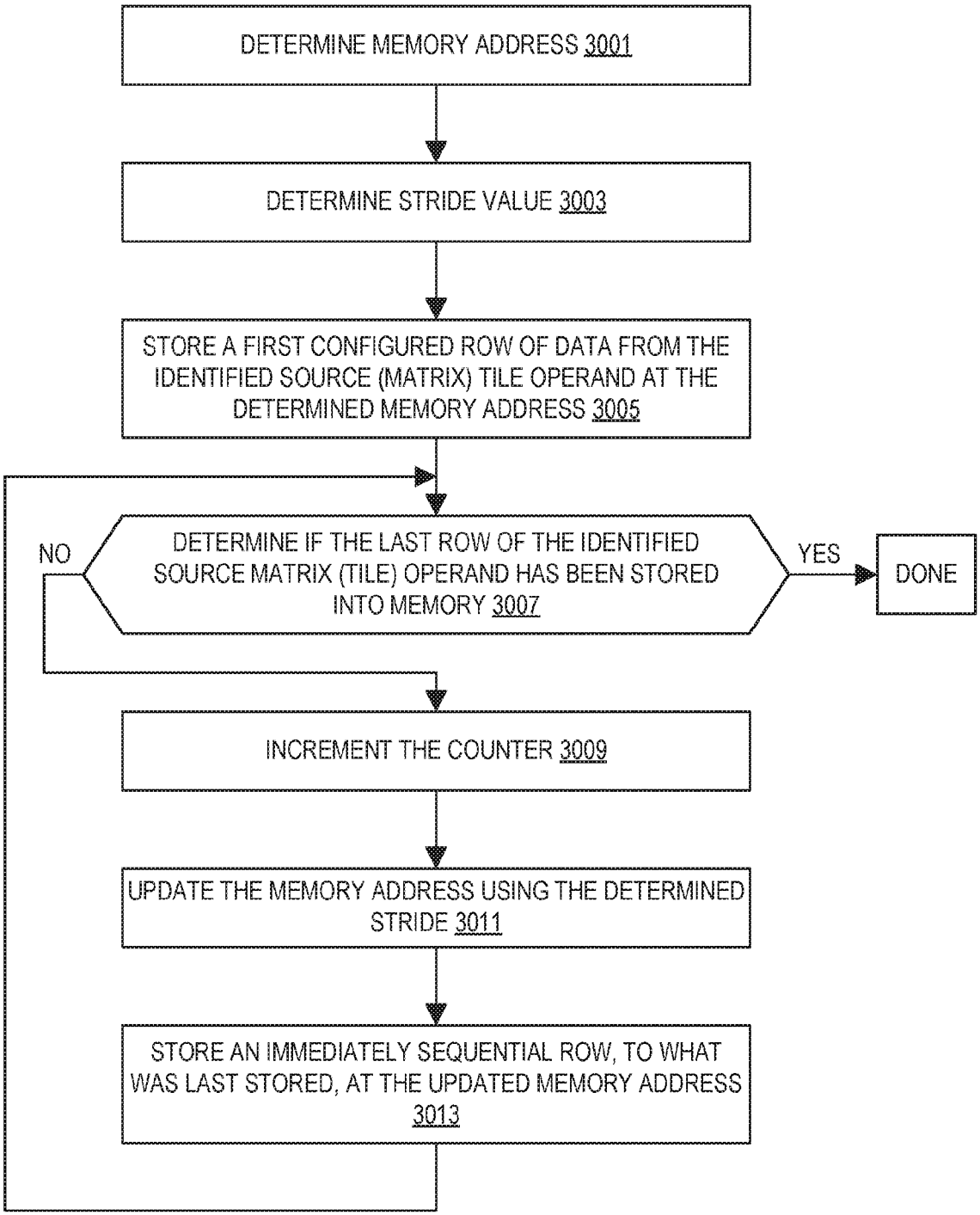

DETERMINE MEMORY ADDRESS 3001
DETERMINE STRIDE VALUE 3003
STORE A FIRST CONFIGURED ROW OF DATA FROM THE IDENTIFIED SOURCE (MATRIX) TILE OPERAND AT THE DETERMINED MEMORY ADDRESS 3005
DETERMINE IF THE LAST ROW OF THE IDENTIFIED SOURCE MATRIX (TILE) OPERAND HAS BEEN STORED INTO MEMORY 3007
NO
YES
DONE
INCREMENT THE COUNTER 3009
UPDATE THE MEMORY ADDRESS USING THE DETERMINED STRIDE 3011
STORE AN IMMEDIATELY SEQUENTIAL ROW, TO WHAT WAS LAST STORED, AT THE UPDATED MEMORY ADDRESS 3013

FIG. 30

```
TILESTORED TSIB, TSRC

#GP IF TILES_CONFIGURED == 0
#GP IF TSRC.COLS * 4 > PALETTE_TABLE[PALETTE_ID].BYTES_PER_ROW
START := TILECONFIG.STARTM
#GP IF START >= TDEST.ROWS

MEMBEGIN := TSIB.BASE + DISPLACEMENT
STRIDE := TSIB.INDEX << TSIB.SCALE

WHILE START < TDEST.ROWS:
MEMPTR := MEMBEGIN + START * STRIDE
WRITE_MEMORY(MEMPTR, 4*TDEST.COLS, TSRC.ROW[START])
START := START + 1
ZERO_TILECONFIG_START()
// IN THE CASE OF A MEMORY FAULT IN THE MIDDLE OF AN INSTRUCTION, THE
// TILECONFIG.STARTM := START
```

FIG. 31(A)

```
TILESTOREQ TSIB, TSRC

#GP IF TILES_CONFIGURED == 0
#GP IF TSRC.COLS * 8 > PALETTE_TABLE[PALETTE_ID].BYTES_PER_ROW
START := TILECONFIG.STARTM
#GP IF START >= TDEST.ROWS

MEMBEGIN := TSIB.BASE + DISPLACEMENT
STRIDE := TSIB.INDEX << TSIB.SCALE

WHILE START < TDEST.ROWS:
MEMPTR := MEMBEGIN + START * STRIDE
WRITE_MEMORY(MEMPTR, 8*TDEST.COLS, TSRC.ROW[START])
START := START + 1
ZERO_TILECONFIG_START()
// IN THE CASE OF A MEMORY FAULT IN THE MIDDLE OF AN INSTRUCTION, THE
// TILECONFIG.STARTM := START
```

FIG. 31(B)

```
TILESTOREW TSIB, TSRC

#GP IF TILES_CONFIGURED == 0
#GP IF TSRC.COLS * 2 > PALETTE_TABLE[PALETTE_ID].BYTES_PER_ROW
START := TILECONFIG.STARTM
#GP IF START >= TDEST.ROWS

MEMBEGIN := TSIB.BASE + DISPLACEMENT
STRIDE := TSIB.INDEX << TSIB.SCALE

WHILE START < TDEST.ROWS:
MEMPTR := MEMBEGIN + START * STRIDE
WRITE_MEMORY(MEMPTR, 2*TDEST.COLS, TSRC.ROW[START])
START := START + 1
ZERO_TILECONFIG_START()
// IN THE CASE OF A MEMORY FAULT IN THE MIDDLE OF AN INSTRUCTION, THE
// TILECONFIG.STARTM := START
```

FIG. 31(C)

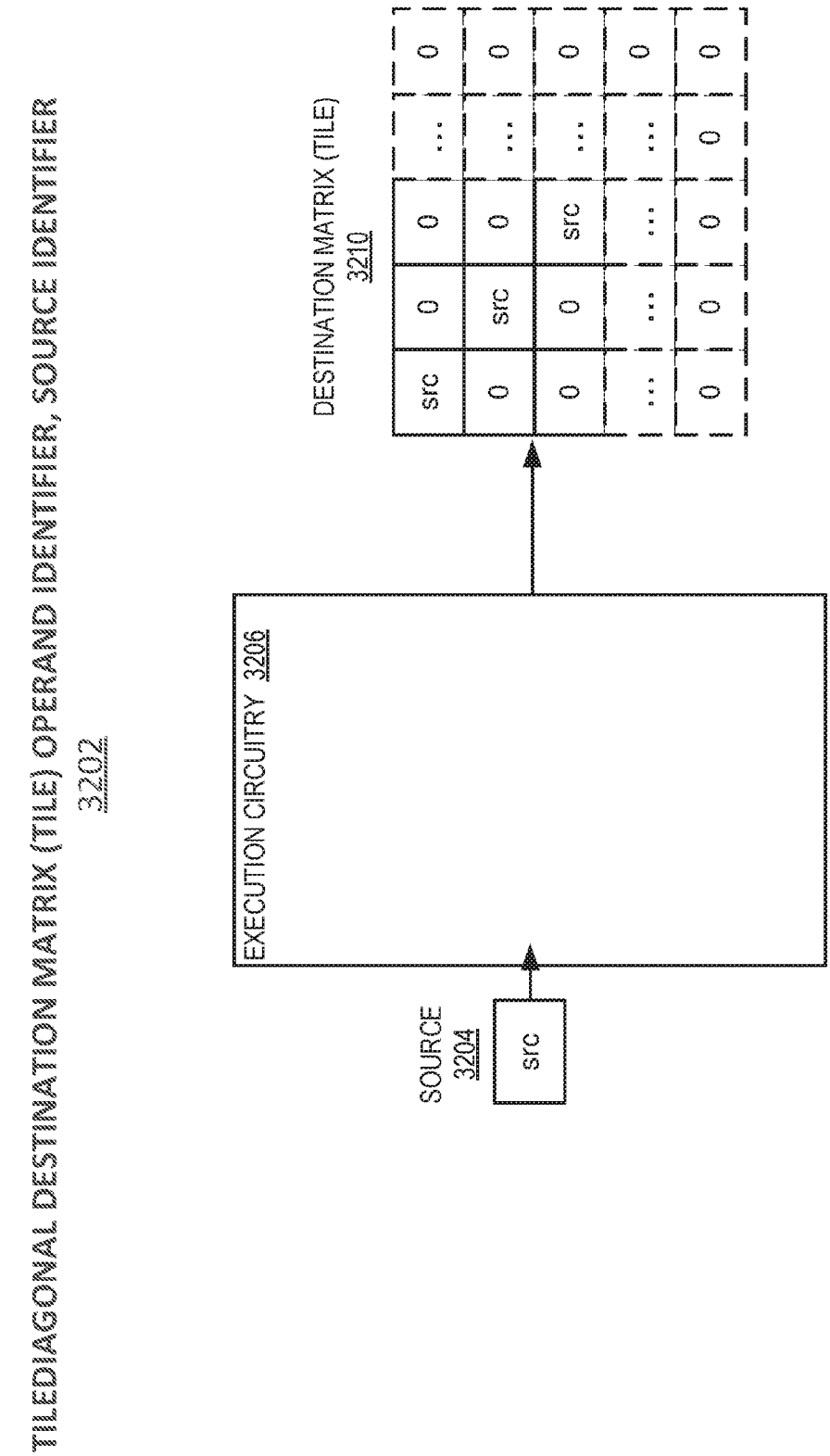

TILEDIAGONAL DESTINATION MATRIX (TILE) OPERAND IDENTIFIER, SOURCE IDENTIFIER
3202
DESTINATION MATRIX (TILE)
3210
src 0 0 ... 0
0 src 0 ... 0
0 0 src ... 0
... ...
0 0 0 0 0
EXECUTION CIRCUITRY 3206
SOURCE
3204
src

FIG. 32

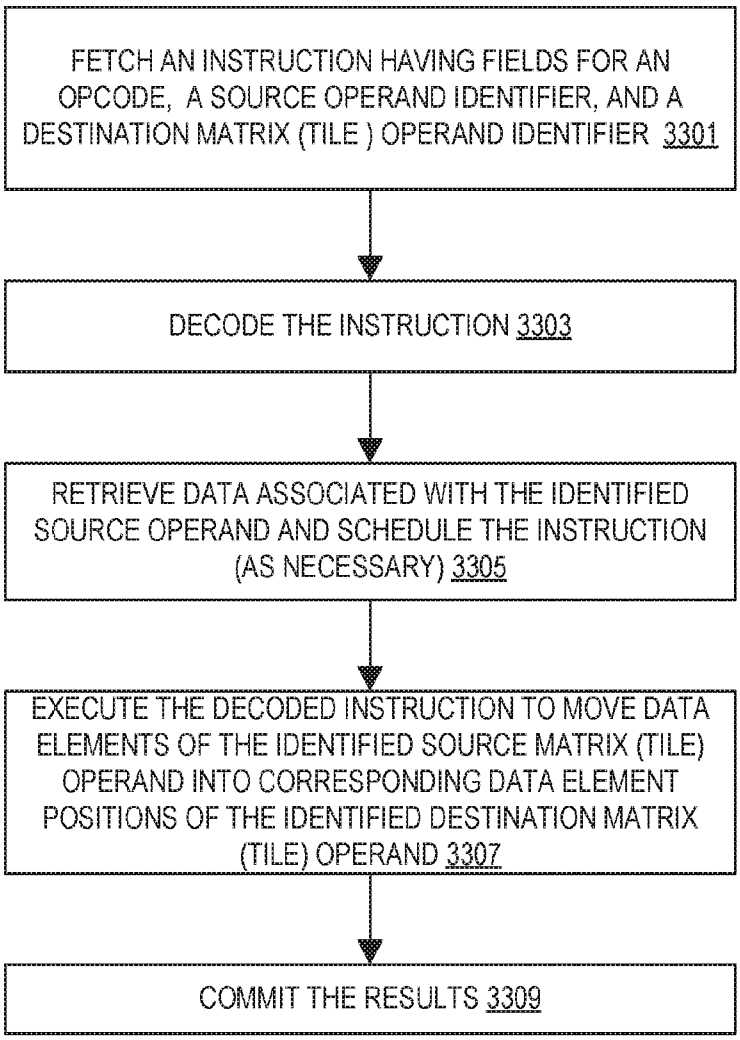

FETCH AN INSTRUCTION HAVING FIELDS FOR AN OPCODE, A SOURCE OPERAND IDENTIFIER, AND A DESTINATION MATRIX (TILE) OPERAND IDENTIFIER 3301
DECODE THE INSTRUCTION 3303
RETRIEVE DATA ASSOCIATED WITH THE IDENTIFIED SOURCE OPERAND AND SCHEDULE THE INSTRUCTION (AS NECESSARY) 3305
EXECUTE THE DECODED INSTRUCTION TO MOVE DATA ELEMENTS OF THE IDENTIFIED SOURCE MATRIX (TILE) OPERAND INTO CORRESPONDING DATA ELEMENT POSITIONS OF THE IDENTIFIED DESTINATION MATRIX (TILE) OPERAND 3307
COMMIT THE RESULTS 3309

FIG. 33

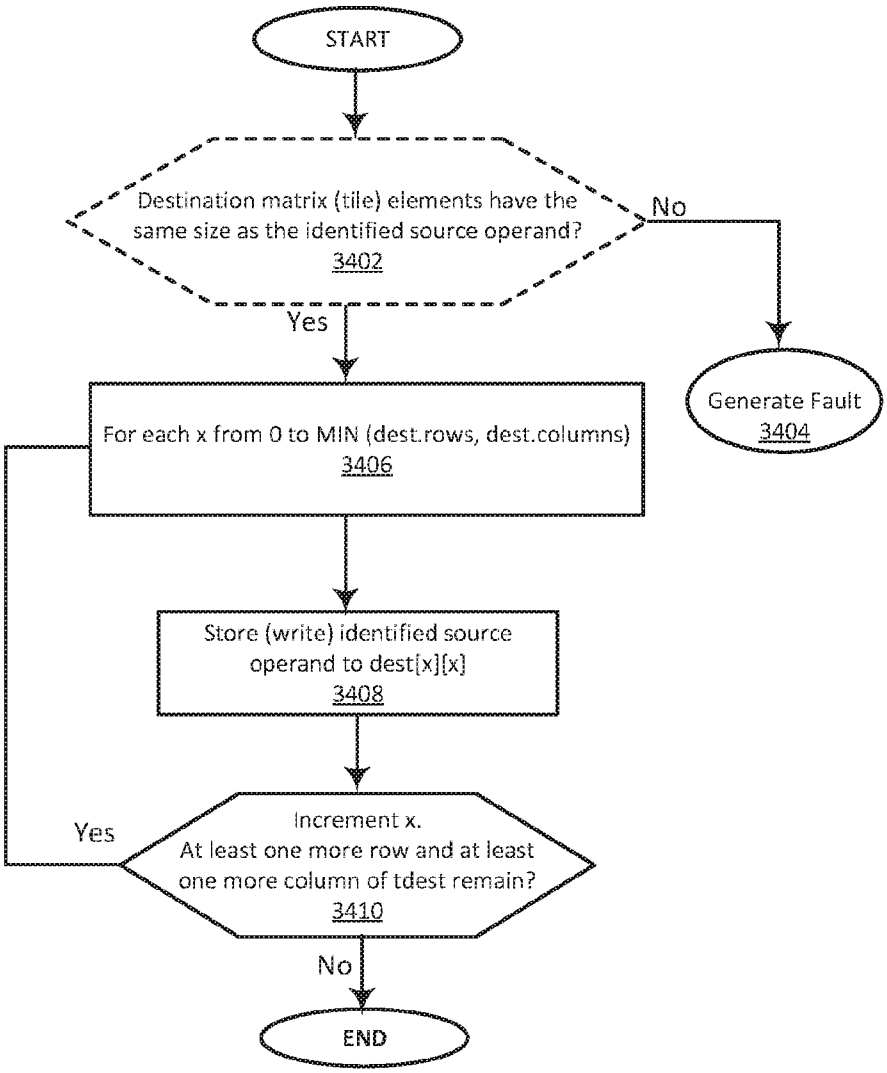

START
Destination matrix (tile) elements have the same size as the identified source operand?
3402
No
Yes
Generate Fault
3404
For each x from 0 to MIN (dest.rows, dest.columns)
3406
Store (write) identified source operand to dest[x][x]
3408
Increment x.
At least one more row and at least one more column of tdest remain?
3410
Yes
No
END

FIG. 34

```
3502
 TILEDIAGONALD TDEST,  SRC
#GP IF TILES_CONFIGURED == 0
#GP IF TDEST.NUM_BYTES_PER_ELEMENT != 4
#GP IF TDEST.NUM_BYTES_PER_ELEMENT != SRC.NUM_BYTES

LOOP_ITERATIONS = MIN(TDEST.NUM_ROWS, TDEST.NUM_COLUMNS)

FOR X IN 0...LOOP_ITERATIONS-1:
        TDEST[X]X] ← SRC.
        ZERO_UNUSED_ELEMENTS (TDEST)
```

```
3504
 TILEDIAGONALW TDEST,  SRC
 #GP IF TILES_CONFIGURED == 0
 #GP IF TDEST.NUM_BYTES_PER_ELEMENT != 2
 #GP IF TDEST.NUM_BYTES_PER_ELEMENT != SRC.NUM_BYTES

 LOOP_ITERATIONS = MIN(TDEST.NUM_ROWS, TDEST.NUM_COLUMNS)

 FOR X IN 0...LOOP_ITERATIONS-1:
         TDEST[X]X] ← SRC.
         ZERO_UNUSED_ELEMENTS (TDEST)
```

FIG. 35

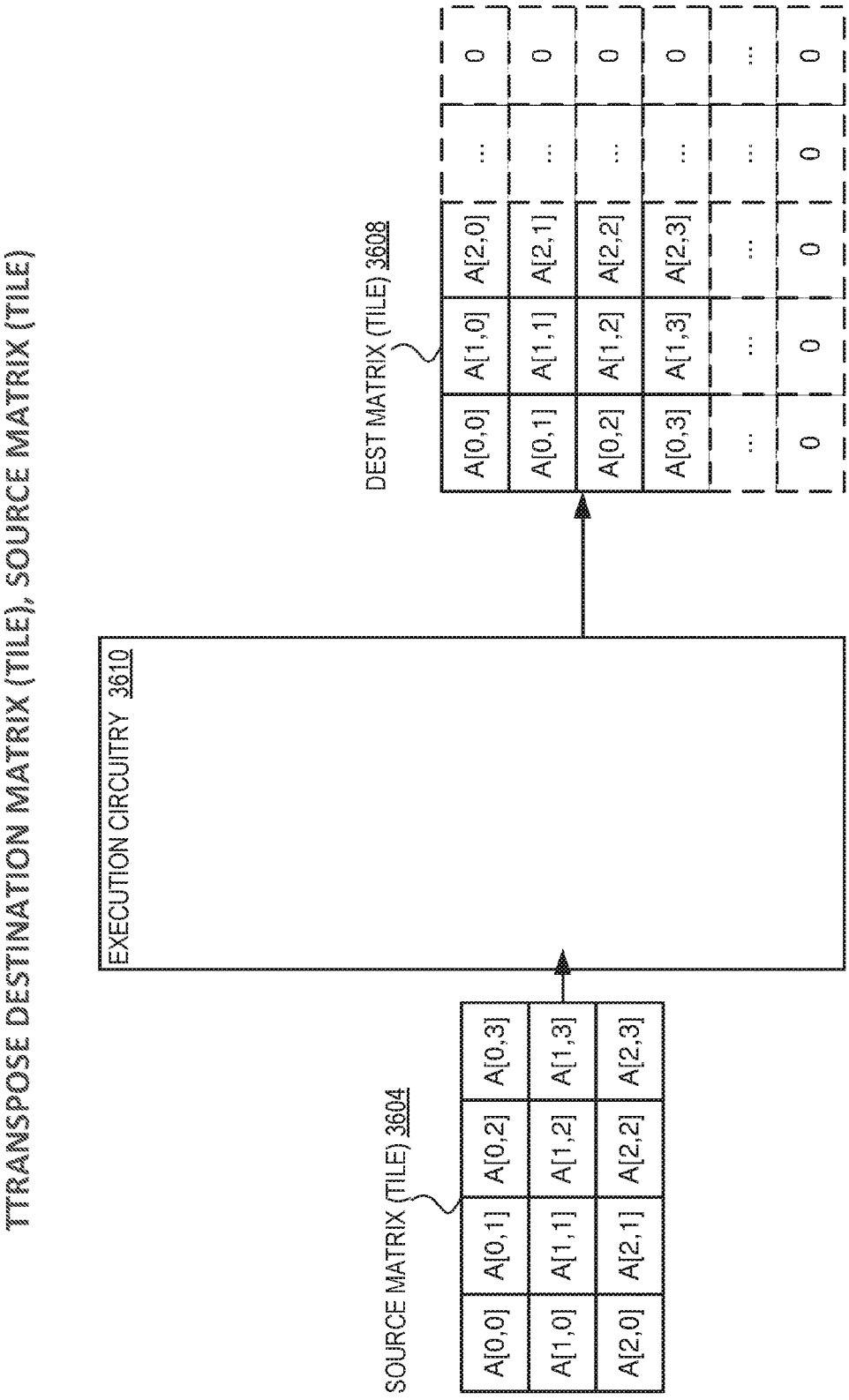

TTRANSPOSE DESTINATION MATRIX (TILE), SOURCE MATRIX (TILE)
SOURCE MATRIX (TILE) 3604
A[0,0] A[0,1] A[0,2] A[0,3]
A[1,0] A[1,1] A[1,2] A[1,3]
A[2,0] A[2,1] A[2,2] A[2,3]
EXECUTION CIRCUITRY 3610
DEST MATRIX (TILE) 3608
A[0,0] A[1,0] A[2,0] ... 0
A[0,1] A[1,1] A[2,1] ... 0
A[0,2] A[1,2] A[2,2] ... 0
A[0,3] A[1,3] A[2,3] ... 0
... ...
0 0 0 0 0

FIG. 36

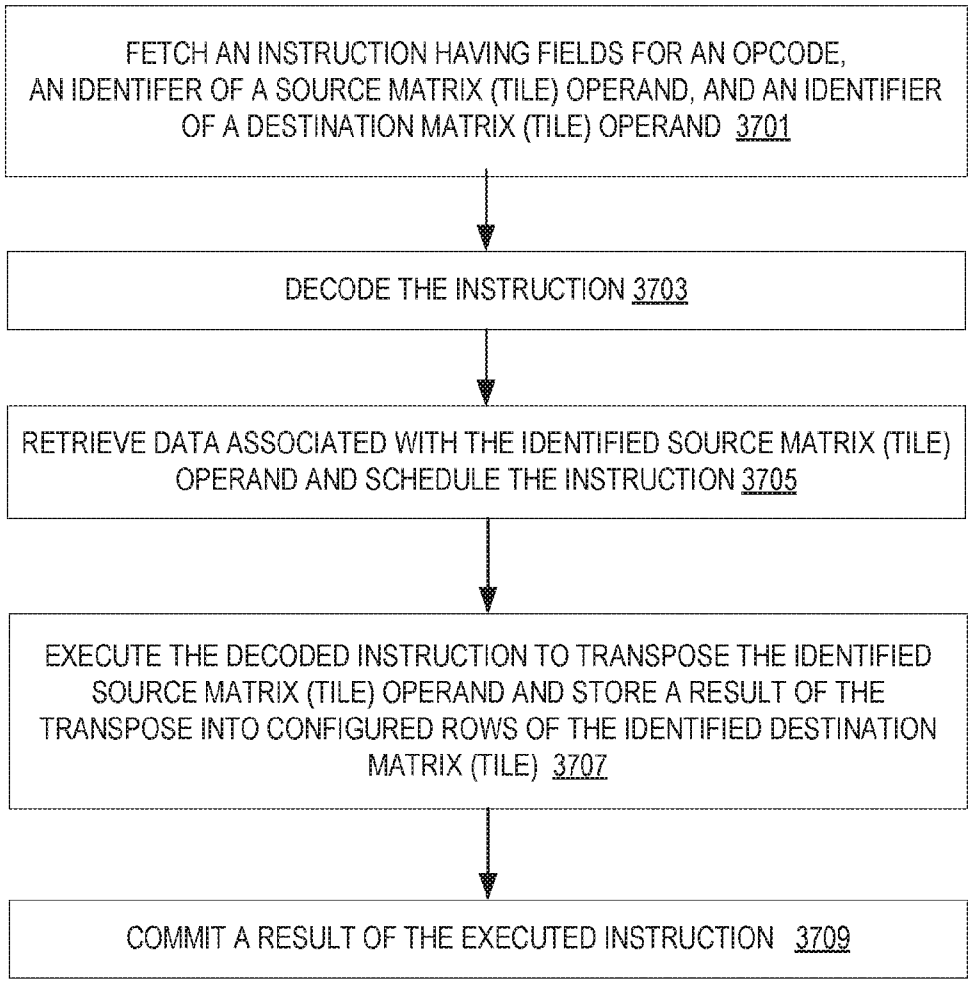

FETCH AN INSTRUCTION HAVING FIELDS FOR AN OPCODE, AN IDENTIFER OF A SOURCE MATRIX (TILE) OPERAND, AND AN IDENTIFIER OF A DESTINATION MATRIX (TILE) OPERAND 3701
DECODE THE INSTRUCTION 3703
RETRIEVE DATA ASSOCIATED WITH THE IDENTIFIED SOURCE MATRIX (TILE) OPERAND AND SCHEDULE THE INSTRUCTION 3705
EXECUTE THE DECODED INSTRUCTION TO TRANSPOSE THE IDENTIFIED SOURCE MATRIX (TILE) OPERAND AND STORE A RESULT OF THE TRANSPOSE INTO CONFIGURED ROWS OF THE IDENTIFIED DESTINATION MATRIX (TILE) 3707
COMMIT A RESULT OF THE EXECUTED INSTRUCTION 3709

FIG. 37

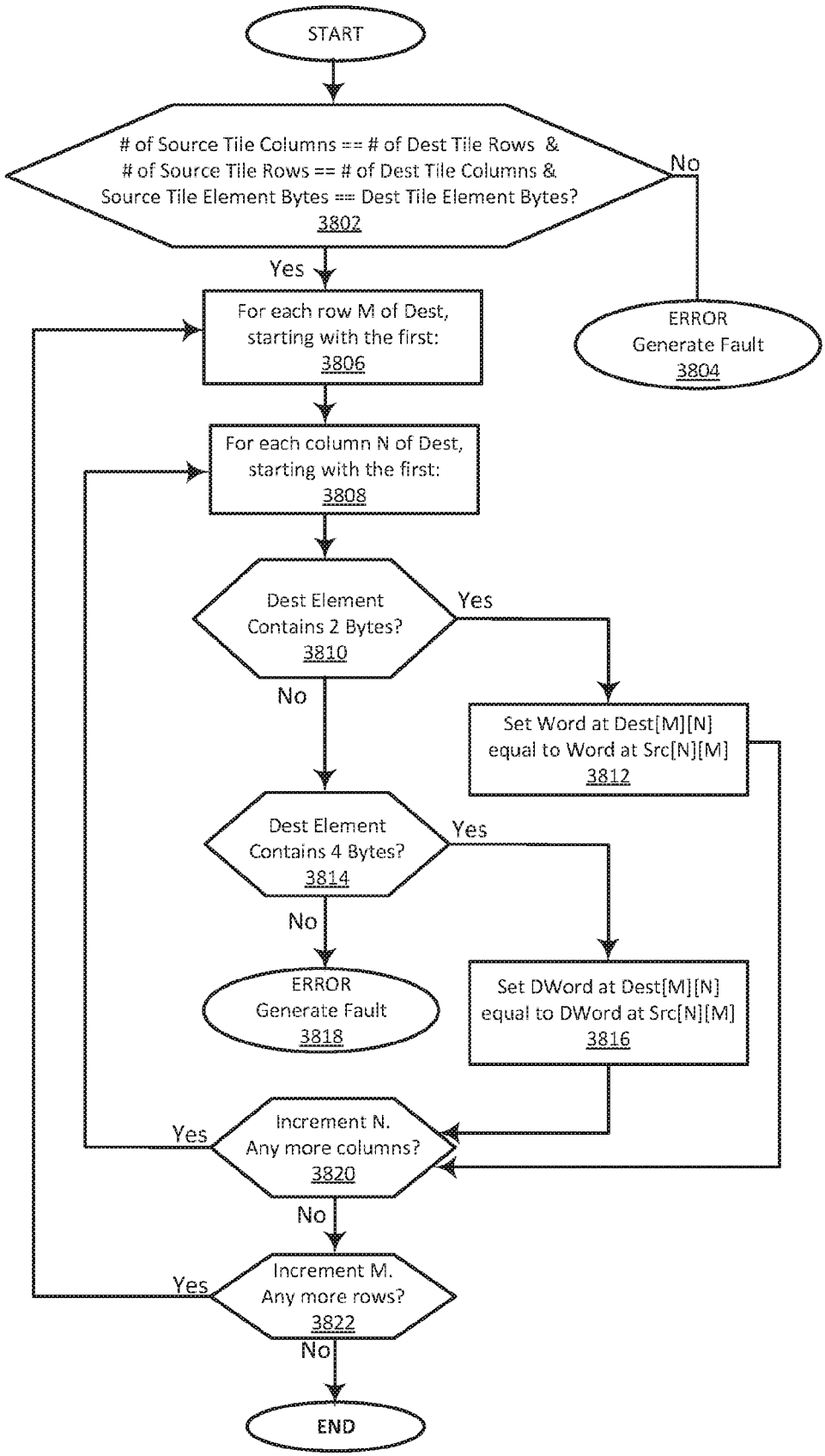

START
# of Source Tile Columns == # of Dest Tile Rows &
# of Source Tile Rows == # of Dest Tile Columns &
Source Tile Element Bytes == Dest Tile Element Bytes?
3802
No
Yes
ERROR
Generate Fault
3804
For each row M of Dest,
starting with the first:
3806
For each column N of Dest,
starting with the first:
3808
Dest Element
Contains 2 Bytes?
3810
Yes
No
Set Word at Dest[M][N]
equal to Word at Src[N][M]
3812
Dest Element
Contains 4 Bytes?
3814
Yes
No
ERROR
Generate Fault
3818
Set DWord at Dest[M][N]
equal to DWord at Src[N][M]
3816
Increment N.
Any more columns?
3820
Yes
No
Increment M.
Any more rows?
3822
Yes
No
END

FIG. 38

```
3902
 TILETRANSPOSED TDEST,  TSRC
#GP IF TILES_CONFIGURED == 0
#GP IF TSRC.COLS != TDEST.ROWS
#GP IF TDEST.COLS * 4 > PALETTE_TABLE[PALETTE_ID].BYTES_PER_ROW

FOR I IN 0...TDEST.ROWS-1:
FOR J IN 0..TDEST.COLS-1:
TMP.DWORD[J] := TSRC.ROW[J].DWORD[I]
WRITE_ROW_AND_ZERO(TDEST, I, TMP, 4 * TDEST.COLS)
ZERO_UPPER_ROWS(TDEST, TDEST.ROWS)
```

```
3904
 TILETRANSPOSEW TDEST,  TSRC
#GP IF TILES_CONFIGURED == 0
#GP IF TSRC.COLS != TDEST.ROWS
#GP IF TDEST.COLS * 2 > PALETTE_TABLE[PALETTE_ID].BYTES_PER_ROW

FOR I IN 0...TDEST.ROWS-1:
FOR J IN 0..TDEST.COLS-1:
TMP.DWORD[J] := TSRC.ROW[J].DWORD[I]
WRITE_ROW_AND_ZERO(TDEST, I, TMP, 2 * TDEST.COLS)
ZERO_UPPER_ROWS(TDEST, TDEST.ROWS)
```

FIG. 39

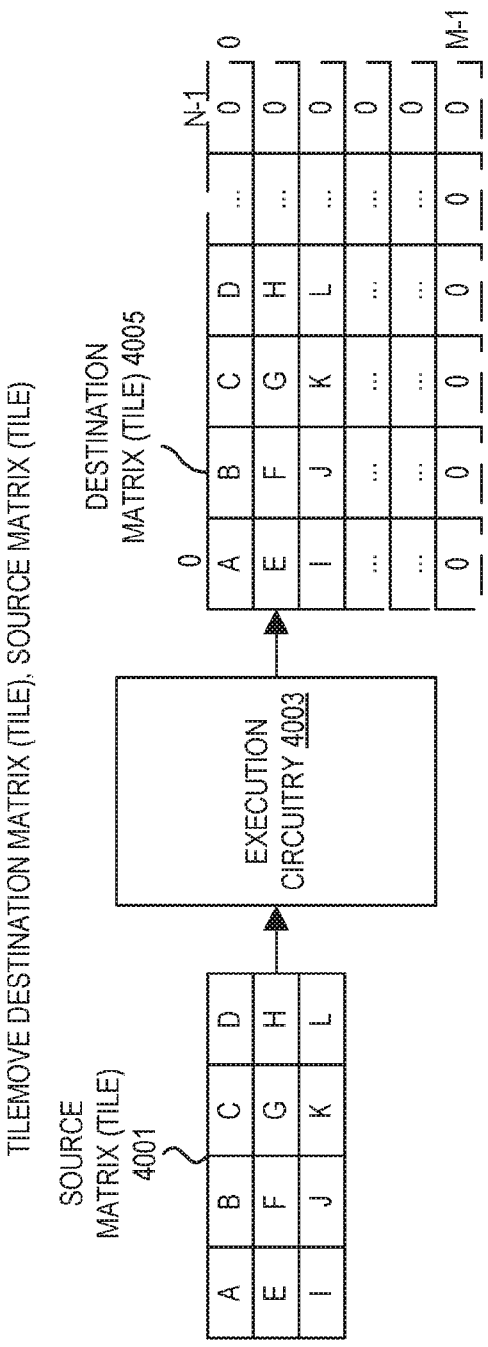

TILEMOVE DESTINATION MATRIX (TILE), SOURCE MATRIX (TILE)
SOURCE MATRIX (TILE) 4001
A B C D
E F G H
I J K L
EXECUTION CIRCUITRY 4003
DESTINATION MATRIX (TILE) 4005
0
N-1
M-1
A B C D ... 0
E F G H ... 0
I J K L ... 0
... ... 0
... ... 0
0 0 0 0 0 0

FIG. 40

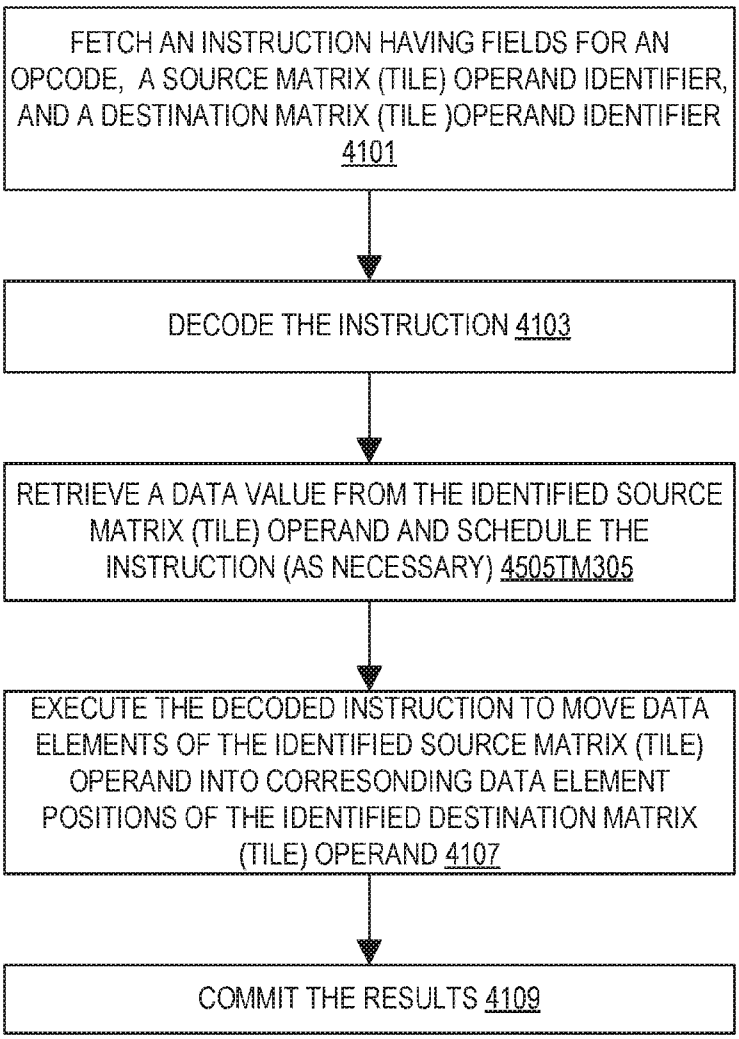

FETCH AN INSTRUCTION HAVING FIELDS FOR AN OPCODE, A SOURCE MATRIX (TILE) OPERAND IDENTIFIER, AND A DESTINATION MATRIX (TILE)OPERAND IDENTIFIER 4101
DECODE THE INSTRUCTION 4103
RETRIEVE A DATA VALUE FROM THE IDENTIFIED SOURCE MATRIX (TILE) OPERAND AND SCHEDULE THE INSTRUCTION (AS NECESSARY) 4505TM305
EXECUTE THE DECODED INSTRUCTION TO MOVE DATA ELEMENTS OF THE IDENTIFIED SOURCE MATRIX (TILE) OPERAND INTO CORRESONDING DATA ELEMENT POSITIONS OF THE IDENTIFIED DESTINATION MATRIX (TILE) OPERAND 4107
COMMIT THE RESULTS 4109

FIG. 41

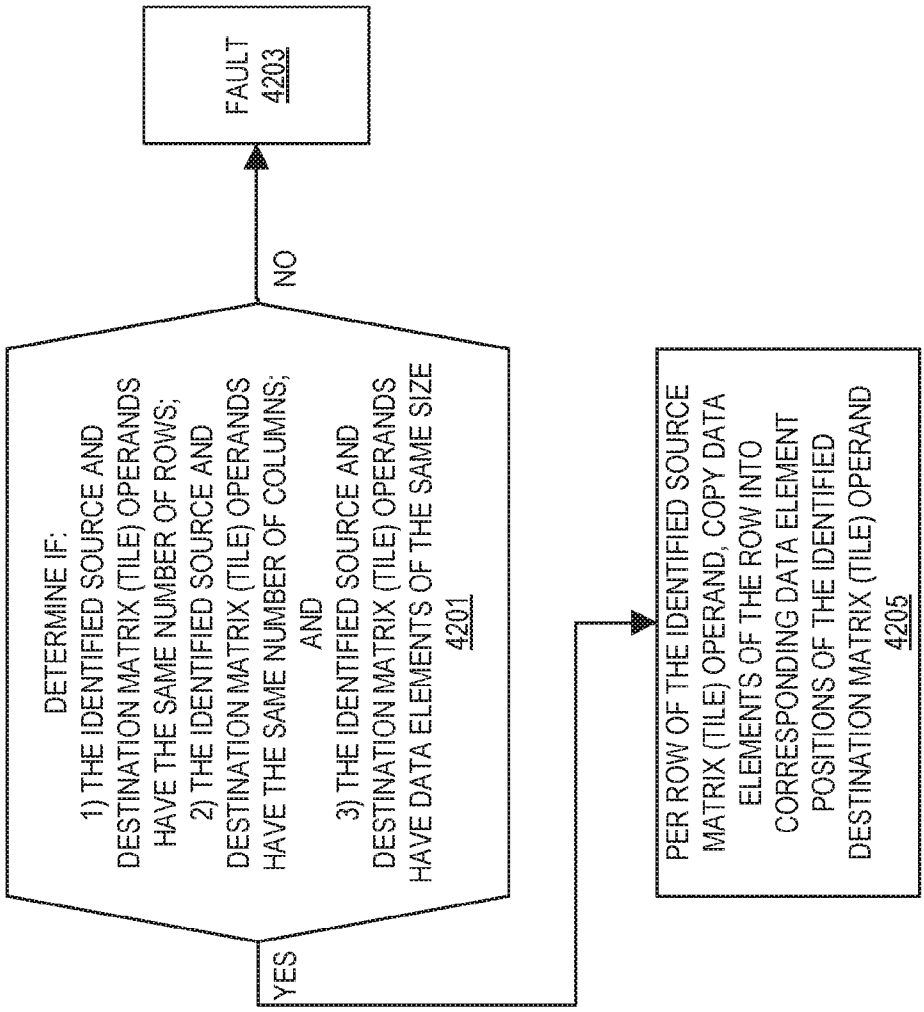

DETERMINE IF:
1) THE IDENTIFIED SOURCE AND DESTINATION MATRIX (TILE) OPERANDS HAVE THE SAME NUMBER OF ROWS;
2) THE IDENTIFIED SOURCE AND DESTINATION MATRIX (TILE) OPERANDS HAVE THE SAME NUMBER OF COLUMNS; AND
3) THE IDENTIFIED SOURCE AND DESTINATION MATRIX (TILE) OPERANDS HAVE DATA ELEMENTS OF THE SAME SIZE
4201
NO
FAULT
4203
YES
PER ROW OF THE IDENTIFIED SOURCE MATRIX (TILE) OPERAND, COPY DATA ELEMENTS OF THE ROW INTO CORRESPONDING DATA ELEMENT POSITIONS OF THE IDENTIFIED DESTINATION MATRIX (TILE) OPERAND
4205

FIG. 42

```
TILEMOVED TDEST, TSRC

 #GP IF TILES_CONFIGURED == 0
 #GP IF TSRC.COLS != TDEST.COLS
 #GP IF TSRC.ROWS != TDEST.ROWS
 #GP IF TDEST.COLS * 4 >
PALETTE_TABLE[PALETTE_ID].BYTES_PER_ROW

 FOR I IN 0..TDEST.ROWS-1:
 WRITE_ROW_AND_ZERO(TDEST, I, TSRC.ROW[I], 4 * TDEST.COLS)
 ZERO_UPPER_ROWS(TDEST, TDEST.ROWS)


TILEMOVEW TDEST, TSRC

 #GP IF TILES_CONFIGURED == 0
 #GP IF TSRC.COLS != TDEST.COLS
 #GP IF TSRC.ROWS != TDEST.ROWS
 #GP IF TDEST.COLS * 2 >
PALETTE_TABLE[PALETTE_ID].BYTES_PER_ROW

 FOR I IN 0..TDEST.ROWS-1:
 WRITE_ROW_AND_ZERO(TDEST, I, TSRC.ROW[I], 2 * TDEST.COLS)
 ZERO_UPPER_ROWS(TDEST, TDEST.ROWS)
```

FIG. 43

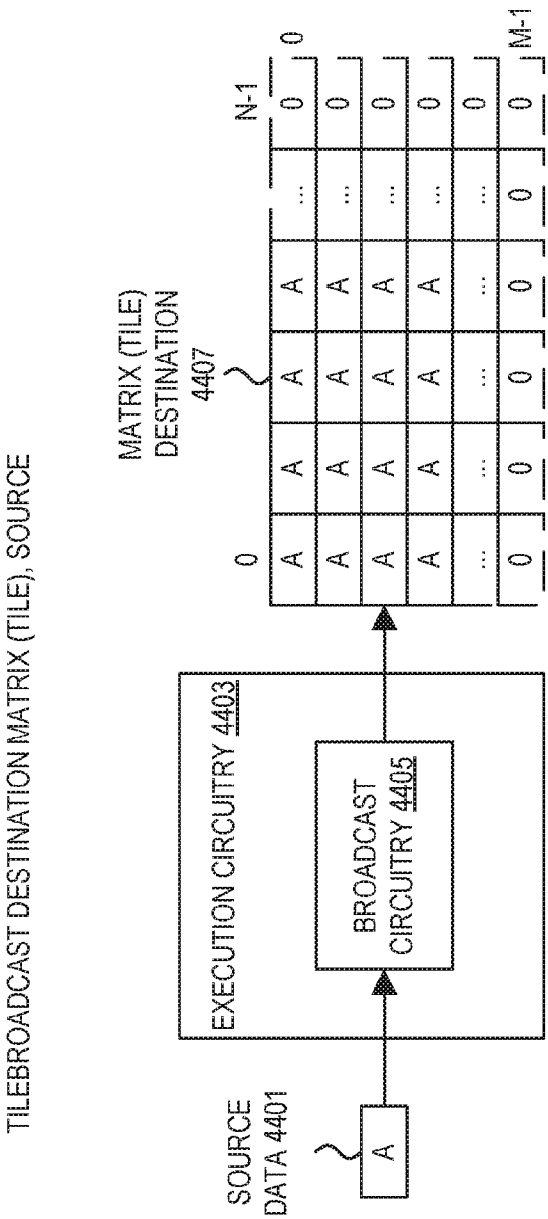

TILEBROADCAST DESTINATION MATRIX (TILE), SOURCE
SOURCE DATA 4401
A
EXECUTION CIRCUITRY 4403
BROADCAST CIRCUITRY 4405
MATRIX (TILE) DESTINATION 4407
0
N-1
M-1
A
0

FIG. 44

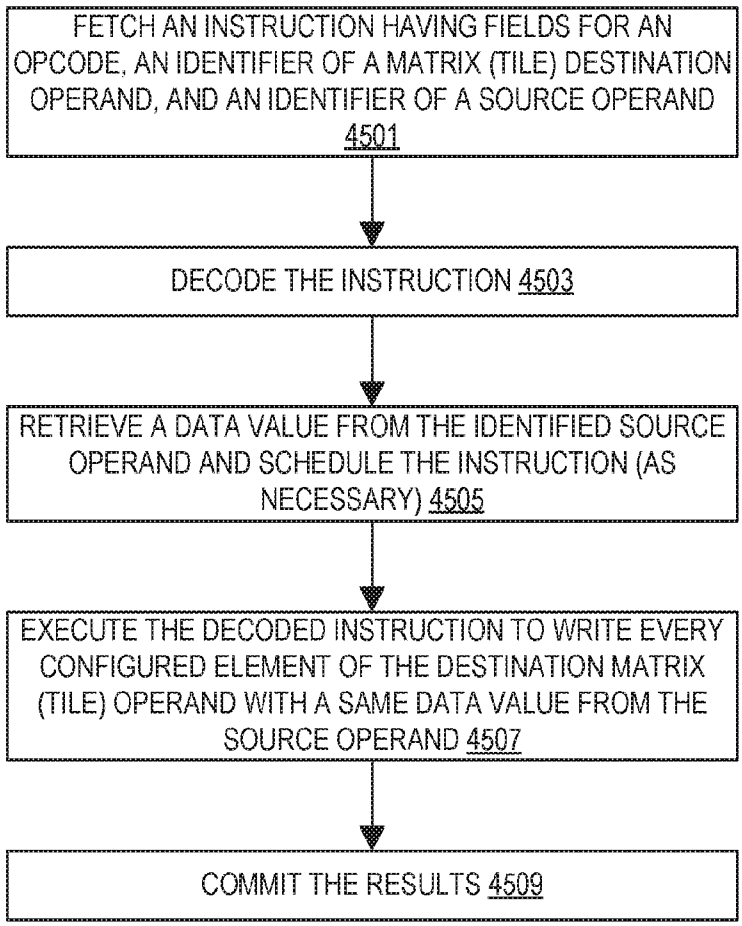

FETCH AN INSTRUCTION HAVING FIELDS FOR AN OPCODE, AN IDENTIFIER OF A MATRIX (TILE) DESTINATION OPERAND, AND AN IDENTIFIER OF A SOURCE OPERAND 4501
DECODE THE INSTRUCTION 4503
RETRIEVE A DATA VALUE FROM THE IDENTIFIED SOURCE OPERAND AND SCHEDULE THE INSTRUCTION (AS NECESSARY) 4505
EXECUTE THE DECODED INSTRUCTION TO WRITE EVERY CONFIGURED ELEMENT OF THE DESTINATION MATRIX (TILE) OPERAND WITH A SAME DATA VALUE FROM THE SOURCE OPERAND 4507
COMMIT THE RESULTS 4509

FIG. 45

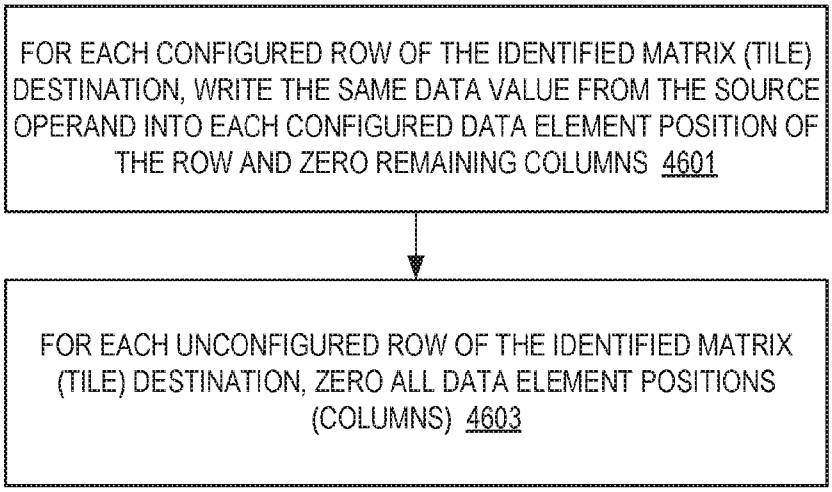

FOR EACH CONFIGURED ROW OF THE IDENTIFIED MATRIX (TILE) DESTINATION, WRITE THE SAME DATA VALUE FROM THE SOURCE OPERAND INTO EACH CONFIGURED DATA ELEMENT POSITION OF THE ROW AND ZERO REMAINING COLUMNS 4601
FOR EACH UNCONFIGURED ROW OF THE IDENTIFIED MATRIX (TILE) DESTINATION, ZERO ALL DATA ELEMENT POSITIONS (COLUMNS) 4603

FIG. 46

```
TILEBROADCASTD TDEST, SRC
#GP IF TILES_CONFIGURED == 0
#GP IF TDEST.COLS * 4 >
PALETTE_TABLE[PALETTE_ID].BYTES_PER_ROW

 FOR I IN 0..TDEST.ROWS-1:
 FOR J IN 0...TDEST.COLS-1:
 TMP.DWORD[J] := SRC
WRITE_ROW_AND_ZERO(TDEST, I, TMP, 4 *
TDEST.COLS)
ZERO_UPPER_ROWS(TDEST, TDEST.ROWS)
```

```
TILEBROADCASTW TDEST, SRC
#GP IF TILES_CONFIGURED == 0
#GP IF TDEST.COLS * 2 >
PALETTE_TABLE[PALETTE_ID].BYTES_PER_ROW

 FOR I IN 0..TDEST.ROWS-1:
 FOR J IN 0...TDEST.COLS-1:
 TMP.WORD[J] := SRC
 WRITE_ROW_AND_ZERO(TDEST, I, TMP, 2 *
TDEST.COLS)
 ZERO_UPPER_ROWS(TDEST, TDEST.ROWS)
```

FIG. 47

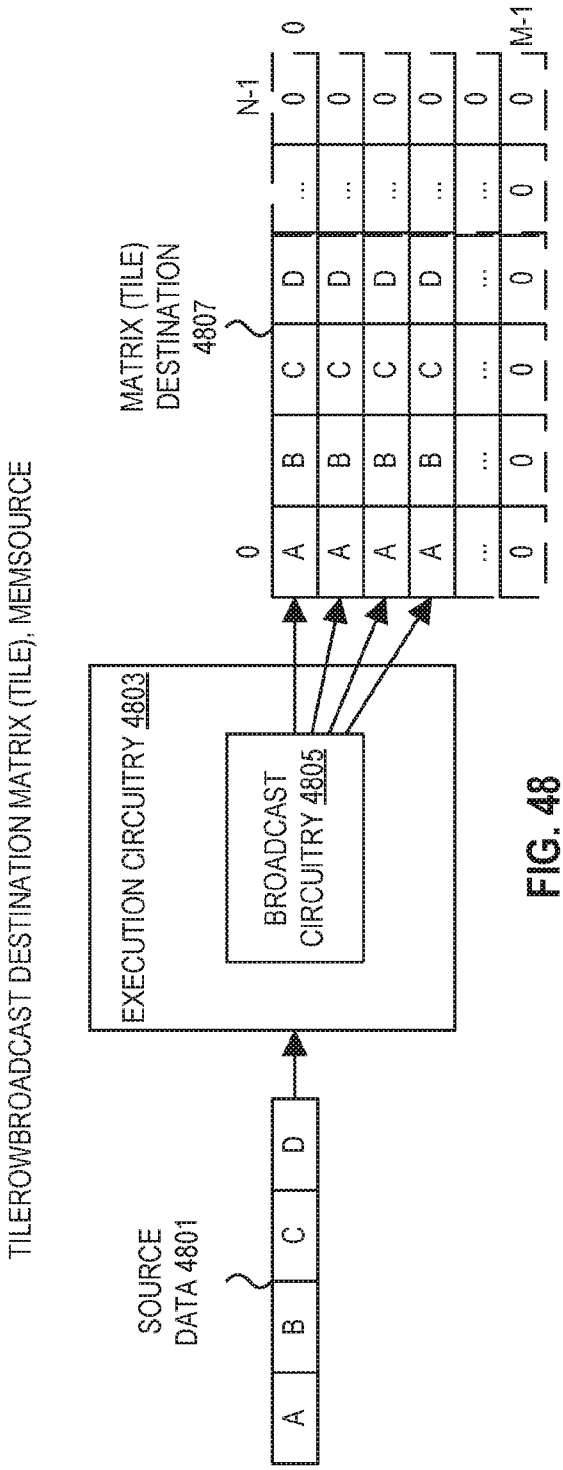

TILEROWBROADCAST DESTINATION MATRIX (TILE), MEMSOURCE
SOURCE DATA 4801
A
B
C
D
EXECUTION CIRCUITRY 4803
BROADCAST CIRCUITRY 4805
MATRIX (TILE) DESTINATION 4807
0
N-1
M-1

FIG. 48

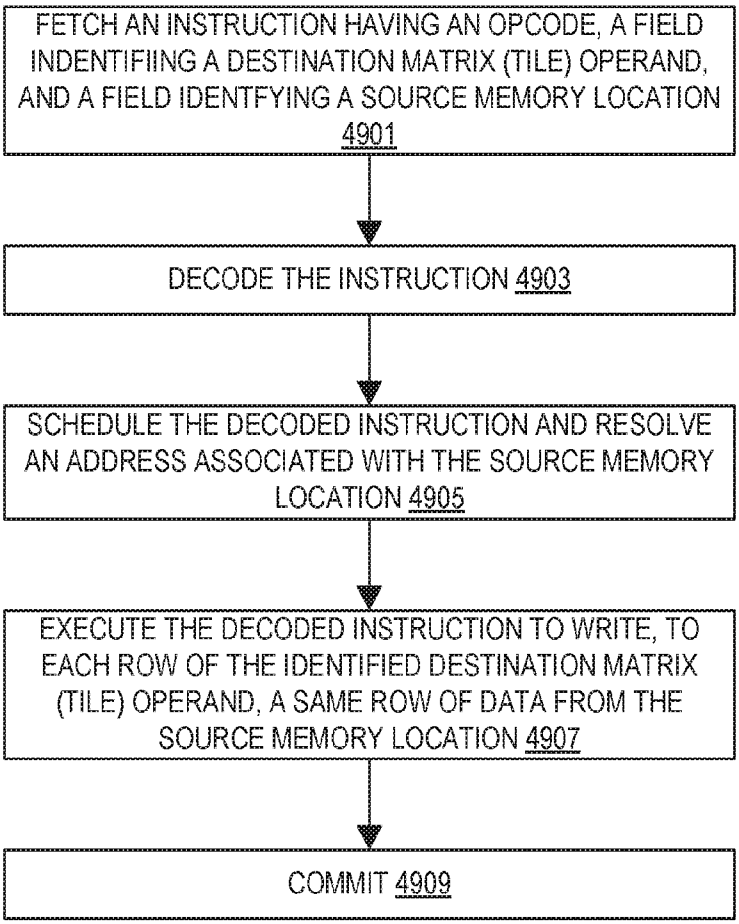

FETCH AN INSTRUCTION HAVING AN OPCODE, A FIELD INDENTIFIING A DESTINATION MATRIX (TILE) OPERAND, AND A FIELD IDENTFYING A SOURCE MEMORY LOCATION 4901
DECODE THE INSTRUCTION 4903
SCHEDULE THE DECODED INSTRUCTION AND RESOLVE AN ADDRESS ASSOCIATED WITH THE SOURCE MEMORY LOCATION 4905
EXECUTE THE DECODED INSTRUCTION TO WRITE, TO EACH ROW OF THE IDENTIFIED DESTINATION MATRIX (TILE) OPERAND, A SAME ROW OF DATA FROM THE SOURCE MEMORY LOCATION 4907
COMMIT 4909

FIG. 49

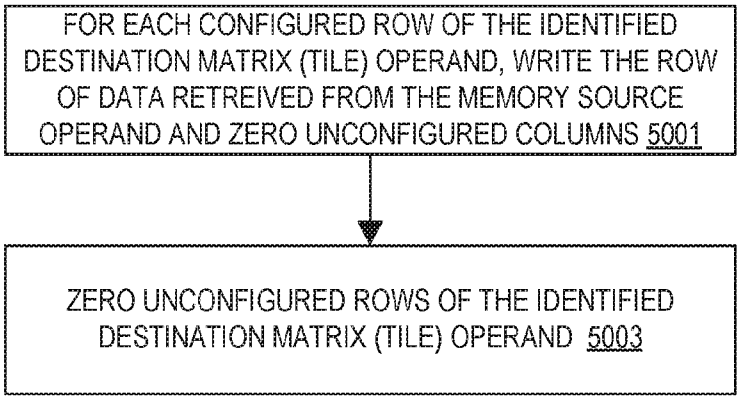

FOR EACH CONFIGURED ROW OF THE IDENTIFIED DESTINATION MATRIX (TILE) OPERAND, WRITE THE ROW OF DATA RETREIVED FROM THE MEMORY SOURCE OPERAND AND ZERO UNCONFIGURED COLUMNS 5001
ZERO UNCONFIGURED ROWS OF THE IDENTIFIED DESTINATION MATRIX (TILE) OPERAND 5003

FIG. 50

```
TILECOLBROADCASTD TDEST, TMEMCOL
 #GP IF TILES_CONFIGURED == 0
 #GP IF TDEST.COLS * 4 >
PALETTE_TABLE[PALETTE_ID].BYTES_PER_ROW

 TDATA := READ_MEMORY(TMEMCOL, 4 * TDEST.ROWS)

 FOR I IN 0 ... TDEST.ROWS-1:
 // COPY EACH ELEMENT TO A WHOLE TEMP ROW.
 // THEN WRITE THE TEMP ROW TO THE TILE.
 FOR J IN 0 ... TDEST.COLS-1 :
 TMP.DWORD[J] := TDATA.DWORD[I]
 WRITE_ROW_AND_ZERO(TDEST, I, TMP, 4 * TDEST.COLS)
 ZERO_UPPER_ROWS(TDEST, TDEST.ROWS)


TILECOLBROADCASTW TDEST, TMEMCOL
 #GP IF TILES_CONFIGURED == 0
 #GP IF TDEST.COLS * 2 >
PALETTE_TABLE[PALETTE_ID].BYTES_PER_ROW

 TDATA := READ_MEMORY(TMEMCOL, 2 * TDEST.ROWS)

 FOR I IN 0 ... TDEST.ROWS-1:
 // COPY EACH ELEMENT TO A WHOLE TEMP ROW.
 // THEN WRITE THE TEMP ROW TO THE TILE.
 FOR J IN 0 ... TDEST.COLS-1 :
 TMP.WORD[J] := TDATA.WORD[I]
 WRITE_ROW_AND_ZERO(TDEST, I, TMP, 2 * TDEST.COLS)
 ZERO_UPPER_ROWS(TDEST, TDEST.ROWS)
```

FIG. 51

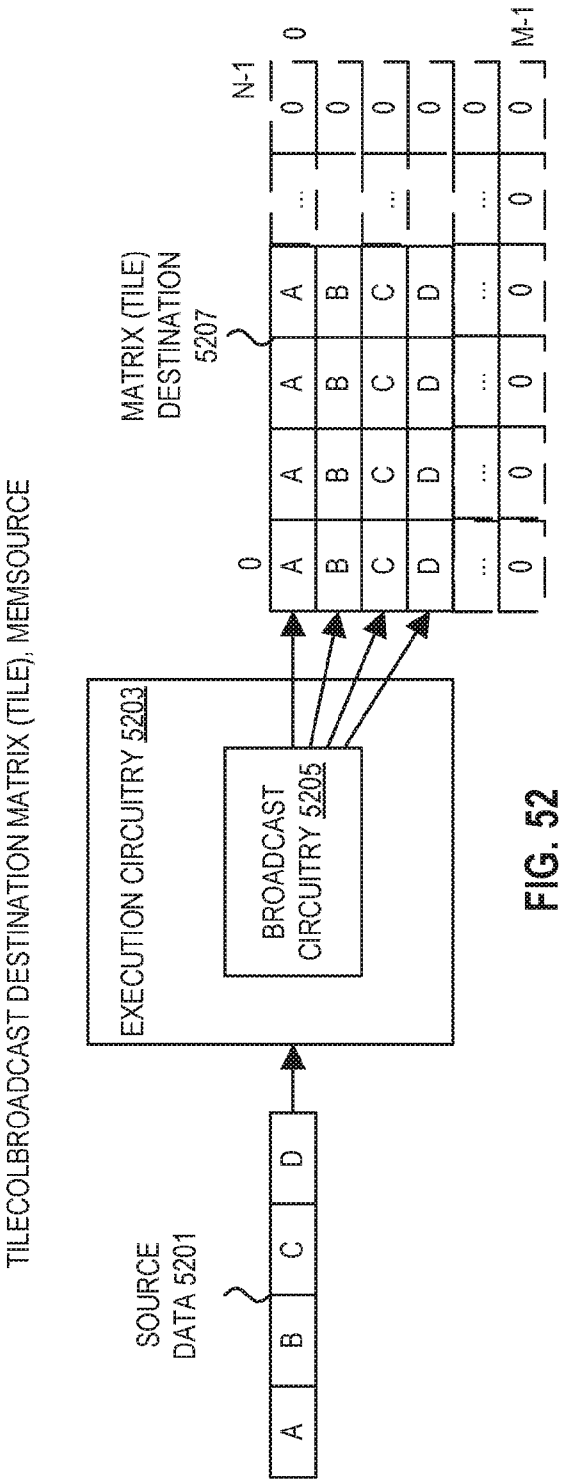

TILECOLBROADCAST DESTINATION MATRIX (TILE), MEMSOURCE
SOURCE DATA 5201
A
B
C
D
EXECUTION CIRCUITRY 5203
BROADCAST CIRCUITRY 5205
MATRIX (TILE) DESTINATION 5207
0
N-1
M-1

FIG. 52

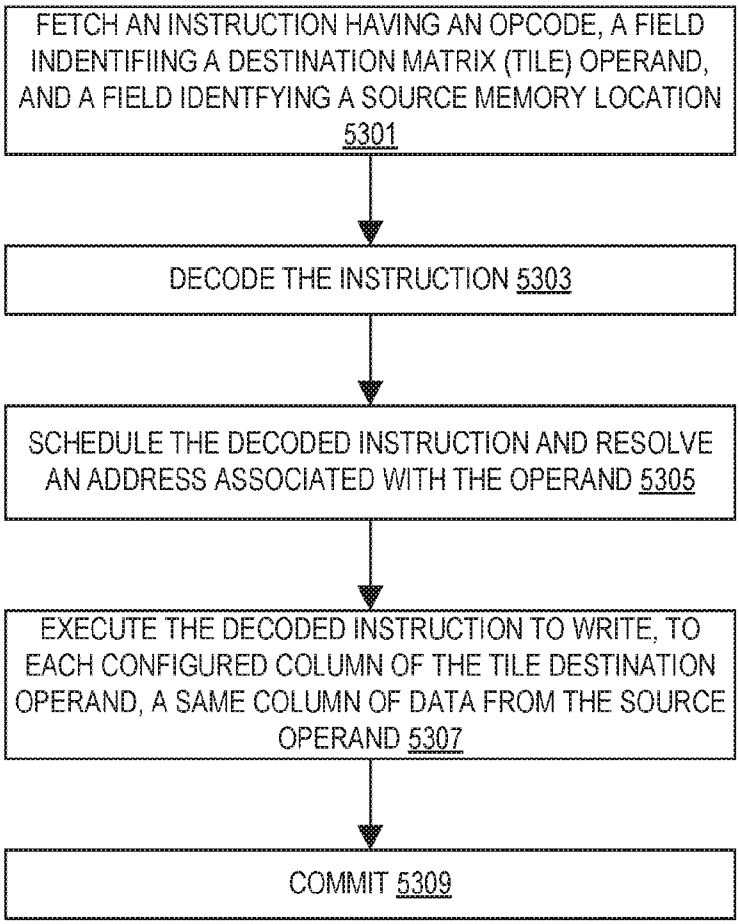

FETCH AN INSTRUCTION HAVING AN OPCODE, A FIELD INDENTIFIING A DESTINATION MATRIX (TILE) OPERAND, AND A FIELD IDENTFYING A SOURCE MEMORY LOCATION 5301
DECODE THE INSTRUCTION 5303
SCHEDULE THE DECODED INSTRUCTION AND RESOLVE AN ADDRESS ASSOCIATED WITH THE OPERAND 5305
EXECUTE THE DECODED INSTRUCTION TO WRITE, TO EACH CONFIGURED COLUMN OF THE TILE DESTINATION OPERAND, A SAME COLUMN OF DATA FROM THE SOURCE OPERAND 5307
COMMIT 5309

FIG. 53

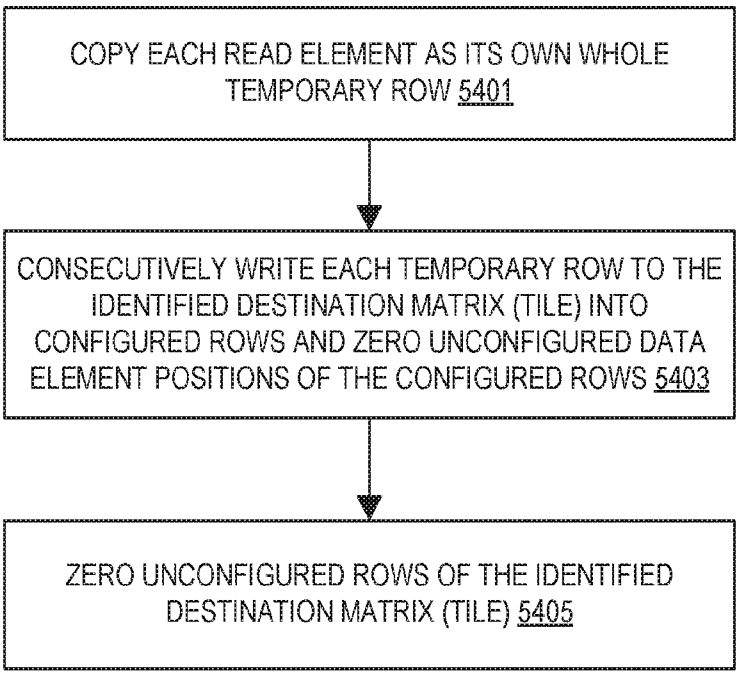

COPY EACH READ ELEMENT AS ITS OWN WHOLE TEMPORARY ROW 5401
CONSECUTIVELY WRITE EACH TEMPORARY ROW TO THE IDENTIFIED DESTINATION MATRIX (TILE) INTO CONFIGURED ROWS AND ZERO UNCONFIGURED DATA ELEMENT POSITIONS OF THE CONFIGURED ROWS 5403
ZERO UNCONFIGURED ROWS OF THE IDENTIFIED DESTINATION MATRIX (TILE) 5405

FIG. 54

```
TILECOLBROADCASTD TDEST, TMEMCOL
 #GP IF TILES_CONFIGURED == 0
 #GP IF TDEST.COLS * 4 >
PALETTE_TABLE[PALETTE_ID].BYTES_PER_ROW

 TDATA := READ_MEMORY(TMEMCOL, 4 * TDEST.ROWS)

 FOR I IN 0 ... TDEST.ROWS-1:
 // COPY EACH ELEMENT TO A WHOLE TEMP ROW.
 // THEN WRITE THE TEMP ROW TO THE TILE.
 FOR J IN 0 ... TDEST.COLS-1 :
 TMP.DWORD[J] := TDATA.DWORD[I]
 WRITE_ROW_AND_ZERO(TDEST, I, TMP, 4 * TDEST.COLS)
 ZERO_UPPER_ROWS(TDEST, TDEST.ROWS)
```

```
TILECOLBROADCASTW TDEST, TMEMCOL
 #GP IF TILES_CONFIGURED == 0
 #GP IF TDEST.COLS * 2 >
PALETTE_TABLE[PALETTE_ID].BYTES_PER_ROW

 TDATA := READ_MEMORY(TMEMCOL, 2 * TDEST.ROWS)

 FOR I IN 0 ... TDEST.ROWS-1:
// COPY EACH ELEMENT TO A WHOLE TEMP ROW.
 // THEN WRITE THE TEMP ROW TO THE TILE.
 FOR J IN 0 ... TDEST.COLS-1 :
 TMP.WORD[J] := TDATA.WORD[I]
 WRITE_ROW_AND_ZERO(TDEST, I, TMP, 2 * TDEST.COLS)
 ZERO_UPPER_ROWS(TDEST, TDEST.ROWS)
```

FIG. 55

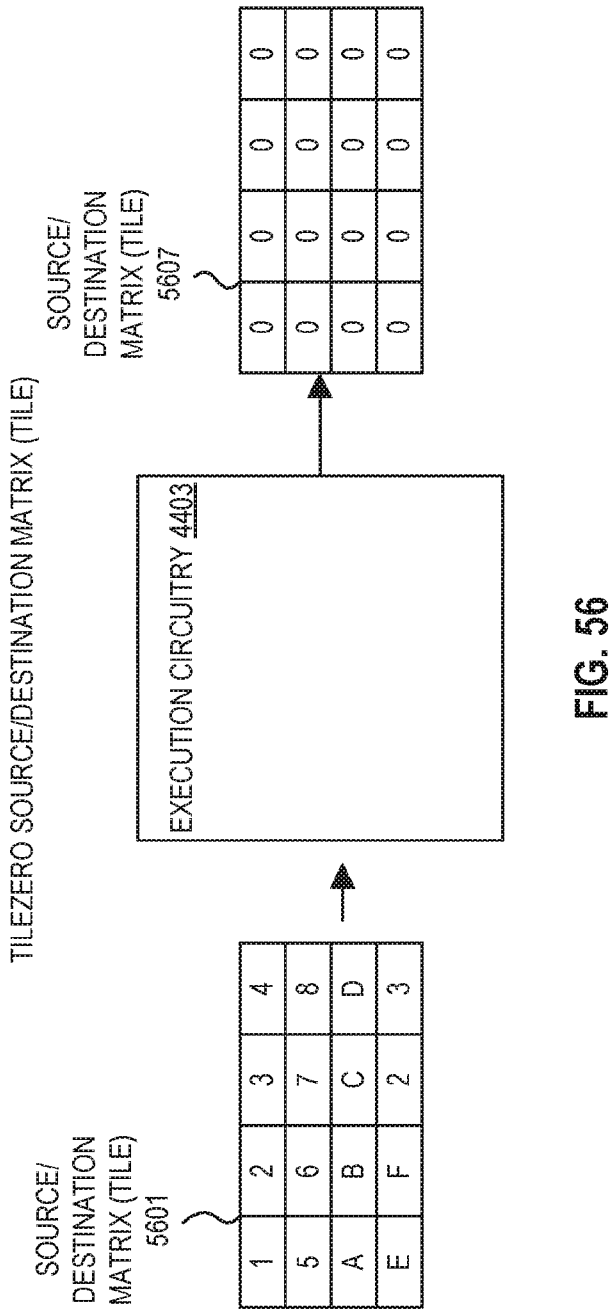

TILEZERO SOURCE/DESTINATION MATRIX (TILE)
SOURCE/ DESTINATION MATRIX (TILE) 5601
1 2 3 4
5 6 7 8
A B C D
E F 2 3
EXECUTION CIRCUITRY 4403
SOURCE/ DESTINATION MATRIX (TILE) 5607
0 0 0 0
0 0 0 0
0 0 0 0
0 0 0 0

FIG. 56

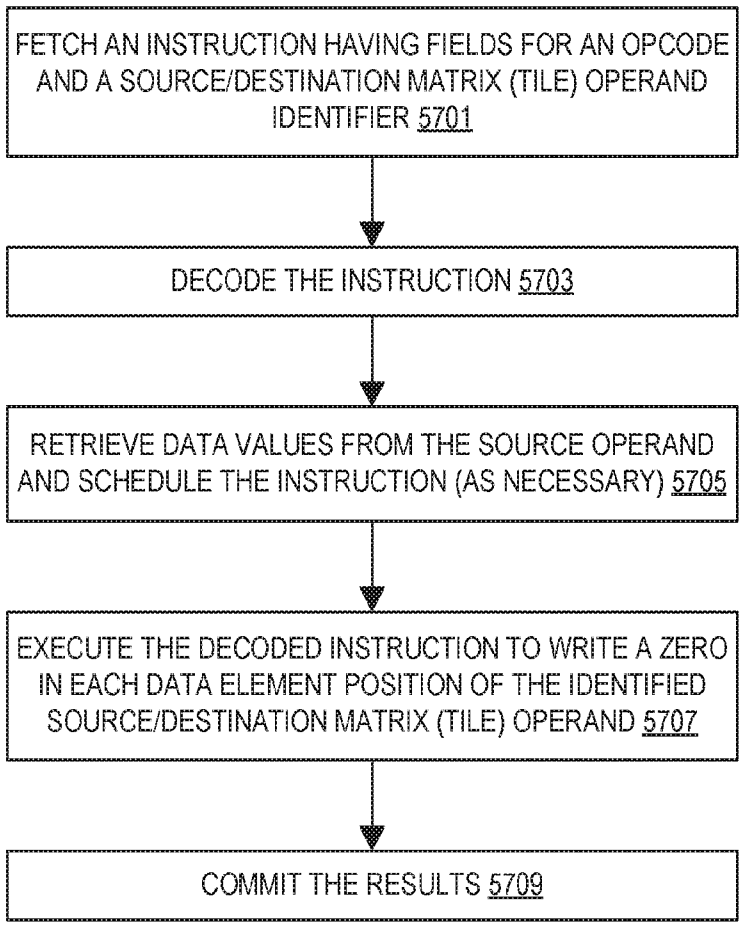

FETCH AN INSTRUCTION HAVING FIELDS FOR AN OPCODE AND A SOURCE/DESTINATION MATRIX (TILE) OPERAND IDENTIFIER 5701
DECODE THE INSTRUCTION 5703
RETRIEVE DATA VALUES FROM THE SOURCE OPERAND AND SCHEDULE THE INSTRUCTION (AS NECESSARY) 5705
EXECUTE THE DECODED INSTRUCTION TO WRITE A ZERO IN EACH DATA ELEMENT POSITION OF THE IDENTIFIED SOURCE/DESTINATION MATRIX (TILE) OPERAND 5707
COMMIT THE RESULTS 5709

FIG. 57

```
TILEZERO TDEST
#GP IF TILES_CONFIGURED == 0

 NBYTES :=
PALETTE_TABLE[PALETTE_ID].BYTES_PER_ROW
MAX_NUM_ROWS :=
PALETTE_TABLE[PALETTE_ID].TILE_BYTES /
PALETTE_TABLE[PALETTE_ID].BYTES_PER_ROW
FOR I IN 0 ... MAX_NUM_ROWS-1:
FOR J IN 0 ... NBYTES-1:
TDEST.BYTE[J] := 0
```

FIG. 58

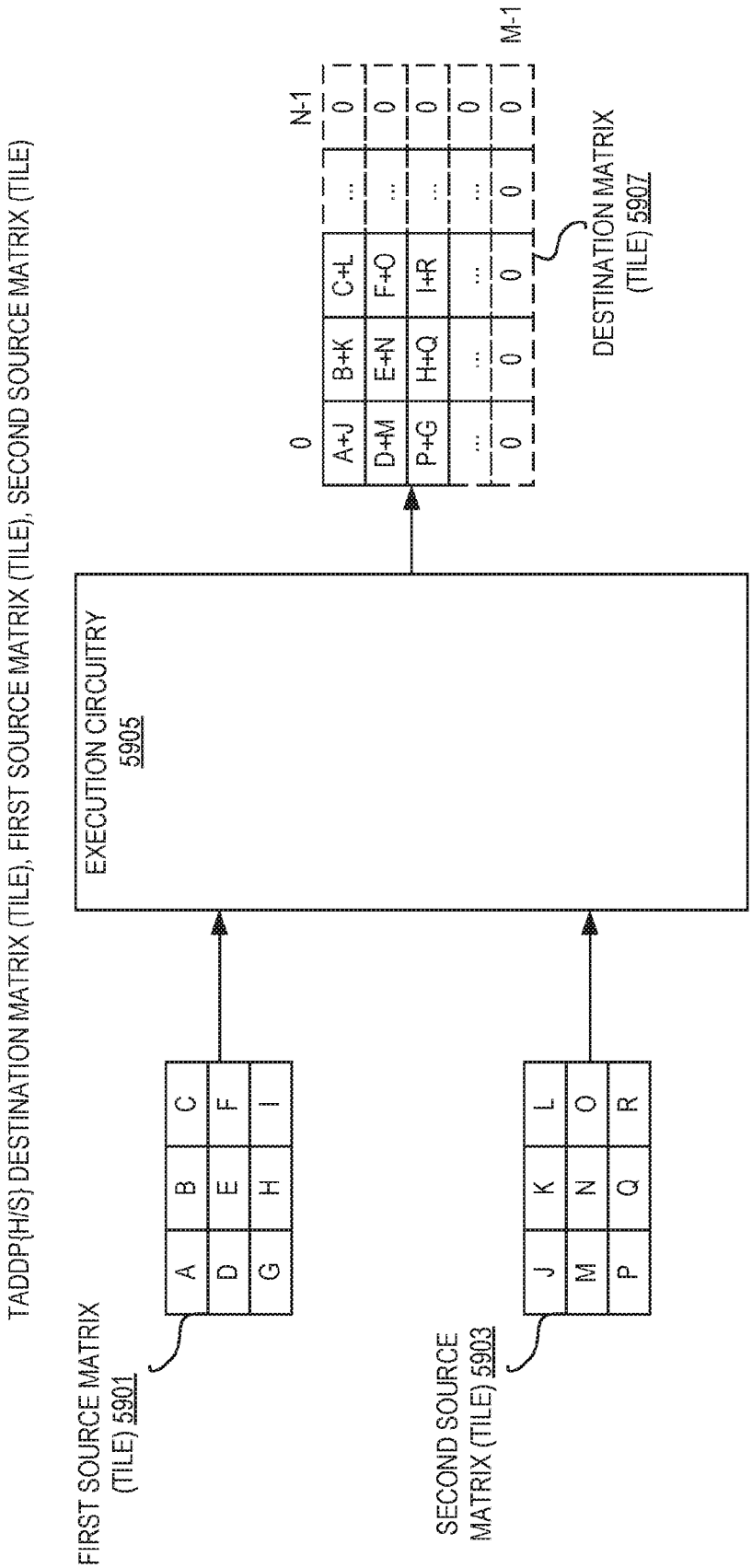

TADDP(H/S) DESTINATION MATRIX (TILE), FIRST SOURCE MATRIX (TILE), SECOND SOURCE MATRIX (TILE)
FIRST SOURCE MATRIX (TILE) 5901
A B C
D E F
G H I
SECOND SOURCE MATRIX (TILE) 5903
J K L
M N O
P Q R
EXECUTION CIRCUITRY 5905
0
N-1
M-1
A+J B+K C+L ... 0
D+M E+N F+O ... 0
P+G H+Q I+R ... 0
... ... 0
0 0 0 0 0
DESTINATION MATRIX (TILE) 5907

FIG. 59

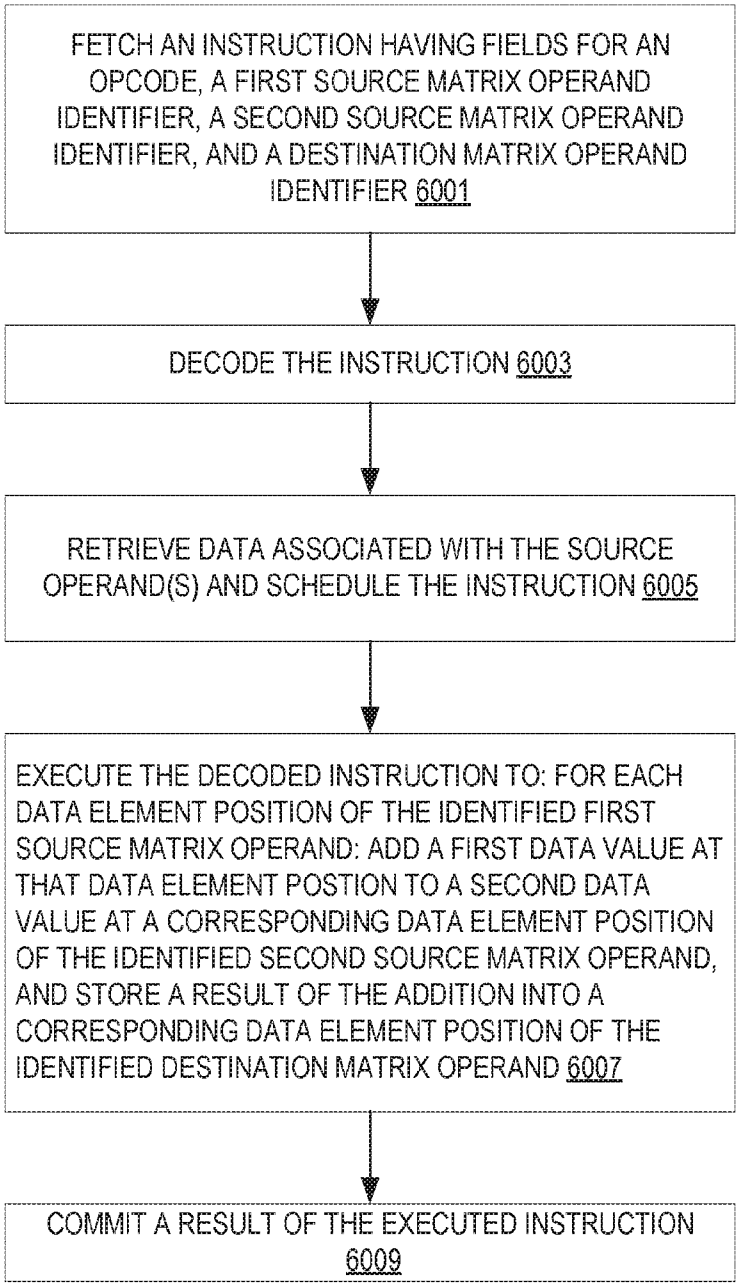

FETCH AN INSTRUCTION HAVING FIELDS FOR AN OPCODE, A FIRST SOURCE MATRIX OPERAND IDENTIFIER, A SECOND SOURCE MATRIX OPERAND IDENTIFIER, AND A DESTINATION MATRIX OPERAND IDENTIFIER 6001
DECODE THE INSTRUCTION 6003
RETRIEVE DATA ASSOCIATED WITH THE SOURCE OPERAND(S) AND SCHEDULE THE INSTRUCTION 6005
EXECUTE THE DECODED INSTRUCTION TO: FOR EACH DATA ELEMENT POSITION OF THE IDENTIFIED FIRST SOURCE MATRIX OPERAND: ADD A FIRST DATA VALUE AT THAT DATA ELEMENT POSTION TO A SECOND DATA VALUE AT A CORRESPONDING DATA ELEMENT POSITION OF THE IDENTIFIED SECOND SOURCE MATRIX OPERAND, AND STORE A RESULT OF THE ADDITION INTO A CORRESPONDING DATA ELEMENT POSITION OF THE IDENTIFIED DESTINATION MATRIX OPERAND 6007
COMMIT A RESULT OF THE EXECUTED INSTRUCTION 6009

FIG. 60

6101 TILE ADD operation (half-precision)

```
TADDPH tdest, tsrc1, tsrc2
#GP if TILES_CONFIGURED == 0
#GP if tdest.cols != tsrc1.cols
#GP if tdest.cols != tsrc2.cols
#GP if tdest.rows != tsrc1.rows
#GP if tdest.rows != tsrc2.rows
#GP if tdest.cols * 2 > impl.tmul_maxn
#GP if tsrc1.cols * 2 > impl.tmul_maxn
#GP if tsrc2.cols * 2 > impl.tmul_maxn
start := tileconfig.startM
#GP if start >= tdest.rows

while start < tdest.rows:
     for n in 0 ... tdest.cols-1:
          tmp.fp16[n] := tsrc1.row[start].fp16[n] + tsrc2.row[start].fp16[n]
     write_row_and_zero(tdest, start, tmp, 2 * tdest.cols)
     start := start + 1

zero_upper_rows(tdest, tdest.rows)
zero_tileconfig_start()

// If an unmasked FP exception occurs, tileconfig.startM and the TILE
// contain a consistent state from an earlier, fault-free
// iteration. Instruction emulation from this initial condition will
// result in the fault condition(s) present in MXCSR.
```

FIG. 61

6201 TILE ADD operation (single-precision)

```
TADDPS tdest, tsrc1, tsrc2
#GP if TILES_CONFIGURED == 0
#GP if tdest.cols != tsrc1.cols
#GP if tdest.cols != tsrc2.cols
#GP if tdest.rows != tsrc1.rows
#GP if tdest.rows != tsrc2.rows
#GP if tdest.cols * 4 > impl.tmul_maxn
#GP if tsrc1.cols * 4 > impl.tmul_maxn
#GP if tsrc2.cols * 4 > impl.tmul_maxn
start := tileconfig.startM
#GP if start >= tdest.rows

while start < tdest.rows:
     for n in 0 ... tdest.cols-1:
          tmp.fp32[n] := tsrc1.row[start].fp32[n] + tsrc2.row[start].fp32[n]
     write_row_and_zero(tdest, start, tmp, 4 * tdest.cols)
     start := start + 1
zero_upper_rows(tdest, tdest.rows)
zero_tileconfig_start()

// If an unmasked FP exception occurs, tileconfig.startM and the TILE
// contain a consistent state from an earlier, fault-free
// iteration. Instruction emulation from this initial condition will
// result in the fault condition(s) present in MXCSR.
```

FIG. 62

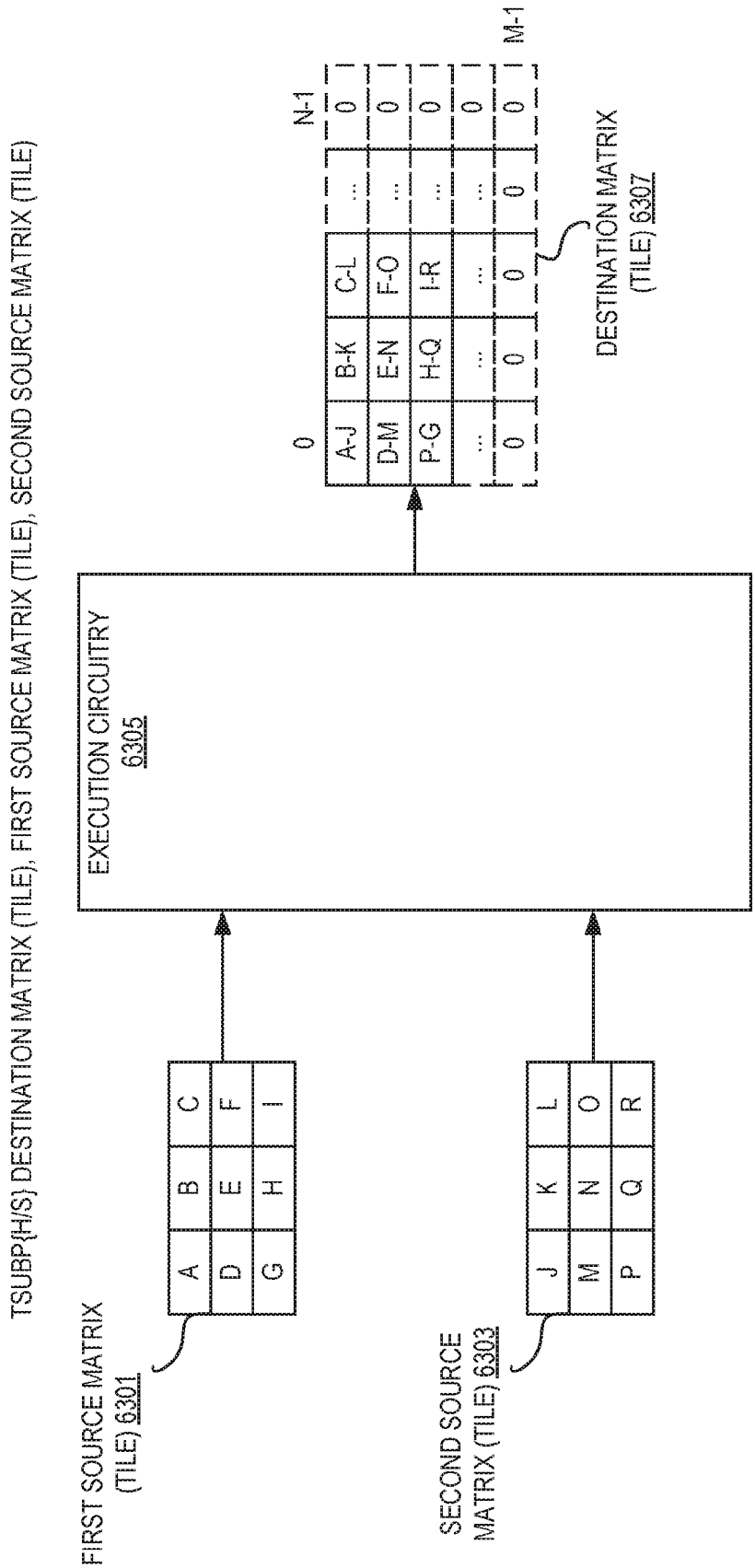

TSUBP{H/S} DESTINATION MATRIX (TILE), FIRST SOURCE MATRIX (TILE), SECOND SOURCE MATRIX (TILE)
FIRST SOURCE MATRIX (TILE) 6301
A B C
D E F
G H I
SECOND SOURCE MATRIX (TILE) 6303
J K L
M N O
P Q R
EXECUTION CIRCUITRY 6305
0
N-1
M-1
A-J B-K C-L ... 0
D-M E-N F-O ... 0
P-G H-Q I-R ... 0
... ... 0
0 0 0 0 0
DESTINATION MATRIX (TILE) 6307

FIG. 63

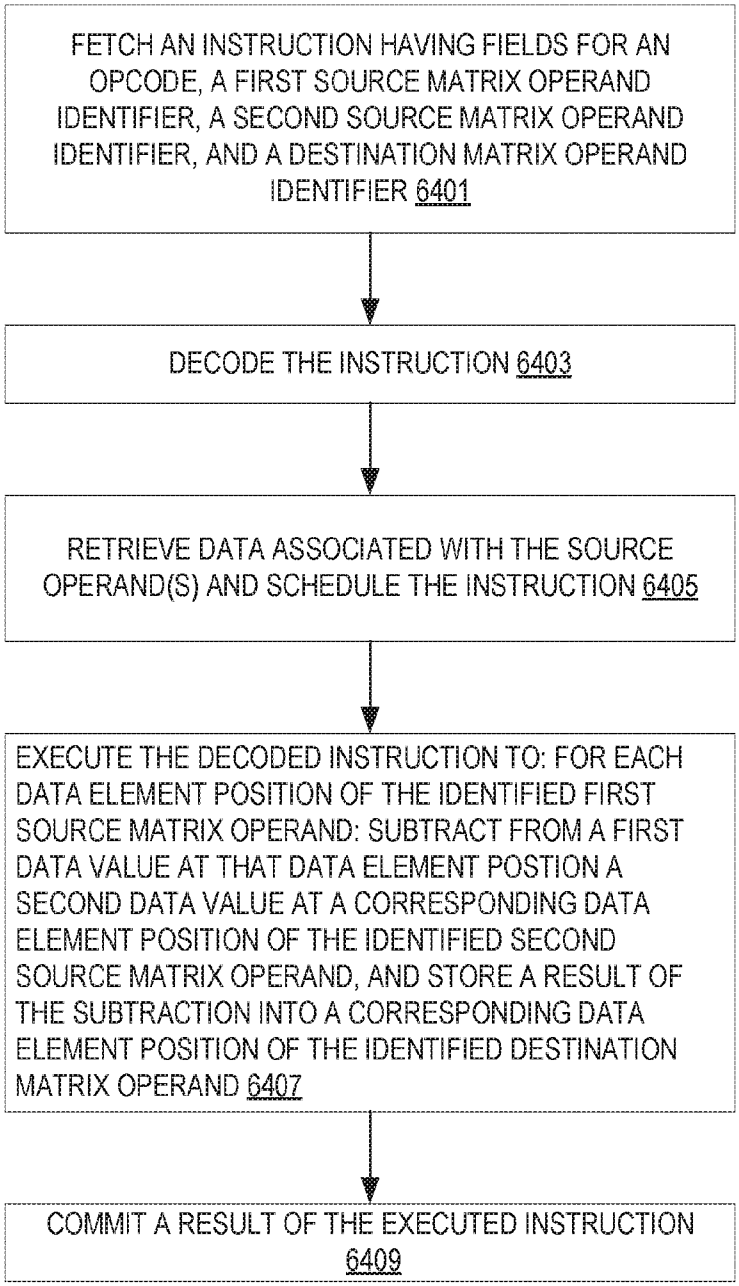

FETCH AN INSTRUCTION HAVING FIELDS FOR AN OPCODE, A FIRST SOURCE MATRIX OPERAND IDENTIFIER, A SECOND SOURCE MATRIX OPERAND IDENTIFIER, AND A DESTINATION MATRIX OPERAND IDENTIFIER 6401
DECODE THE INSTRUCTION 6403
RETRIEVE DATA ASSOCIATED WITH THE SOURCE OPERAND(S) AND SCHEDULE THE INSTRUCTION 6405
EXECUTE THE DECODED INSTRUCTION TO: FOR EACH DATA ELEMENT POSITION OF THE IDENTIFIED FIRST SOURCE MATRIX OPERAND: SUBTRACT FROM A FIRST DATA VALUE AT THAT DATA ELEMENT POSTION A SECOND DATA VALUE AT A CORRESPONDING DATA ELEMENT POSITION OF THE IDENTIFIED SECOND SOURCE MATRIX OPERAND, AND STORE A RESULT OF THE SUBTRACTION INTO A CORRESPONDING DATA ELEMENT POSITION OF THE IDENTIFIED DESTINATION MATRIX OPERAND 6407
COMMIT A RESULT OF THE EXECUTED INSTRUCTION 6409

FIG. 64

6501 TILE SUBTRACT operation (half-precision)

```
TSUBPH tdest, tsrc1, tsrc2
#GP if TILES_CONFIGURED == 0
#GP if tdest.cols != tsrc1.cols
#GP if tdest.cols != tsrc2.cols
#GP if tdest.rows != tsrc1.rows
#GP if tdest.rows != tsrc2.rows
#GP if tdest.cols * 2 > impl.tmul_maxn
#GP if tsrc1.cols * 2 > impl.tmul_maxn
#GP if tsrc2.cols * 2 > impl.tmul_maxn
start := tileconfig.startM
#GP if start >= tdest.rows

while start < tdest.rows:
     for n in 0 ... tdest.cols-1:
          tmp.fp16[n] := tsrc1.row[start].fp16[n] - tsrc2.row[start].fp16[n]
     write_row_and_zero(tdest, start, tmp, 2 * tdest.cols)
     start := start + 1

zero_upper_rows(tdest, tdest.rows)
zero_tileconfig_start()

// If an unmasked FP exception occurs, tileconfig.startM and the TILE
// contain a consistent state from an earlier, fault-free
// iteration. Instruction emulation from this initial condition will
// result in the fault condition(s) present in MXCSR.
```

FIG. 65

6601 TILE SUBTRACT operation (single-precision)

```
TSUBPS tdest, tsrc1, tsrc2
#GP if TILES_CONFIGURED == 0
#GP if tdest.cols != tsrc1.cols
#GP if tdest.cols != tsrc2.cols
#GP if tdest.rows != tsrc1.rows
#GP if tdest.rows != tsrc2.rows
#GP if tdest.cols * 4 > impl.tmul_maxn
#GP if tsrc1.cols * 4 > impl.tmul_maxn
#GP if tsrc2.cols * 4 > impl.tmul_maxn
start := tileconfig.startM
#GP if start >= tdest.rows

while start < tdest.rows:
     for n in 0 ... tdest.cols-1:
          tmp.fp32[n] := tsrc1.row[start].fp32[n] - tsrc2.row[start].fp32[n]
     write_row_and_zero(tdest, start, tmp, 4 * tdest.cols)
     start := start + 1
zero_upper_rows(tdest, tdest.rows)
zero_tileconfig_start()

// If an unmasked FP exception occurs, tileconfig.startM and the TILE
// contain a consistent state from an earlier, fault-free
// iteration. Instruction emulation from this initial condition will
// result in the fault condition(s) present in MXCSR.
```

FIG. 66

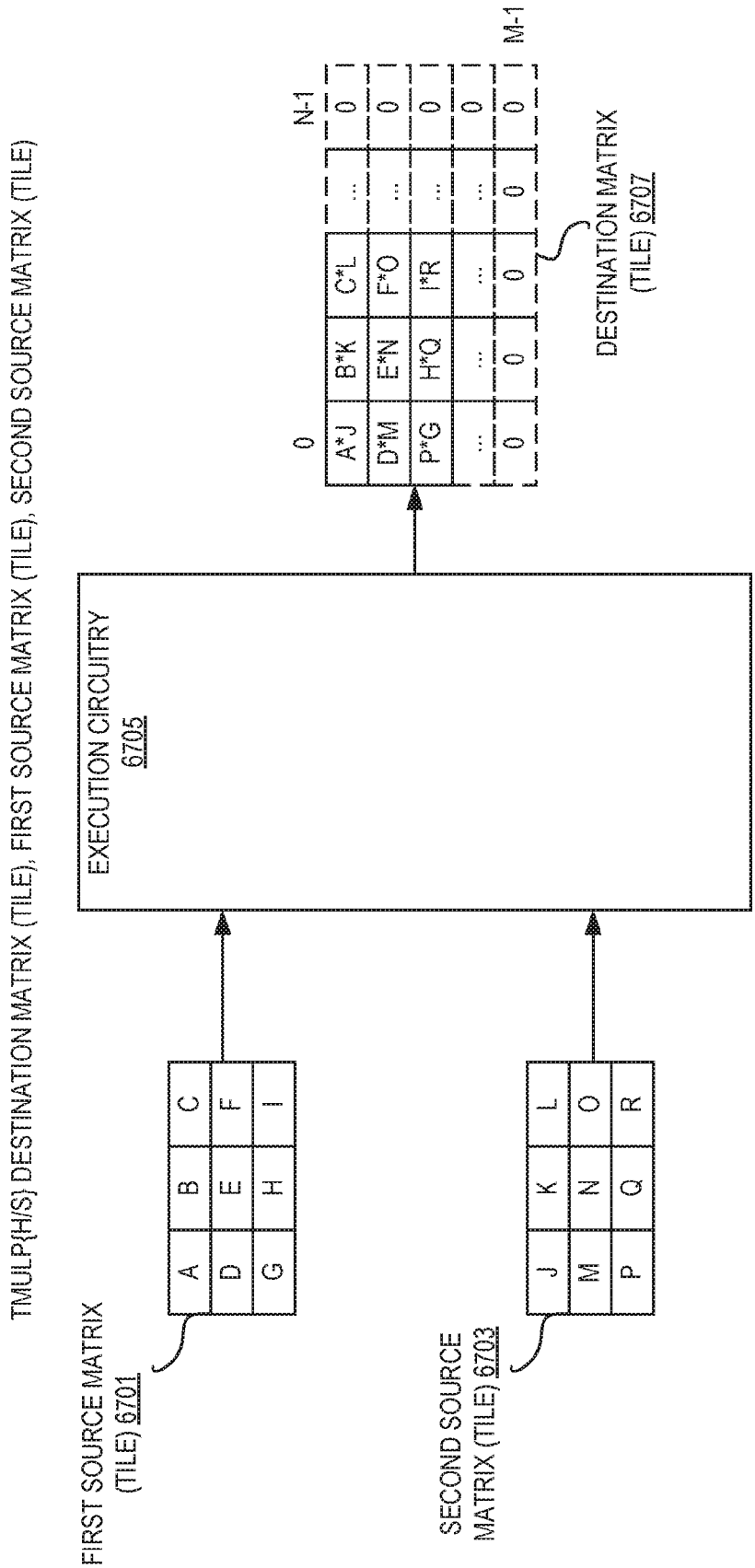

TMULP{H/S} DESTINATION MATRIX (TILE), FIRST SOURCE MATRIX (TILE), SECOND SOURCE MATRIX (TILE)
FIRST SOURCE MATRIX (TILE) 6701
A B C
D E F
G H I
SECOND SOURCE MATRIX (TILE) 6703
J K L
M N O
P Q R
EXECUTION CIRCUITRY 6705
0
N-1
M-1
A*J B*K C*L ... 0
D*M E*N F*O ... 0
P*G H*Q I*R ... 0
... ... 0
0 0 0 0 0
DESTINATION MATRIX (TILE) 6707

FIG. 67

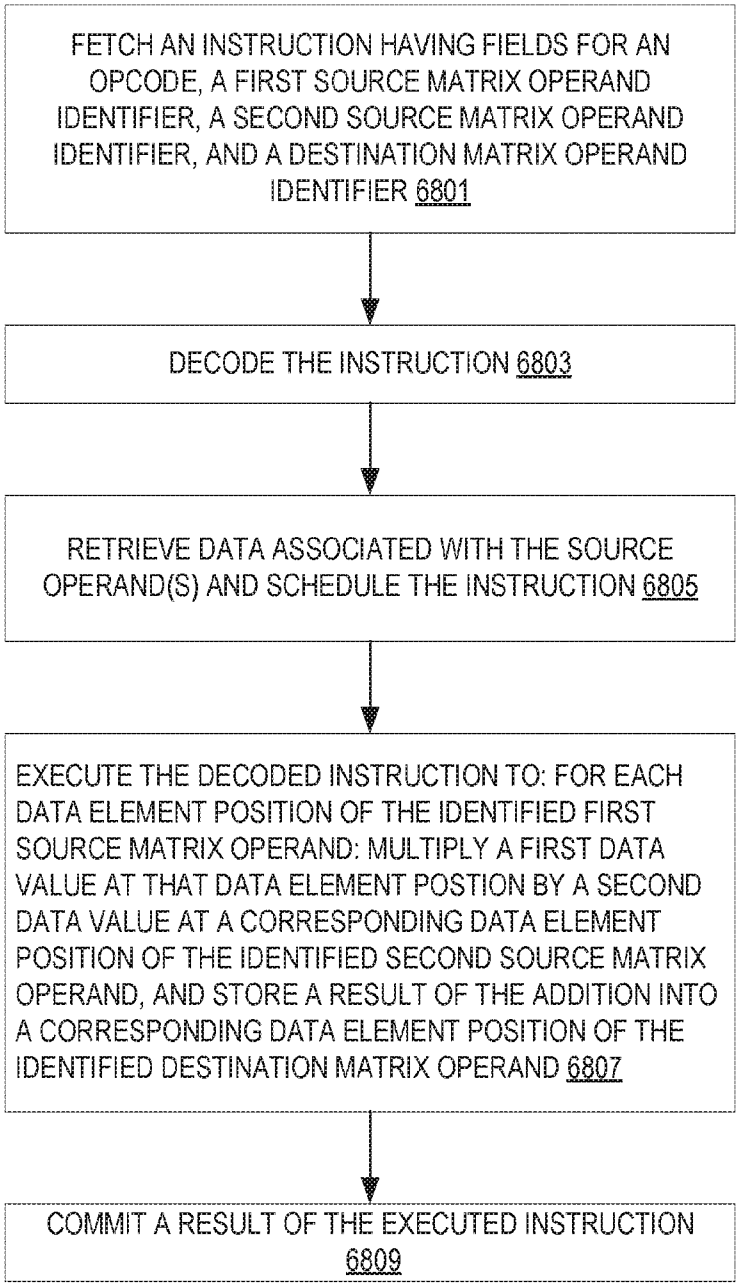

FETCH AN INSTRUCTION HAVING FIELDS FOR AN OPCODE, A FIRST SOURCE MATRIX OPERAND IDENTIFIER, A SECOND SOURCE MATRIX OPERAND IDENTIFIER, AND A DESTINATION MATRIX OPERAND IDENTIFIER 6801
DECODE THE INSTRUCTION 6803
RETRIEVE DATA ASSOCIATED WITH THE SOURCE OPERAND(S) AND SCHEDULE THE INSTRUCTION 6805
EXECUTE THE DECODED INSTRUCTION TO: FOR EACH DATA ELEMENT POSITION OF THE IDENTIFIED FIRST SOURCE MATRIX OPERAND: MULTIPLY A FIRST DATA VALUE AT THAT DATA ELEMENT POSTION BY A SECOND DATA VALUE AT A CORRESPONDING DATA ELEMENT POSITION OF THE IDENTIFIED SECOND SOURCE MATRIX OPERAND, AND STORE A RESULT OF THE ADDITION INTO A CORRESPONDING DATA ELEMENT POSITION OF THE IDENTIFIED DESTINATION MATRIX OPERAND 6807
COMMIT A RESULT OF THE EXECUTED INSTRUCTION 6809

FIG. 68

6901 TILE MUL operation (half-precision)

```
TMULPH tdest, tsrc1, tsrc2
#GP if TILES_CONFIGURED == 0
#GP if tdest.cols != tsrc1.cols
#GP if tdest.cols != tsrc2.cols
#GP if tdest.rows != tsrc1.rows
#GP if tdest.rows != tsrc2.rows
#GP if tdest.cols * 2 > impl.tmul_maxn
#GP if tsrc1.cols * 2 > impl.tmul_maxn
#GP if tsrc2.cols * 2 > impl.tmul_maxn
start := tileconfig.startM
#GP if start >= tdest.rows

while start < tdest.rows:
     for n in 0 ... tdest.cols-1:
          tmp.fp16[n] := tsrc1.row[start].fp16[n] * tsrc2.row[start].fp16[n]
     write_row_and_zero(tdest, start, tmp, 2 * tdest.cols)
     start := start + 1

zero_upper_rows(tdest, tdest.rows)
zero_tileconfig_start()

// If an unmasked FP exception occurs, tileconfig.startM and the TILE
// contain a consistent state from an earlier, fault-free
// iteration. Instruction emulation from this initial condition will
// result in the fault condition(s) present in MXCSR.
```

FIG. 69

7001 TILE MUL operation (single-precision)

```
TMULPS tdest, tsrc1, tsrc2
#GP if TILES_CONFIGURED == 0
#GP if tdest.cols != tsrc1.cols
#GP if tdest.cols != tsrc2.cols
#GP if tdest.rows != tsrc1.rows
#GP if tdest.rows != tsrc2.rows
#GP if tdest.cols * 4 > impl.tmul_maxn
#GP if tsrc1.cols * 4 > impl.tmul_maxn
#GP if tsrc2.cols * 4 > impl.tmul_maxn
start := tileconfig.startM
#GP if start >= tdest.rows

while start < tdest.rows:
     for n in 0 ... tdest.cols-1:
          tmp.fp32[n] := tsrc1.row[start].fp32[n] * tsrc2.row[start].fp32[n]
     write_row_and_zero(tdest, start, tmp, 4 * tdest.cols)
     start := start + 1
zero_upper_rows(tdest, tdest.rows)
zero_tileconfig_start()

// If an unmasked FP exception occurs, tileconfig.startM and the TILE
// contain a consistent state from an earlier, fault-free
// iteration. Instruction emulation from this initial condition will
// result in the fault condition(s) present in MXCSR.
```

FIG. 70

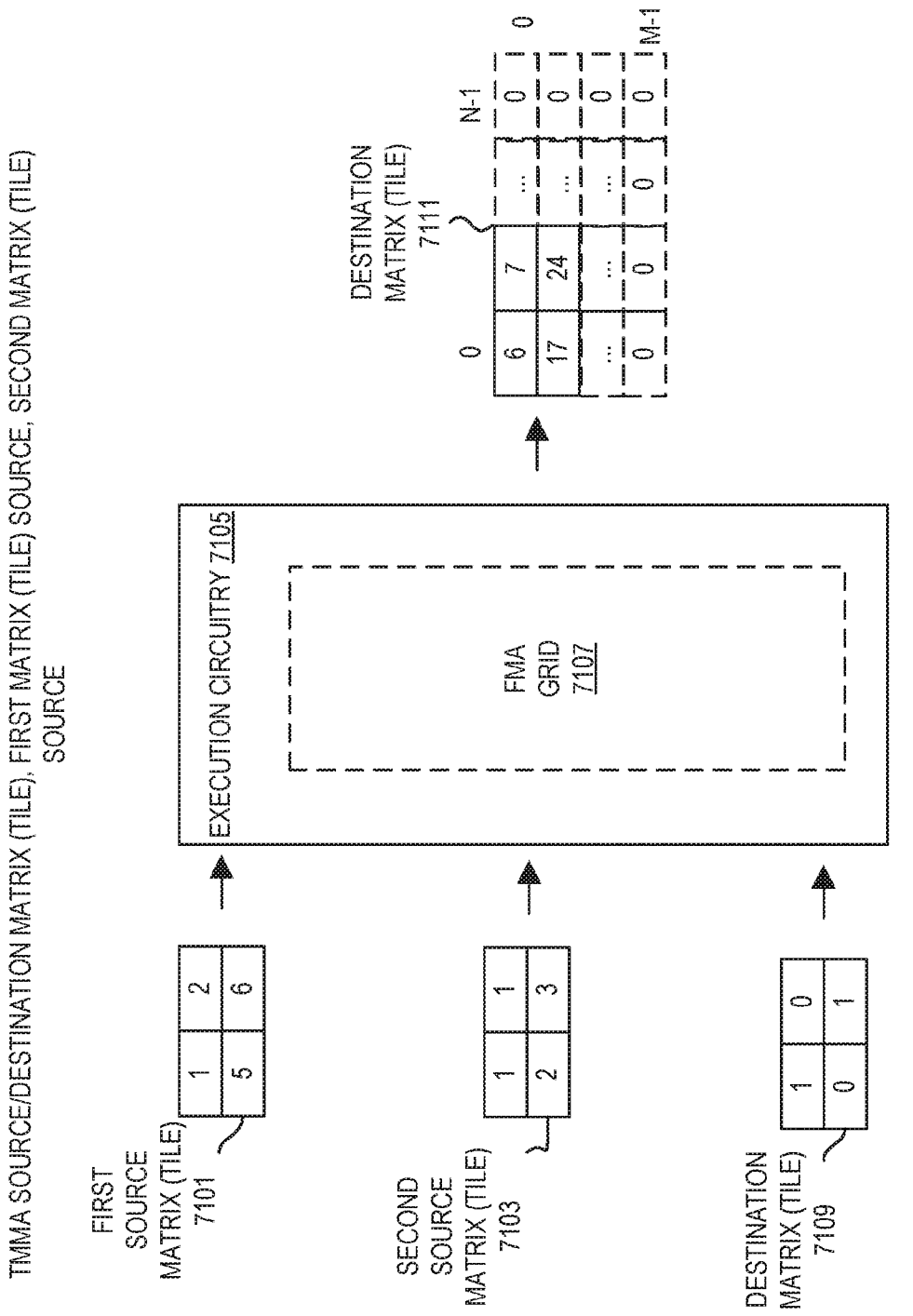

TMMA SOURCE/DESTINATION MATRIX (TILE), FIRST MATRIX (TILE) SOURCE, SECOND MATRIX (TILE) SOURCE
FIRST SOURCE MATRIX (TILE) 7101
1
2
5
6
SECOND SOURCE MATRIX (TILE) 7103
1
1
2
3
DESTINATION MATRIX (TILE) 7109
1
0
0
1
EXECUTION CIRCUITRY 7105
FMA GRID 7107
DESTINATION MATRIX (TILE) 7111
0
N-1
M-1
6
7
17
24
0
0
0
0
0
0
0
0

FIG. 71

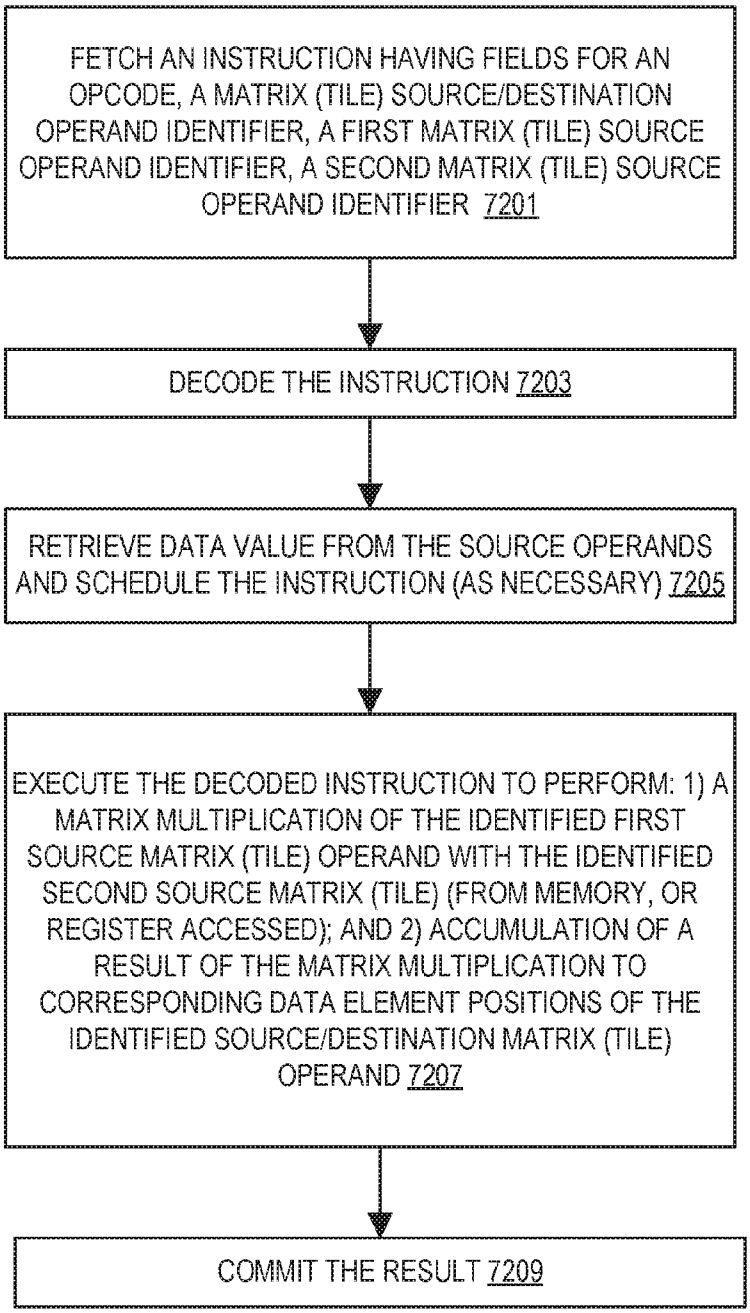

FETCH AN INSTRUCTION HAVING FIELDS FOR AN OPCODE, A MATRIX (TILE) SOURCE/DESTINATION OPERAND IDENTIFIER, A FIRST MATRIX (TILE) SOURCE OPERAND IDENTIFIER, A SECOND MATRIX (TILE) SOURCE OPERAND IDENTIFIER 7201
DECODE THE INSTRUCTION 7203
RETRIEVE DATA VALUE FROM THE SOURCE OPERANDS AND SCHEDULE THE INSTRUCTION (AS NECESSARY) 7205
EXECUTE THE DECODED INSTRUCTION TO PERFORM: 1) A MATRIX MULTIPLICATION OF THE IDENTIFIED FIRST SOURCE MATRIX (TILE) OPERAND WITH THE IDENTIFIED SECOND SOURCE MATRIX (TILE) (FROM MEMORY, OR REGISTER ACCESSED); AND 2) ACCUMULATION OF A RESULT OF THE MATRIX MULTIPLICATION TO CORRESPONDING DATA ELEMENT POSITIONS OF THE IDENTIFIED SOURCE/DESTINATION MATRIX (TILE) OPERAND 7207
COMMIT THE RESULT 7209

FIG. 72

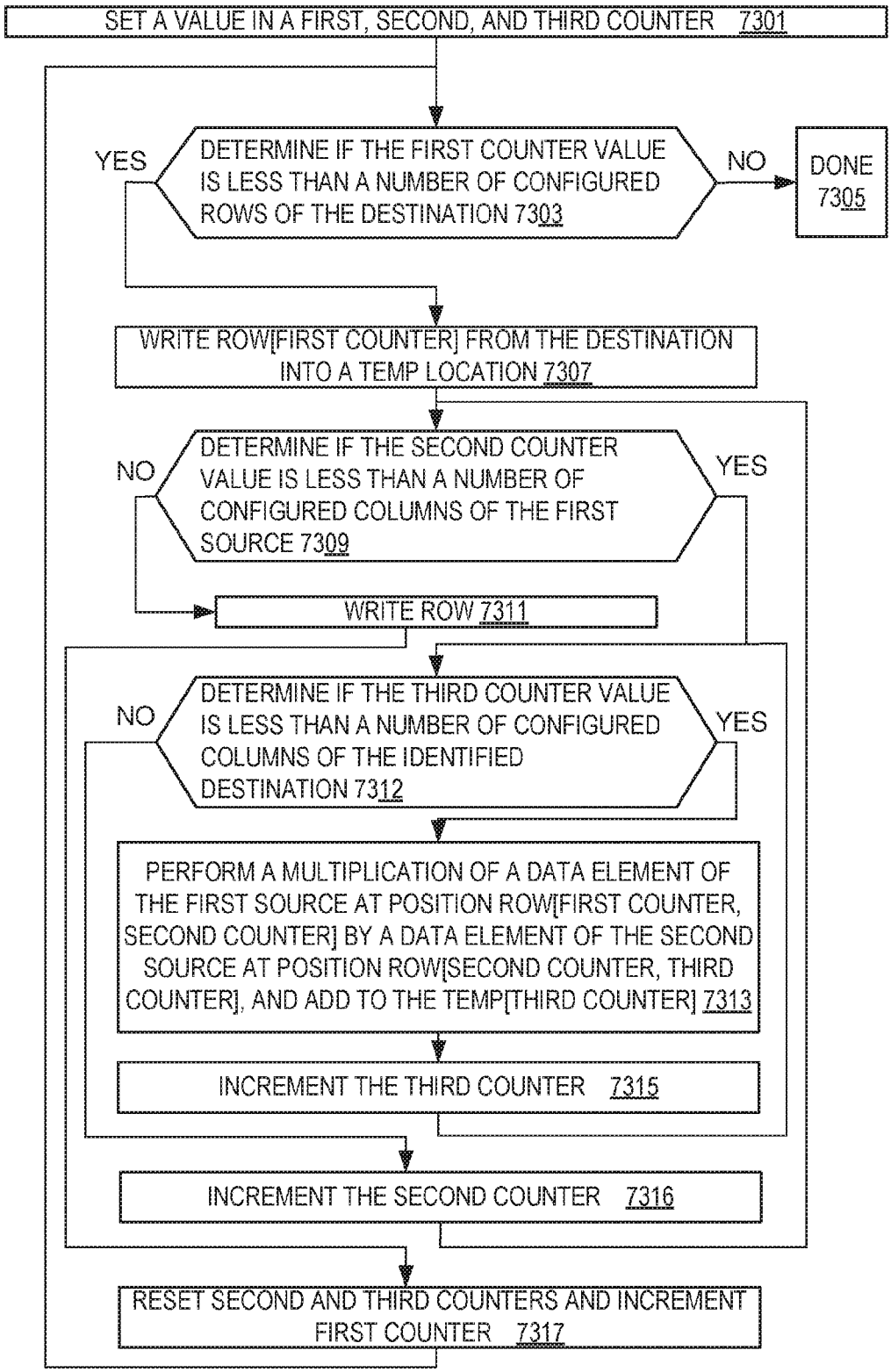

SET A VALUE IN A FIRST, SECOND, AND THIRD COUNTER 7301
YES
DETERMINE IF THE FIRST COUNTER VALUE IS LESS THAN A NUMBER OF CONFIGURED ROWS OF THE DESTINATION 7303
NO
DONE 7305
WRITE ROW[FIRST COUNTER] FROM THE DESTINATION INTO A TEMP LOCATION 7307
NO
DETERMINE IF THE SECOND COUNTER VALUE IS LESS THAN A NUMBER OF CONFIGURED COLUMNS OF THE FIRST SOURCE 7309
YES
WRITE ROW 7311
NO
DETERMINE IF THE THIRD COUNTER VALUE IS LESS THAN A NUMBER OF CONFIGURED COLUMNS OF THE IDENTIFIED DESTINATION 7312
YES
PERFORM A MULTIPLICATION OF A DATA ELEMENT OF THE FIRST SOURCE AT POSITION ROW[FIRST COUNTER, SECOND COUNTER] BY A DATA ELEMENT OF THE SECOND SOURCE AT POSITION ROW[SECOND COUNTER, THIRD COUNTER], AND ADD TO THE TEMP[THIRD COUNTER] 7313
INCREMENT THE THIRD COUNTER 7315
INCREMENT THE SECOND COUNTER 7316
RESET SECOND AND THIRD COUNTERS AND INCREMENT FIRST COUNTER 7317

FIG. 73

```
    DEFINE FMAOP(C,A,B): // OPERATES ON
    ONE ELEMENT
    // FUSED MULTIPLY ADD
     IF *NEGATIVE VERSION*:
     C := C - A * B
     ELSE:
     C := C + A * B


T[,N]MMAPS TSRCDEST, TSRC1, TSRC2
// C = M X N (TSRCDEST), A = M X K (TSRC1), B = K X N (TSRC2)
#GP IF TILES_CONFIGURED == 0
#GP IF TSRC1.COLS != TSRC2.ROWS
#GP IF TSRCDEST.ROWS != TSRC1.ROWS
#GP IF TSRCDEST.COLS != TSRC2.COLS

#GP IF TSRC2.ROWS > IMPL.TMUL_MAXK
#GP IF TSRC2.COLS * 4 > IMPL.TMUL_MAXN
#GP IF TSRCDEST.COLS * 4 > IMPL.TMUL_MAXN

STARTK := TILECONFIG.STARTK
STARTM := TILECONFIG.STARTM
#GP IF STARTM >= TSRCDEST.ROWS
#GP IF STARTK >= TSRC2.ROWS
WHILE STARTM < TSRCDEST.ROWS:
TMP := TSRCDEST.ROW[STARTM]
WHILE STARTK < TSRC1.COLS:
FOR N IN 0 ... TSRCDEST.COLS-1: // SIMD LOOP
FMAOP( TMP.FP32[N],
TSRC1.ROW[STARTM].FP32[STARTK],
TSRC2.ROW[STARTK].FP32[N] )
STARTK := STARTK + 1
WRITE_ROW_AND_ZERO(TSRCDEST, STARTM, TMP, 4 * TSRCDEST.COLS)
STARTK := 0
STARTM := STARTM + 1
ZERO_UPPER_ROWS(TSRCDEST, TSRCDEST.ROWS)
ZERO_TILECONFIG_START()

// IN THE CASE OF AN UNMASKED FLOATING POINT EXCEPTION
// IN THE MIDDLE OF AN INSTRUCTION:
// TILECONFIG.STARTM := STARTM AND TILECONFIG.STARTK := STARTK
```

FIG. 74

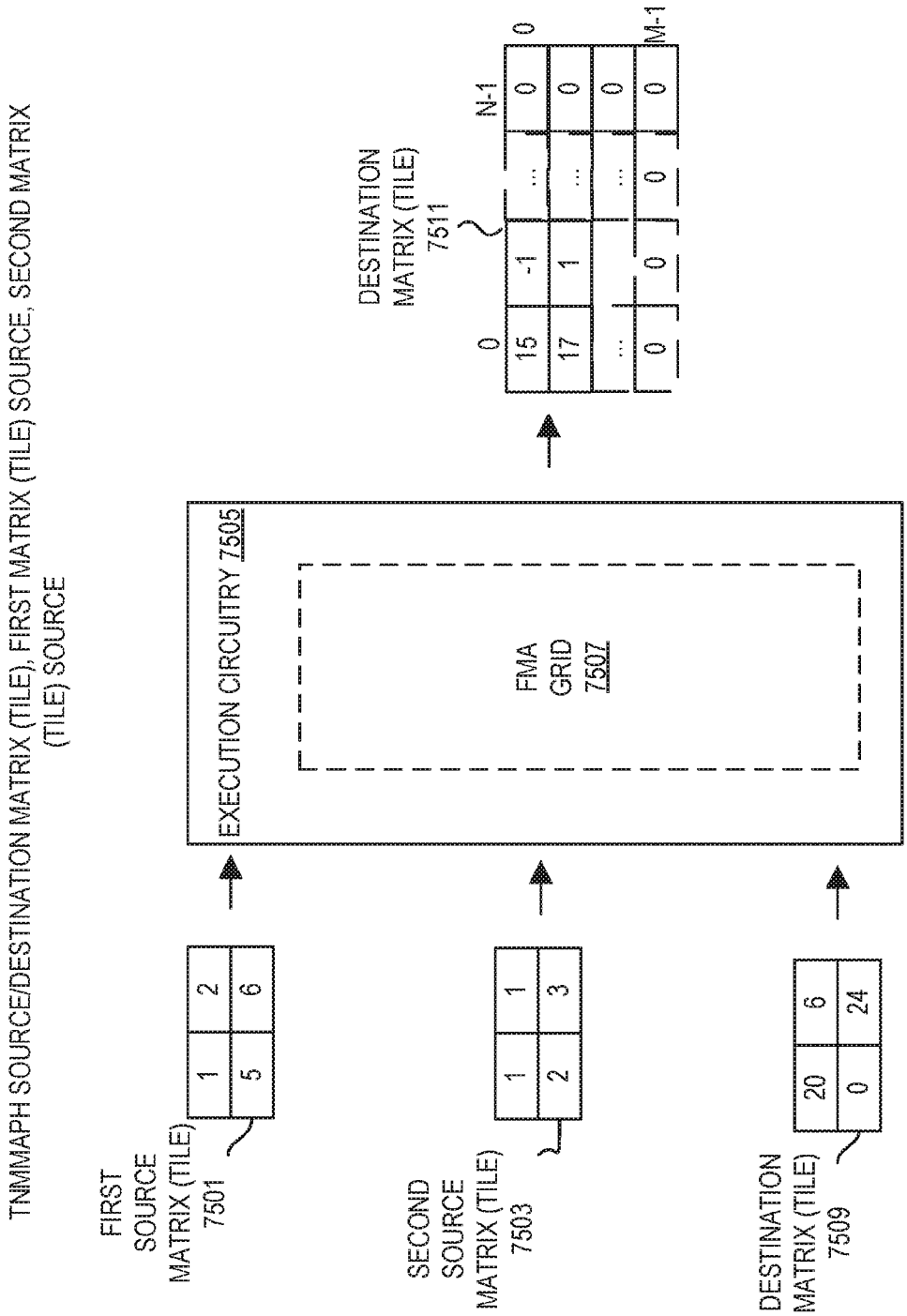

TNMMAPH SOURCE/DESTINATION MATRIX (TILE), FIRST MATRIX (TILE) SOURCE, SECOND MATRIX (TILE) SOURCE
FIRST SOURCE MATRIX (TILE) 7501
1
2
5
6
SECOND SOURCE MATRIX (TILE) 7503
1
1
2
3
DESTINATION MATRIX (TILE) 7509
20
6
0
24
EXECUTION CIRCUITRY 7505
FMA GRID 7507
DESTINATION MATRIX (TILE) 7511
0
N-1
0
M-1
15
-1
17
1
0
0
0
0
0
0
0
0

FIG. 75

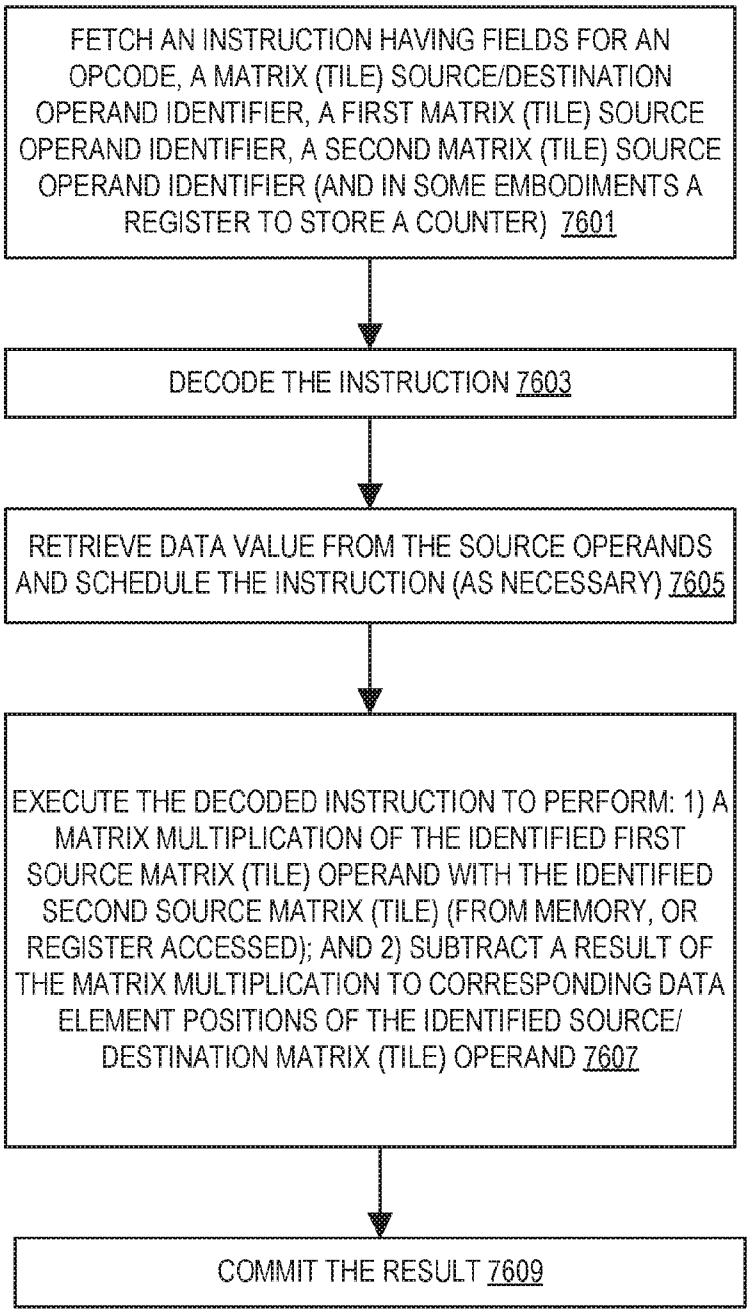

FETCH AN INSTRUCTION HAVING FIELDS FOR AN OPCODE, A MATRIX (TILE) SOURCE/DESTINATION OPERAND IDENTIFIER, A FIRST MATRIX (TILE) SOURCE OPERAND IDENTIFIER, A SECOND MATRIX (TILE) SOURCE OPERAND IDENTIFIER (AND IN SOME EMBODIMENTS A REGISTER TO STORE A COUNTER) 7601
DECODE THE INSTRUCTION 7603
RETRIEVE DATA VALUE FROM THE SOURCE OPERANDS AND SCHEDULE THE INSTRUCTION (AS NECESSARY) 7605
EXECUTE THE DECODED INSTRUCTION TO PERFORM: 1) A MATRIX MULTIPLICATION OF THE IDENTIFIED FIRST SOURCE MATRIX (TILE) OPERAND WITH THE IDENTIFIED SECOND SOURCE MATRIX (TILE) (FROM MEMORY, OR REGISTER ACCESSED); AND 2) SUBTRACT A RESULT OF THE MATRIX MULTIPLICATION TO CORRESPONDING DATA ELEMENT POSITIONS OF THE IDENTIFIED SOURCE/ DESTINATION MATRIX (TILE) OPERAND 7607
COMMIT THE RESULT 7609

FIG. 76

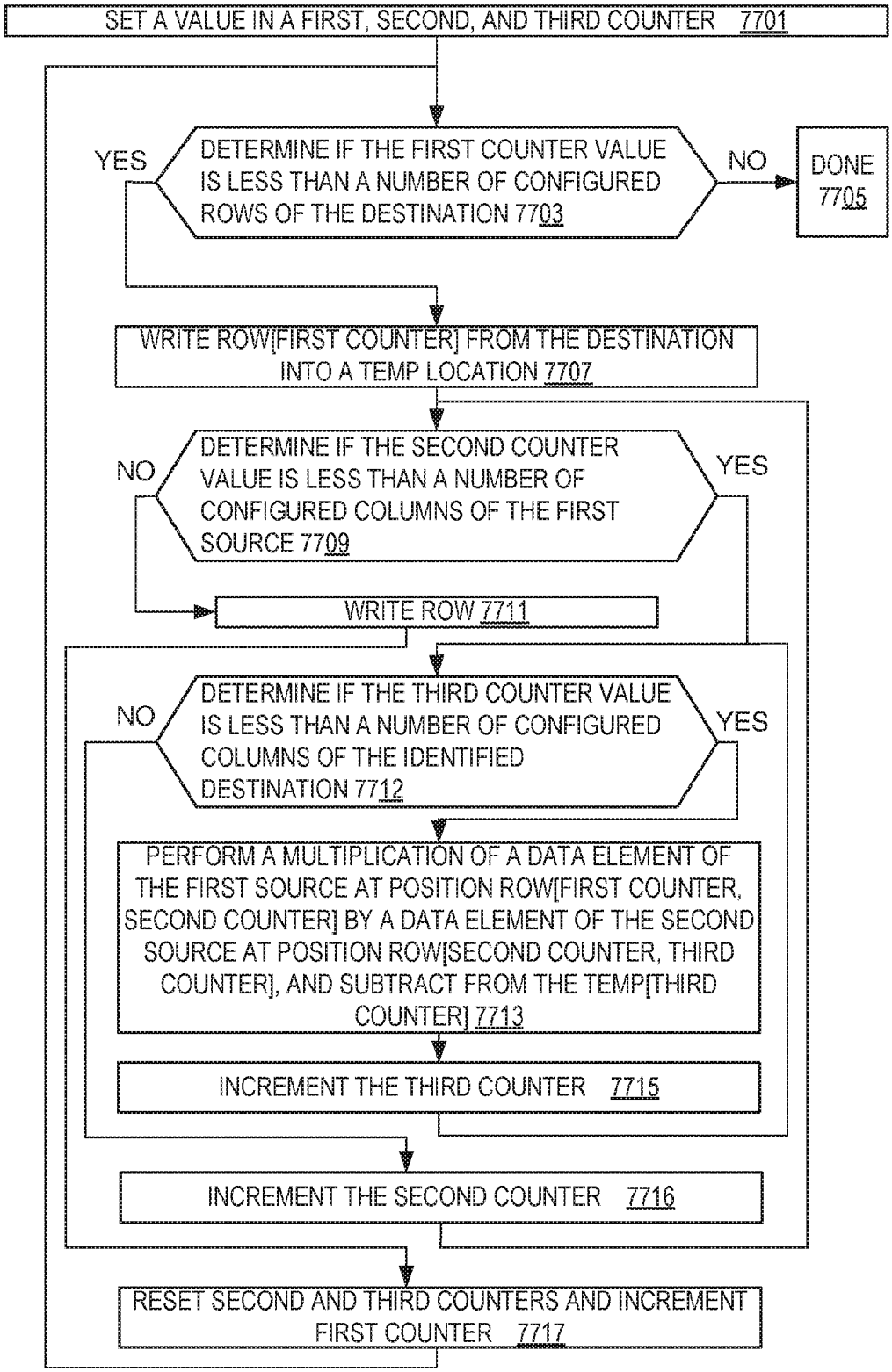

SET A VALUE IN A FIRST, SECOND, AND THIRD COUNTER 7701
YES
DETERMINE IF THE FIRST COUNTER VALUE IS LESS THAN A NUMBER OF CONFIGURED ROWS OF THE DESTINATION 7703
NO
DONE 7705
WRITE ROW[FIRST COUNTER] FROM THE DESTINATION INTO A TEMP LOCATION 7707
NO
DETERMINE IF THE SECOND COUNTER VALUE IS LESS THAN A NUMBER OF CONFIGURED COLUMNS OF THE FIRST SOURCE 7709
YES
WRITE ROW 7711
NO
DETERMINE IF THE THIRD COUNTER VALUE IS LESS THAN A NUMBER OF CONFIGURED COLUMNS OF THE IDENTIFIED DESTINATION 7712
YES
PERFORM A MULTIPLICATION OF A DATA ELEMENT OF THE FIRST SOURCE AT POSITION ROW[FIRST COUNTER, SECOND COUNTER] BY A DATA ELEMENT OF THE SECOND SOURCE AT POSITION ROW[SECOND COUNTER, THIRD COUNTER], AND SUBTRACT FROM THE TEMP[THIRD COUNTER] 7713
INCREMENT THE THIRD COUNTER 7715
INCREMENT THE SECOND COUNTER 7716
RESET SECOND AND THIRD COUNTERS AND INCREMENT FIRST COUNTER 7717

FIG. 77

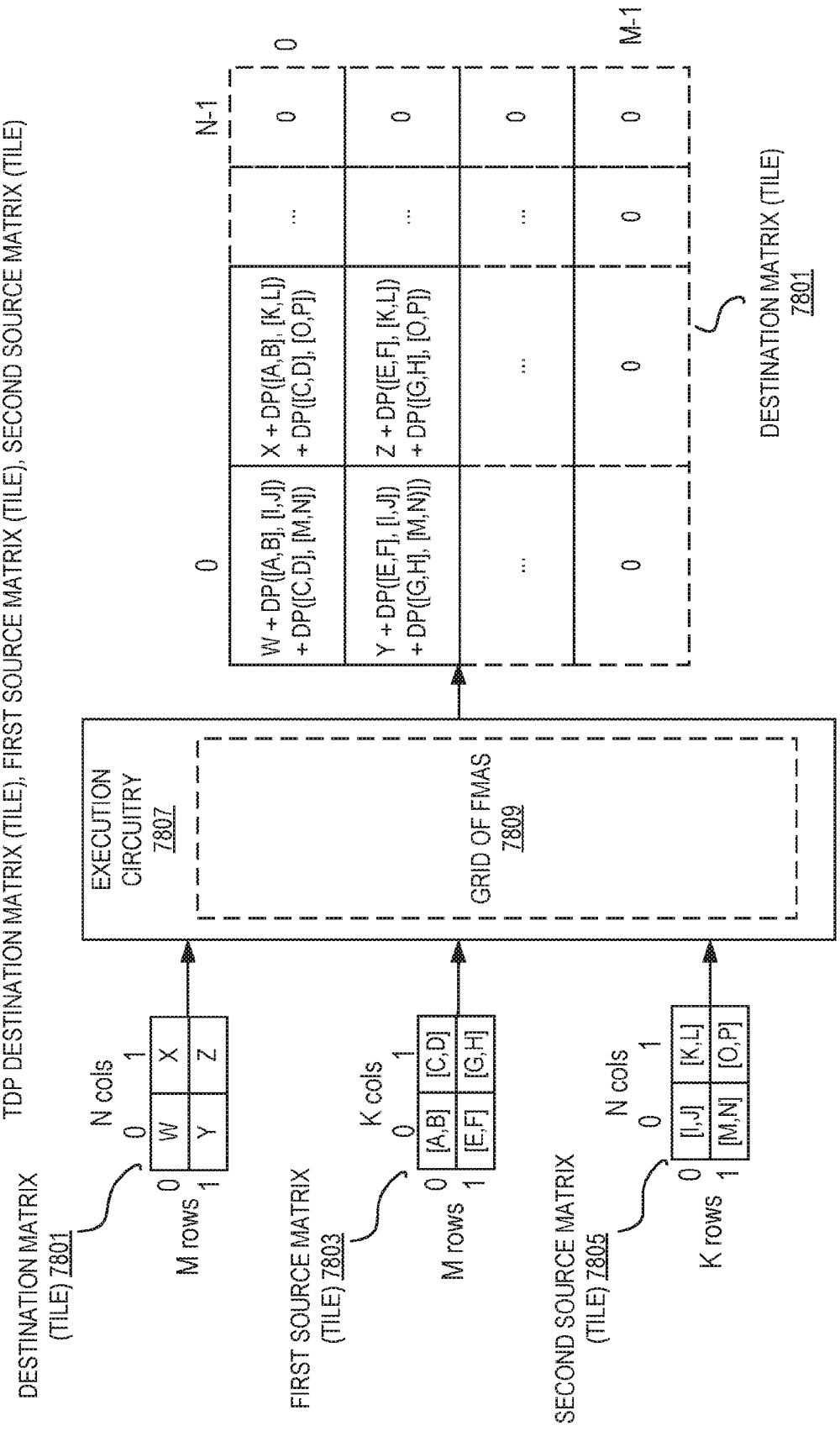

TDP DESTINATION MATRIX (TILE), FIRST SOURCE MATRIX (TILE), SECOND SOURCE MATRIX (TILE)
DESTINATION MATRIX (TILE) 7801
N cols
M rows
W
X
Y
Z
FIRST SOURCE MATRIX (TILE) 7803
K cols
M rows
[A,B]
[C,D]
[E,F]
[G,H]
SECOND SOURCE MATRIX (TILE) 7805
N cols
K rows
[I,J]
[K,L]
[M,N]
[O,P]
EXECUTION CIRCUITRY 7807
GRID OF FMAS 7809
W + DP([A,B], [I,J]) + DP([C,D], [M,N])
X + DP([A,B], [K,L]) + DP([C,D], [O,P])
Y + DP([E,F], [I,J]) + DP([G,H], [M,N]))
Z + DP([E,F], [K,L]) + DP([G,H], [O,P])
DESTINATION MATRIX (TILE) 7801
0
N-1
M-1

FIG. 78

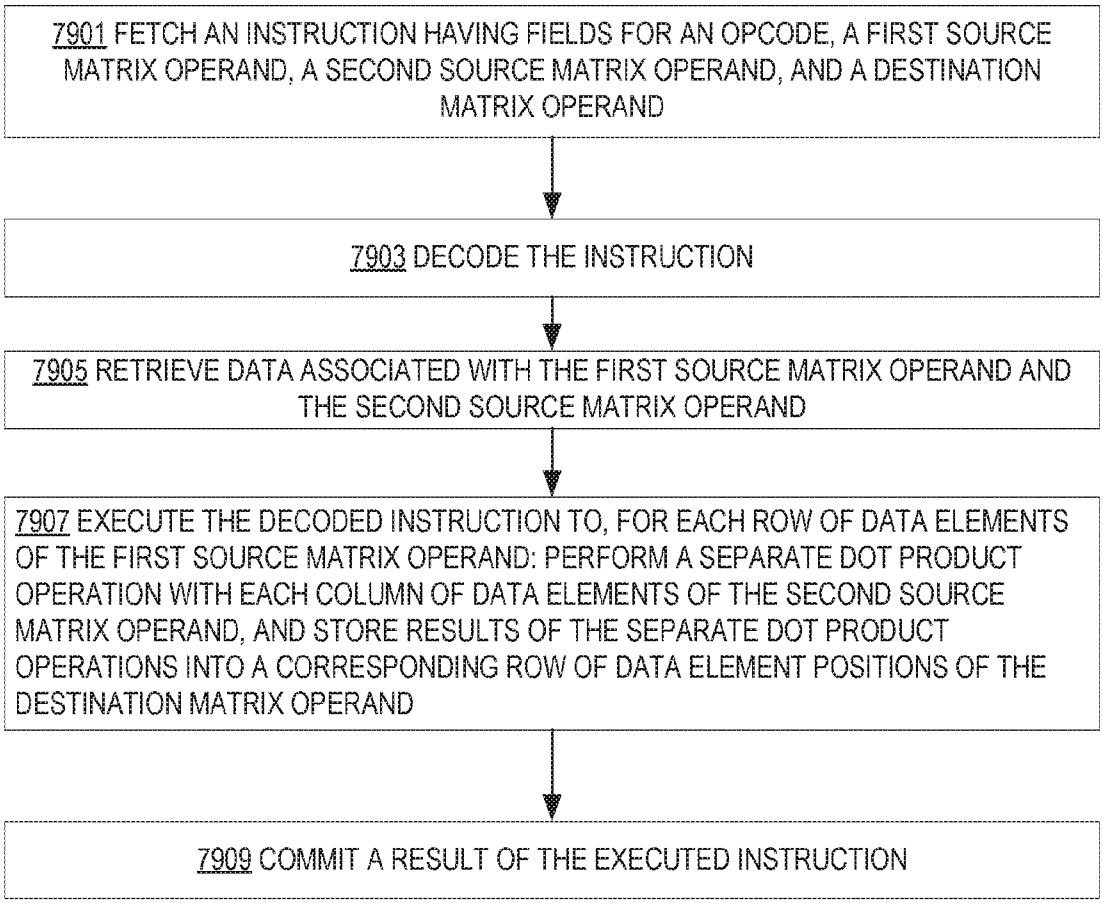

7901 FETCH AN INSTRUCTION HAVING FIELDS FOR AN OPCODE, A FIRST SOURCE MATRIX OPERAND, A SECOND SOURCE MATRIX OPERAND, AND A DESTINATION MATRIX OPERAND
7903 DECODE THE INSTRUCTION
7905 RETRIEVE DATA ASSOCIATED WITH THE FIRST SOURCE MATRIX OPERAND AND THE SECOND SOURCE MATRIX OPERAND
7907 EXECUTE THE DECODED INSTRUCTION TO, FOR EACH ROW OF DATA ELEMENTS OF THE FIRST SOURCE MATRIX OPERAND: PERFORM A SEPARATE DOT PRODUCT OPERATION WITH EACH COLUMN OF DATA ELEMENTS OF THE SECOND SOURCE MATRIX OPERAND, AND STORE RESULTS OF THE SEPARATE DOT PRODUCT OPERATIONS INTO A CORRESPONDING ROW OF DATA ELEMENT POSITIONS OF THE DESTINATION MATRIX OPERAND
7909 COMMIT A RESULT OF THE EXECUTED INSTRUCTION

FIG. 79

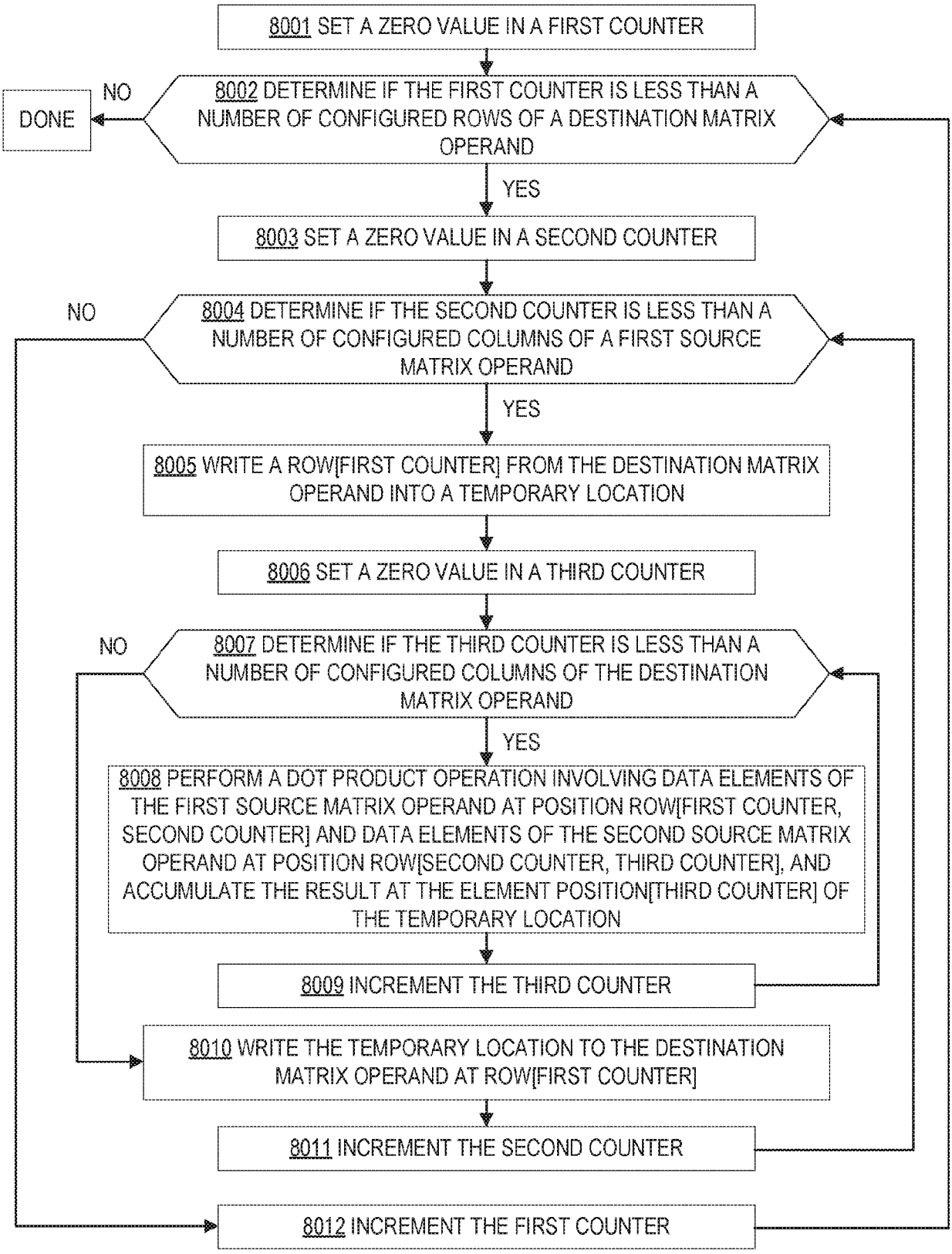

8001 SET A ZERO VALUE IN A FIRST COUNTER
8002 DETERMINE IF THE FIRST COUNTER IS LESS THAN A NUMBER OF CONFIGURED ROWS OF A DESTINATION MATRIX OPERAND
NO
DONE
YES
8003 SET A ZERO VALUE IN A SECOND COUNTER
8004 DETERMINE IF THE SECOND COUNTER IS LESS THAN A NUMBER OF CONFIGURED COLUMNS OF A FIRST SOURCE MATRIX OPERAND
NO
YES
8005 WRITE A ROW[FIRST COUNTER] FROM THE DESTINATION MATRIX OPERAND INTO A TEMPORARY LOCATION
8006 SET A ZERO VALUE IN A THIRD COUNTER
8007 DETERMINE IF THE THIRD COUNTER IS LESS THAN A NUMBER OF CONFIGURED COLUMNS OF THE DESTINATION MATRIX OPERAND
NO
YES
8008 PERFORM A DOT PRODUCT OPERATION INVOLVING DATA ELEMENTS OF THE FIRST SOURCE MATRIX OPERAND AT POSITION ROW[FIRST COUNTER, SECOND COUNTER] AND DATA ELEMENTS OF THE SECOND SOURCE MATRIX OPERAND AT POSITION ROW[SECOND COUNTER, THIRD COUNTER], AND ACCUMULATE THE RESULT AT THE ELEMENT POSITION[THIRD COUNTER] OF THE TEMPORARY LOCATION
8009 INCREMENT THE THIRD COUNTER
8010 WRITE THE TEMPORARY LOCATION TO THE DESTINATION MATRIX OPERAND AT ROW[FIRST COUNTER]
8011 INCREMENT THE SECOND COUNTER
8012 INCREMENT THE FIRST COUNTER

FIG. 80

8101 TDPWSSDS DOT PRODUCT instruction helper function

```
define DPWSS(c,x,y): // arguments are dwords
     p1dword := SIGN_EXTEND(x.word[0]) * SIGN_EXTEND(y.word[0])
     p2dword := SIGN_EXTEND(x.word[1]) * SIGN_EXTEND(y.word[1])

     c := SIGNED_DWORD_SATURATE(c + p1dword + p2dword)
```

8103 TDPWSSDS DOT PRODUCT operation

```
TDPWSSDS tsrcdest, tsrc1, tsrc2
// C = m x n (tsrcdest), A = m x k (tsrc1), B = k x n (tsrc2)
#GP if TILES_CONFIGURED == 0
#GP if tsrc1.cols != tsrc2.rows
#GP if tsrcdest.rows != tsrc1.rows
#GP if tsrcdest.cols != tsrc2.cols

#GP if tsrc2.rows > impl.tmul_maxk
#GP if tsrc2.cols * 4 > impl.tmul_maxn
#GP if tsrcdest.cols * 4 > impl.tmul_maxn

for m in 0 ... tsrcdest.rows-1:
     for k in 0 ... tsrc1.cols-1:
          tmp := tsrcdest.row[m]
          for n in 0 ... tsrcdest.cols-1:
               DPWSS(tmp.dword[n],
                    tsrc1.row[m].dword[k],
                    tsrc2.row[k].dword[n])
          write_row_and_zero(tsrcdest, m, tmp, 4 * tsrcdest.cols)

zero_upper_rows(tsrcdest, tsrcdest.rows)
zero_tileconfig_start()
```

FIG. 81A

FIG. 81B

8105 TDPWSSDS DOT PRODUCT instruction helper function

```
define DPWSSQ(c,x,y): // c is a qword. x and y are qwords containing 4 int16
      p1dword := SIGN_EXTEND(x.word[0]) * SIGN_EXTEND(y.word[0])
      p2dword := SIGN_EXTEND(x.word[1]) * SIGN_EXTEND(y.word[1])
      p3dword := SIGN_EXTEND(x.word[2]) * SIGN_EXTEND(y.word[2])
      p4dword := SIGN_EXTEND(x.word[3]) * SIGN_EXTEND(y.word[3])

      c := SIGNED_QWORD_SATURATE(c + p1dword + p2dword + p3dword + p4dword)
```

8107 TDPWSSDS DOT PRODUCT operation

```
TDPWSSQS tsrcdest, tsrc1, tsrc2
// C = m x n (tsrcdest), A = m x k (tsrc1), B = k x n (tsrc2)
// Tile A is an even/odd pair of tile regs indicated by tsrc1.
// The tiles[.] notation denotes an array representing all the tile registers.

#GP if TILES_CONFIGURED == 0
// reg_id_tsrc1 is the register field in the encoding for the tsrc1 operand.
t1_left := (reg_id_tsrc1 & ~1)
t1_right := (reg_id_tsrc1 & ~1) + 1

startK := tileconfig.startK
startM := tileconfig.startM

#GP if tsrcdest.rows != tiles[t1_left].rows
#GP if tsrcdest.rows != tiles[t1_right].rows
#GP if tsrcdest.cols != tsrc2.cols

#GP if startM >= tsrcdest.rows
#GP if startK >= tsrc2.rows

#GP if tsrc2.rows        > impl.tmul_maxk
#GP if tsrc2.cols * 8    > impl.tmul_maxn
#GP if tsrcdest.cols * 8 > impl.tmul_maxn

if tsrc2.rows > impl.tmul_maxk / 2:
      lim_left := impl.tmul_maxk/2
      lim_right := tsrc2.rows - lim_left
      #GP if tiles[t1_left].cols != lim_left
      #GP if tiles[t1_right].cols != lim_right
else:
      lim_left := tsrc2.rows
      lim_right := 0 // ignore t1_right if not required!
      #GP if tiles[t1_left].cols != lim_left
```

(continued at FIG. 81C)

8107 TDPWSSDS DOT PRODUCT operation (continued)

```
while startM < tsrcdest.rows:
     while startK < tsrc2.rows:
          tempC := tsrcdest.row[startM]

          tk := startK % (impl.tmul_maxk/2)
          if startK < impl.tmul_maxk/2:
               tempA := tiles[t1_left].row[m].qword[tk]
          else:
               tempA := tiles[t1_right].row[m].qword[tk]

          for n := 0 ... tsrcdest.cols-1: //SIMD loop
               DPWSSQ(tempC.qword[n],
                      tempA,
                      tsrc2.row[startK].qword[n])
          write_row_and_zero(tsrcdest, startM, tempC, 8 * tsrcdest.cols)
          startK := startK + 1
     startK := 0
     startM := startM + 1

zero_upper_rows(tsrcdest, tsrcdest.rows)
zero_tileconfig_start()
```

FIG. 81C

8109 TDPB[SS,UU,US,SU]DS DOT PRODUCT operation helper function

```
define DPBD(c,x,y): // arguments are dwords
      if *UU version*:
            saturation_fn := UNSIGNED_SATURATION
      else:
            saturation_fn := SIGNED_SATURATION

      if *x operand is signed*:
            extend_src1 := SIGN_EXTEND
      else:
            extend_src1 := ZERO_EXTEND

      if *y operand is signed*:
            extend_src2 := SIGN_EXTEND
      else:
            extend_src2 := ZERO_EXTEND

      p0word := extend_src1(x.byte[0]) * extend_src2(y.byte[0])
      p1word := extend_src1(x.byte[1]) * extend_src2(y.byte[1])
      p2word := extend_src1(x.byte[2]) * extend_src2(y.byte[2])
      p3word := extend_src1(x.byte[3]) * extend_src2(y.byte[3])

      c := saturation_fn(c + p0word + p1word + p2word + p3word)
```

8111 TDPB[SS,UU,US,SU]DS DOT PRODUCT operation

```
TDPB[SS,SU,US,SS]DS tsrcdest, tsrc1, tsrc2
// C = m x n (tsrcdest), A = m x k (tsrc1), B = k x n (tsrc2)
#GP if TILES_CONFIGURED == 0
#GP if tsrc1.cols != tsrc2.rows
#GP if tsrcdest.rows != tsrc1.rows
#GP if tsrcdest.cols != tsrc2.cols

#GP if tsrc2.rows > impl.tmul_maxk
#GP if tsrc2.cols * 4 > impl.tmul_maxn
#GP if tsrcdest.cols * 4 > impl.tmul_maxn

for m in 0 ... tsrcdest.rows-1:
      for k in 0 ... tsrc1.cols-1:
            tmp := tsrcdest.row[m]
            for n in 0 ... tsrcdest.cols-1:
                  DPBD(tmp.dword[n],
                        tsrc1.row[m].dword[k],
                        tsrc2.row[k].dword[n])
            write_row_and_zero(tsrcdest, m, tmp, 4 * tsrcdest.cols)

zero_upper_rows(tsrcdest, tsrcdest.rows)
zero_tileconfig_start()
```

FIG. 81D

FIG. 81E

8113 TDP8B[SS,UU,US,SU]4BITDS DOT PRODUCT operation

```
TDP8B[SS,SU,US,UU]4BITDS tsrcdest, tsrc1, tsrc2
// tsrdest (m x n) := tilepair (m x k, bytes) * tsrc2 (k x n, 4b values)
// tilepair is an even/odd pair of tile regs indicated by tsrc1.
// The tiles[.] notation denotes an array representing all the tile registers.

#GP if TILES_CONFIGURED == 0
// reg_id_tsrc1 is the register field in the encoding for the tsrc1 operand.
t1_left := (reg_id_tsrc1 & ~1)
t1_right := (reg_id_tsrc1 & ~1) + 1

#GP if tsrcdest.rows != tiles[t1_left].rows
#GP if tsrcdest.rows != tiles[t1_right].rows
#GP if tsrcdest.cols != tsrc2.cols

#GP if tsrc2.rows        > impl.tmul_maxk
#GP if tsrc2.cols * 4    > impl.tmul_maxn
#GP if tsrcdest.cols * 4 > impl.tmul_maxn

if tsrc2.rows > impl.tmul_maxk / 2:
      lim_left := impl.tmul_maxk/2
      lim_right := tsrc2.rows - lim_left
      #GP if tiles[t1_left].cols != lim_left
      #GP if tiles[t1_right].cols != lim_right
else:
      lim_left := tsrc2.rows
      lim_right := 0 // ignore t1_right if not required!
      #GP if tiles[t1_left].cols != lim_left

if *tsrc2 operand is signed*:
      extend_src2 := SIGN_EXTEND
else:
      extend_src2 := ZERO_EXTEND

if *tsrc1 operand is signed*:
      extend_src1 := SIGN_EXTEND
else:
      extend_src1 := ZERO_EXTEND
```

(continued at FIG. 81F)

FIG. 81F

8113 TDP8B[SS,UU,US,SU]4BITDS DOT PRODUCT operation (continued)

```
if *tsrc1 is unsigned and tsrc2 is unsigned*:
      saturation_fn := UNSIGNED_SATURATION
else:
      saturation_fn := SIGNED_SATURATION

for m in 0..tsrcdest.rows-1:
      for k in 0..tsrc2.rows-1:
            tk := k % (impl.tmul_maxk/2)
            if k < impl.tmul_maxk/2:
                  tq := tiles[t1_left].row[m].qword[tk]
            else:
                  tq := tiles[t1_right].row[m].qword[tk]
            for n in 0..tsrcdest.cols-1:
                  for b in 0..7: // 4-bit array index & tq byte index
                        x := tsrc2[k].dword[n].nibble[b] // extract 4 bits
                        tprod.dword[b] := extend_src2(x) * extend_src1(tq.byte[b])
                  // 9-way sum
                  tsrcdest.row[m].dword[n] := saturation_fn(tsrcdst.row[m].dword[n] +
                                              tprod.dword[0] +
                                              tprod.dword[1] +
                                              tprod.dword[2] +
                                              ...
                                              tprod.dword[7] )
zero_upper_rows(tsrcdest, tsrcdest.rows)
zero_tileconfig_start()
```

8115 TDP4BIT[S,U][S,U]DS DOT PRODUCT operation

```
TDP4BIT[SS,SU,US,UU]DS tsrcdest, tsrc1, tsrc2
// tsrdest (m x n) += tsrc1 (m x k, bytes) op tsrc2 (k x n, 4b values)

#GP if TILES_CONFIGURED == 0
#GP if tsrcdest.cols != tsrc2.cols
#GP if tsrc2.rows > impl.tmul_maxk
#GP if tsrc2.cols * 4 > impl.tmul_maxn
#GP if tsrcdest.cols * 4 > impl.tmul_maxn

startK := tileconfig.startK
startM := tileconfig.startM
#GP if startM >= tsrcdest.rows
#GP if startK >= tsrc2.rows

if *src2 operand is signed*:
      extend_src2 := SIGN_EXTEND
else:
      extend_src2 := ZERO_EXTEND

if *tmem operand is signed*:
      extend_mem := SIGN_EXTEND
else:
      extend_mem := ZERO_EXTEND

if *src1 is unsigned and src2 is unsigned*:
      saturation_fn := UNSIGNED_SATURATION
else:
      saturation_fn := SIGNED_SATURATION

while startM < tsrcdest.rows:
      while startK < tsrc2.rows:
            td := tsrc1.row[startM].dword[startK]
            for n in 0..tsrcdest.cols-1:
                  for b in 0..7:
                        x := tsrc2.row[startK].dword[n].nibble[b]
                        tprod.dword[b] := extend_src2(x) * extend_mem(td.nibble[b])
                  // 9-way sum
                  tsrcdest.row[startM].dword[n] :=
                              saturation_fn(tsrcdst.row[startM].dword[n] +
                              tprod.dword[0] +
                              tprod.dword[1] +
                              tprod.dword[2] +
                              ...
                              tprod.dword[7])
      startK := 0
      startM := startM + 1
zero_upper_rows(tsrcdest, tsrcdest.rows)
zero_tileconfig_start()
```

FIG. 81G

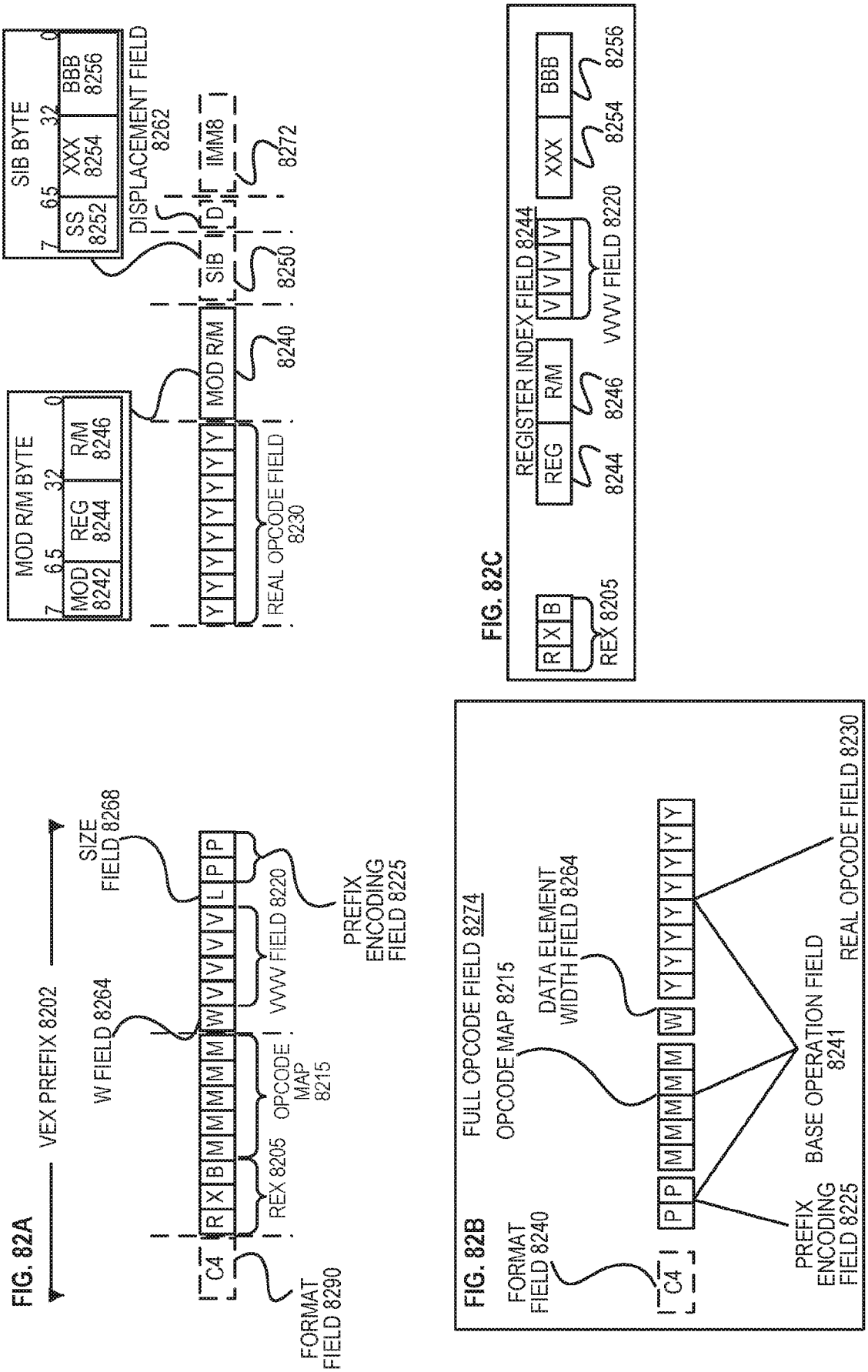


FIG. 82A
VEX PREFIX 8202
SIZE FIELD 8268
W FIELD 8264
C4
R X B M M M M W V V V V L P P
REX 8205
OPCODE MAP 8215
VVV FIELD 8220
PREFIX ENCODING FIELD 8225
FORMAT FIELD 8290
MOD R/M BYTE
7 6 5 3 2 0
MOD 8242
REG 8244
R/M 8246
SIB BYTE
SS 8252
XXX 8254
BBB 8256
DISPLACEMENT FIELD 8262
Y Y Y Y Y Y Y Y
REAL OPCODE FIELD 8230
MOD R/M
SIB
D
IMM8
8240
8250
8272
FIG. 82C
REGISTER INDEX FIELD 8244
R X B
REX 8205
REG
R/M
V V V V
VVV FIELD 8220
XXX
BBB
8244
8246
8254
8256
FIG. 82B
FULL OPCODE FIELD 8274
FORMAT FIELD 8240
OPCODE MAP 8215
DATA ELEMENT WIDTH FIELD 8264
C4
P P
M M M M
W
Y Y Y Y Y Y Y Y
PREFIX ENCODING FIELD 8225
BASE OPERATION FIELD 8241
REAL OPCODE FIELD 8230

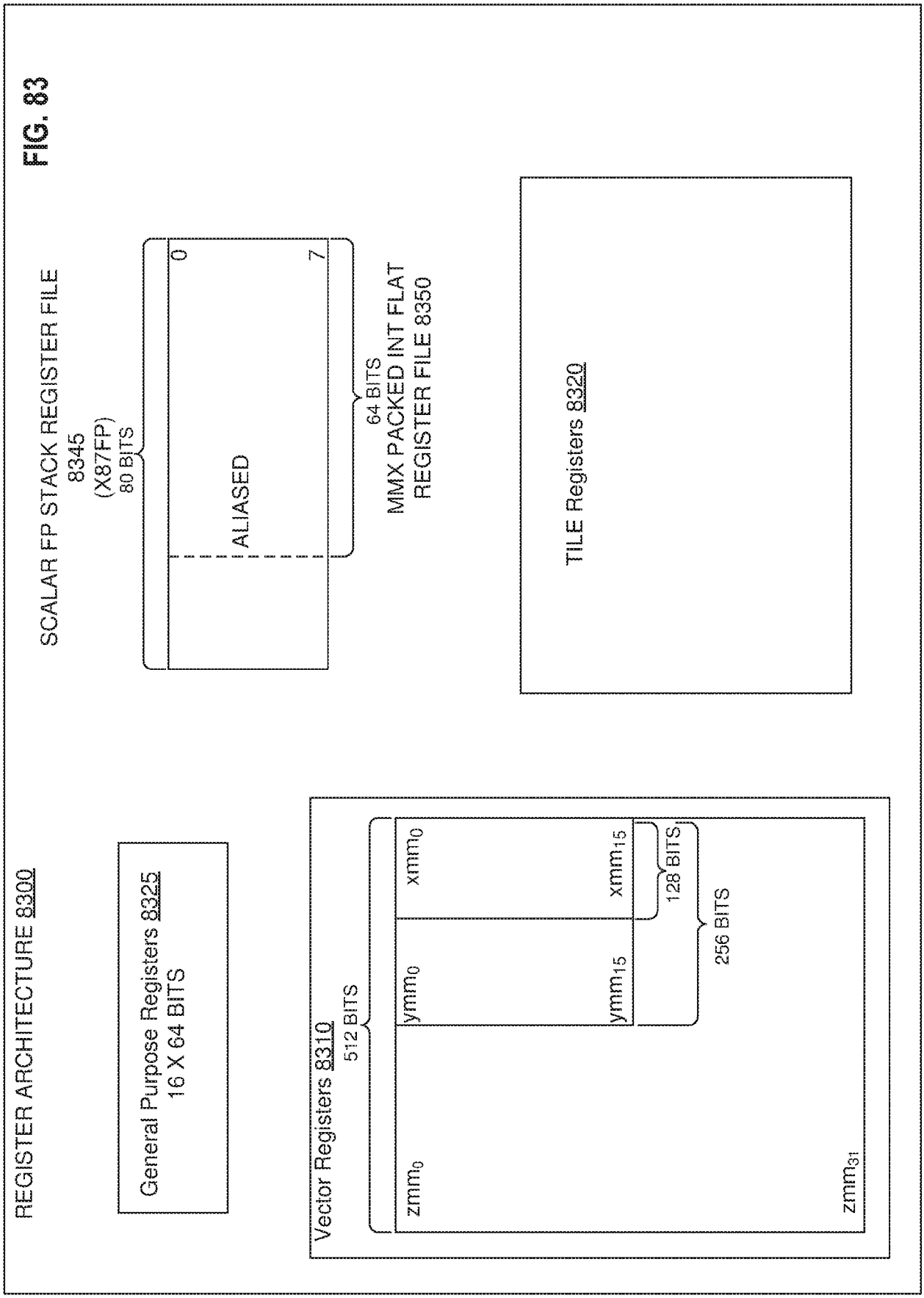

REGISTER ARCHITECTURE 8300
General Purpose Registers 8325
16 X 64 BITS
SCALAR FP STACK REGISTER FILE
8345
(X87FP)
80 BITS
0
7
ALIASED
64 BITS
MMX PACKED INT FLAT
REGISTER FILE 8350
TILE Registers 8320
Vector Registers 8310
512 BITS
zmm0
ymm0
xmm0
ymm15
xmm15
128 BITS
256 BITS
zmm31

FIG. 83

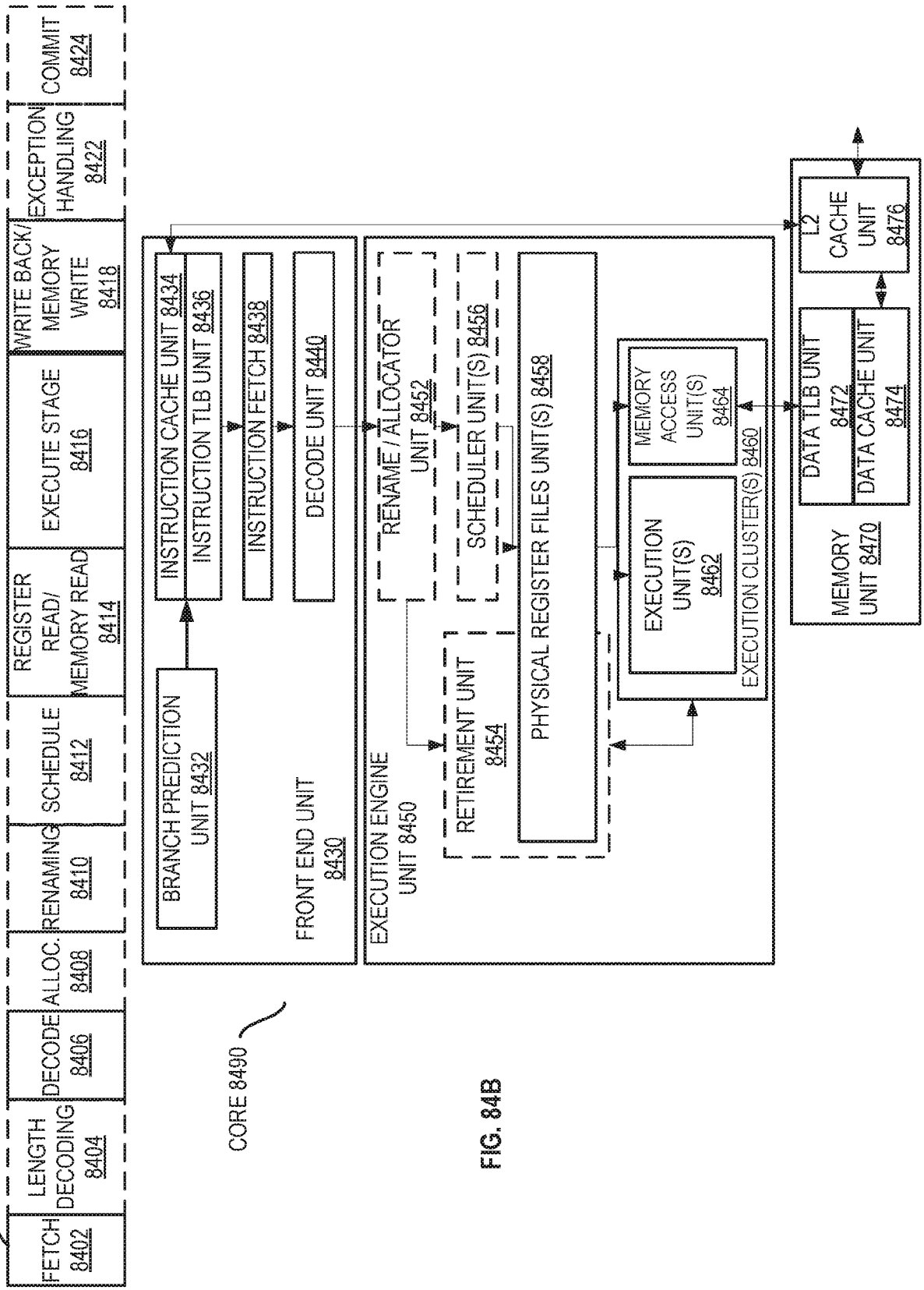


FIG. 84A

FIG. 84B

FIG. 85B

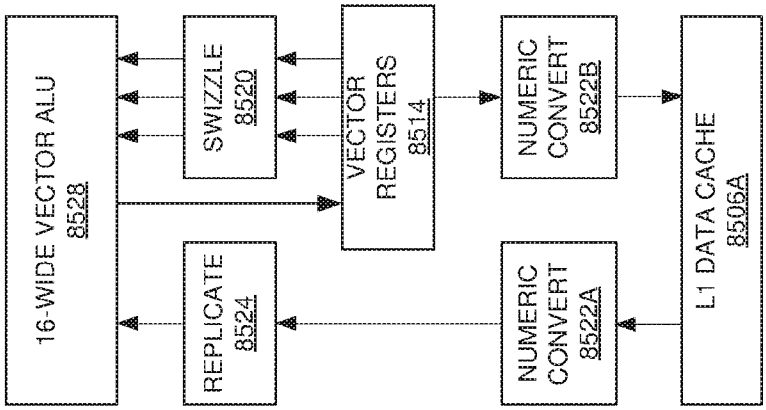
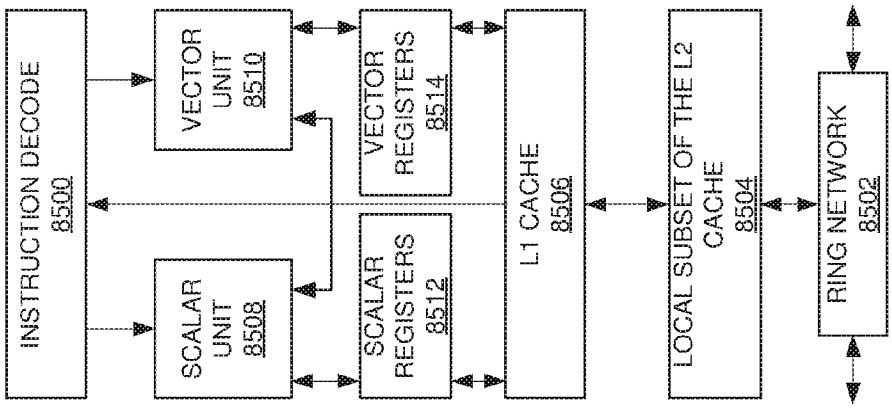


FIG. 85A



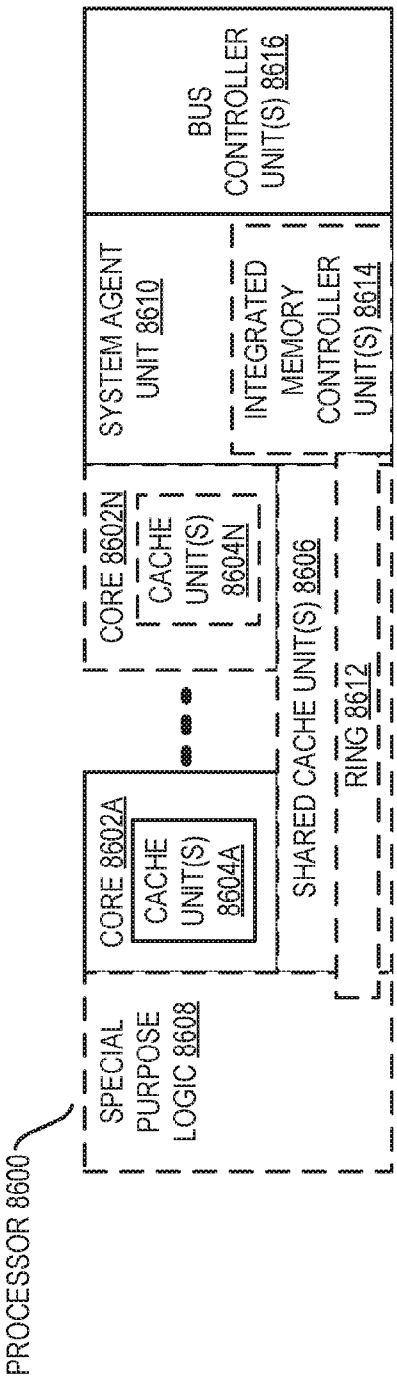

PROCESSOR 8600
SPECIAL PURPOSE LOGIC 8608
CORE 8602A
CACHE UNIT(S) 8604A
CORE 8602N
CACHE UNIT(S) 8604N
SHARED CACHE UNIT(S) 8606
RING 8612
SYSTEM AGENT UNIT 8610
INTEGRATED MEMORY CONTROLLER UNIT(S) 8614
BUS CONTROLLER UNIT(S) 8616

FIG. 86

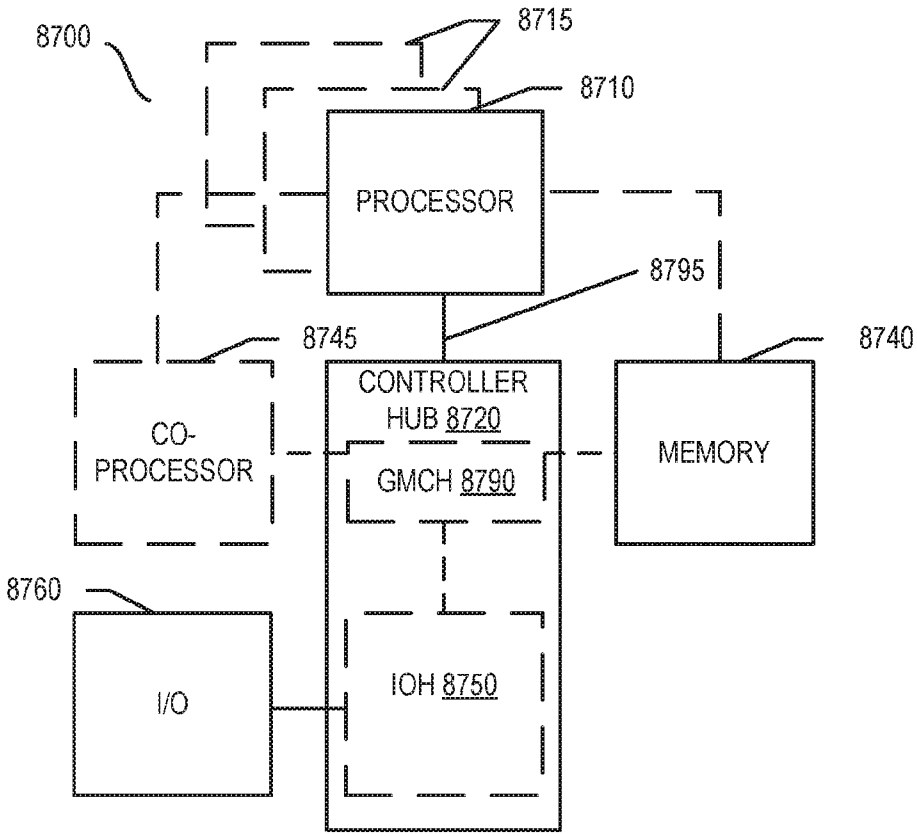

8700
8715
8710
PROCESSOR
8795
8745
8740
CONTROLLER
HUB 8720
CO-
PROCESSOR
GMCH 8790
MEMORY
8760
I/O
IOH 8750

FIG. 87

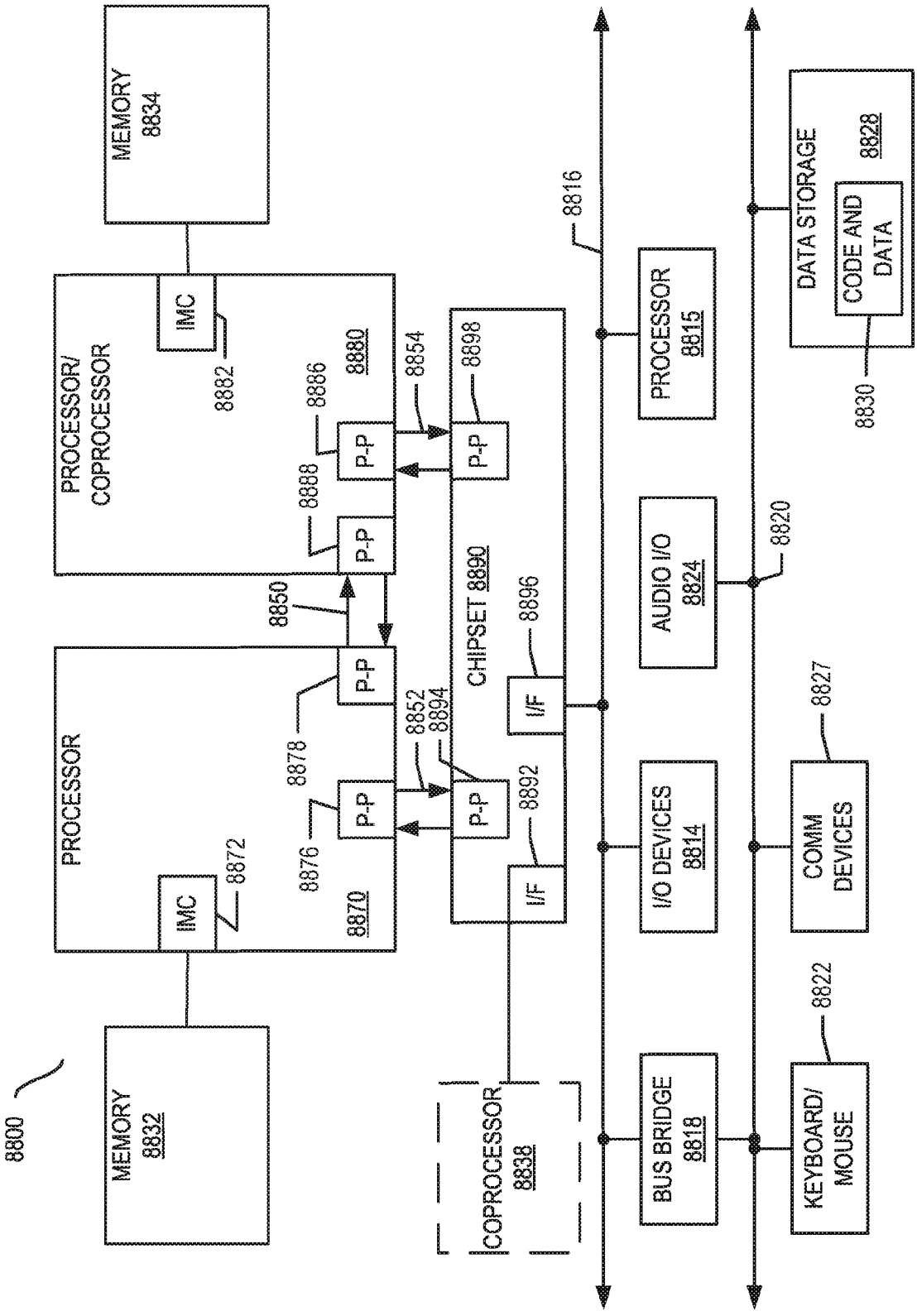

8800
MEMORY 8832
MEMORY 8834
PROCESSOR
IMC
8872
8870
PROCESSOR/ COPROCESSOR
IMC
8882
8880
P-P
8876
8878
8886
8888
8850
8852
8854
8894
8892
8896
8898
CHIPSET 8890
I/F
I/F
COPROCESSOR 8838
8816
BUS BRIDGE 8818
I/O DEVICES 8814
PROCESSOR 8815
AUDIO I/O 8824
8820
KEYBOARD/ MOUSE
8822
COMM DEVICES
8827
DATA STORAGE
CODE AND DATA
8828
8830

FIG. 88

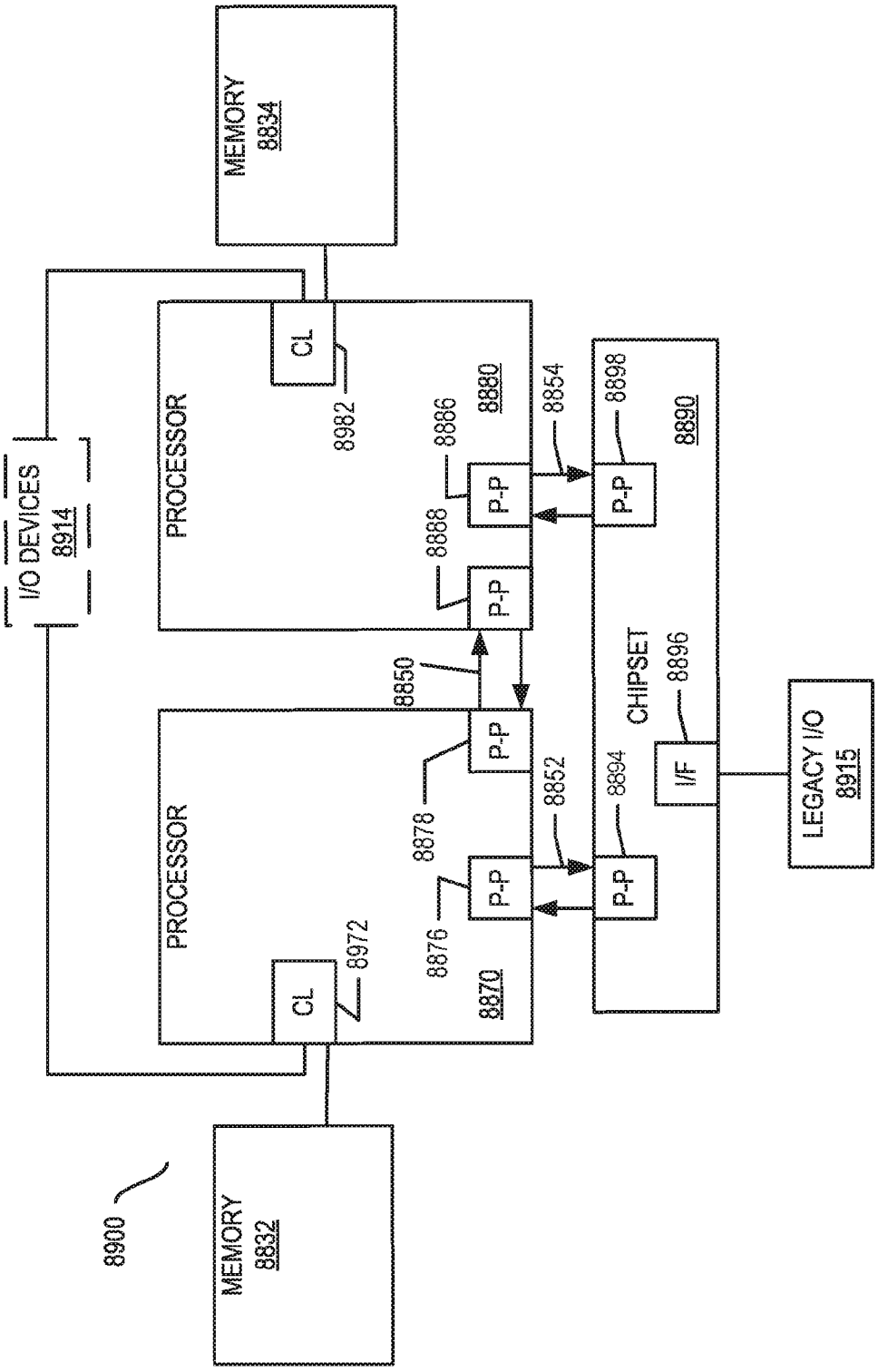

8900
MEMORY
8832
MEMORY
8834
I/O DEVICES
8914
PROCESSOR
CL
8972
8870
PROCESSOR
CL
8982
8880
P-P
8876
P-P
8878
P-P
8888
P-P
8886
8850
8852
8854
CHIPSET
8890
P-P
8894
P-P
8898
I/F
8896
LEGACY I/O
8915

FIG. 89

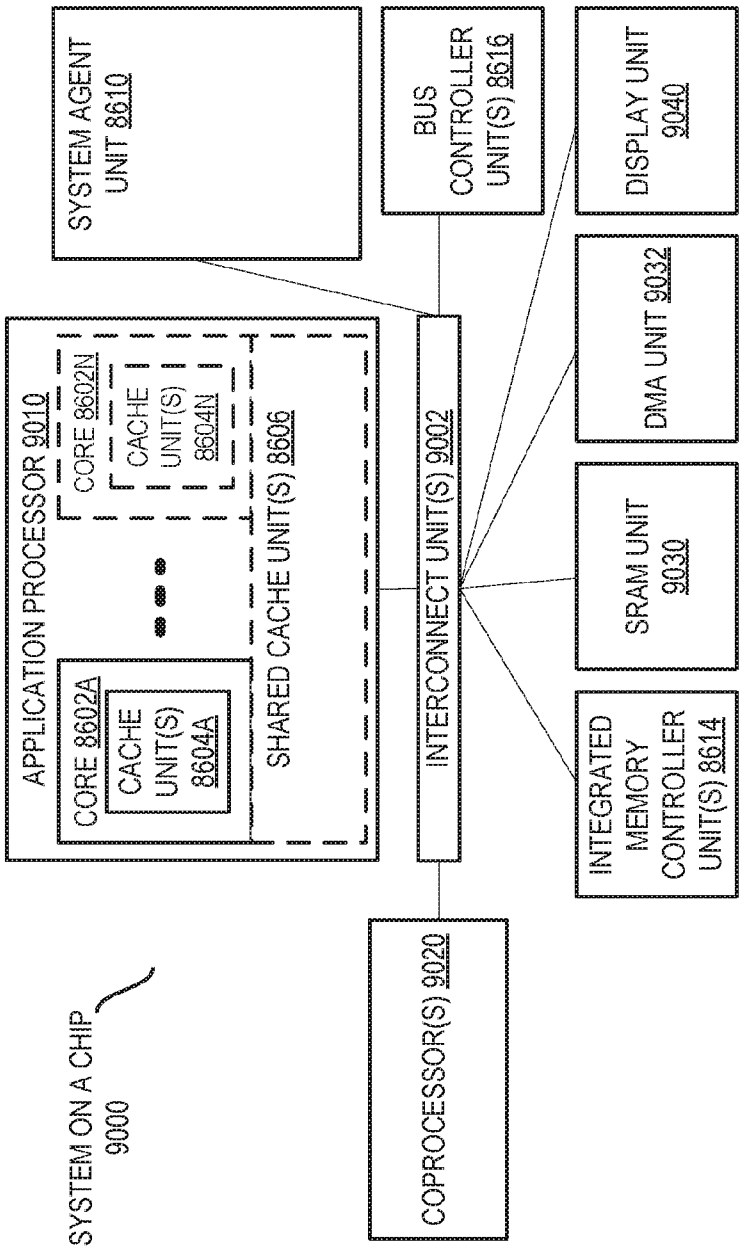

SYSTEM ON A CHIP
9000
APPLICATION PROCESSOR 9010
CORE 8602A
CACHE UNIT(S) 8604A
CORE 8602N
CACHE UNIT(S) 8604N
SHARED CACHE UNIT(S) 8606
SYSTEM AGENT UNIT 8610
BUS CONTROLLER UNIT(S) 8616
INTERCONNECT UNIT(S) 9002
COPROCESSOR(S) 9020
INTEGRATED MEMORY CONTROLLER UNIT(S) 8614
SRAM UNIT 9030
DMA UNIT 9032
DISPLAY UNIT 9040

FIG. 90

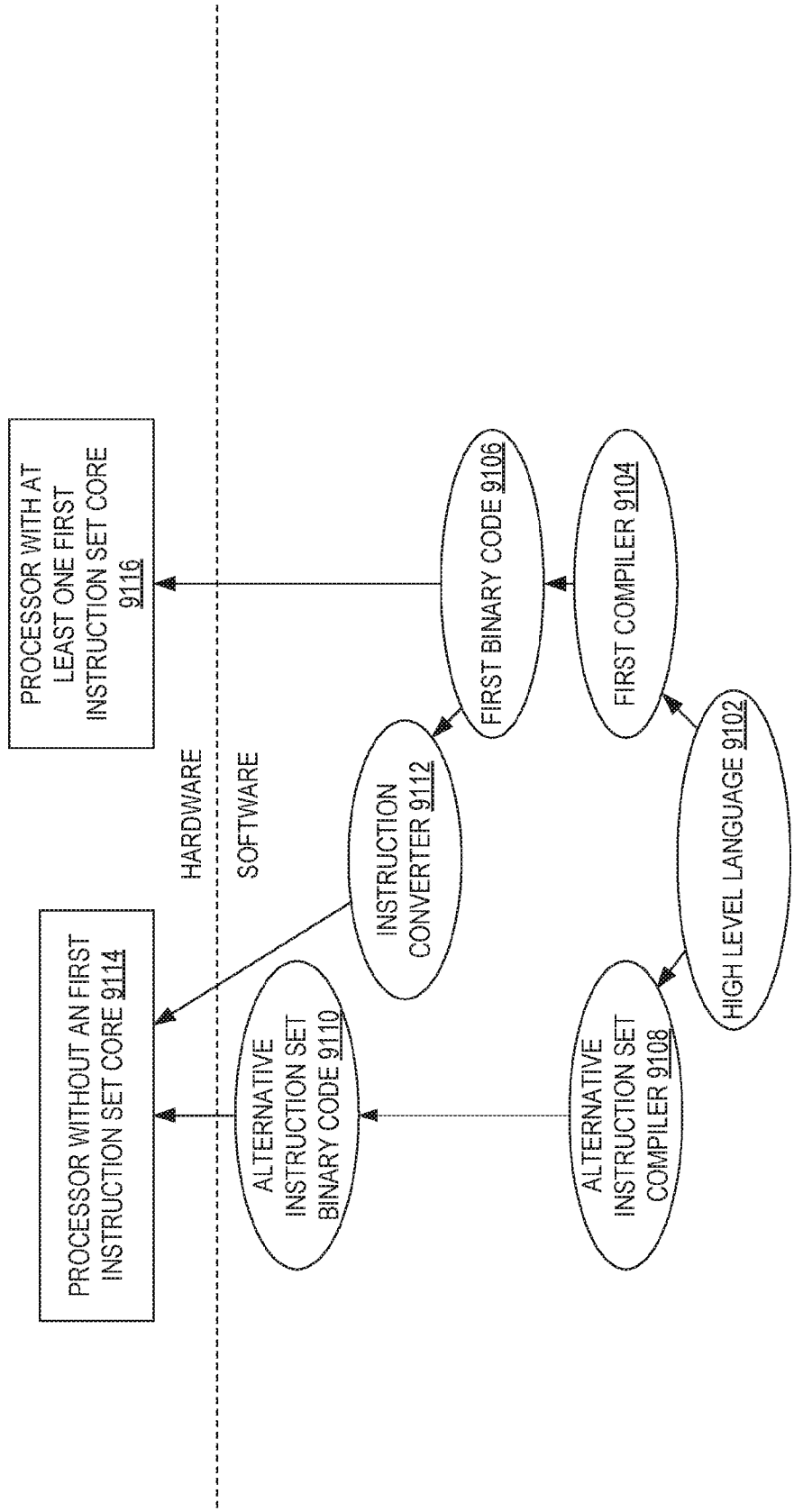

PROCESSOR WITH AT LEAST ONE FIRST INSTRUCTION SET CORE 9116
HARDWARE
SOFTWARE
FIRST BINARY CODE 9106
FIRST COMPILER 9104
INSTRUCTION CONVERTER 9112
HIGH LEVEL LANGUAGE 9102
PROCESSOR WITHOUT AN FIRST INSTRUCTION SET CORE 9114
ALTERNATIVE INSTRUCTION SET BINARY CODE 9110
ALTERNATIVE INSTRUCTION SET COMPILER 9108

FIG. 91

INTERNATIONAL SEARCH REPORT

International application No.
PCT/US2017/040546

A. CLASSIFICATION OF SUBJECT MATTER

G06F 9/30(2006.01)i

According to International Patent Classification (IPC) or to both national classification and IPC

B. FIELDS SEARCHED

Minimum documentation searched (classification system followed by classification symbols)
G06F 9/30; G06F 15/00; G06F 7/52; G06T 15/00; G06F 12/00; G06F 7/57; G06T 1/00; G09G 5/00

Documentation searched other than minimum documentation to the extent that such documents are included in the fields searched
Korean utility models and applications for utility models
Japanese utility models and applications for utility models

Electronic data base consulted during the international search (name of data base and, where practicable, search terms used)
eKOMPASS(KIPO internal) & Keywords: matrix, operation, instruction, two-dimensional, execute

C. DOCUMENTS CONSIDERED TO BE RELEVANT

Category*	Citation of document, with indication, where appropriate, of the relevant passages	Relevant to claim No.
X	US 7932910 B2 (CRAIG HANSEN et al.) 26 April 2011 See column 4, lines 2-4; column 27, lines 3-6; column 30, lines 63-64; column 86, lines 51-58; column 135, lines 34-38; column 152, lines 35-37; claims 1, 15; and figure 1.	1-8
Y		9-17
Y	US 5892962 A (JOCELYN CLOUTIER) 06 April 1999 See column 5, lines 7-8; and figure 1.	9-17
A	WO 2016-003740 A1 (VIA ALLIANCE SEMICONDUCTOR CO., LTD.) 07 January 2016 See claims 1-6; and figures 1-2.	1-17
A	US 2012-0113133 A1 (SHAI SHPIGELBLAT) 10 May 2012 See paragraphs [0066]-[0076]; and figure 6.	1-17
A	US 6332186 B1 (MATTHEW PAUL ELWOOD et al.) 18 December 2001 See column 28, lines 20-31; and figure 5.	1-17

☐ Further documents are listed in the continuation of Box C.

☒ See patent family annex.

* Special categories of cited documents:
"A" document defining the general state of the art which is not considered to be of particular relevance
"E" earlier application or patent but published on or after the international filing date
"L" document which may throw doubts on priority claim(s) or which is cited to establish the publication date of another citation or other special reason (as specified)
"O" document referring to an oral disclosure, use, exhibition or other means
"P" document published prior to the international filing date but later than the priority date claimed
"T" later document published after the international filing date or priority date and not in conflict with the application but cited to understand the principle or theory underlying the invention
"X" document of particular relevance; the claimed invention cannot be considered novel or cannot be considered to involve an inventive step when the document is taken alone
"Y" document of particular relevance; the claimed invention cannot be considered to involve an inventive step when the document is combined with one or more other such documents,such combination being obvious to a person skilled in the art
"&" document member of the same patent family

Date of the actual completion of the international search	Date of mailing of the international search report
24 January 2018 (24.01.2018)	**24 January 2018 (24.01.2018)**

Name and mailing address of the ISA/KR
International Application Division
Korean Intellectual Property Office
189 Cheongsa-ro, Seo-gu, Daejeon, 35208, Republic of Korea
Facsimile No. +82-42-481-8578

Authorized officer
YANG, Jeong Rok
Telephone No. +82-42-481-5709

INTERNATIONAL SEARCH REPORT

Information on patent family members

International application No.

PCT/US2017/040546

Patent document cited in search report	Publication date	Patent family member(s)	Publication date
US 7932910 B2	26/04/2011	AT 484022 T	15/10/2010
		AT 511137 T	15/06/2011
		AT 528712 T	15/10/2011
		AT 532130 T	15/11/2011
		AT 548691 T	15/03/2012
		AT 557342 T	15/05/2012
		AT 557343 T	15/05/2012
		AU 2002-339867 A1	18/03/2003
		AU 6771696 A	12/03/1997
		CA 2573292 A1	27/01/2005
		CA 2573292 C	17/01/2012
		CA 2735354 A1	27/01/2005
		DE 96928129 T1	10/05/2007
		EP 0845120 A1	04/05/2005
		EP 0845120 B1	06/10/2010
		EP 1105792 A1	13/06/2001
		EP 1105792 B1	05/05/2010
		EP 1236090 A1	04/09/2002
		EP 1652090 A2	03/05/2006
		EP 1873628 A2	02/01/2008
		EP 1873628 A3	21/05/2008
		EP 1873629 A2	02/01/2008
		EP 1873629 A3	30/07/2008
		EP 1873630 A2	02/01/2008
		EP 1873630 A3	28/05/2008
		EP 1873630 B1	02/11/2011
		EP 1873653 A2	02/01/2008
		EP 1873653 A3	04/06/2008
		EP 1873654 A2	02/01/2008
		EP 1873654 A3	06/08/2008
		EP 1876524 A2	09/01/2008
		EP 1876524 A3	19/03/2008
		EP 1876524 B1	12/10/2011
		EP 1876535 A2	09/01/2008
		EP 1876535 A3	09/04/2008
		EP 1876536 A2	09/01/2008
		EP 1876536 A3	25/06/2008
		EP 1879102 A2	16/01/2008
		EP 1879102 A3	25/02/2009
		EP 1879102 B1	25/05/2011
		EP 1879103 A2	16/01/2008
		EP 1879103 A3	24/12/2008
		EP 1879103 B1	07/03/2012
		EP 1879104 A2	16/01/2008
		EP 1879104 A3	13/08/2008
		EP 1879397 A2	16/01/2008
		EP 1879397 A3	23/04/2008
		EP 1879398 A2	16/01/2008
		EP 1879398 A3	10/12/2008

INTERNATIONAL SEARCH REPORT

Information on patent family members

International application No.

PCT/US2017/040546

Patent document cited in search report	Publication date	Patent family member(s)	Publication date
		EP 1879398 B1	06/06/2012
		EP 2226725 A2	08/09/2010
		EP 2226725 A3	03/11/2010
		EP 2226726 A2	08/09/2010
		EP 2226726 A3	20/10/2010
		EP 2228727 A2	15/09/2010
		EP 2228727 A3	20/10/2010
		EP 2241968 A2	20/10/2010
		EP 2241968 A3	03/11/2010
		EP 2241968 B1	27/06/2012
		EP 2284709 A1	16/02/2011
		EP 2284709 B1	26/11/2014
		EP 2302510 A1	30/03/2011
		EP 2302510 B1	09/05/2012
		EP 2309382 A1	13/04/2011
		EP 2309382 B1	03/12/2014
		EP 2309383 A1	13/04/2011
		EP 2309383 B1	09/05/2012
		ES 2275453 T1	16/06/2007
		ES 2527937 T3	02/02/2015
		JP 03482172 B2	22/12/2003
		JP 03827659 B2	27/09/2006
		JP 04933693 B2	16/05/2012
		JP 05005342 B2	22/08/2012
		JP 2000-314954 A	14/11/2000
		JP 2002-528786 A	03/09/2002
		JP 2004-004941 A	08/01/2004
		JP 2007-531072 A	01/11/2007
		US 2002-0133682 A1	19/09/2002
		US 2003-0110197 A1	12/06/2003
		US 2004-0015533 A1	22/01/2004
		US 2004-0049663 A1	11/03/2004
		US 2004-0098548 A1	20/05/2004
		US 2004-0098567 A1	20/05/2004
		US 2004-0103266 A1	27/05/2004
		US 2004-0107410 A1	03/06/2004
		US 2004-0153632 A1	05/08/2004
		US 2004-0156248 A1	12/08/2004
		US 2004-0158689 A1	12/08/2004
		US 2004-0199750 A1	07/10/2004
		US 2004-0205096 A1	14/10/2004
		US 2004-0205323 A1	14/10/2004
		US 2004-0205324 A1	14/10/2004
		US 2004-0205325 A1	14/10/2004
		US 2004-0210745 A1	21/10/2004
		US 2004-0210746 A1	21/10/2004
		US 2004-0215942 A1	28/10/2004
		US 2005-0289500 A1	29/12/2005
		US 2008-0040584 A1	14/02/2008
		US 2008-0059766 A1	06/03/2008

INTERNATIONAL SEARCH REPORT

Information on patent family members

International application No.

PCT/US2017/040546

Patent document cited in search report	Publication date	Patent family member(s)	Publication date
		US 2008-0059767 A1	06/03/2008
		US 2008-0065860 A1	13/03/2008
		US 2008-0065862 A1	13/03/2008
		US 2008-0072020 A1	20/03/2008
		US 2008-0091758 A1	17/04/2008
		US 2008-0091925 A1	17/04/2008
		US 2008-0104375 A1	01/05/2008
		US 2008-0104376 A1	01/05/2008
		US 2008-0162882 A1	03/07/2008
		US 2008-0177986 A1	24/07/2008
		US 2008-0189512 A1	07/08/2008
		US 2008-0222398 A1	11/09/2008
		US 2009-0019419 A1	15/01/2009
		US 2009-0031105 A1	29/01/2009
		US 2009-0083498 A1	26/03/2009
		US 2009-0089540 A1	02/04/2009
		US 2009-0094309 A1	09/04/2009
		US 2009-0100227 A1	16/04/2009
		US 2009-0106536 A1	23/04/2009
		US 2009-0113176 A1	30/04/2009
		US 2009-0113185 A1	30/04/2009
		US 2009-0113187 A1	30/04/2009
		US 2009-0158012 A1	18/06/2009
		US 2011-0107069 A1	05/05/2011
		US 2012-0117441 A1	10/05/2012
		US 2012-0204013 A1	09/08/2012
		US 2012-0215826 A1	23/08/2012
		US 2012-0311303 A1	06/12/2012
		US 2012-0317400 A1	13/12/2012
		US 2013-0013901 A1	10/01/2013
		US 2013-0173888 A1	04/07/2013
		US 2014-0351565 A1	27/11/2014
		US 2016-0048393 A1	18/02/2016
		US 2016-0321071 A1	03/11/2016
		US 5742840 A	21/04/1998
		US 5778419 A	07/07/1998
		US 5794060 A	11/08/1998
		US 5794061 A	11/08/1998
		US 5809321 A	15/09/1998
		US 5822603 A	13/10/1998
		US 5953241 A	14/09/1999
		US 6006318 A	21/12/1999
		US 6295599 B1	25/09/2001
		US 6378060 B1	23/04/2002
		US 6584482 B1	24/06/2003
		US 6643765 B1	04/11/2003
		US 6691297 B1	10/02/2004
		US 6725356 B2	20/04/2004
		US 7103870 B2	05/09/2006
		US 7213131 B2	01/05/2007

INTERNATIONAL SEARCH REPORT

Information on patent family members

International application No.

PCT/US2017/040546

Patent document cited in search report	Publication date	Patent family member(s)	Publication date
		US 7216217 B2	08/05/2007
		US 7222225 B2	22/05/2007
		US 7260708 B2	21/08/2007
		US 7301541 B2	27/11/2007
		US 7353367 B2	01/04/2008
		US 7386706 B2	10/06/2008
		US 7404165 B2	22/07/2008
		US 7430655 B2	30/09/2008
		US 7464252 B2	09/12/2008
		US 7483935 B2	27/01/2009
		US 7509366 B2	24/03/2009
		US 7516308 B2	07/04/2009
		US 7526635 B2	28/04/2009
		US 7565515 B2	21/07/2009
		US 7653806 B2	26/01/2010
		US 7660972 B2	09/02/2010
		US 7660973 B2	09/02/2010
		US 7730287 B2	01/06/2010
		US 7818548 B2	19/10/2010
		US 7843459 B2	30/11/2010
		US 7849291 B2	07/12/2010
		US 7889204 B2	15/02/2011
		US 7932911 B2	26/04/2011
		US 7940277 B2	10/05/2011
		US 7948496 B2	24/05/2011
		US 7952587 B2	31/05/2011
		US 7987344 B2	26/07/2011
		US 8001360 B2	16/08/2011
		US 8018464 B2	13/09/2011
		US 8095894 B2	10/01/2012
		US 8117426 B2	14/02/2012
		US 8195735 B2	05/06/2012
		US 8269784 B2	18/09/2012
		US 8289335 B2	16/10/2012
		US 8683182 B2	25/03/2014
		US 8769248 B2	01/07/2014
		US 8812821 B2	19/08/2014
		US 8943119 B2	27/01/2015
		US 9229713 B2	05/01/2016
		US 9378018 B2	28/06/2016
US 5892962 A	06/04/1999	CA 2215598 C	14/08/2001
WO 2016-003740 A1	07/01/2016	CN 105849690 A	10/08/2016
		CN 106126189 A	16/11/2016
		CN 106293610 A	04/01/2017
		CN 106325810 A	11/01/2017
		CN 106325811 A	11/01/2017
		CN 106339202 A	18/01/2017
		CN 106406810 A	15/02/2017

INTERNATIONAL SEARCH REPORT

Information on patent family members

International application No.

PCT/US2017/040546

Patent document cited in search report	Publication date	Patent family member(s)	Publication date
		EP 2963538 A1	06/01/2016
		EP 2963539 A1	06/01/2016
		JP 2016-535360 A	10/11/2016
		JP 2017-010512 A	12/01/2017
		JP 6207574 B2	04/10/2017
		TW 201612739 A	01/04/2016
		TW 201617849 A	16/05/2016
		TW 201617850 A	16/05/2016
		TW 201617857 A	16/05/2016
		TW 201617927 A	16/05/2016
		TW 201617928 A	16/05/2016
		TW 201617929 A	16/05/2016
		US 2016-0004504 A1	07/01/2016
		US 2016-0004505 A1	07/01/2016
		US 2016-0004506 A1	07/01/2016
		US 2016-0004507 A1	07/01/2016
		US 2016-0004508 A1	07/01/2016
		US 2016-0004509 A1	07/01/2016
		US 2016-0004665 A1	07/01/2016
		US 9778907 B2	03/10/2017
		US 9778908 B2	03/10/2017
		US 9798519 B2	24/10/2017
US 2012-0113133 A1	10/05/2012	None	
US 6332186 B1	18/12/2001	CN 1191535 C	02/03/2005
		CN 1303501 A	11/07/2001
		DE 69901708 T2	05/12/2002
		EP 1080420 A1	07/03/2001
		EP 1080420 B1	05/06/2002
		GB 2338094 A	08/12/1999
		GB 2338094 B	28/05/2003
		GB 2339936 A	09/02/2000
		GB 2339936 B	25/09/2002
		IL 139250 D0	25/11/2001
		JP 04166367 B2	15/10/2008
		JP 11-353305 A	24/12/1999
		JP 2000-010959 A	14/01/2000
		JP 2002-517037 A	11/06/2002
		KR 10-0563219 B1	22/03/2006
		MY 125652 A	30/08/2006
		RU 2212049 C2	10/09/2003
		TW 413766 A	01/12/2000
		US 6282634 B1	28/08/2001
		US 6360189 B1	19/03/2002
		WO 99-61996 A1	02/12/1999